



CAPSTONE PROJECT REPORT

Report 3 – Software Requirement Specification

Table of Contents

1. Product Overview	9
2. User Requirements	10
2.1 Actors	10
2.2 Use Cases	11
3. Functional Requirements	20
3.1 System Functional Overview	20
3.2 Authenticate	34
3.3 Manage Password	45
3.4 Manage User Profile	52
3.5 Manage Users (Global & Assign Fixed Roles)	54
3.6 View Flood Map (Real-time Water Levels)	60
3.7 Filter and Toggle Map Layer	63
3.8 View Flood Conditions by Sensor Stations	68
3.9 Register Monitored Areas (My Areas)	72
3.10 Manage Monitored Areas	78
3.11 Evaluate Administrative Area-based Flood Updates	88
3.12 Receive Flood Alerts (Push / SMS / Email)	90
3.13 View Flood Alert History	93
3.14 View Flood History & Trends	94
3.15 View AI-based Flood Prediction	97
3.16 Compare AI Predictions with Actual Data	101
3.17 Identify High-risk Flood Zones (AI)	102
3.18 Request Safe Route Suggestions	103
3.19 Basic safe route	110
3.20 Compare Alternative Routes	115
3.21 Report Flood Points (Photo / Video)	117
3.22 View Community Flood Reports	124
3.23 Vote Flood Reports	127
3.24 Hide Invalid Flood Report	130
3.25 Create Alert Message Templates	132
3.26 View Alert Message Templates	134

3.27 Update Alert Message Templates	136
3.28 Delete Alert Message Templates	138
3.29 Preview Alert Message Templates	140
3.30 Create Sensor Stations	141
3.31 View Sensor Stations	143
3.32 Update Sensor Stations	146
3.33 Delete Sensor Stations	148
3.34 Monitor Sensor & Device Status	150
3.35 View Subscription Plans	152
3.36 Upgrade Subscription	155
3.37 Cancel Subscription	159
3.38 View Subscription Status & Billing History	162
3.39 Create Subscription Plans	166
3.40 View Subscription Plans for Web	168
3.41 Update Subscription Plans	169
3.42 Delete Subscription Plan	170
3.43 Handle Subscription Disputes	172
3.44 View Billing & Payment Records	173
3.45 Publish NewsFeed	175
3.46 View Newsfeed	177
3.47 Create Newsfeed	179
3.48 Update Newsfeed	181
3.49 Delete Newsfeed	182
3.50 Use Offline Mode (Mobile)	183
3.51 View Operational Logs (Sensors / Alerts / Moderation)	186
4. Non-Functional Requirements	186
4.1 External Interfaces	186
4.2 Quality Attributes	187
5. Requirement Appendix	188
5.1 Business Rules	188
5.2 Common Requirements	189
5.3 Application Messages List	189

Table of Figures

Figure 1 System Context Diagram	10
Figure 2 Use Case Diagram Authorization	11
Figure 3 Use Case Diagram Citizen	12
Figure 4 Use Case Diagram Moderator	13
Figure 5 Use Case Diagram Admin	14
Figure 6 Business Workflow Diagram - Safe Route	18
Figure 7 Business Workflow Diagram -AI	18
Figure 8 Business Workflow Diagram – Monitor Area	19
Figure 9 Business Workflow Diagram – Flood Monitoring & Alert	19
Figure 10 Business Workflow Diagram – Premium	20
Figure 11 Screen Flow Guest	20
Figure 12 Screen Flow Citizen	21
Figure 13 Screen Flow Admin & Super Admin	22
Figure 14 Screen Flow Moderator	23
Figure 15 Entity Relationship Diagram	33
Figure 16 Login Screen layout for Mobile	35
Figure 17 Login Screen layout for Website	36
Figure 18 Sign Out Screen layout for Mobile	38
Figure 19 Sign Out Screen layout for Mobile	39
Figure 20 Sign Out Screen layout for Website	40
Figure 21 Google OAuth Screen layout for Mobile	42
Figure 22 Google OAuth Screen layout for Mobile	43
Figure 23 : Google OAuth Screen layout for Website	44
Figure 24 Set Password	46
Figure 25 Set Password	47
Figure 26 Manage Password Screen layout for Mobile	48
Figure 27 Manage Password Screen layout for Website	49
Figure 28 Reset Password	51
Figure 29 Manage User Profile Screen layout for Mobile	53
Figure 30 Manage User Profile Screen layout for Website	54
Figure 31 View User List	55
Figure 32 Create User	57
Figure 33 Update User	58
Figure 34 Ban User	60
Figure 35 Flood Map	62
Figure 36 Filter and Toggle Map Layer	65
Figure 37 Filter and Toggle Map Layer	66
Figure 38 Filter and Toggle Map Layer	67
Figure 39 View Flood Conditions by Sensor Stations	69
Figure 40 View legend	70
Figure 41 View Flood Conditions by Sensor Stations	71
Figure 42 Register Monitored Areas	73

Figure 43 Register Monitored Areas	74
Figure 44 Register Monitored Areas	75
Figure 45 Register Monitored Areas	76
Figure 46 Register Monitored Areas	77
Figure 47 Areas Tab: My Areas sub-tab with WaterLevelAreaCards and flood status badges	80
Figure 48 Area Detail Screen: animated header, water level gauge, contributing sensors list	81
Figure 49 Area Detail Screen: animated header, water level gauge, contributing sensors list	82
Figure 50 Create Area Step 1: Map picker screen — user pins a location on the Mapbox map	83
Figure 51 Create Area Step 2: Name, address, and radius input form with inline validation	84
Figure 52 Edit button	85
Figure 53 Edit Area: EditAreaSheet bottom sheet pre-filled with existing area data	86
Figure 54 Delete Area: ConfirmDeleteModal with area name, warning, Confirm / Cancel buttons	87
Figure 55 Evaluate Administrative Area-based Flood Updates	89
Figure 56 Receive Flood Alerts	92
Figure 57 View Flood Alert History	94
Figure 58 View Flood History	96
Figure 59 View AI-based Flood Prediction	98
Figure 60 View AI-based Flood Prediction	99
Figure 61 View AI-based Flood Prediction	100
Figure 62 View AI-based Flood Prediction	101
Figure 63 Compare AI Prediction with Actual Data	102
Figure 64 Identify High-risk Flood Zones (AI)	103
Figure 65 Request Safe Route Suggestions	105
Figure 66 Request Safe Route Suggestions	106
Figure 67 Request Safe Route Suggestions	107
Figure 68 Request Safe Route Suggestions	108
Figure 69 Request Safe Route Suggestions	109
Figure 70 Input Origin and Destination	111
Figure 71 Input Origin and Destination	112
Figure 72 Input Origin and Destination	113
Figure 73 Input Origin and Destination	114
Figure 74 Compare Alternative Routes	116
Figure 75 Report Flood Points	119
Figure 76 Report Flood Points	120
Figure 77 Report Flood Points	121
Figure 78 Report Flood Points	122
Figure 79 Report Flood Points	123
Figure 80 View Community Flood Reports	126
Figure 81 Vote Flood Reports	129
Figure 82 Hide Invalid Flood Report	131
Figure 83 Create Alert Message Templates	133
Figure 84 View Alert Message Templates	135
Figure 85 Update Alert Message Templates	137
Figure 86 Delete Alert Message Templates	139
Figure 87 Preview Alert Message Templates	141

Figure 88 Create Sensor Stations	142
Figure 89 View Sensor Station	145
Figure 90 Update Sensor Station	147
Figure 91 Delete Sensor Station	149
Figure 92 Monitor Sensor and Device Status	152
Figure 93 View Subscription Plans	154
Figure 94 Upgrade Subscription	157
Figure 95 Current Subscription	158
Figure 96 Cancel Subscription	161
Figure 97 View Subscription Status	164
Figure 98 View Billing History	165
Figure 99 Create Subscription Plans	167
Figure 100 View Subscription Plans for web	168
Figure 101 Update Subscription Plans	170
Figure 102 Delete Subscription Plans	171
Figure 103 Handle Subscription Disputes	173
Figure 104 View Billing and Payment Records	174
Figure 105 Publish NewsFeed	176
Figure 106 New Feed of Mobile	178
Figure 107 Create Newsfeed	180
Figure 108 Update Newsfeed	181
Figure 109 Delete Newsfeed	183
Figure 110 Use Offline Mode (Mobile)	185
Figure 111 View Operational Logs	186

Table of Tables

Table 1 Record of Changes	8
Table 2 Actors	11
Table 3 UseCase	17
Table 4 Screen Descriptions	25
Table 5 Screen Authorization	26
Table 6 Non-Screen Functions	27
Table 7 Entities Description	29
Table 8 Business Rules	184
Table 9 Application Messages List	185

I. Record of Changes

Date	A* M, D	In charge	Change Description
10/02/2026	A	PhucTNH	Create base for Report 3
12/02/2026	A	PhucTNH	Add Product Overview
25/02/2026	A	PhucTNH	Add User Requirement
01/03/2026	A	HoaTX	Add Functional Requirement
13/03/2026	M	PhucTNH	Update Screen Flow
14/04/2026	M	TuanTA	Update Use Case Diagram

Table 1 Record of Changes

*A - Added M - Modified D - Deleted

II. Software Requirement Specification

1. Product Overview

FDA is a real-time flood monitoring and safe routing platform for Da Nang, built on NextJS, React Native, ASP.NET Core, and PostgreSQL. By integrating IoT sensors and AI-based prediction, it delivers accurate flood data and personalized alerts to citizens, moderators, and administrators. The system automates data collection, replacing manual reporting with a comprehensive admin dashboard. Its scalable architecture enhances public safety and offers a flexible foundation for expansion across Vietnam.

Key Objectives of the System

1. **Unified real-time flood monitoring platform:** Collects IoT water-level data and visualizes flood conditions on dynamic web and mobile maps.
2. **Automate data collection and minimize manual processes:** Use IoT sensor networks and automated data pipelines to replace manual reporting, ensuring faster, more accurate, and more consistent flood information.
3. **Provide intelligent flood-aware routing:** Analyze real-time sensor data and leverage Mapbox Directions API to suggest safer paths and compare safe versus alternative routes during floods.
4. **Deliver personalized alerting and flood warnings:** Offer user-defined alert zones, push/SMS/email notifications, and real-time threshold-based alerts to enhance public safety.
5. **Support short-term AI-based flood prediction:** Utilize historical and real-time data to forecast flood conditions 3, 5, or 7 hours in advance, giving users and administrators more time to react and prepare.
6. **Enable community participation:** Allow citizens to report flood points, view and vote on community flood reports, and contribute to real-time situational awareness.
7. **Provide timely news and announcements:** Allow Moderators and Admins to publish official flood updates and announcements, keeping citizens informed through a dedicated newsfeed.
8. **Improve operational efficiency for administrators:** Empower administrators with tools to manage sensors, monitor device health, configure system thresholds, and access analytics or exportable reports.
9. **Ensure scalability:** Design a flexible architecture to easily expand sensor networks and adapt to other flood-prone cities in Vietnam.

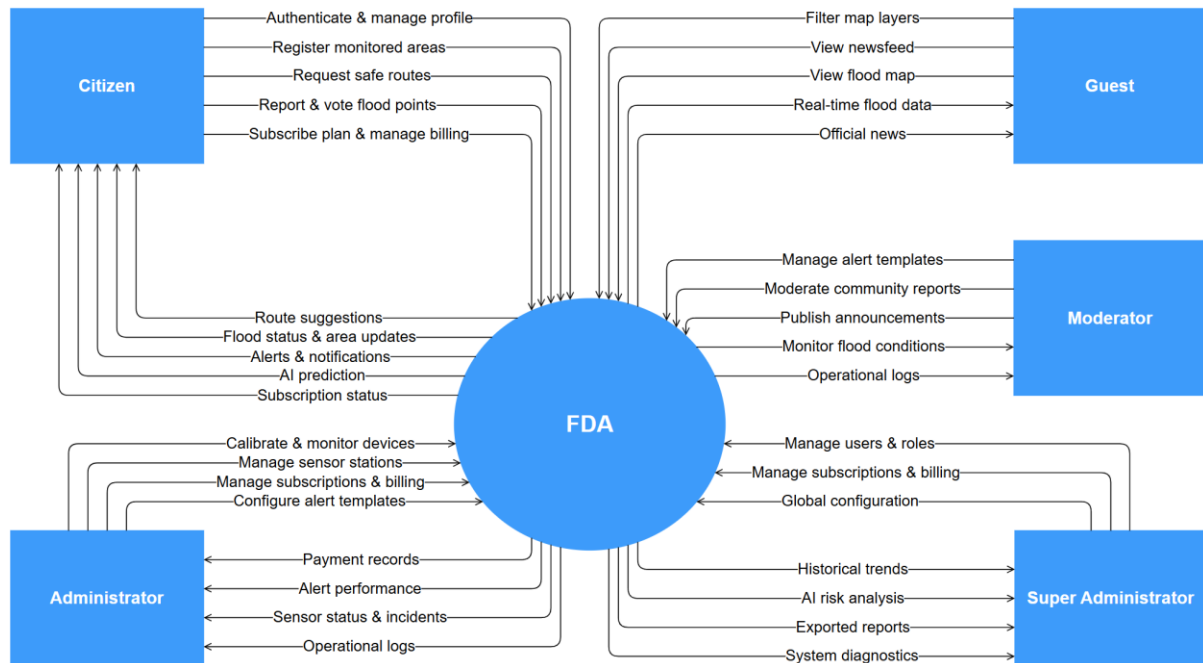


Figure 1 System Context Diagram

2. User Requirements

2.1 Actors

#	Actor	Description
1	Guest	A visitor who accesses the FloodGuard system without authentication. This actor can only view publicly available information and does not have access to personalized or interactive features.
2	Citizen	A registered citizen who uses the FloodGuard system to monitor flood conditions, receive alerts, plan safe routes, and contribute community flood information. This actor represents the main group of end users affected by flooding.
3	Moderator	An official user representing government agencies or emergency response organizations. This actor uses FloodGuard as a decision-support system to monitor flood situations, validate community data, coordinate flood response actions, and configure alert templates for flood notifications.
4	Admin	A system operator responsible for managing the operational components of FloodGuard, including IoT infrastructure, flood event records, announcements, subscriptions, and billing. This actor ensures system availability and data accuracy.
5	Super Admin	The highest-privileged system administrator with full control over FloodGuard. This actor manages global users, roles, permissions, and system-wide configurations to ensure security, compliance, and reliability.

Table 2 Actors

2.2 Use Cases

2.2.1 Diagram(s)

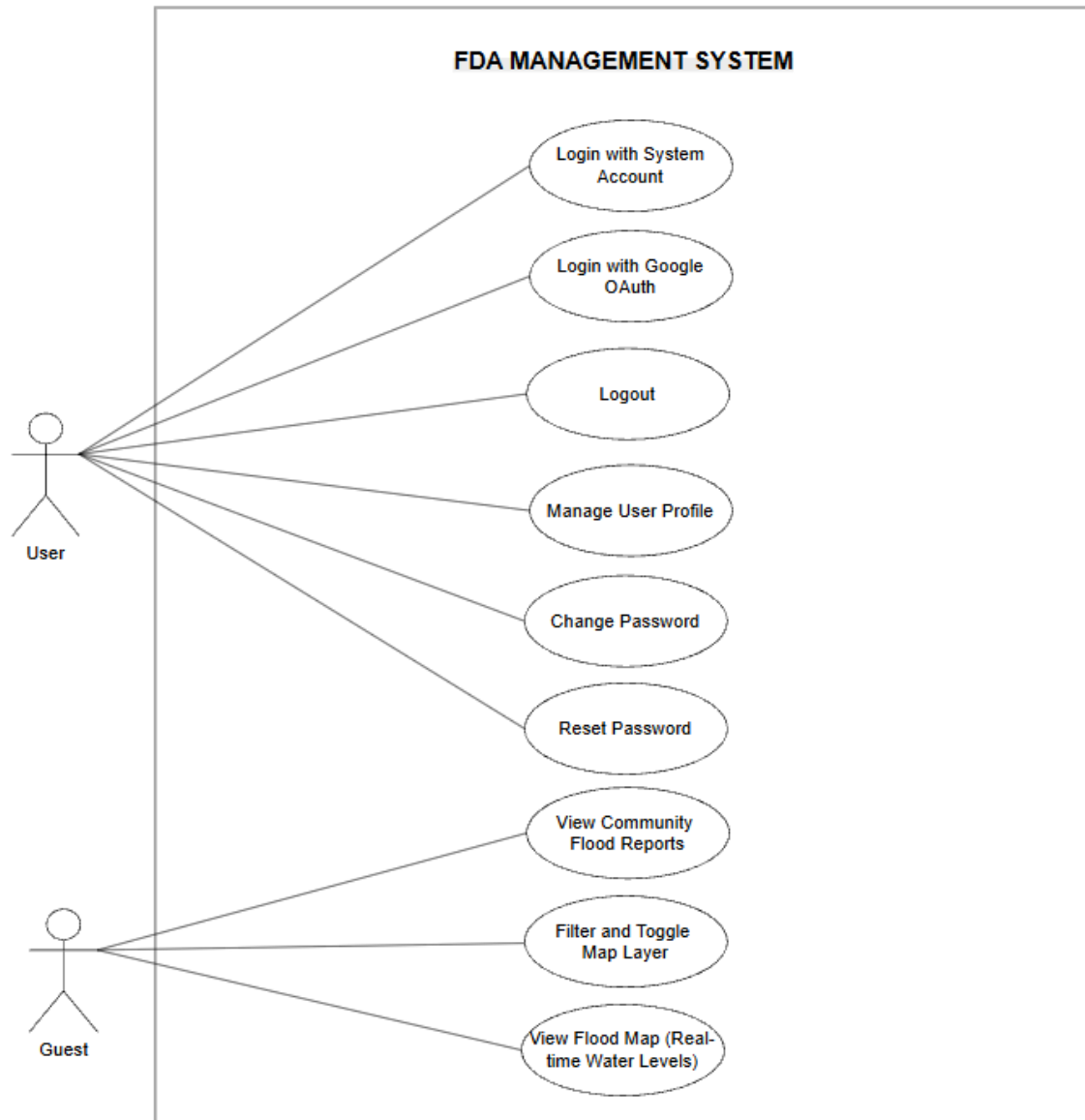


Figure 2 Use Case Diagram Authorization

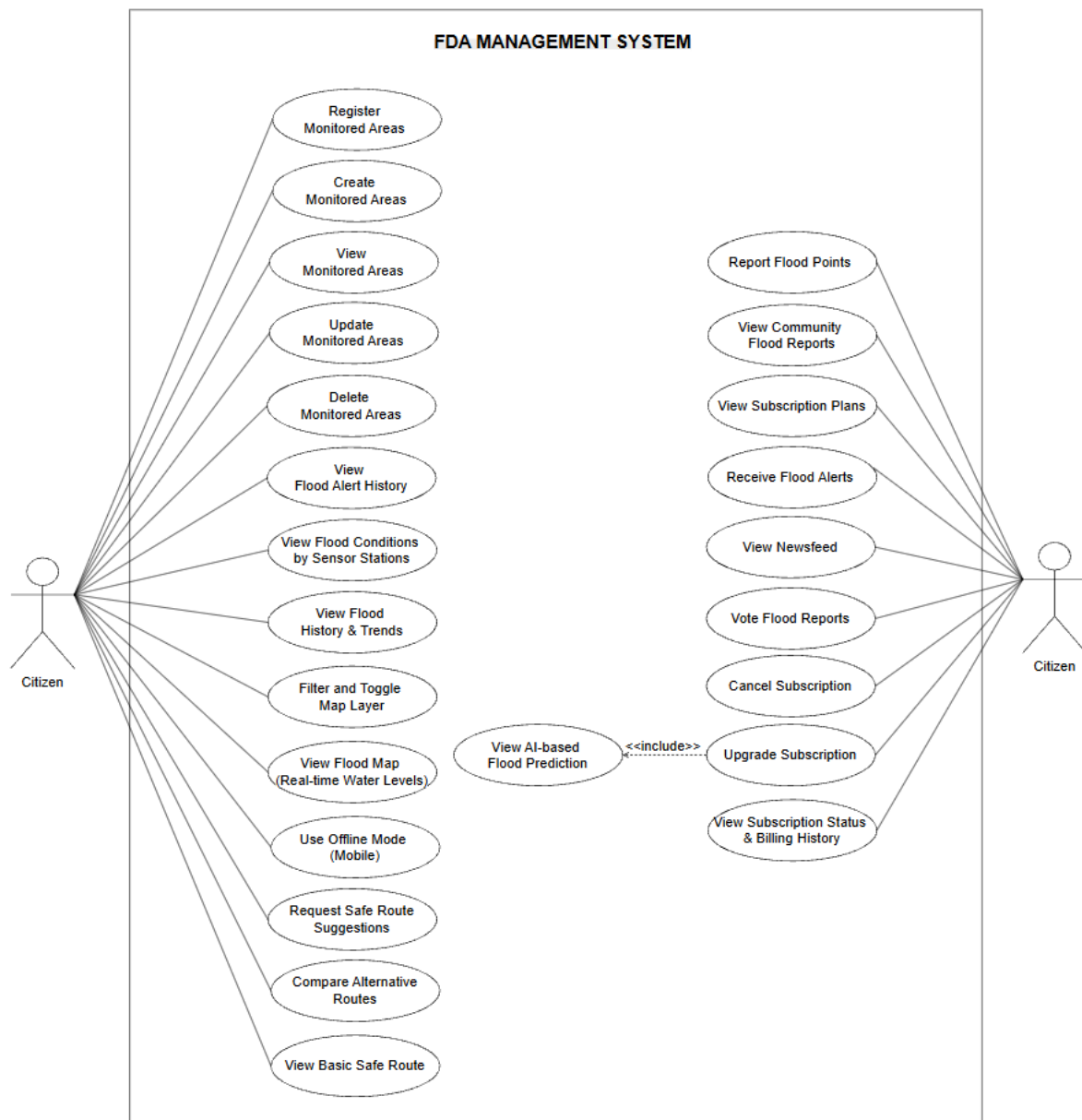


Figure 3 Use Case Diagram Citizen

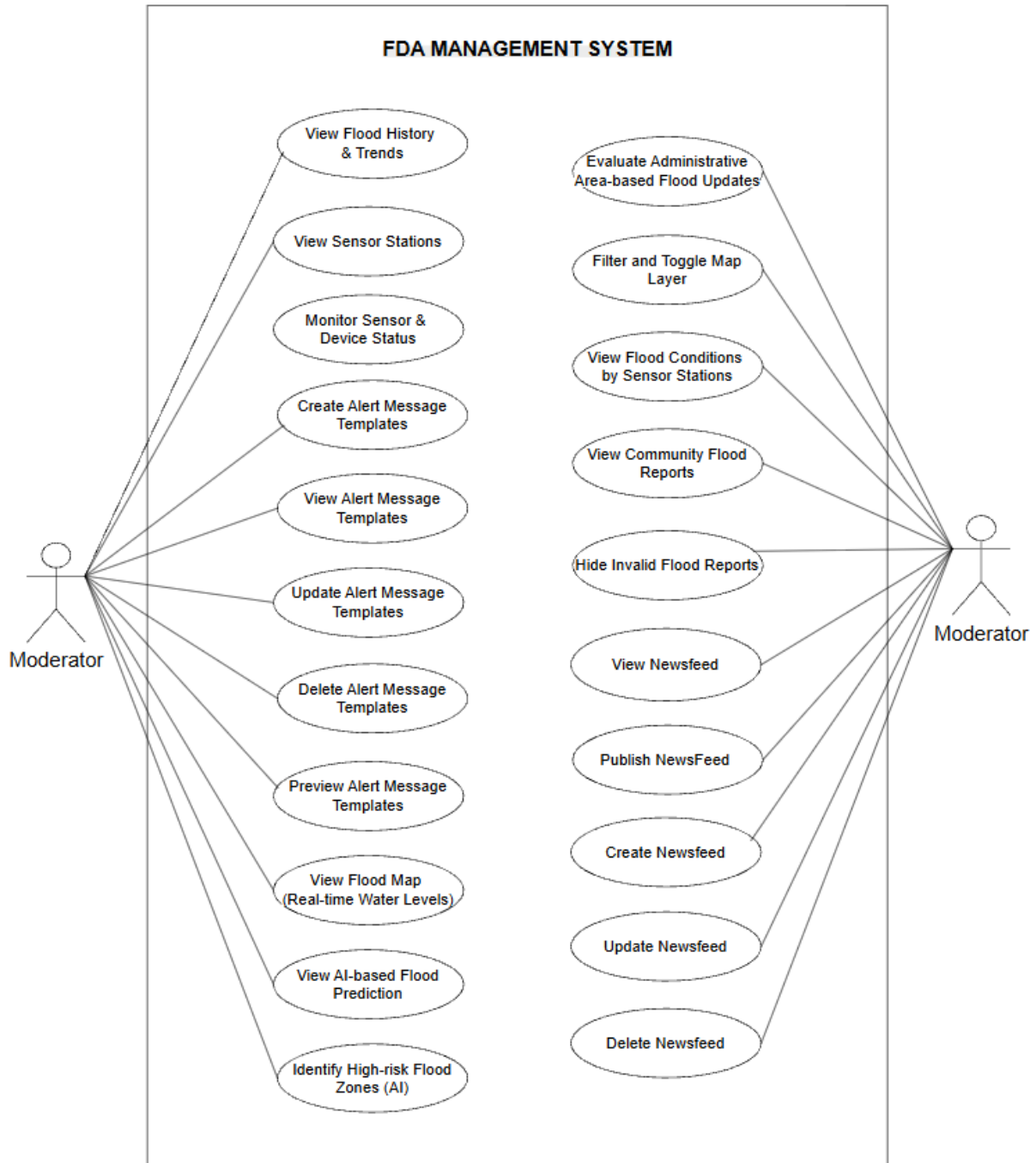


Figure 4 Use Case Diagram Moderator



Figure 5 Use Case Diagram Admin

2.2.2 Descriptions

ID	Use Case	Actors	Use Case Description
FE-01	Login with System Account	Citizen, Moderator, Admin	Allows users to authenticate into the system to access personalized and role-based features.
FE-02	Login with Google OAuth	Citizen, Moderator, Admin	Enables users to log in using Google OAuth for faster and secure authentication.
FE-03	Logout	Citizen, Moderator, Admin	Allow users to logout.
FE-04	Change Password	Citizen, Moderator, Admin	Allows users to update their password.
FE-05	Reset Password	Citizen, Moderator, Admin	Allows users to recover/reset their password when forgotten.
FE-06	Manage User Profile	Citizen, Moderator, Admin	Allows users to view and update their personal profile information.
FE-07	Create User	Super Admin	Allows Super Admin to create user accounts and assign fixed roles (Citizen, Monitor, Admin)
FE-08	View User	Super Admin	Allows Super Admin to view list of user accounts
FE-09	Update User	Super Admin	Allows Super Admin to update user accounts (account created by Admin)
FE-10	Delete User	Super Admin	Allows Super Admin to delete user accounts (account created by Admin)
FE-11	Ban User	Super Admin	Allows Super Admin to ban user accounts
FE-12	View Flood Map (Real-time Water Levels)	Guest, Citizen, Moderator, Admin	Displays an interactive real-time flood map showing water levels, flooded areas, sensor locations, and flood severity.
FE-13	Filter and Toggle Map Layers	Guest, Citizen, Moderator, Admin	Allows users to enable or disable map layers such as flood severity, traffic, satellite, and weather.
FE-14	View Flood Conditions by Sensor Stations	Citizen, Moderator, Admin	Displays flood conditions and latest readings collected from sensor stations.
FE-15	Register Monitored Areas (My Areas)	Citizen	Allows citizens to register areas they want to monitor for flood conditions.
FE-16	Create Monitored Areas	Citizen	Enables citizens to add monitored areas.
FE-17	View Monitored Areas	Citizen	Enables citizens to view monitored areas.
FE-18	Update Monitored Areas	Citizen	Enables citizens to edit monitored areas.
FE-19	Delete Monitored Areas	Citizen	Enables citizens to remove monitored areas.
FE-20	Evaluate Administrative Area-based Flood Updates	Admin, Moderator	Enables the evaluation of administrative area-based flood updates, utilizing water

ID	Use Case	Actors	Use Case Description
			level data from monitoring stations located within each respective area
FE-21	Receive Flood Alerts (Push / SMS / Email)	Citizen	Sends flood alerts through multiple notification channels.
FE-22	View Flood Alert History	Citizen	Allows citizens to review previously received flood alerts.
FE-23	View Flood History & Trends	Citizen, Moderator,	Displays historical flood data and trends by region over time.
FE-24	View AI-based Flood Prediction	Citizen (Premium), Moderator, Admin	Displays AI-based flood predictions 3-5-7 hours.
FE-25	Compare AI Predictions with Actual Data	Admin	Allows evaluate AI prediction accuracy using real flood data and save data for AI training
FE-26	Identify High-risk Flood Zones (AI)	Moderator, Admin	Identifies flood-prone areas using AI-based risk analysis.
FE-27	Request Safe Route Suggestions	Citizen	Generates safe travel routes that avoid flooded areas.
FE-28	View Basic Safe Route	Citizen	Allows Citizen to View Basic Safe Route after analyzing
FE-29	Compare Alternative Routes	Citizen	Displays multiple route options with safety comparisons.
FE-30	Report Flood Points (Photo / Video)	Citizen	Allows citizens to submit flood reports with multimedia evidence.
FE-31	View Community Flood Reports	Citizen, Moderator	Displays flood reports submitted by the community.
FE-32	Hide Invalid Flood Report	Moderator	Allows moderator to hide invalid flood reports
FE-33	Vote Flood Reports	Citizen	Allows citizens to vote community flood reports for verification.bô
FE-34	Create Alert Message Templates	Moderator	Allows create alert message templates for flood notifications.
FE-35	View Alert Message Templates	Moderator	Allows view alert message templates for flood notifications.
FE-36	Update Alert Message Templates	Moderator	Allows update alert message templates for flood notifications.
FE-37	Delete Alert Message Templates	Moderator	Allows delete alert message templates for flood notifications.
FE-38	Preview Alert Message Templates	Moderator	Allows to check UI of message template for each channel before sending
FE-39	Create Sensor Stations	Admin	Allows administrators to add sensor stations.
FE-40	View Sensor Stations	Admin, Moderator	Allows administrators to view sensor stations.
FE-41	Update Sensor Stations	Admin	Allows administrators to edit sensor stations.
FE-42	Delete Sensor Stations	Admin	Allows administrators to remove sensor stations.

ID	Use Case	Actors	Use Case Description
FE-43	Monitor Sensor & Device Status	Admin	Displays real-time sensor/device status including connectivity and battery level.
FE-44	View Subscription Plans	Citizen	Allows citizens to view available subscription plans and features.
FE-45	Upgrade Subscription	Citizen	Allows citizens to upgrade their subscription plan.
FE-46	Cancel Subscription	Citizen	Allows citizens to cancel their active subscription.
FE-47	View Subscription Status & Billing History	Citizen	Displays subscription status and billing history.
FE-48	Create Subscription Plans	Admin	Allows administrators to create pricing and feature limits.
FE-49	View Subscription Plans	Admin	Allows administrators to view pricing and feature limits.
FE-50	Update Subscription Plans	Admin	Allows administrators to edit pricing and feature limits.
FE-51	Delete Subscription Plans	Admin	Allows administrators to remove pricing and feature limits.
FE-52	Handle Subscription Disputes	Admin	Enables handling of subscription and payment disputes.
FE-53	View Billing & Payment Records	Admin	Allows administrators to view billing and payment records.
FE-54	Publish Newsfeed	Moderator	Allows posting of official flood-related news and announcements.
FE-55	View Newsfeed	Guest, Citizen, Moderator	Displays official flood-related news and announcements.
FE-56	Create Newsfeed	Moderator	Create official flood-related news and announcements.
FE-57	Update Newsfeed	Moderator	Update official flood-related news and announcements.
FE-58	Delete Newsfeed	Moderator	Delete official flood-related news and announcements.
FE-59	Use Offline Mode (Mobile)	Citizen	Allows citizens to access cached flood information when offline.
FE-60	View Operational Logs (Sensors / Alerts / Moderation)	Admin	Displays operational logs related to sensors, alerts, and moderation activities.

Table 3 UseCase

2.2.3. Business Workflow

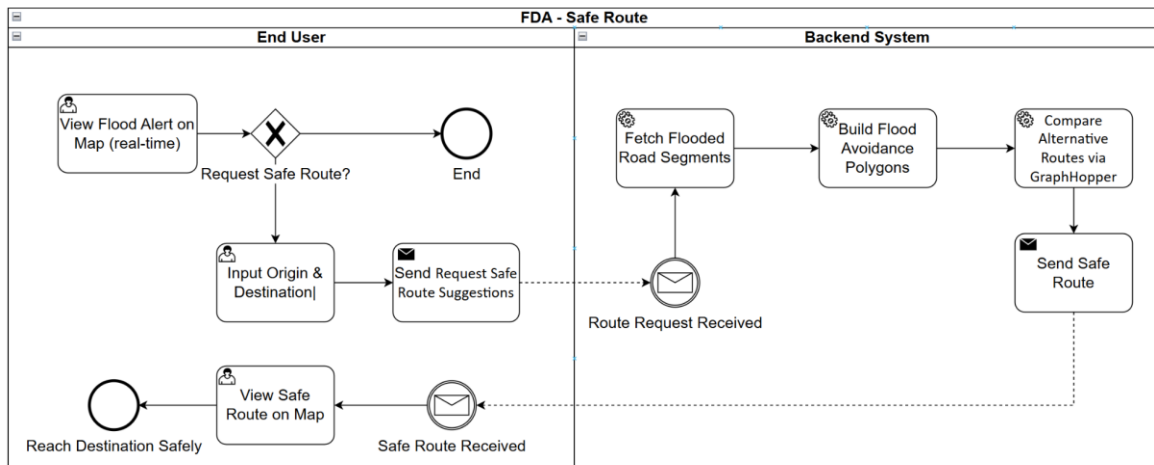


Figure 6 Business Workflow Diagram - Safe Route

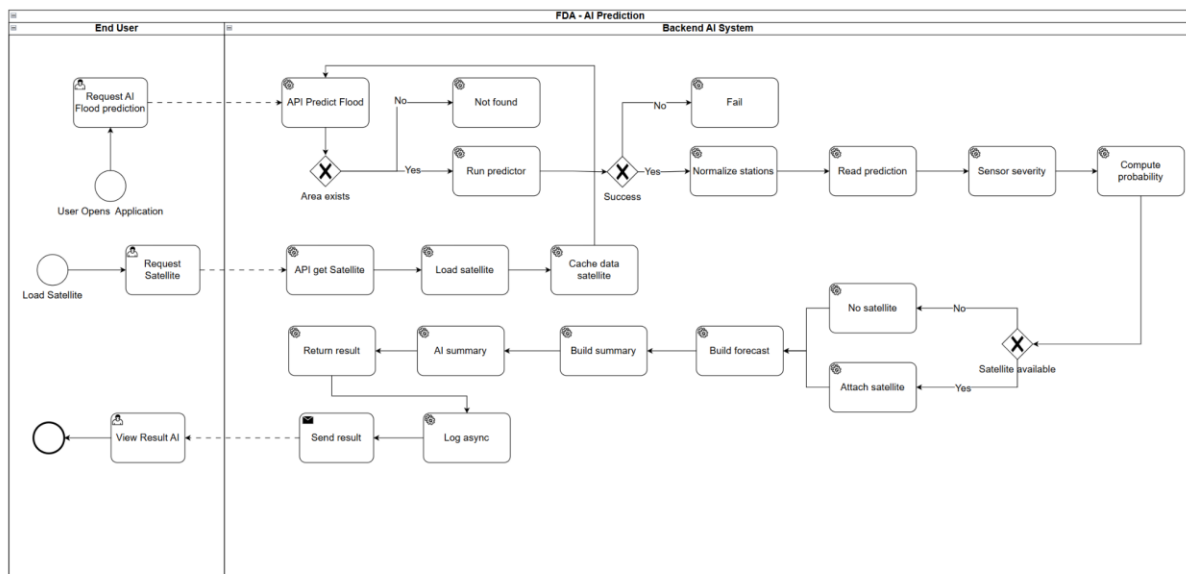


Figure 7 Business Workflow Diagram -AI

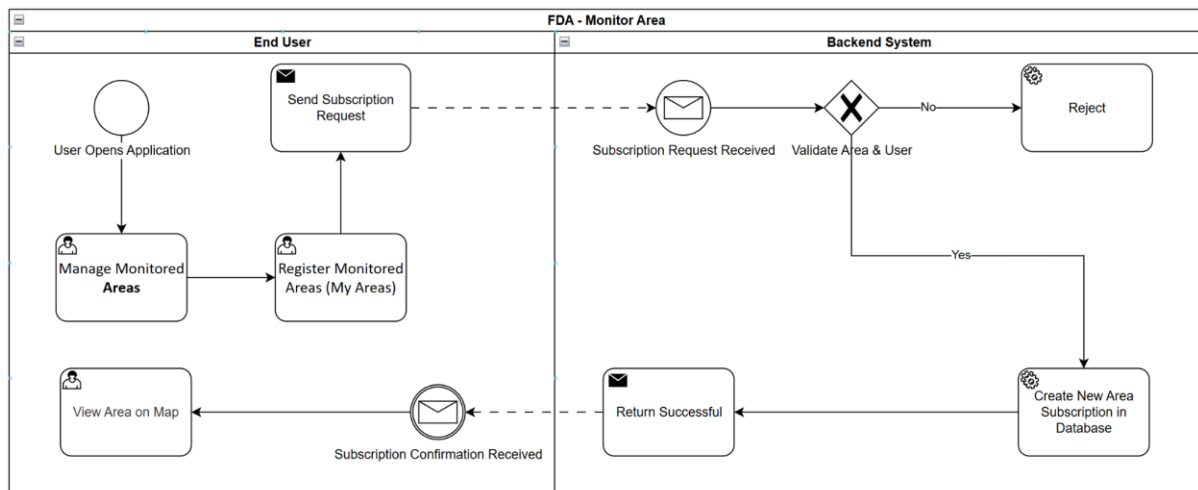


Figure 8 Business Workflow Diagram – Monitor Area

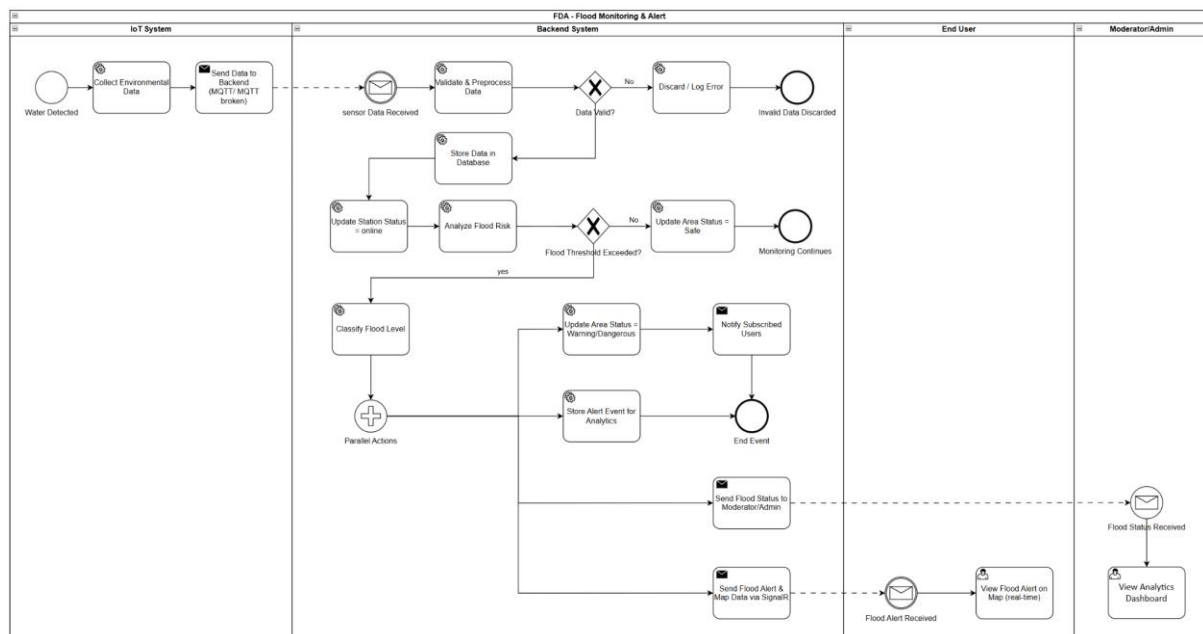


Figure 9 Business Workflow Diagram – Flood Monitoring & Alert

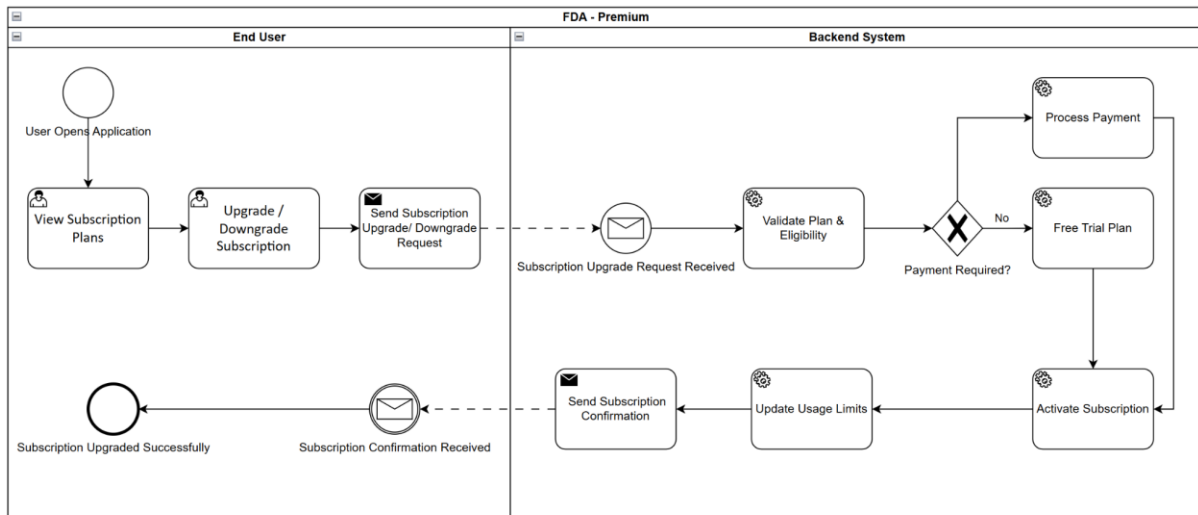


Figure 10 Business Workflow Diagram – Premium

3. Functional Requirements

3.1 System Functional Overview

3.1.1 Screens Flow

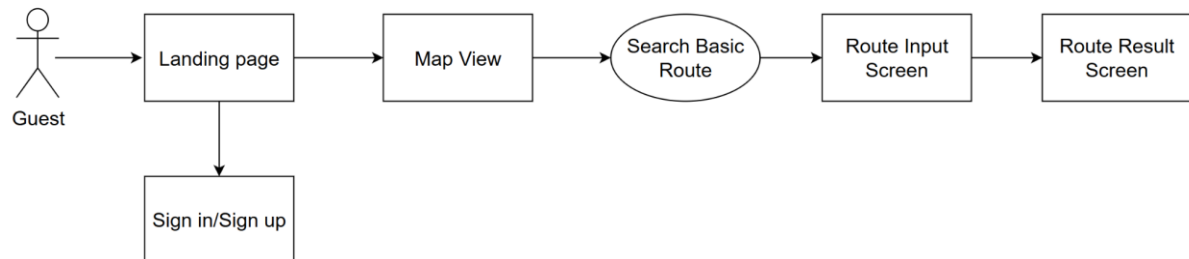


Figure 11 Screen Flow Guest

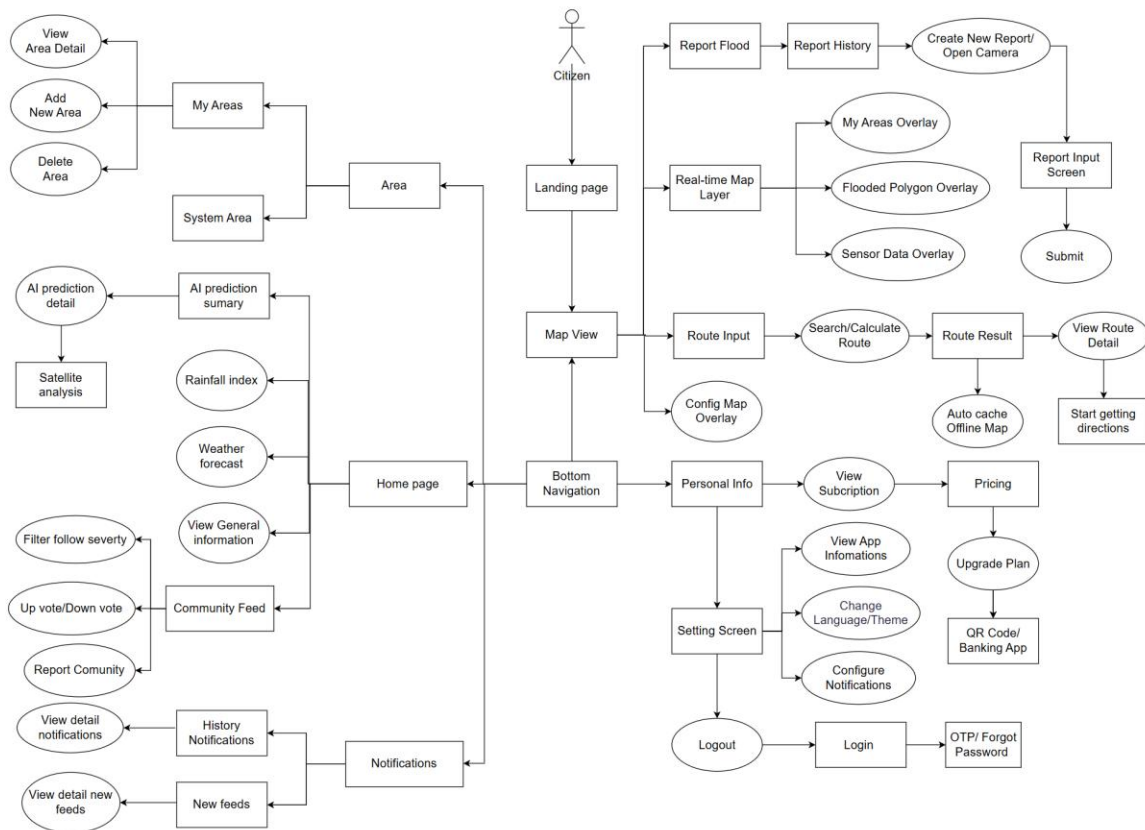


Figure 12 Screen Flow Citizen

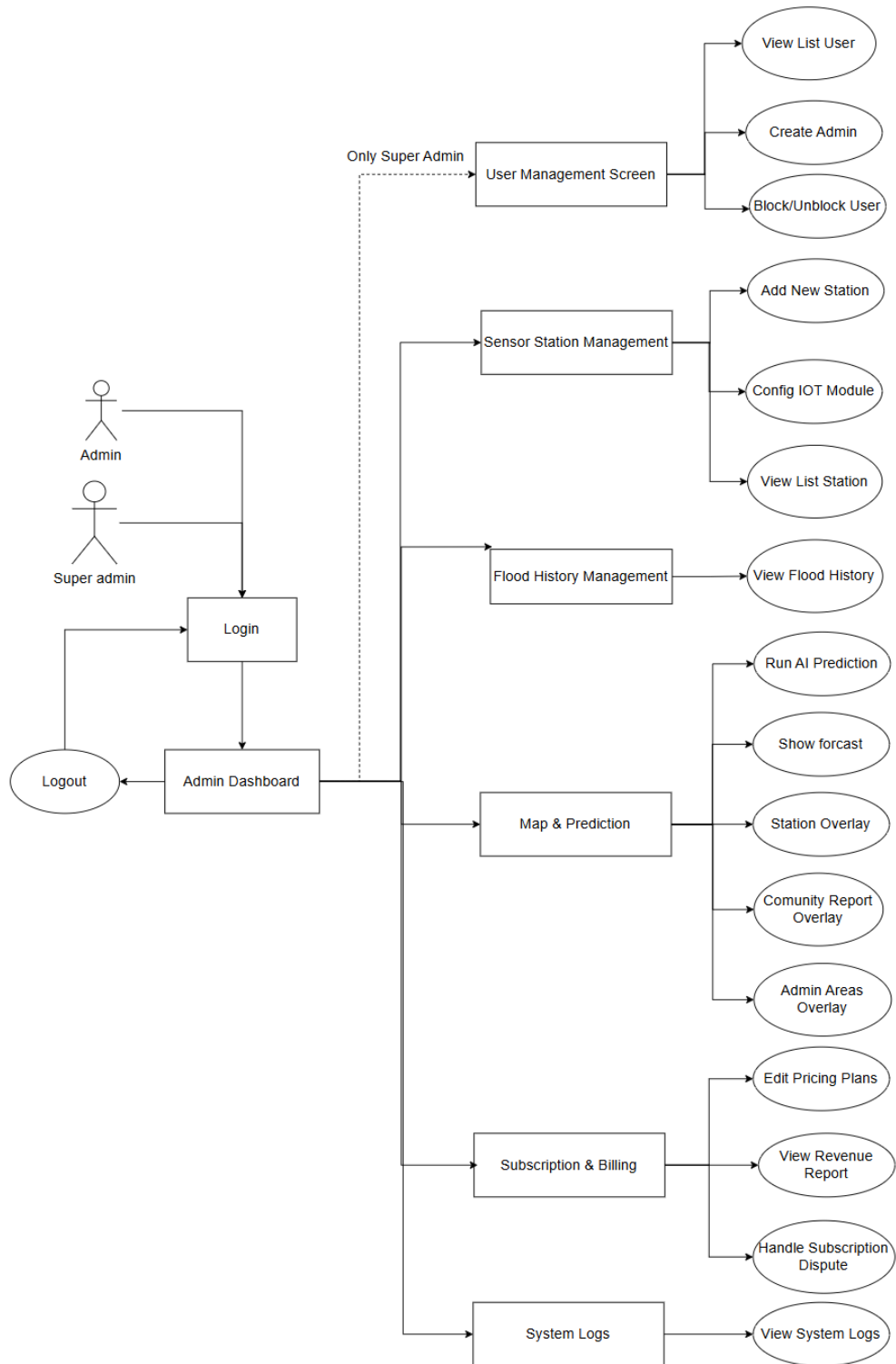


Figure 13 Screen Flow Admin & Super Admin

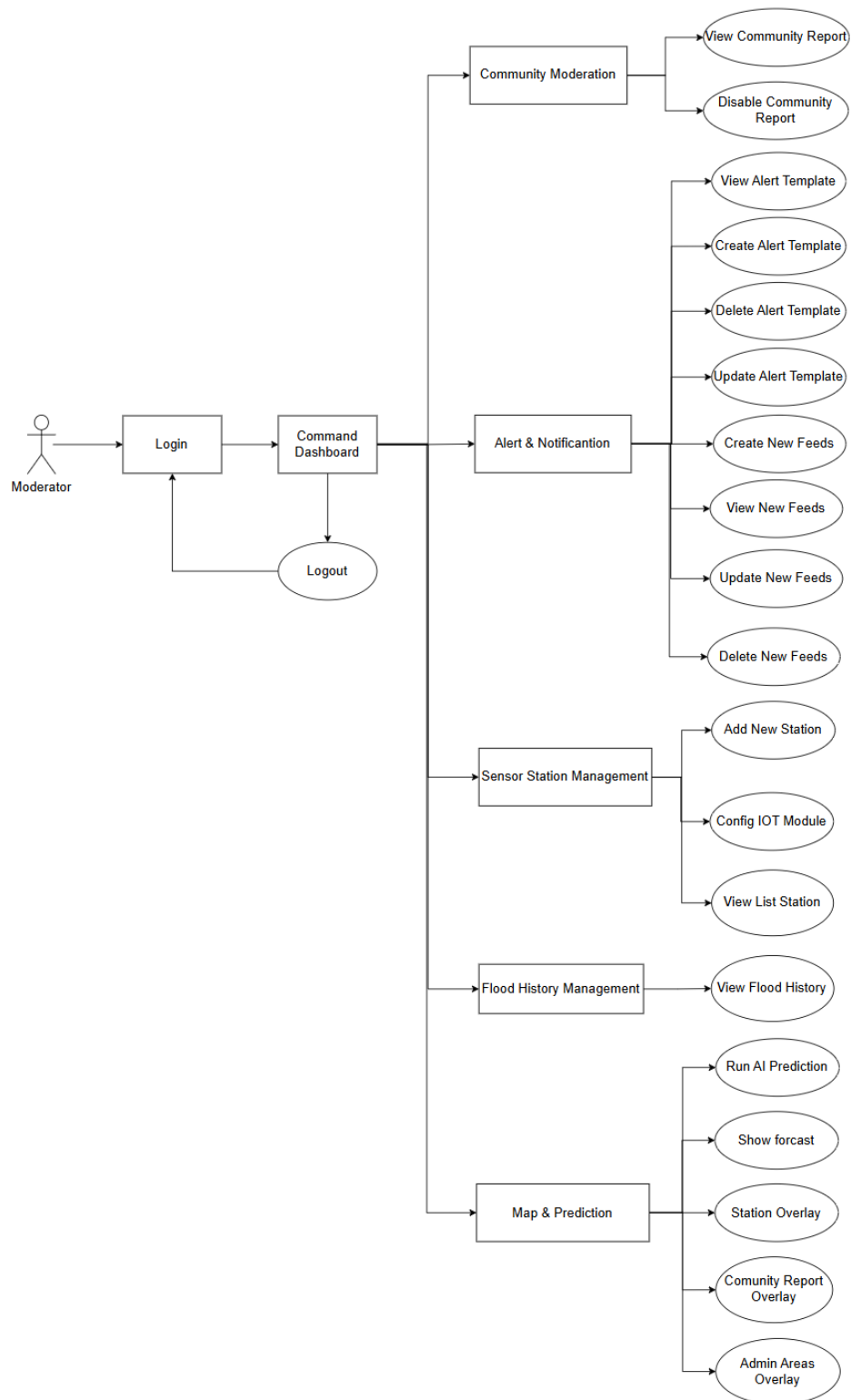


Figure 14 Screen Flow Moderator

3.1.2 Screen Descriptions

#	Feature	Screen	Description
1	Authentication & Account	Login	Screen where users enter their credentials such as email, phone number, password, or OTP to access the system.
2	Authentication & Account	Register / Sign Up	Screen for creating a new user account with basic personal information and verification.
3	Authentication & Account	OTP / Forgot Password	Screen that allows users to request an OTP or reset their password when they cannot log in.
4	Core Platform	Landing Page	First screen shown after opening the application, providing entry points to the main map, authentication, and key features.
5	Core Platform	Home Page	Main home screen showing general flood information, AI prediction summary, community feed, and notification/news access.
6	Core Platform	Bottom Navigation	Persistent bottom navigation bar that allows users to switch between main sections such as Home, Map, Notifications, and Profile.
7	Flood Map Core	Map View	Main interactive map screen displaying the base map, user location, flood-related layers, and route access.
8	Flood Map Core	Real-time Map Layer	Dynamic map layer displaying live flood status, sensor information, and real-time geospatial flood data.
9	Flood Map Core	My Areas Overlay	Map overlay showing the user's saved areas on the map.
10	Flood Map Core	Flooded Polygon Overlay	Map overlay showing inundated or flood-affected regions.
11	Flood Map Core	Sensor Data Overlay	Map overlay displaying real-time data from IoT sensors, such as water level or rainfall readings.

#	Feature	Screen	Description
12	Flood Map Core	Configure Map Overlay	Screen or control panel that allows users to enable or disable map overlays.
13	Area Management	Area	Main area section where users access saved areas and system-defined flood monitoring areas.
14	Area Management	My Areas	Screen for managing saved locations such as home, school, office, or frequently visited places.
15	Area Management	View Area Detail	Screen showing detailed flood status, prediction, and related information for a selected saved area.
16	Area Management	Add New Area	Screen for adding a new saved area by selecting a location or entering an address.
17	Area Management	Delete Area	Function that allows users to remove a saved area from their personal area list.
18	Area Management	System Area	Screen listing official flood monitoring areas defined by the system or administrators.
19	AI Prediction	AI Prediction Summary	Summary section showing predicted flood risk, severity level, rainfall index, and forecast overview.
20	AI Prediction	AI Prediction Detail	Detailed screen explaining flood prediction results, including supporting data and analysis.
21	AI Prediction	Satellite Analysis	Screen showing satellite-based flood or rainfall analysis for selected regions.
22	AI Prediction	Rainfall Index	Screen or component showing rainfall intensity indicators used to support flood prediction.
23	AI Prediction	Weather Forecast / Show Forecast	Screen displaying weather forecast information related to flood risk and prediction.
24	Information	View General Information	Screen showing general flood-related information, safety guidance, or system-provided public information.

#	Feature	Screen	Description
25	Community Reporting	Community Feed	Screen listing community-submitted flood reports or posts with severity, location, and interaction options.
26	Community Reporting	Filter by Severity	Filter function that allows users to view community reports based on flood severity level.
27	Community Reporting	Upvote / Downvote	Function that allows users to vote on community reports to reflect usefulness or credibility.
28	Community Reporting	Report Community Post	Function that allows users to report inappropriate, false, or problematic community content.
29	Community Reporting	Report Flood	Entry screen or function for citizens to submit a new flood incident report.
30	Community Reporting	Report History	Screen showing all flood reports submitted by the user, including status and timestamps.
31	Community Reporting	Create New Report / Open Camera	Screen or action that opens the camera to capture evidence for a new flood report.
32	Community Reporting	Report Input Screen	Form where citizens enter flood report details such as location, image, severity, and description.
33	Community Reporting	Submit Report	Action that submits the completed flood report to the system for processing or moderation.
34	Safe Routing	Route Input Screen	Screen where users enter or select origin and destination points to calculate a route.
35	Safe Routing	Search / Calculate Route	Function that calculates a safe route based on flood data, map information, and routing rules.
36	Safe Routing	Search Basic Route	Basic route search function available to guest users with limited features.
37	Safe Routing	Route Result Screen	Screen displaying recommended route results based on safety, distance, and travel time.

#	Feature	Screen	Description
38	Safe Routing	View Route Detail	Screen showing detailed route information such as distance, estimated time, safety level, and flood risks.
39	Safe Routing	Start Getting Directions	Function that starts turn-by-turn navigation based on the selected route.
40	Safe Routing	Auto Cache Offline Map	Function that automatically stores route or map data for offline access after route calculation.
41	Notifications	Notifications	Main notification screen showing alerts, warnings, and system messages.
42	Notifications	Notification History	Screen listing previous notifications received by the user.
43	Notifications	View Notification Detail	Screen showing full details of a selected notification or alert.
44	News	News Feed	Screen listing official flood-related news, updates, and announcements.
45	News	View News Feed Detail	Screen showing the full content of a selected news item.
46	Account & Profile	Personal Info	Profile screen where users view or edit personal information and account details.
47	Subscription	View Subscription	Screen showing the user's current subscription plan, benefits, and status.
48	Subscription	Pricing	Public pricing screen showing available subscription packages and plan comparisons.
49	Subscription	Upgrade Plan	Screen or action that allows users to upgrade to a selected subscription plan.

#	Feature	Screen	Description
50	Subscription	QR Code / Banking App Payment	Payment screen showing QR code or banking app payment instructions for plan upgrade.
51	Settings	Setting Screen	In-app settings screen where users configure language, theme, notifications, app information, and logout.
52	Settings	View App Information	Screen showing app version, product information, terms, policy, or support information.
53	Settings	Change Language / Theme	Function that allows users to change application language and visual theme.
54	Settings	Configure Notifications	Screen where users configure notification preferences and alert settings.
55	Authentication & Account	Logout	Function that signs the user out and returns them to the login screen.
56	Administration	Admin Dashboard	Main dashboard for administrators showing key metrics, alerts, and shortcuts to management modules.
57	Administration	Command Dashboard	Main dashboard for moderators to access moderation, alert, notification, station, flood history, and map functions.
58	User Management	User Management Screen	Management screen where super admins manage users, admins, roles, and account access.
59	User Management	View List User	Screen listing all users in the system with role, status, and account information.
60	User Management	Create Admin	Screen or function that allows a super admin to create a new admin account.
61	User Management	Block / Unblock User	Function that allows a super admin to block or unblock user accounts.

#	Feature	Screen	Description
62	IoT & Sensor Operations	Sensor Station Management	Screen for managing sensor stations, assigning them to areas, and monitoring station status.
63	IoT & Sensor Operations	Add New Station	Screen or function for adding a new sensor station to the system.
64	IoT & Sensor Operations	Configure IoT Module	Screen for configuring IoT module settings, connection parameters, and device behavior.
65	IoT & Sensor Operations	View List Station	Screen listing all sensor stations with status, location, and basic operational information.
66	Flood Data Management	Flood History Management	Management screen for accessing and organizing historical flood records.
67	Flood Data Management	View Flood History	Screen showing historical flood data, events, and trends by area or time range.
68	Map & Prediction	Map & Prediction	Screen that visualizes flood maps, prediction outputs, stations, reports, and administrative overlays.
69	Map & Prediction	Run AI Prediction	Function that triggers the AI model to generate or update flood prediction results.
70	Map & Prediction	Station Overlay	Map overlay displaying sensor station locations and related status information.
71	Map & Prediction	Community Report Overlay	Map overlay displaying flood reports submitted by citizens or the community.
72	Map & Prediction	Admin Areas Overlay	Map overlay displaying administrative or system-managed flood monitoring areas.
73	Subscription & Billing	Subscription & Billing	Admin screen for managing subscription records, billing status, payments, and related information.
74	Subscription & Billing	Edit Pricing Plans	Screen or function for editing available pricing plans, subscription tiers, and feature sets.
75	Subscription & Billing	View Revenue Report	Screen showing revenue reports, payment statistics, and subscription-related financial information.

#	Feature	Screen	Description
76	Subscription & Billing	Handle Subscription Dispute	Screen or function for reviewing and resolving subscription or payment disputes.
77	System Monitoring	System Logs	Screen for viewing system logs, errors, events, and administrative audit history.
78	System Monitoring	View System Logs	Screen showing detailed system activity logs for monitoring and troubleshooting.
79	Community Moderation	Community Moderation	Screen where moderators review, manage, and control community-submitted flood reports or posts.
80	Community Moderation	View Community Report	Screen showing detailed information about a community-submitted report.
81	Community Moderation	Disable Community Report	Function that allows moderators to disable or hide inappropriate, false, or invalid community reports.
82	Alert & Notification	Alert & Notification	Central screen for managing alert templates, flood warning messages, and news feed content.
83	Alert & Notification	View Alert Template	Screen listing existing alert templates used for flood warnings or notifications.
84	Alert & Notification	Create Alert Template	Screen for creating a new reusable alert or flood warning template.
85	Alert & Notification	Update Alert Template	Screen for editing an existing alert template.
86	Alert & Notification	Delete Alert Template	Function that allows moderators to remove an unused or outdated alert template.
87	News Management	Create News Feed	Screen for creating a new official news feed or public announcement.
88	News Management	View News Feed	Screen listing existing news feeds or announcements created by moderators.
89	News Management	Update News Feed	Screen for editing the content of an existing news feed.
90	News Management	Delete News Feed	Function that allows moderators to remove outdated or invalid news feed items.

Table 4 Screen Descriptions

3.1.3 Screen Authorization

Screen	Super Admin	Admin	Moderator	Citizen	Guest
Register					X
Login	X	X	X	X	X
OTP/Forgot Password	X	X	X	X	
Landing Page				X	X
Map View	X	X	X	X	X
Input: From - To				X	X
Real-time Map Layer	X	X	X	X	X
Flood History Screen	X	X	X		
Manage Alert Messages			X		
Moderate Community Reports			X		
Flood History and Trends	X	X	X		
Flood AI Prediction Screen	X	X	X	X	
Flood AI Satellite Screen	X	X	X	X	
Personal Info	X	X	X	X	
Manage Billing & Payment Records	X	X			
Manage Subscription Plans	X	X			
View Operational Details	X	X	X		
Sensor Station Management	X	X			
Sensor Management	X	X			
User Management	X	X			
Community Report Management			X		
Setting Screen				X	
Sensor Data Overlay	X	X	X	X	
Available Area Overlay	X	X	X	X	
Community Report Overlay	X	X	X	X	
Normal Routes				X	
Bottom Tabs				X	
Safe Route Suggestion				X	
User's Areas Screen				X	
Community Flood Report			X	X	
Subscription and Billing	X	X			
Pricing Package(month, year)				X	
Payment Screen				X	
Rating Flood Report Score				X	
Manage Alert and Template			X		
User's History Alert Screen				X	
Manage News			X		
User's Newsfeed Screen				X	
System area Screen	X	X	X	X	
Weatherforecast Screen				X	
General Info Screen				X	

Table 5 Screen Authorization

3.1.4 Non-Screen Functions

#	Feature	System Function	Description
1	Sensor Data Processing	Data Ingestion Service	Receives real-time data from IoT sensors (JSN-SR04T, ESP32) via MQTT, validates JSON payload, stores in sensor_readings table, logs to devices_logs. Latency <3s.
2	Alert Generation	Alert Engine Worker	Analyzes sensor data against alert thresholds (water_level > max_level), generates alerts, triggers push notifications via FCM to Mobile App.
3	Data Aggregation	Daily Statistics Cron Job	Runs daily at 00:00, calculates avg water_level, total rainfall, max flow_rate → sensor_daily_agg, updates flood_statistics and station risk_level.
4	Flood Prediction	ML Prediction Worker	Runs every prediction trigger to save to db for next update version
5	Risk Assessment	Flood Risk Update Service	Maps stations → areas via area_stations table, computes risk_level (safe/warning/danger) for areas and flood_routes_cache for map display.
6	Notification Delivery	Notification Queue Worker	Processes alert queue (Redis/RabbitMQ), sends push notifications based on user subscriptions, retry logic, rate limiting 10 msg/hour/user.
7	API Security	API Gateway Service	Validates JWT/API key, rate limiting (100 req/min), CORS for Mobile App/Admin endpoints (/stations, /alerts, /areas).
8	System Monitoring	Health Check Service	Internal API /health checks MQTT broker, DB connections, queue length, sensor uptime → monitoring dashboard.
9	Cache Management	Cache Invalidation Service	Event-driven, clears Redis cache when sensor data or risk_level changes, ensures Mobile App data freshness <3s.

Table 6 Non-Screen Functions

3.1.5 Entity Relationship Diagram

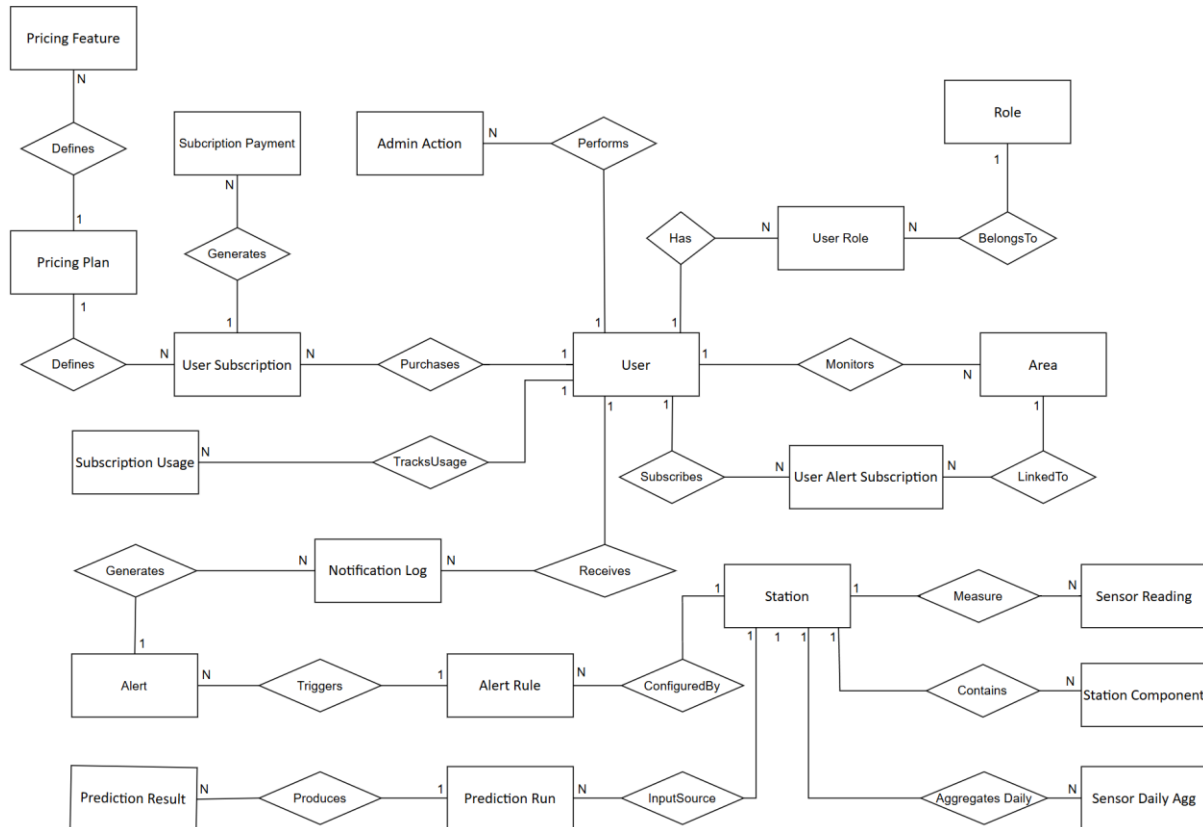


Figure 15 Entity Relationship Diagram

Entities Description

#	Entity	Description
1	User	Represents a user account within the system. Stores personal details such as email, full name, avatar, authentication provider (Local/Google), and account status.
2	Role	Defines the specific roles within the system to manage access control. Roles include: Administrator, Government Official, and Resident (User).
3	User Role	Associates users with specific roles. Handles the many-to-many relationship between Users and Roles.
4	Pricing Plan	Defines the available SaaS subscription tiers (e.g., Free, Premium). Includes pricing details, feature sets, and associated resource quotas.
5	Pricing Feature	Details specific features and limits (e.g., "Max 5 Areas", "SMS Enabled") associated with each Pricing Plan.
6	User Subscription	Tracks the validity period of user subscriptions. Includes start dates, expiration dates, and renewal status to determine feature access.
7	Subscription Payment	Records financial transactions related to subscription purchases. Includes payment methods, transaction IDs, and amounts.
8	Area	Manages specific geographic zones (Geofences) registered by users for monitoring. Includes coordinates (latitude/longitude), radius, and a descriptive label.

#	Entity	Description
9	Station	Represents physical monitoring stations installed in the field. Stores station codes, location details, coordinates, and operational status (Online/Maintenance).
10	Sensor Reading	Stores historical time-series data transmitted by sensors. Includes raw values, calibrated values, and data quality flags for analysis and visualization.
11	Sensor Daily Agg	Stores pre-calculated daily statistics (Min, Max, Avg levels, Total Rainfall) for each station to optimize reporting performance.
12	Alert Rule	Defines the monitoring logic and threshold configurations for each station. Sets specific water levels (Yellow, Orange, Red) to trigger automated alerts.
13	Alert	Stores the history of triggered flood events. Includes severity level, start time, end time, and peak water level recorded during the event.
14	Notification Log	Records a log of all notifications sent to users via various channels (Push, SMS). Used for auditing delivery success rates and calculating SMS costs.
15	User Alert Subscription	Manages the association between users and specific stations or areas. Determines which users receive alerts from which locations and at what severity level.
16	Prediction Log	Runs every prediction trigger to save to db for next update version
17	Subscription Usage	Tracks the actual resource consumption of a user (e.g., SMS sent, Areas created) against the limits defined in their subscription plan.

Table 7 Entities Description

3.2 Authenticate

3.2.1 Sign in

Function trigger:

- **Navigation path:** /sign-in
- **Timing Frequency:** On demand (whenever a user wants to log in to the system).

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:** Allow users to log in to the system to use any feature in the system.
- **Interface:**
 - **Sign-in Screen:**
 - **“Số Điện Thoại”:** Input fields for PhoneNumber (Available for Citizen accounts only).
 - **“Email”:** Input fields for Email (Available for Moderator and Admin).
 - **“Mật Khẩu”:** Input fields for Password.
- **Data processing:**
 - When a user is not logged in, the system will auto redirect to the Login page.
 - The user submits login credentials (Phone + Password or Email + Password).
 - The system queries the **Users** table to verify credentials.
 - If authentication succeeds:

- The system retrieves user information: UserId, FullName, PhoneNumber, Email, AvatarUrl, Provider, Status.
- The system joins with UserRoles and Roles tables to determine the user's role code.
- A valid access token/session is generated.
- The user is redirected to the appropriate dashboard based on their role.

Screen layout:

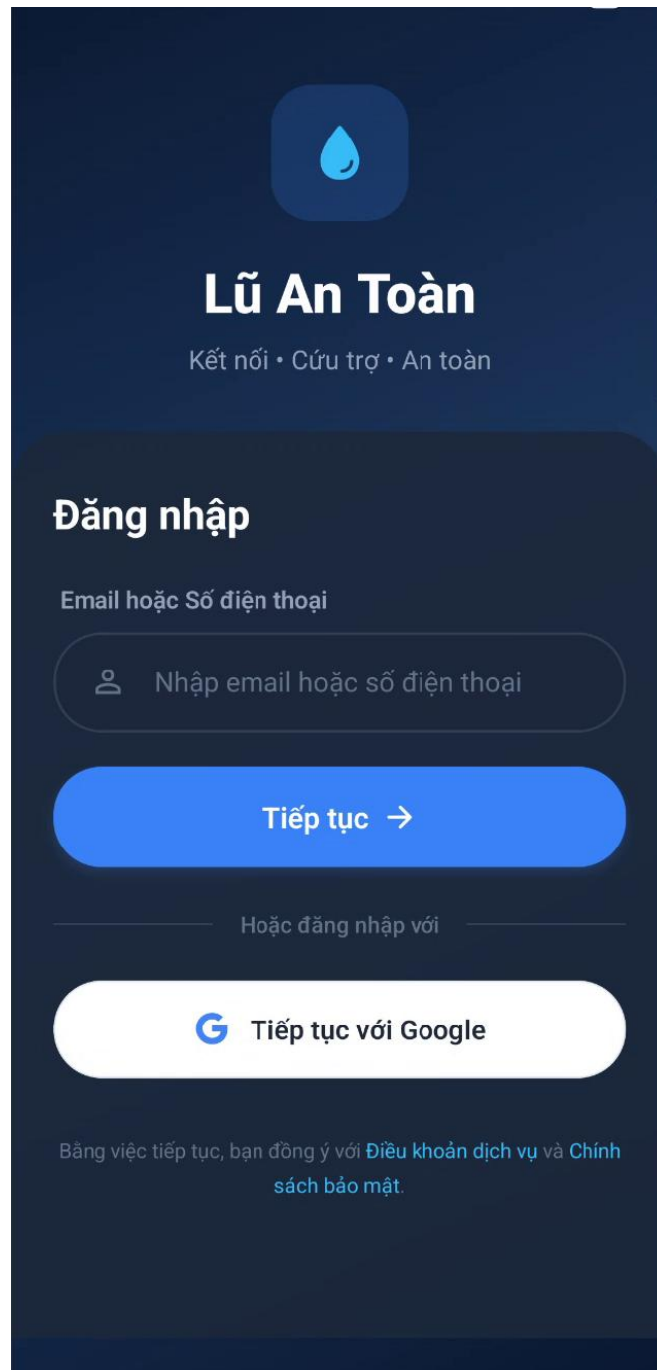


Figure 16 Login Screen layout for Mobile

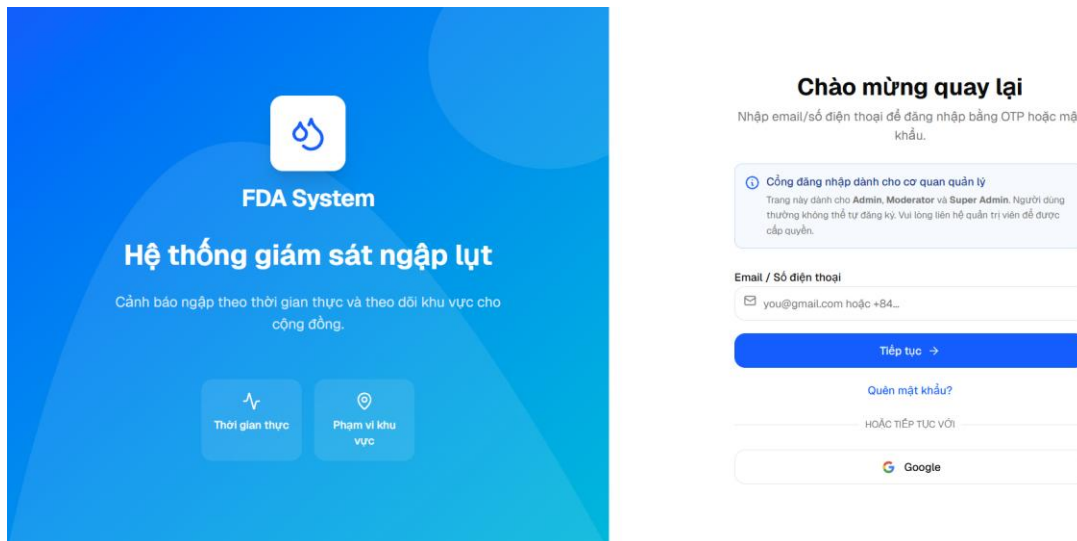


Figure 17 Login Screen layout for Website

Function details:

- **Data:** User ID, Full name, Phone Number, Email, Avatar URL, Role code, Account Status.
- **Validation:**
 - **“Số Điện Thoại”:** Must follow a valid phone number format, applied only for Citizen login.
 - **“Email”:** Must be in valid email format.
 - **“Mật Khẩu”:** Must match stored hashed password.
- **Business rules:** Users can only log in using credentials associated with their role type. Disabled or inactive accounts cannot sign in. Role-based access control is applied immediately after login.
- **Error Handling:**
 - Invalid credentials → Display a generic error message (e.g., “Invalid email/phone number or password”).
 - Inactive or suspended account → Display an account status warning.
 - System or database failure → Display a generic system error message.
- **Normal case:** The user enters valid credentials. The system authenticates successfully. The user is redirected to the authorized dashboard. A success notification is displayed
- **Abnormal case:**
 - **Invalid credentials:** Authentication fails => The system displays an error message.
 - **Invalid input format:** The system rejects submission => An error message is displayed next to the invalid field.
 - **Account inactive:** Login is blocked => The system informs the user of account status.

3.2.2 Sign Out

Function trigger:

- Navigation path: /sign-out (via user menu / logout button)
- Timing Frequency: On demand (whenever a user chooses to log out).

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:** Allow authenticated users to securely terminate their session and prevent further access to protected system resources.
- **Interface:**
 - **Sign-out Action:**
 - **Access point:** User avatar dropdown / system menu
 - **Action Button:** “Đăng Xuất”
- **Data processing:**
 - The user initiates the **Sign Out** action.
 - The system validates that the user currently has an active session or access token.
 - The system invalidates the authentication session:
 - Access token is revoked or removed from client storage.
 - Refresh token (if any) is invalidated in the system.
 - User session data is cleared from memory/cache.
 - The system redirects the user to the **Sign-in page**.
 - Any attempt to access protected resources after sign-out triggers re-authentication.

Screen layout:

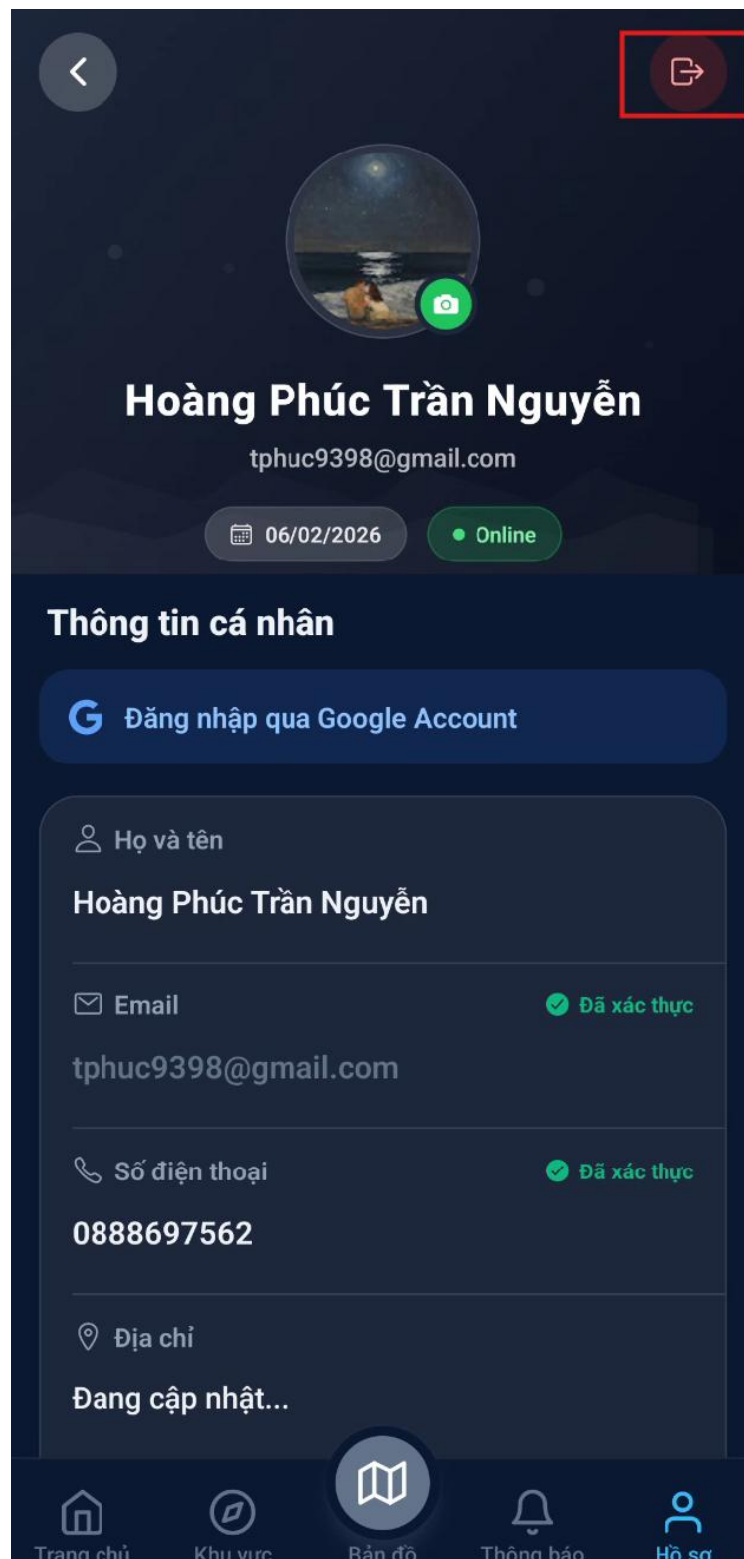


Figure 18 Sign Out Screen layout for Mobile

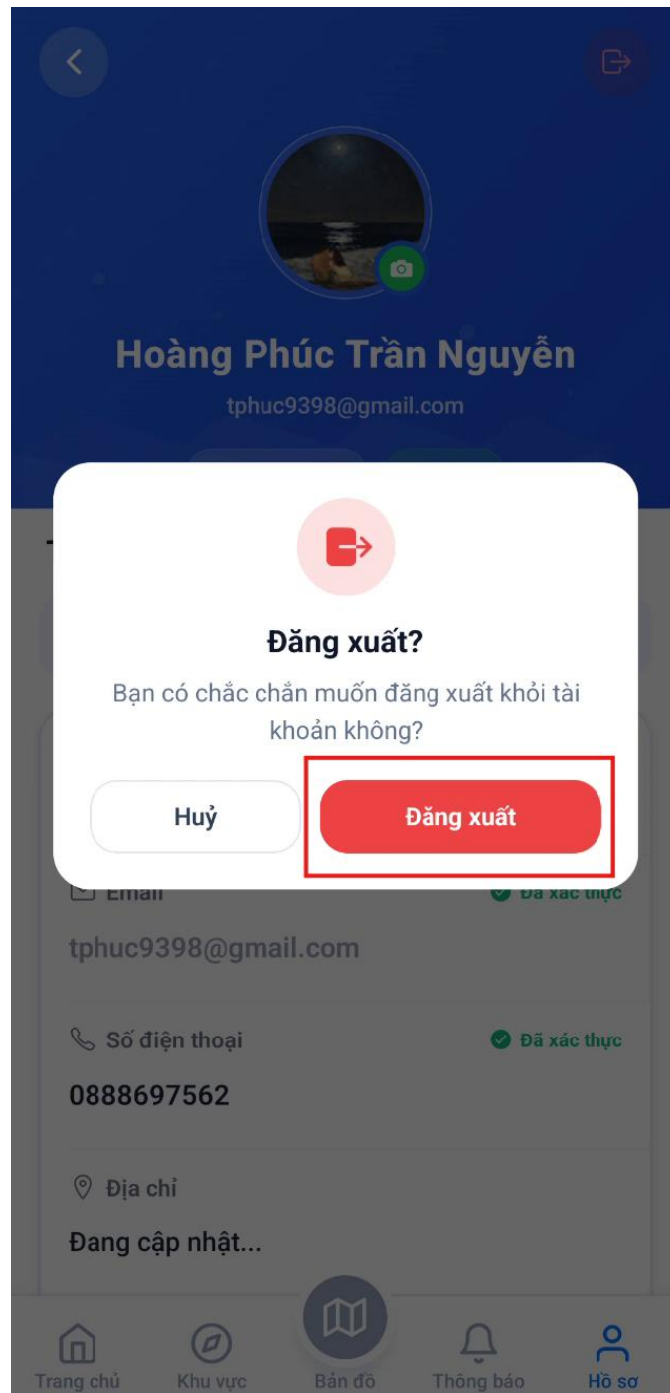


Figure 19 Sign Out Screen layout for Mobile

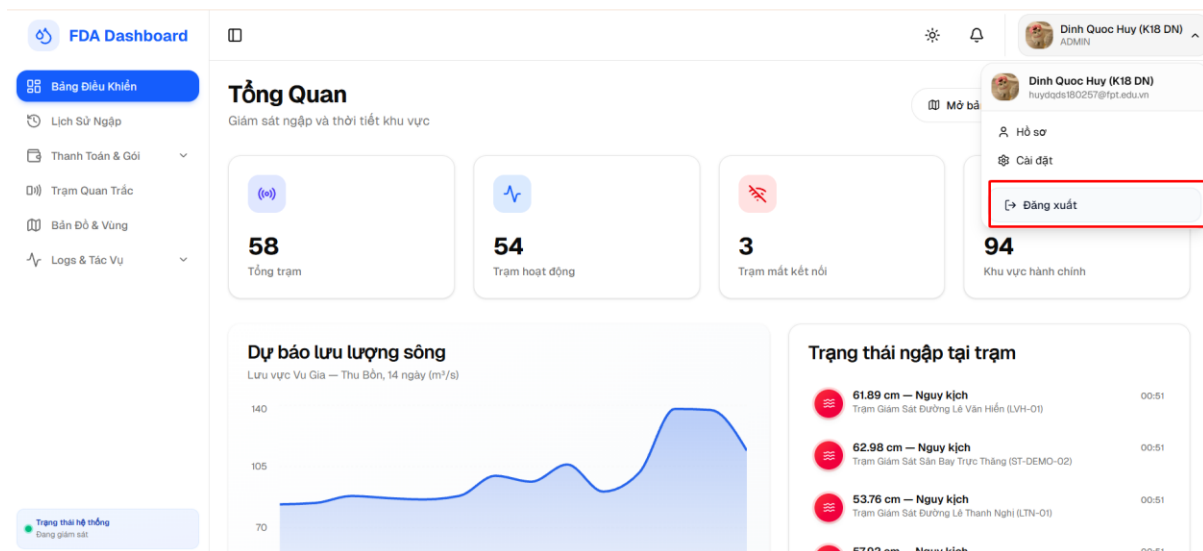


Figure 20 Sign Out Screen layout for Website

Function details:

- **Data:** User ID, Full name, Access token, Refresh token, Session status.
- **Validation:**
 - The user must be authenticated to perform Sign Out.
 - The system verifies token/session validity before invalidation.
- **Business rules:**
 - A signed-out user must not access any protected system feature.
 - Each Sign Out action terminates only the current session.
 - Multi-device sessions (if supported) remain unaffected unless explicitly revoked.
- **Error Handling:**
 - Session/token not found → The system still redirects the user to the Sign-in page.
 - Token invalidation failure → The system clears client-side session data and logs errors.
 - System failure → Display a generic error message and force redirect to Sign-in.
- **Normal case:**
 - The user clicks **"Đăng Xuất"**
 - The system successfully invalidates the session/token
 - The user is redirected to the Sign-in screen
 - A sign-out success notification is displayed
- **Abnormal case:**
 - Expired or missing session
 - The system treats the user as already signed out.
 - Redirects to the Sign-in page.
 - Backend failure during token revocation
 - Client session is cleared.
 - User is logged out for security reasons.

3.2.3 Authentication with OAuth

Function trigger:

- Navigation path: /sign-in → *Login with Google*

- Timing Frequency: On demand (whenever a user wants to log in to the system).

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:** Allow users to authenticate using their Google account via OAuth 2.0, reducing friction during login while ensuring secure identity verification through **Google**.
- **Interface:**
 - Sign-in Screen:
 - Button: “**Tiếp tục với Google**”.
- **Data processing:**
 - The user clicks **Login with Google**.
 - The system redirects the user to Google’s OAuth authorization endpoint.
 - The user authenticates with Google and grants requested permissions.
 - Google redirects back to the system with an authorization code.
 - The backend exchanges the authorization code for an access token and retrieves user profile data from Google, including: Email, Full name, Avatar URL, OAuth provider ID
 - The system checks the users table:
 - If the email already exists: Link the account to Google as an authentication provider (if not already linked).
 - If the email does not exist: Create a new user record with provider = **google**.
 - The system assigns a role based on predefined rules (default role if new user).
 - The system generates an access token/session.
 - The user is redirected to the appropriate dashboard based on role.

Screen layout:

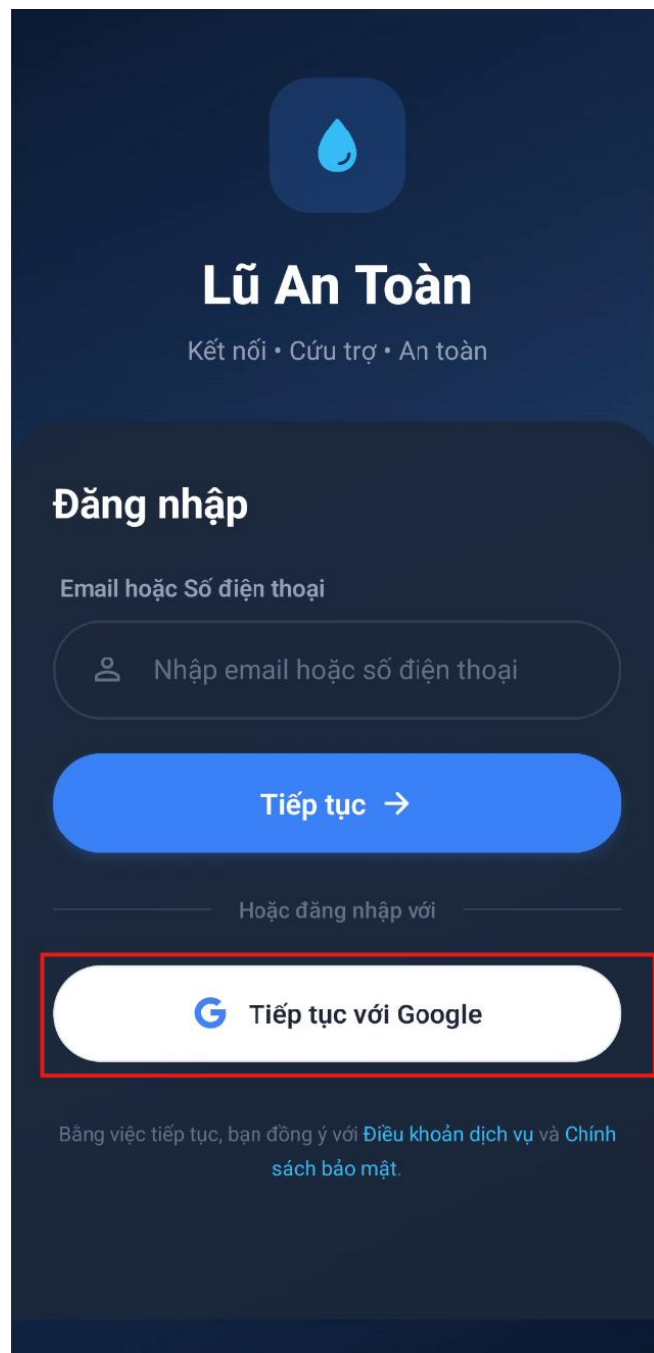


Figure 21 Google OAuth Screen layout for Mobile

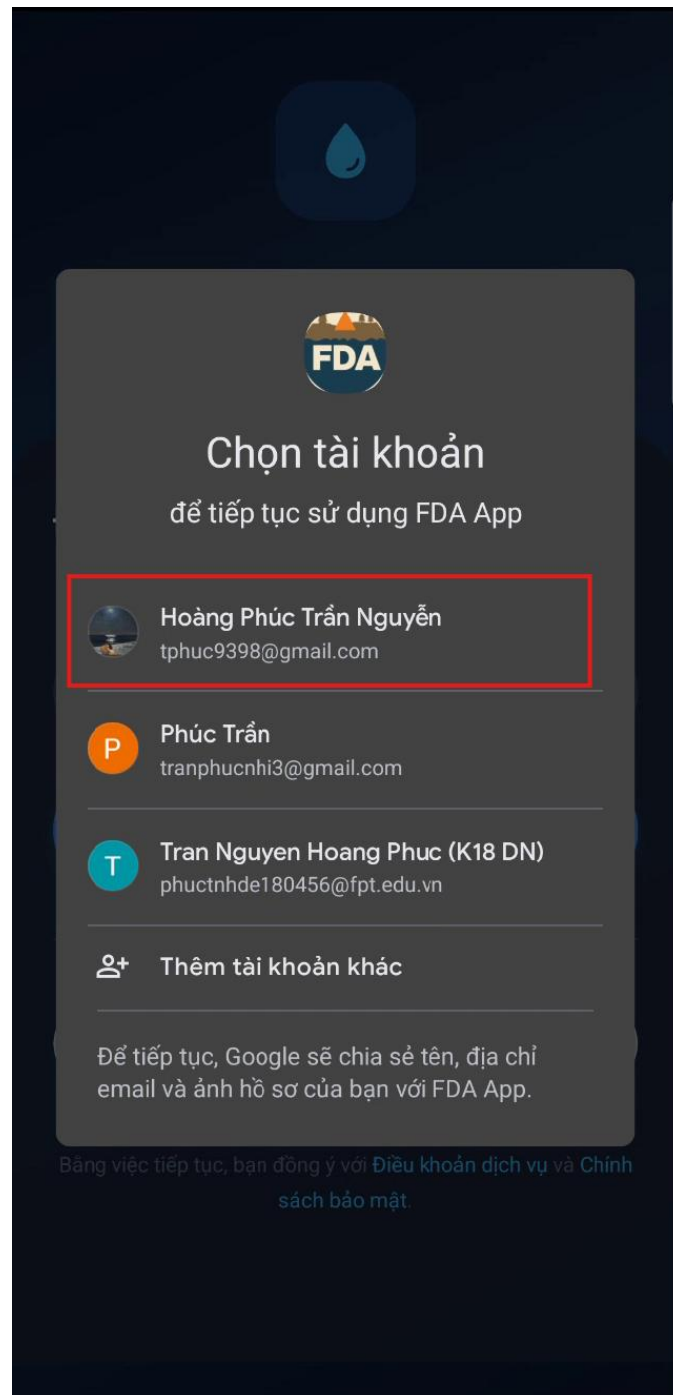


Figure 22 Google OAuth Screen layout for Mobile

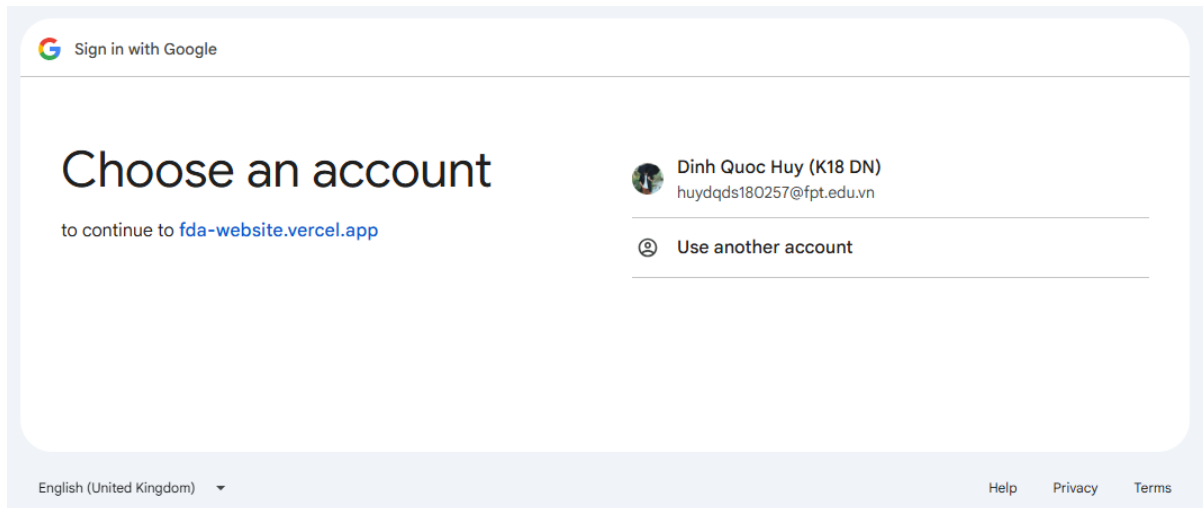


Figure 23 : Google OAuth Screen layout for Website

Function details:

- **Data:** User ID, Full name, Phone Number, Email, Avatar URL, OAuth provider, Role code, Account status.
- **Validation:**
 - Email retrieved from Google must be verified.
 - OAuth token must be valid and not expired.
 - The Google account email must be unique per user record.
- **Business rules:**
 - Password is not required for OAuth-based login.
 - One email corresponds to one user account, regardless of login method.
 - Users authenticated via Google inherit the same role-based access control as password-based users.
 - Inactive or suspended accounts cannot log in via OAuth.
- **Error Handling:**
 - User cancels Google authorization → Redirect back to Sign-in page with a cancellation message.
 - Invalid or expired OAuth token → Display a generic authentication error.
 - Email conflict with an existing inactive account → Display an account status warning.
 - System or third-party failure → Display a generic system error message.
- **Normal case:**
 - The user selects **"Tiếp tục với Google"**
 - Google authentication succeeds
 - The system validates or creates the user account
 - A session/token is generated
 - The user is redirected to the authorized dashboard
 - A success notification is displayed
- **Abnormal case:**
 - Authorization denied by user
 - Authentication flow is aborted
 - User is returned to the Sign-in page
 - Google API unavailable
 - Login fails

- System displays a retry or fallback message
- Account inactive
 - Login is blocked
 - The system informs the user of account status

3.3 Manage Password

3.2.1 Set password

Function trigger:

- Navigation path: Settings / Security → Manage Password
- Timing Frequency: On demand (whenever a user wants to log in to the system).

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:** Allow authenticated users to securely set a password (if they signed up via OAuth and do not have one) or change their existing password to maintain account security.
- **Interface:**
 - **Mode A – “Thiết lập mật khẩu”** (*OAuth user without password*)
 - New Password
 - Confirm New Password
 - Button: “**Tạo mật khẩu**”
 - **Mode B – “Đổi mật khẩu”** (*user already has password*)
 - Current Password
 - New Password
 - Confirm New Password
 - Button: “**Tạo mật khẩu**”
 - **Optional**
 - “**Show password**” toggle
 - Password strength indicator
- **Data processing:**
 - The user navigates to Manage Password.
 - The system loads the user’s authentication context from users:
- **UserId, Provider, PasswordHash (exists or not), Status**
 - The system decides screen mode:
- If **PasswordHash** is NULL/empty → **Set Password mode**
- Else → **Change Password mode**
 - User submits the form:
 - **Set Password:** validate new password + confirmation
 - **Change Password:** verify current password, then validate new password + confirmation
 - The system hashes the new password and updates Users:
 - Update PasswordHash
 - Update UpdatedAt
 - Optionally update Provider (e.g., keep google but allow password login as additional method)
 - The system invalidates active sessions if security policy requires:
 - Revoke refresh tokens / force re-login on other devices (optional policy)
 - Display success notification.

Screen layout:

The image shows a mobile application interface for a user named Hoàng Phúc Trần Nguyễn. The screen is titled "Thông tin cá nhân" (Personal Information). At the top, there is a blue header bar with a back arrow, a profile picture, and the user's name. Below the header, there is a light blue button labeled "Đăng nhập qua Google Account". The main content area is a white card with rounded corners, containing several fields: "Họ và tên" (Full Name) with the value "Hoàng Phúc Trần Nguyễn", "Email" with the value "tphuc9398@gmail.com" and a green checkmark indicating it is verified, "Số điện thoại" (Phone Number) with the value "0888697562" and a green checkmark indicating it is verified, and "Địa chỉ" (Address) with the value "Đang cập nhật..." (Being updated...). Below these fields, there is a red rectangular box highlighting a button labeled "Đổi hoặc thiết lập mật khẩu" (Change or set password) with a key icon and a right arrow. At the bottom of the card, there is a large blue button labeled "Lưu Thay Đổi" (Save Changes). The bottom of the screen features a navigation bar with five icons: a house for "Trang chủ" (Home), a compass for "Khu vực" (Area), a map for "Bản đồ" (Map), a bell for "Thông báo" (Notifications), and a person for "Hồ sơ" (Profile).

Hoàng Phúc Trần Nguyễn

Đăng nhập qua Google Account

Họ và tên
Hoàng Phúc Trần Nguyễn

Email ✓ Đã xác thực
tphuc9398@gmail.com

Số điện thoại ✓ Đã xác thực
0888697562

Địa chỉ
Đang cập nhật...

Đổi hoặc thiết lập mật khẩu >

Lưu Thay Đổi

Trang chủ Khu vực Bản đồ Thông báo Hồ sơ

Figure 24 Set Password

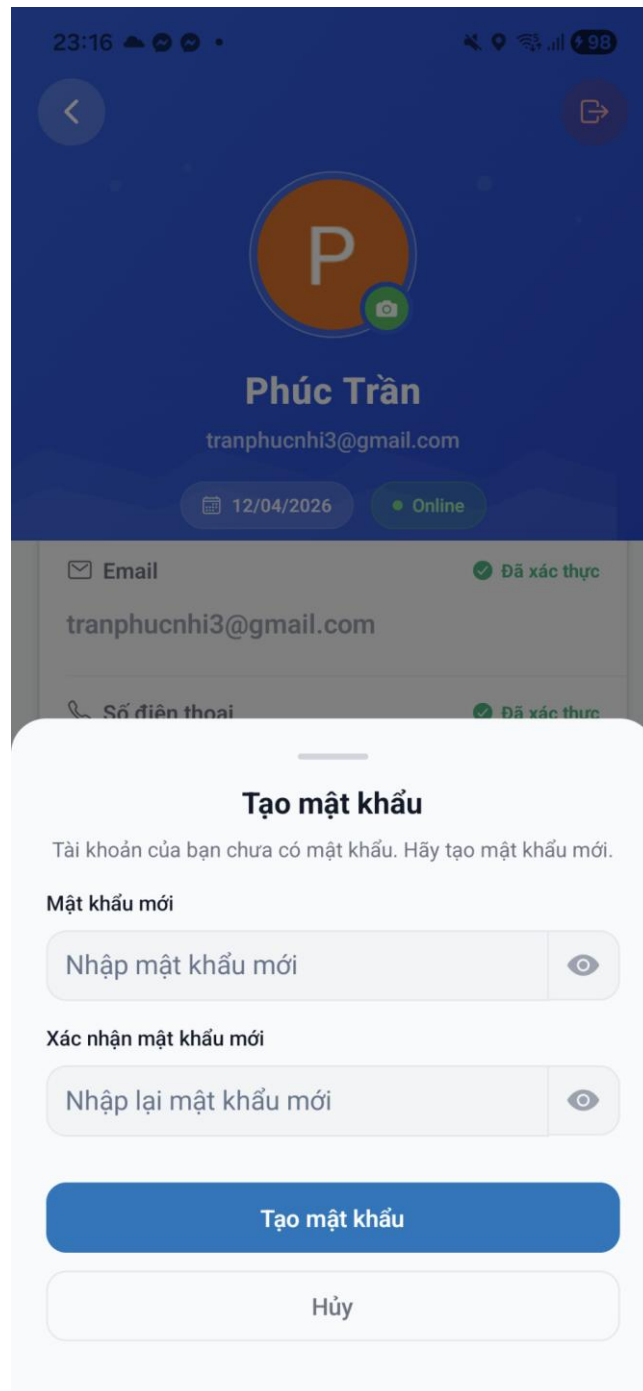


Figure 25 Set Password

The image shows a mobile application interface. At the top, there is a user profile card for 'Hoàng Phúc Trần Nguyễn' with email 'tphuc9398@gmail.com', a date '06/02/2026', and a status 'Online'. Below this, a modal titled 'Đổi mật khẩu' (Change Password) is displayed. The modal contains three input fields, each with a label and a toggle icon: 'Mật khẩu hiện tại' (Current Password) with label 'Nhập mật khẩu hiện tại', 'Mật khẩu mới' (New Password) with label 'Nhập mật khẩu mới', and 'Xác nhận mật khẩu mới' (Confirm New Password) with label 'Nhập lại mật khẩu mới'. A red rectangle highlights these three input fields. At the bottom of the modal are two buttons: 'Cập nhật mật khẩu' (Update Password) and 'Hủy' (Cancel).

<

Hoàng Phúc Trần Nguyễn

tphuc9398@gmail.com

06/02/2026 Online

Đổi mật khẩu

Nhập mật khẩu hiện tại và mật khẩu mới để cập nhật.

Mật khẩu hiện tại

Nhập mật khẩu hiện tại

Mật khẩu mới

Nhập mật khẩu mới

Xác nhận mật khẩu mới

Nhập lại mật khẩu mới

Cập nhật mật khẩu

Hủy

Figure 26 Manage Password Screen layout for Mobile

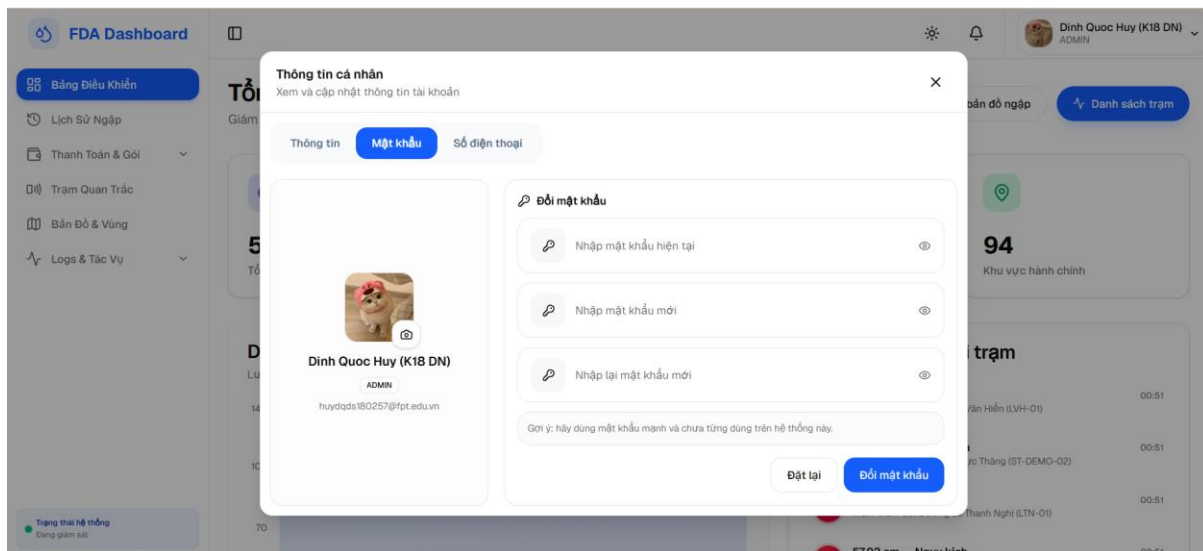


Figure 27 Manage Password Screen layout for Website

Function details:

- **Data:** User ID, Provider , Password hash, Updated timestamp, Session tokens.
- **Validation:**
 - **Current Password (Change mode)**
 - Must match stored PasswordHash
 - **New Password**
 - Minimum length (e.g., ≥ 8).
 - Must meet complexity policy (e.g., uppercase/lowercase/number/special) *(define per system policy)*.
 - Must not be the same as current password *(recommended)*.
 - **Confirm New Password**
 - Must match New Password.
- **Business rules:**
 - Only the authenticated user can set/change their own password.
 - Suspended/inactive accounts cannot perform password management.
 - OAuth-only users are allowed to set a password to enable email/password login (hybrid login).
 - After password change, the system may require re-authentication depending on security policy.
- **Error Handling:**
 - Wrong current password → Display error message next to “**Mật khẩu hiện tại**”.
 - New password fails policy validation → Display validation error message.
 - Confirm password mismatch → Display error message next to confirmation field.
 - Database update failure → Display a generic error message and log the incident.
- **Normal case:**
 - **Case A – “Thiết lập mật khẩu OAuth”**
 - User enters New Password + Confirm
 - System updates PasswordHash, UpdatedAt
 - System displays success notification
 - **Case B – “Đổi mật khẩu”**
 - User enters Current Password correctly

- User enters valid New Password + Confirm
 - System updates PasswordHash, UpdatedAt
 - System displays success notification
- **Abnormal case:**
 - Current password incorrect → Reject submission and show field-level error.
 - Weak/invalid new password → Reject submission and show validation hints.
 - Confirm mismatch → Reject submission and show mismatch error.
 - Account inactive → Block action and show account status warning.

3.3.2 Reset Password

Function trigger:

- On the Login screen, the user selects **“Quên mật khẩu?”**.
- The user submits their email or phone number, receives an OTP, verifies it, and is prompted to set a new password.

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:**
 - Restore account access without requiring administrator intervention.
 - Ensure password reset is protected by OTP-based verification.
- **Interface:**
 - Entry screen: `GET /auth/login` (Login flow).
 - Identifier field: “Email / Phone number”
 - Action: **“Quên mật khẩu”**
 - OTP verification step:
 - OTP input (6 digits)
 - Actions: Verify OTP, Resend OTP (after expiry)
 - Reset password modal (shown after OTP verification on a protected portal screen):
 - New password
 - Confirm new password
 - Password policy checklist (minimum length and complexity indicators)
 - Action: **“Đặt mật khẩu”**
- **Data processing:**
 - The user submits an identifier (email/phone).
 - The system triggers OTP delivery:
 - Endpoint: `POST /auth/send-otp` with `{ identifier }` (unauthenticated).
 - The user enters OTP and submits:
 - Endpoint: `POST /auth/login` with `{ identifier, otpCode }` (unauthenticated).
 - If OTP verification succeeds and the flow is “forgot password”, the frontend marks the session as requiring a password reset and opens the reset-password modal after redirect to a protected portal.
 - Implementation note: the modal guard is currently wired in the Admin portal providers; equivalent guards for Citizen/Moderator portals should follow the same pattern (Screen layout: TBD if not present).
 - The user submits a new password and confirmation:

- Endpoint: `POST /auth/reset-password` with `{ newPassword, confirmPassword }` (requires access token from OTP login).
- Backend updates stored password hash and returns success; frontend clears the reset flag and closes the modal.

Screen layout:

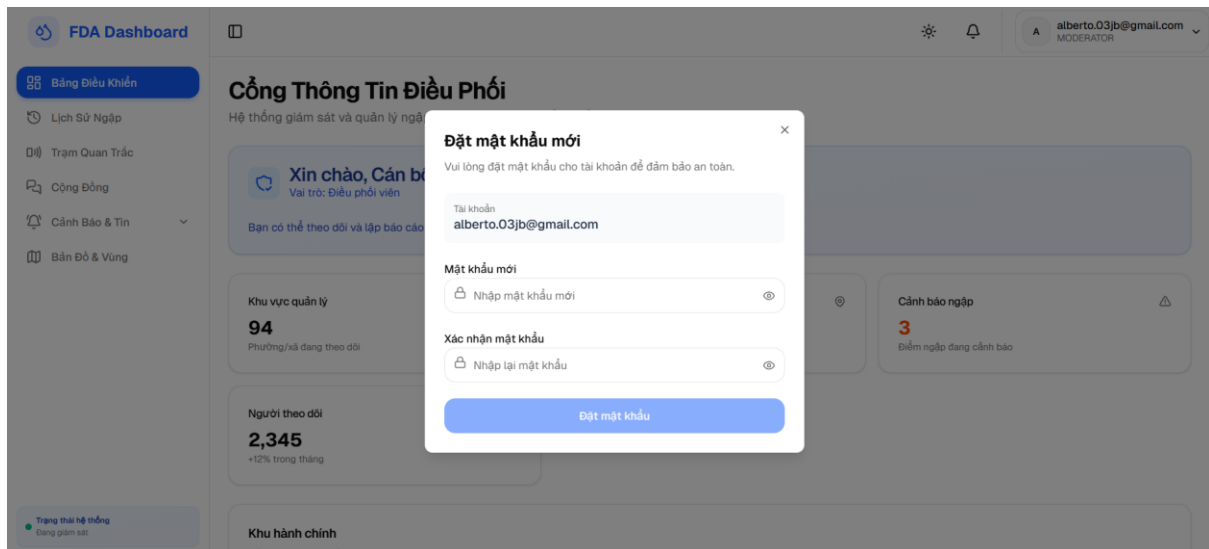


Figure 28 Reset Password

Function details:

- **Data:**
 - Identifier (email or phone number)
 - OTP code
 - Access token (post OTP login)
 - New password and confirmation
- **Validation:**
 - Identifier:
 - Must be non-empty after trimming.
 - OTP:
 - Must be exactly 6 digits.
 - New password:
 - Minimum length: 8 characters
 - Contains uppercase letter
 - Contains lowercase letter
 - Contains number
 - Contains special character
 - Confirm password:
 - Must match new password.
- **Business rules:**
 - Only the account owner who can verify OTP for the identifier may reset the password.
 - The reset operation updates the password hash (plain text passwords are never stored).

- If the account is inactive/banned (per system policy), the system may block reset or require additional verification (policy-dependent).
- OTP resend rate limits should be enforced by backend policy (recommended).
- **Error Handling:**
 - Unknown identifier or account not found:
 - Display: “Account does not exist” (or equivalent backend message).
 - OTP invalid/expired:
 - Display: OTP validation error; do not proceed to reset step.
 - Password policy failure:
 - Display password rule failures and prevent submission.
 - Confirm mismatch:
 - Display mismatch error and prevent submission.
 - Backend reset failure:
 - Display a generic error and log the incident server-side.
- **Normal case:**
 - User selects “Forgot password”.
 - User enters email/phone and requests OTP.
 - System sends OTP successfully.
 - User enters OTP correctly and verifies.
 - System authenticates OTP login and prompts password reset.
 - User enters a valid new password and matching confirmation.
 - System updates password hash and shows success notification; user may continue using the portal.
- **Abnormal case:**
 - User enters empty identifier → system blocks submission and shows a field warning.
 - User enters invalid OTP (not 6 digits) → system blocks submission.
 - OTP expired or incorrect → system rejects verification and prompts retry/resend.
 - Password fails complexity rules → system blocks submission and shows unmet rules.
 - Confirm password does not match → system blocks submission and shows mismatch.
 - API/network error → system shows error and user may retry.

3.4 Manage User Profile

Function trigger:

- **Navigation path:** Profile icon/link
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen, Moderator, Admin
- **Purpose:** Allows the currently logged-in user to review their stored personal information and update editable fields like contact details.
- **Interface:**
 - **Editable Fields:** Input fields for “Họ và Tên”, “Email”, and Avatar URL.
 - **Read-Only Fields:** Display of Email, Account Provider.
 - **“Lưu thay đổi” Button:** Submits changes to the backend.

Data processing:

- User navigates to the profile page.
- The system queries the Users table for the logged-in user's data (FullName, PhoneNumber, AvatarUrl, Email, Provider, Status).
- The system joins with UserRoles and Roles to retrieve the user's role code.
- The user modifies the editable fields and clicks "Save Changes."
- The system updates the user's record in the Users table, updating the changed fields and the updated_at timestamp.

Screen layout:

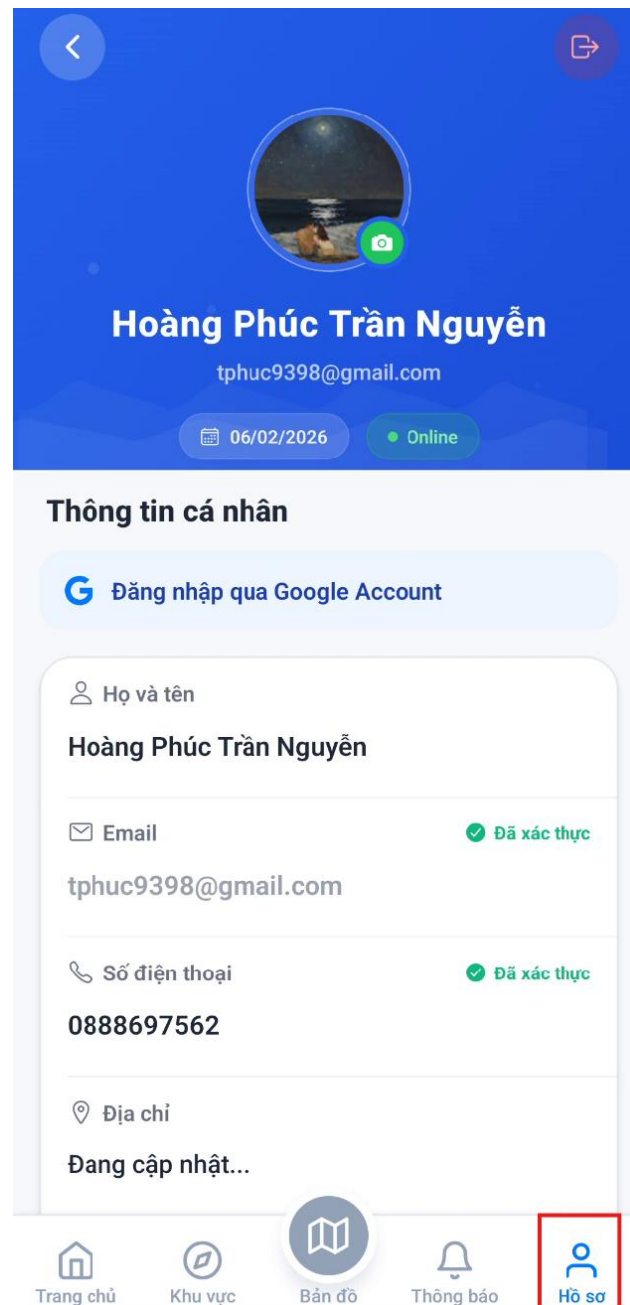


Figure 29 Manage User Profile Screen layout for Mobile

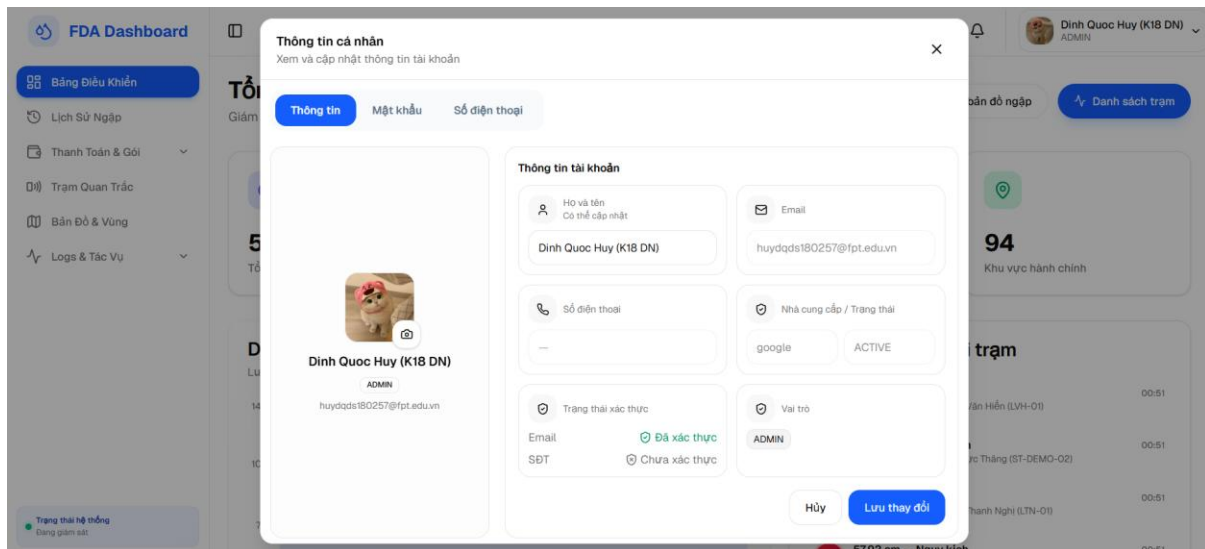


Figure 30 Manage User Profile Screen layout for Website

Function details:

- **Data:** User ID, Name, Phone Number, Avatar URL, Email, Role.
- **Validation:**
 - Phone Number: Ensures the PhoneNumber field in the Users table follows a valid format.
 - Error Handling: The system handles database update failures and displays a generic error message.
- **Business rules:** Only the user themselves can update their profile information.
- **Normal case:** The user successfully updates their phone number => The system updates the PhoneNumber and UpdatedAt columns in the Users table and displays a success notification.
- **Abnormal case:** The user submits a phone number that fails format validation => The system rejects the submission and displays an error message next to the invalid field.

3.5 Manage Users (Global & Assign Fixed Roles)

3.5.1 View User

Function trigger:

- Super Admin opens “Người Dùng & Vai Trò”.
- Super Admin uses search, filters, pagination, or opens a user row for additional detail.

Function description:

- **Actors/Roles:** Super Admin
- **Purpose:**
 - Provide global visibility into registered users and their operational status.
 - Support administrative tasks such as verification, troubleshooting, and governance.
- **Interface:**
 - Route/UI:
 - Users listing page: `GET /admin/users`
 - Table columns: Name, Email, Role(s), Status, Last Login, Actions
 - Search & filter:
 - Search box (name/email)

- Role filter (USER, MODERATOR, ADMIN, SUPERADMIN)
- Status filter (active, inactive, banned)
- Pagination:
 - Query parameters: `page`, `perPage`

Data processing:

- Frontend loads list using:
 - Endpoint:
 - `GET/admin/users?pageNumber=&pageSize=&searchTerm=&role=&status=`
- The backend returns `{ users, totalCount }`.
- Frontend maps backend representation to display model and renders the data table.
- Filters update query parameters and re-fetch data.

Screen layout:

The screenshot displays the 'Quản Lý Người Dùng' (User Management) interface. It features a sidebar with navigation options and a main content area with a table of users. The table includes columns for user information, role, status, and last login time. The user 'alberto.03jb' is highlighted as a Moderator.

Người dùng	Vai trò	Trạng thái	Lần đăng nhập gần nhất
T Trương Anh Tuấn tuan.2004hi@gmail.com	User	Active	Apr 21, 4:09 PM
O 0795993433 0795993433@temp.fda.local	User	Banned	Apr 21, 2:38 PM
S Suong Vo suongvo8999@gmail.com	User	Active	Apr 15, 3:32 AM
S SUPER ADMIN superadmin@gmail.com	User	Active	Apr 14, 10:25 PM
S SUPER ADMIN 0352736100@temp.fda.local	User	Active	Apr 14, 10:21 PM
A alberto.03jb alberto.03jb@gmail.com	Moderator	Active	Apr 25, 11:00 AM

Figure 31 View User List

Function details:

- **Data:**
 - User: id, email, fullName/name, roles, status, isAdminCreated, createdAt, lastLoginAt
 - Query parameters: pageNumber, pageSize, searchTerm, role, status
- **Validation:**
 - Page and page size must be positive integers (frontend defaults apply).
 - Role and status values must be within supported enumerations.
- **Business rules:**
 - Only Super Admin may access global user listing.
 - The system should not expose sensitive authentication artifacts (password hashes, tokens) in listing responses.
- **Error Handling:**
 - 401 Unauthorized → force re-login.
 - 403 Forbidden → show “Insufficient permissions”.

- Fetch failure (network/server) → show error state and allow reload.
- **Normal case:**
 - Super Admin opens `/admin/users`.
 - System fetches paged users from `GET /admin/users`.
 - System shows table and total counts; Super Admin searches and filters.
- **Abnormal case:**
 - Invalid filter values → system ignores or resets invalid filters and refreshes list.
 - Backend error → system shows an error card/state and provides a retry action.

3.5.2 Create User

Function trigger:

- Super Admin selects “**Thêm người dùng**” on the User Management dashboard.

Function description:

The system allows the Super Admin to create a new user account and assign a fixed role at creation time.

- **Actors/Roles:** Super Admin
- **Purpose:**
 - Provision user accounts for system operation, staffing, or controlled onboarding.
 - Ensure roles are assigned explicitly as fixed roles.
- **Interface:**
 - Route/UI: `/admin/users` → Create dialog.
 - Create form fields (current UI):
 - Email (required)
 - Password (required)
 - Full name (required)
 - Phone number (optional)
 - Status (active/inactive/banned)
 - Role (USER/MODERATOR/ADMIN/SUPERADMIN may appear in UI but should be restricted by policy)
 - Action: Submit to create user.

Data processing:

- Super Admin opens the create dialog and enters user data.
- Frontend validates fields and sends:
 - Endpoint: `POST /admin/users`
 - Payload: `{ email, password, fullName, phoneNumber?, status, roleNames: [role] }`
- On success, the system refreshes the user listing.

Screen layout:

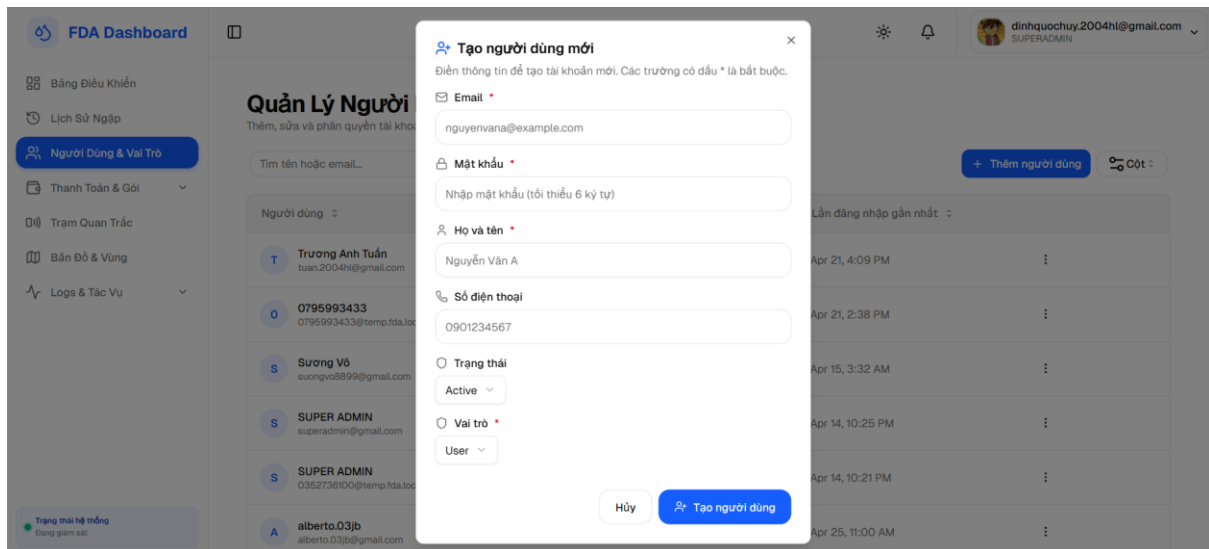


Figure 32 Create User

Function details:

- **Data:** Email, password, full name, phone number, status, roleNames
- **Validation:**
 - Email:
 - Required; must match email format.
 - Password:
 - Required; minimum length at least 6 characters (frontend rule).
 - Full name:
 - Required; non-empty after trimming.
 - Phone number (if provided):
 - Must be 10–11 digits (numeric only).
- **Business rules:**
 - Only Super Admin may create accounts and assign fixed roles.
 - Roles are fixed and limited to predefined values (USER/Citizen, MODERATOR, ADMIN) as defined by the system role model.
 - The system should enforce uniqueness constraints for email/phone at the backend.
- **Error Handling:**
 - Duplicate email/identifier → backend returns error; UI displays create failure message.
 - Validation failures → UI shows field-level error messages and blocks submission.
 - Server/network errors → UI shows error toast and preserves entered data for retry.
- **Normal case:**
 - Super Admin clicks “Create user”.
 - Super Admin enters valid information and selects a role.
 - System creates account and returns success.
 - User list refreshes and includes the new user.
- **Abnormal case:**
 - Email invalid or missing → submission blocked with field error.
 - Password too short → submission blocked with field error.
 - Backend rejects creation (e.g., duplicate email) → show failure and allow correction.

3.5.3 Update User

Function trigger: Super Admin selects “Sửa” on a user row from the user listing.

Function description:

The system allows Super Admin to update user information and administrative attributes (status and role), subject to constraints (e.g., only for accounts created by admins if required by policy).

- **Actors/Roles:** Super Admin
- **Purpose:**
 - Maintain accurate user records.
 - Correct role assignments and operational status.
- **Interface:**
 - Route/UI: `/admin/users` → Row action → Edit dialog.
 - Editable fields (current UI):
 - Full name (required)
 - Phone number (optional)
 - Status (active/inactive/banned)
 - Role (USER/MODERATOR/ADMIN/SUPERADMIN)
 - Read-only fields: Email

Data processing:

- Super Admin opens edit dialog for a selected user.
- Frontend enforces a constraint:
 - If the user is not “admin-created”, the UI blocks edit and shows an error.
- If allowed, frontend sends:
 - Endpoint: `PATCH /admin/users/:id`
 - Payload: `{ fullName, phoneNumber?, status, roleNames: [role] }`
- On success, system refreshes the list view.

Screen layout:

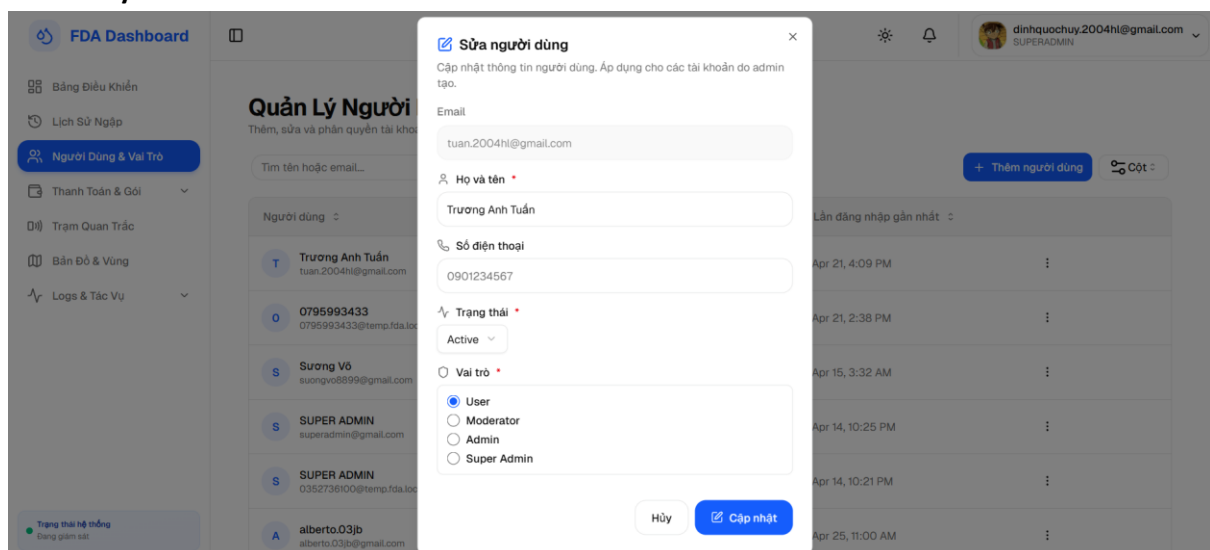


Figure 33 Update User

Function details:

- **Data:** User id, fullName, phoneNumber, status, roleNames

- **Validation:**
 - Full name: required; non-empty after trimming.
 - Phone number (if provided): 10–11 digits.
 - Role: must be selected.
- **Business rules:**
 - Access is restricted to Super Admin.
 - Accounts flagged as non-admin-created may be non-editable in the current UI (implementation constraint).
 - Backend must validate role existence and enforce role/status integrity.
- **Error Handling:**
 - Forbidden edit attempt → show “Only admin-created accounts can be edited” (or equivalent).
 - Backend validation failure → show error and keep dialog open.
 - Network/server errors → show error and allow retry.
- **Normal case:**
 - Super Admin selects “Edit”.
 - System shows edit dialog with current values.
 - Super Admin updates fields and submits.
 - System updates user and shows success message; list refreshes.
- **Abnormal case:**
 - User is not editable (not admin-created) → edit action is blocked.
 - Phone number invalid → UI blocks submission.
 - Backend failure → show error and keep data for retry.

3.5.4 Ban User

Function trigger: Super Admin selects “Ban/Unban” action for a user in the user listing.

Function description:

The system allows Super Admin to ban (lock) a user account to prevent future logins, and unban to restore access.

- **Actors/Roles:** Super Admin
- **Purpose:**
 - Enforce policy compliance and block abusive or compromised accounts.
 - Provide reversible account lockdown without deleting user data.
- **Interface:**
 - Route/UI: `/admin/users` → Row action → Ban dialog confirmation.
 - Action label adapts to current state:
 - If currently banned → “Unban”
 - Else → “Ban”

Data processing:

- Super Admin opens ban dialog.
- System confirms the action.
- Frontend updates status using existing update endpoint:
 - Endpoint: `PATCH /admin/users/:id`

- Payload: `{ status: 'banned' }` to ban, `{ status: 'active' }` to unban.
- List is refreshed after success.

Screen layout:

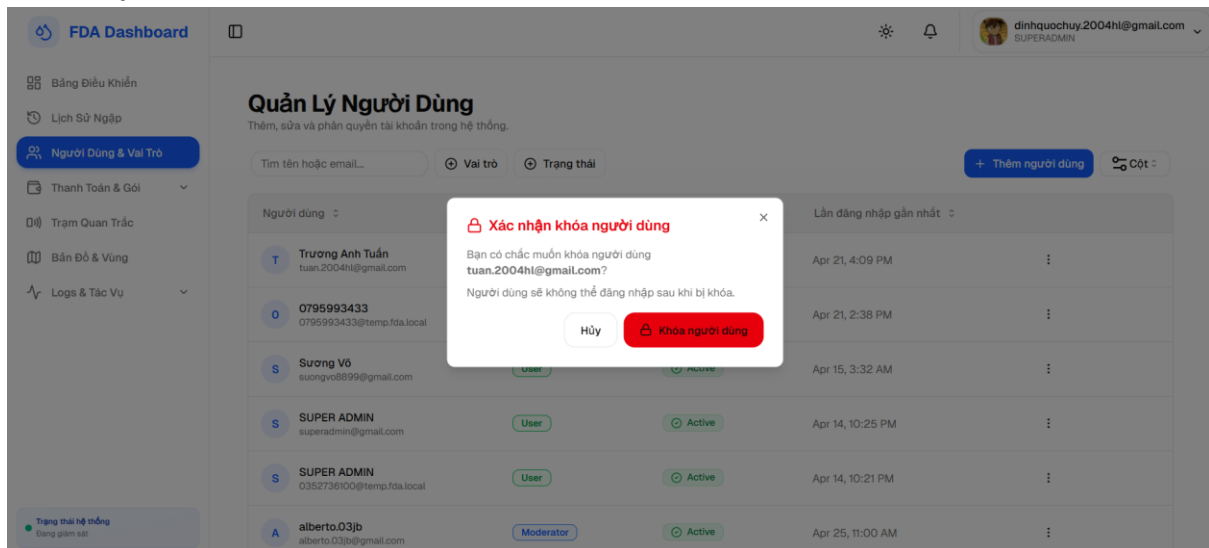


Figure 34 Ban User

Function details:

- **Data:**
 - User id
 - Status value (`active` / `banned`)
 - Audit log record (recommended): actor id, timestamp, reason (optional policy)
- **Validation:**
 - The target user must exist.
 - Status transitions must be valid (`active` ↔ `banned`).
- **Business rules:**
 - Only Super Admin may ban/unban users.
 - The system should prevent self-lockout of the acting Super Admin (recommended).
- **Error Handling:**
 - Backend rejects status change → show error and keep original status.
 - Network/server errors → show error toast and allow retry.
- **Normal case:**
 - Super Admin selects Ban.
 - Confirms ban.
 - System updates status to banned and refreshes list.
- **Abnormal case:**
 - Attempt to ban protected account → reject and show message.
 - Backend failure → show error and do not change UI state.

3.6 View Flood Map (Real-time Water Levels)

Function trigger:

- **Navigation path:** App Launch -> Bottom Tab Bar -> “**Bản Đồ**” tab (default landing tab) OR deep-link to `/(tabs)/map`.

- **Timing Frequency:** On demand. Initial load occurs on screen mount; auto-refresh is triggered via SignalR push events (ReceiveSensorUpdate, ReceiveStationUpdate); background REST refetch occurs when the flood overlay is toggled back on or a new flood zone is detected (React Query staleTime = 30s).

Function description:

- **Actors/Roles:** All Users (Authenticated users + Guest users).
- **Purpose:** To allow users to visualize real-time flood conditions across Da Nang city via a combined GeoJSON layer of sensor station markers and flood-zone polygons, enabling users to assess flood risk around their location or along intended routes.
- **Interface:**
 - **Map Screen (full-screen):** react-native-maps MapView rendered by MapContent.tsx, centered on Da Nang with a current user location FAB.
 - **Station Markers:** Pin markers color-coded by severity (Green: Safe, Yellow: Caution, Orange: Warning, Red: Critical, Gray: Unknown).
 - **Flood Zone Polygons:** Ring polygons around stations with severity $\geq$ warning, rendered with semi-transparent fill.
 - **Legend Card:** Floating card at top-left explaining the 4 severity tiers with icons, labels, and water-level ranges.
 - **Station Detail Sheet:** On marker tap, FloodStationCard.tsx opens showing station name, code, road, water level, alert level, and measured timestamp.

Data processing:

- The system reads useMapSettingsStore to determine if the overlays.flood toggle is enabled.
- When enabled, useFloodSeverityQuery issues GET /api/v1/map/current-status to retrieve GeoJSON FeatureCollection data.
- Simultaneously, useFloodSignalR maintains a hub connection and subscribes to real-time update events.
- useFloodData merges REST GeoJSON with real-time updates; real-time data overrides REST, and new polygons for critical stations trigger a refetch().
- FloodSeverityMarkers.tsx filters out features with invalid coordinates or missing water level data before rendering.

Screen layout:

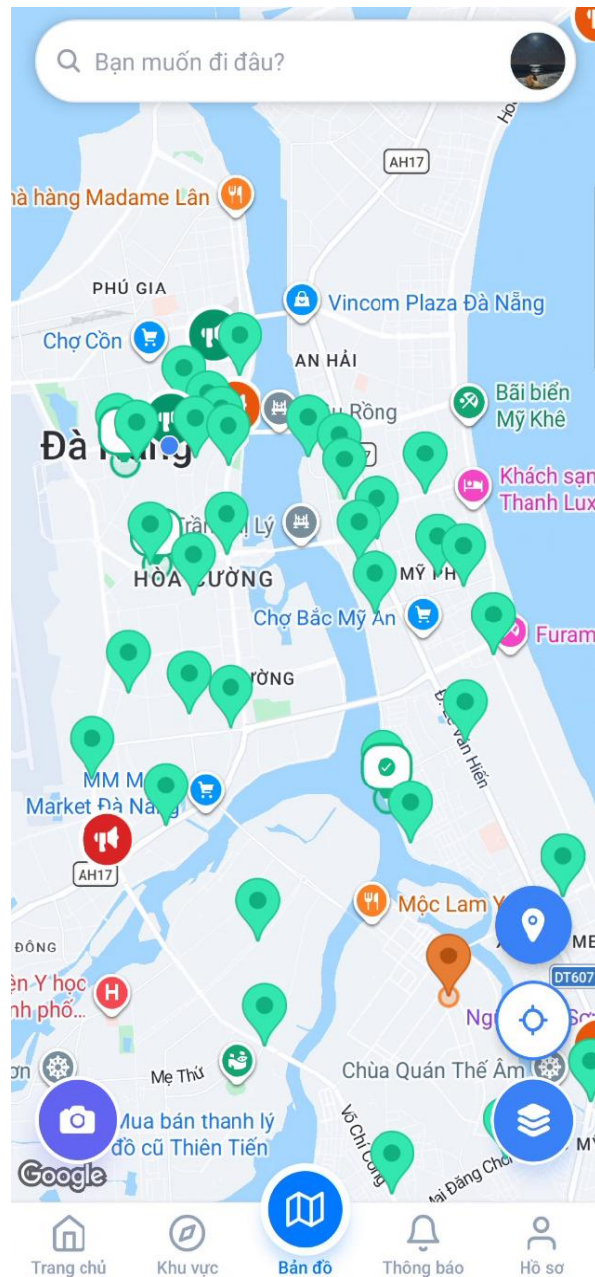


Figure 35 Flood Map

Function details:

- **Data:** User ID, BaseMap, Overlay toggles (Flood/Traffic/Weather), Opacity (Flood/Weather), Station properties (ID, Name, WaterLevel, Severity, Status, MeasuredTimestamp).
- **Validation:**
 - **BaseMap:** Must be one of standard, satellite, or hybrid (API accepts hybrid even if UI shows 2 options).
 - **Opacity:** Flood/Weather must be between 0 and 100.
 - **Coordinates:** Latitude [-90, 90], Longitude [-180, 180].
 - **Water level:** Must be a non-null number to render a marker; missing measurement data results in the station being hidden.
 - **Severity:** Must be one of safe, caution, warning, critical, or unknown.

- **Polygon geometry:** Must contain at least one outer ring with ≥ 3 valid [lng, lat] tuples.
- **Business rules:**
 - Any authenticated user can view and update their own map layer preferences.
 - Any user (authenticated or guest) can view the flood map.
 - Only stations with waterLevel != null are displayed.
 - Flood-zone polygons are only drawn when a station's severity is warning or critical.
 - Real-time updates always override the last REST snapshot for the same stationId.
 - The flood overlay visibility is controlled by the user's overlays.flood preference.
- **Normal case:**
 - User opens the map; the system loads UserPreferences for MapLayers.
 - If Overlays.Flood = true, the system calls GET /api/v1/map/current-status and connects to the SignalR flood hub.
 - Markers and flood-zone polygons are rendered on top of the base map.
 - SignalR pushes ReceiveSensorUpdate events; the map re-renders with updated colors and water levels.
 - User selects Base Map, toggles overlays, adjusts opacity, and saves -> The system validates the settings, serializes to JSON, and upserts the UserPreferences row.
 - The user taps a marker -> FloodStationCard sheet opens with full station details.
 - The user backgrounds the app -> SignalR hub stops. User returns to foreground -> hub restarts and resyncs.
- **Abnormal case:**
 - User submits invalid settings (e.g., opacity out of range or unsupported base map) -> the system rejects the update with a validation error and keeps existing preferences.
 - REST fetch fails: React Query surfaces the error state but the last cached GeoJSON remains visible to avoid a blank map.
 - SignalR connection drops: Existing markers stay visible while the hub logs reconnection states; state is reconciled upon successful reconnection.
 - Invalid coordinates / missing water level: The offending feature is silently filtered out during rendering without a crash.
 - Flood overlay toggled off: Markers and polygons disappear immediately, and the SignalR hub is released.

3.7 Filter and Toggle Map Layer

Function trigger:

- **Navigation path:** Map Screen → Layer Button (layers icon on bottom-right) OR Map Header → "Layers" button
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** All Users (Authenticated users + Guest users)

- **Purpose:** To enable users to customize map display layers according to their preferences, including selecting base map type, toggling overlay visibility, and adjusting layer transparency for optimal flood monitoring experience.
- **Interface:**
 - **Bottom Sheet Modal:** Slide-up panel from bottom, max height 70% of screen.
 - **Base Map Selector:** 2 selectable cards (“Tiêu chuẩn”/ “Vệ tinh”) with icons and active state highlight.
 - **Overlay Toggle List:** List of 3 overlay layers with Switch toggles:
 - “Mức ngập” - Display flood severity markers at monitoring stations
 - “Giao thông” - Show real-time traffic conditions on roads
 - “Thời tiết” - Display rain radar overlay
 - **Opacity Sliders:** Range sliders (0-100%) to adjust transparency for Flood and Weather layers, shown only when respective layer is enabled

Data processing:

- The system queries the UserPreferences table for the loggedin user where PreferenceKey = 'MapLayers' to load saved layer toggles and opacity settings.
- If no record is found or the JSON is invalid, the system falls back to default map layer settings.
- The user toggles layers/opacity and clicks Save.
- The system validates the settings (allowed base map values, opacity range 0–100).
- The system upserts the UserPreferences record, storing the JSON in PreferenceValue and updating audit fields (UpdatedAt, UpdatedBy) or inserting a new row.
- The user applies a map filter (bounds/zoom).
- The system queries the stations table for active stations and filters by latitude/longitude bounds.
- The system returns a GeoJSON layer with per-station severity properties (currently computed from mock water level data).

Screen layout:

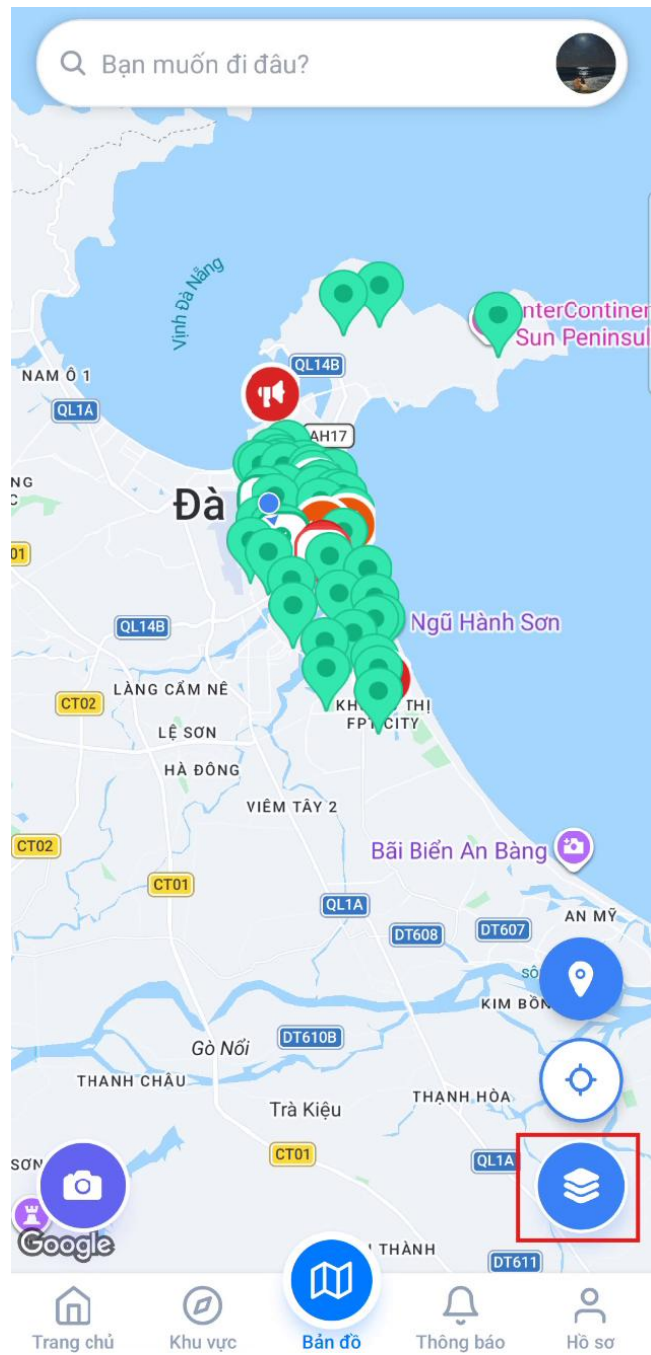


Figure 36 Filter and Toggle Map Layer

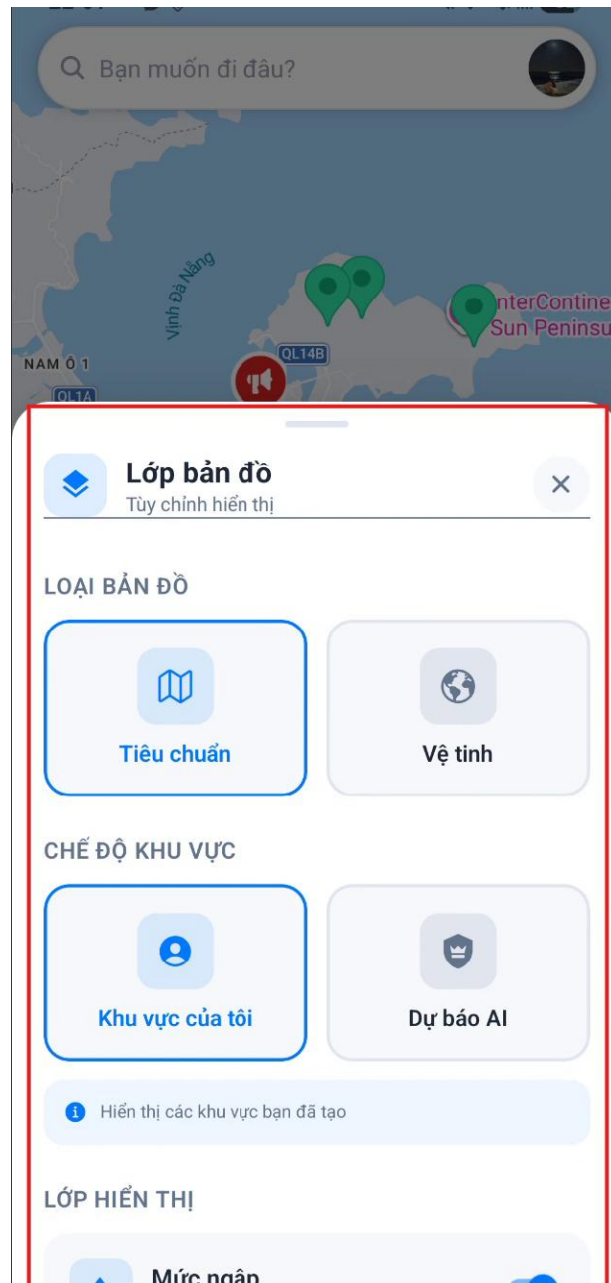


Figure 37 Filter and Toggle Map Layer

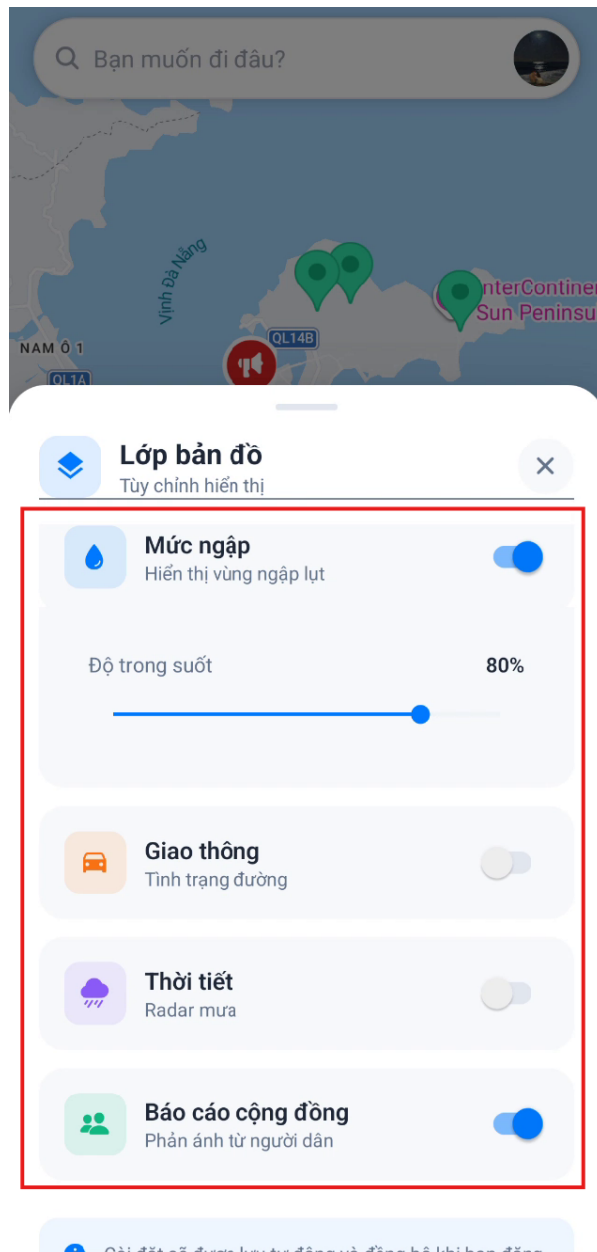


Figure 38 Filter and Toggle Map Layer

Function details:

- **Data:** User ID, BaseMap, Overlay toggles (Flood/Traffic/Weather), Opacity (Flood/Weather).
- **Validation:**
 - **BaseMap:** Must be one of standard, satellite, or hybrid (API accepts hybrid even if UI shows 2 options).
 - **Opacity:** Flood/Weather must be between 0 and 100.
- **Business rules:** Any authenticated user can view and update their own map layer preferences.
- **Normal case:**
 - The user opens the map; the system loads UserPreferences for MapLayers.
 - The user selects Base Map, toggles overlays, adjusts opacity, and saves.

- The system validates the settings, serializes to JSON, and upserts the UserPreferences row (insert if missing, otherwise update preference_value, UpdatedAt, UpdatedBy).
- **Abnormal case:** User submits invalid settings (e.g., opacity out of range or unsupported base map) → the system rejects the update with a validation error and keeps existing preferences.

3.8 View Flood Conditions by Sensor Stations

Function trigger:

- **Navigation path:** Map Screen (Mapbox) → Tap a Sensor Station Marker
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen, Moderator and Admin (Authenticated users)
- **Purpose:** Displays flood conditions and latest readings collected from sensor stations.
- **Interface:**
 - **Map View (Mapbox):** Displays sensor station markers.
 - **Station Markers:** Marker color reflects flood severity, consistent with the Flood Severity legend.
 - **Flood Severity Legend:** Shows severity levels (An toàn / Chú ý / Cảnh báo / Nguy hiểm) with corresponding colors.
 - **Flood Detail Popup Card (on marker tap):** Shows a quick overview of the selected station and its latest reading.
 - **“Chi tiết” button:** Navigates to another use case (Flood History / analytics) for station history & predictions (not detailed in 3.8)

Data processing:

- Actors/Roles: Citizen, Moderator and Admin (Authenticated users)
- If provided, the system applies filters:
 - Status filter (Stations.Status)
 - Map bounds filter (min/max lat/lng) to return only stations in the current viewport.
- The system queries SensorReadings and gets the latest reading per station (by MeasuredAt).
- For each station, the system derives flood severity from the latest water level and maps it to a severity level + marker color (aligned with the Flood Severity legend). If no reading exists, severity is No data/Unknown
- The system returns a GeoJSON FeatureCollection: Each station is a Point feature with properties needed for:
 - Marker rendering (severity, markerColor, alertLevel)
 - Popup flood detail card (station info + latest reading + updated time).

Screen layout:

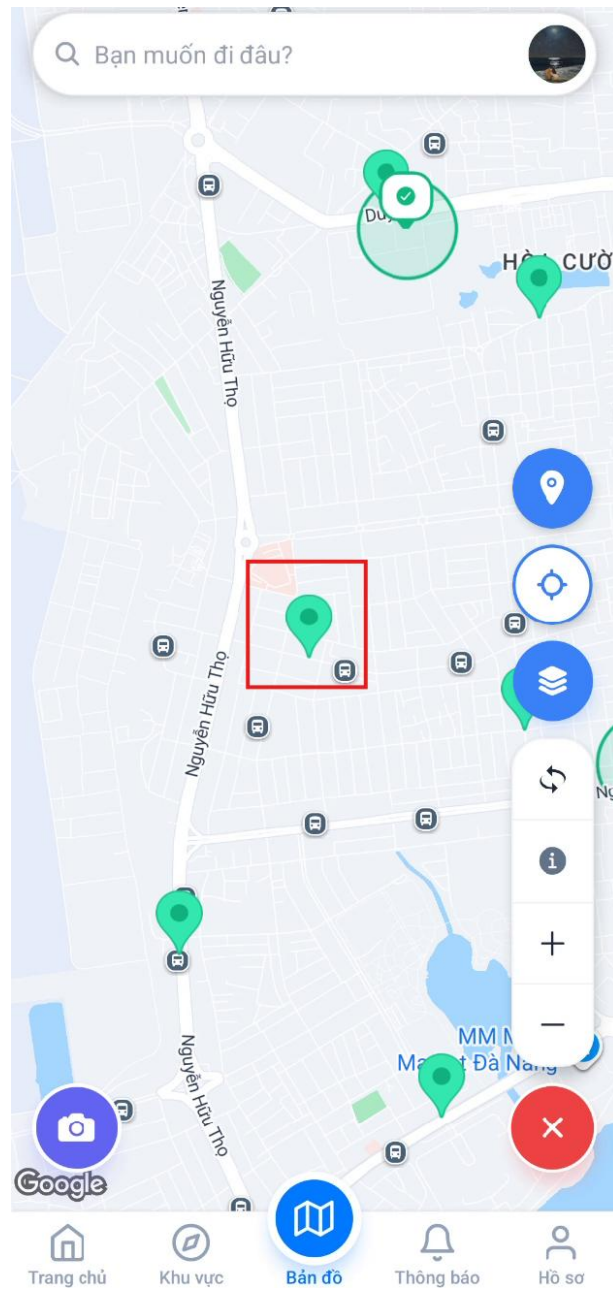


Figure 39 View Flood Conditions by Sensor Stations

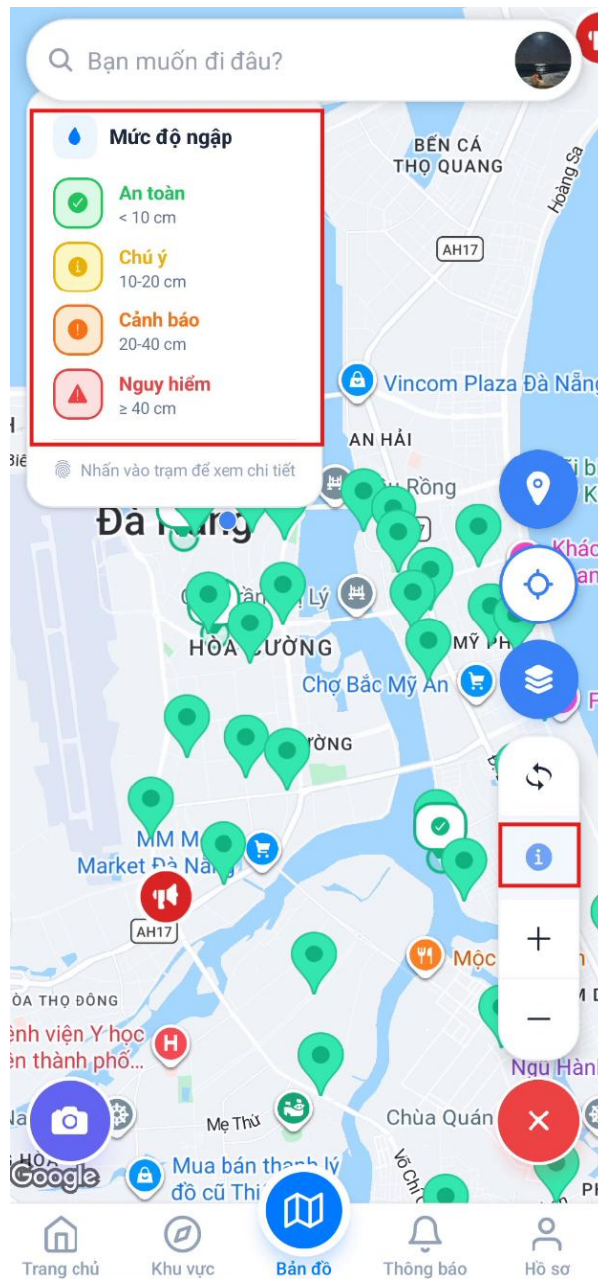


Figure 40 View legend

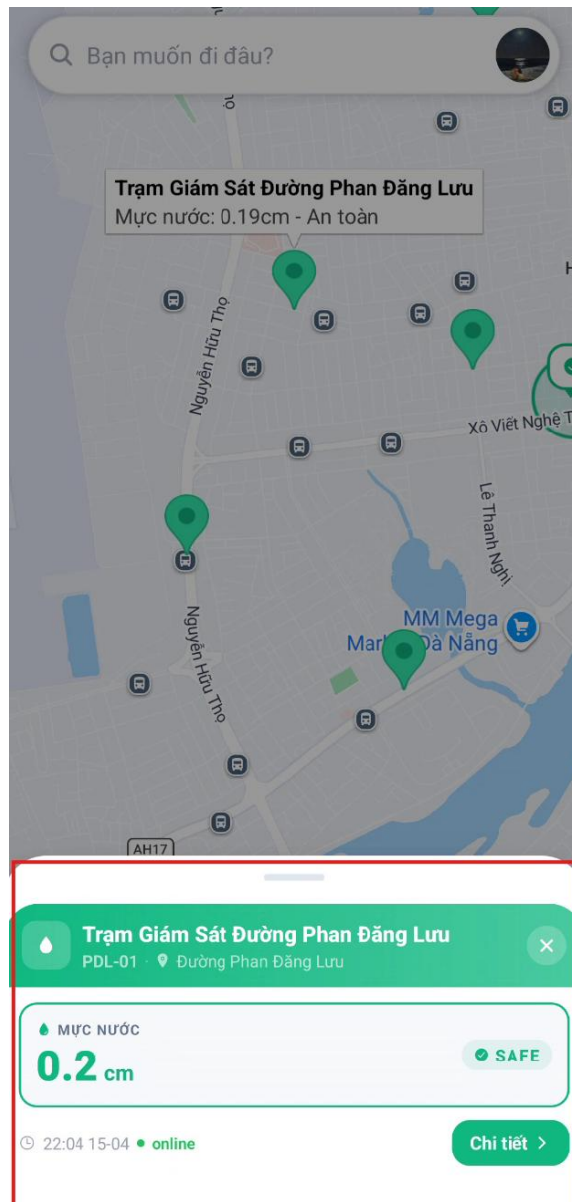


Figure 41 View Flood Conditions by Sensor Stations

Function details:

- **Data:**
 - Geometry: Point [Longitude, Latitude]
 - Station info: StationId, StationCode, StationName, LocationDesc, RoadName
 - Latest reading: WaterLevel, Distance, SensorHeight, Unit, MeasuredAt
 - Flood condition: Severity, SeverityLevel
 - Station status: StationStatus, LastSeenAt
 - Map styling: MarkerColor, AlertLevel
- **Validation:**
 - **Latitude** (MinLat, MaxLat): must be within [-90, 90]
 - **Longitude** (MinLng, MaxLng): must be within [-180, 180]

- **Status** : must be one of active, offline, maintenance
- **Business rules:** Any authenticated user can view flood conditions by sensor stations.
- **Normal case:**
 - The user opens the Map and sees station markers colored by flood severity.
 - The user taps a marker → system shows the Flood Detail Popup Card with station overview + latest reading time/value.
 - The user taps “Chi tiết” → navigates to the Flood History / analytics screen(except Citizen).
- **Abnormal case:**
 - Station has no latest reading → marker/popup displays No data/Unknown state.
 - Invalid bounds/status filter → system returns a validation error and keeps the map in the current state.

3.9 Register Monitored Areas (My Areas)

Function trigger:

- **Navigation path:** Tab “Khu vực” → “Add Area” button / Map screen → Long press or search location
- **Timing Frequency:** Event-driven (User-initiated action when creating, viewing, updating, or deleting monitored areas)

Function description:

- **Actors/Roles:** Citizen (Free: max 5 areas, Premium: unlimited)
- **Purpose:** To allow users to register and manage specific locations for flood monitoring, enabling personalized alerts for areas important to them (home, workplace, school, family locations).
- **Interface:**
 - **Mobile App:** Map view with location picker, radius slider (150m), area list with status indicators.
 - **API:** RESTful endpoints for CRUD operations.

Data processing:

- Areas Stores user's monitored locations with coordinates, radius, and address text
- Users Links areas to authenticated users via UserId foreign key
- UserSubscriptions Determines area limit based on subscription tier (Free: 5, Premium: unlimited)

Screen layout:

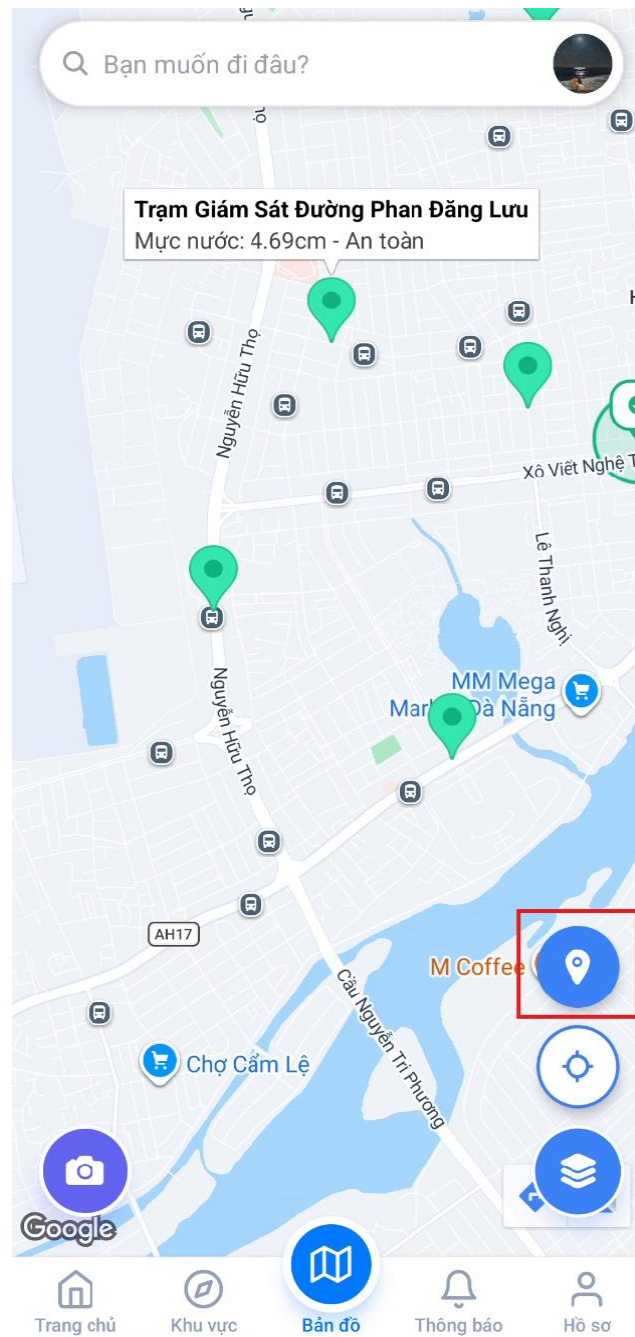


Figure 42 Register Monitored Areas

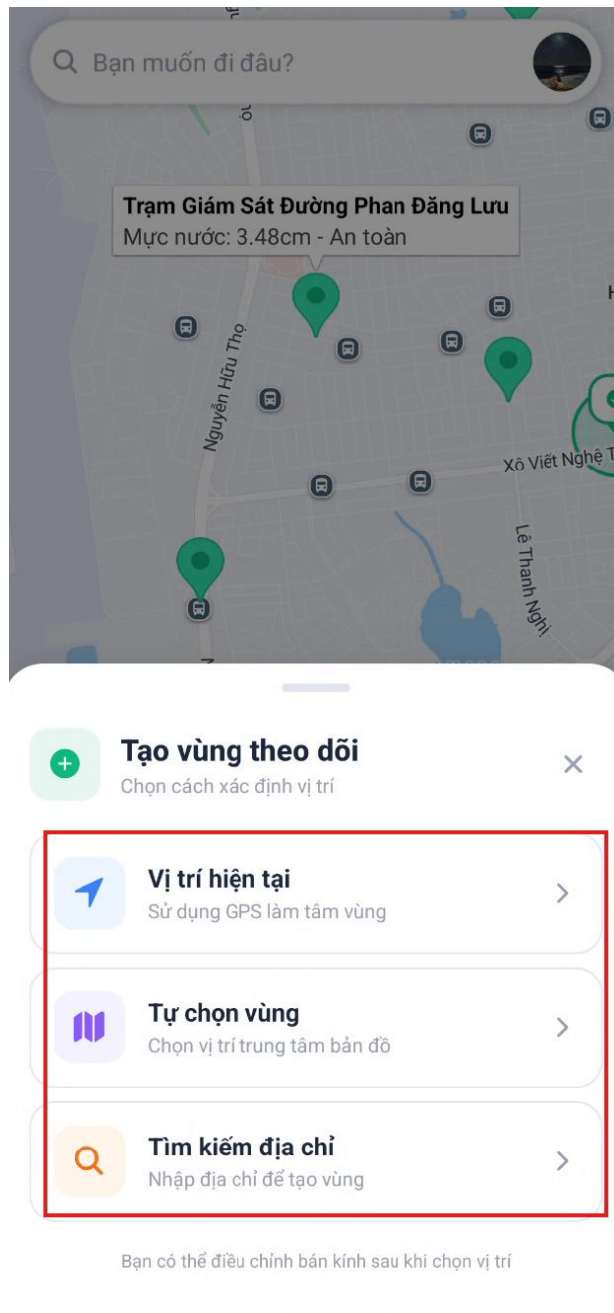


Figure 43 Register Monitored Areas

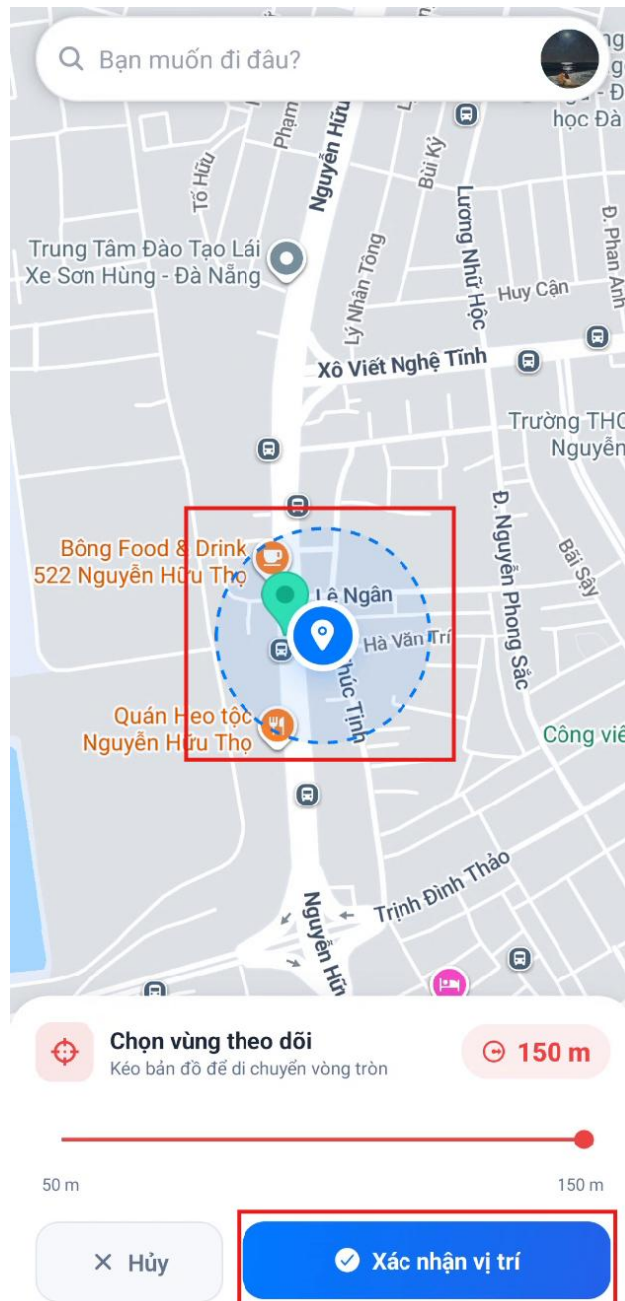


Figure 44 Register Monitored Areas

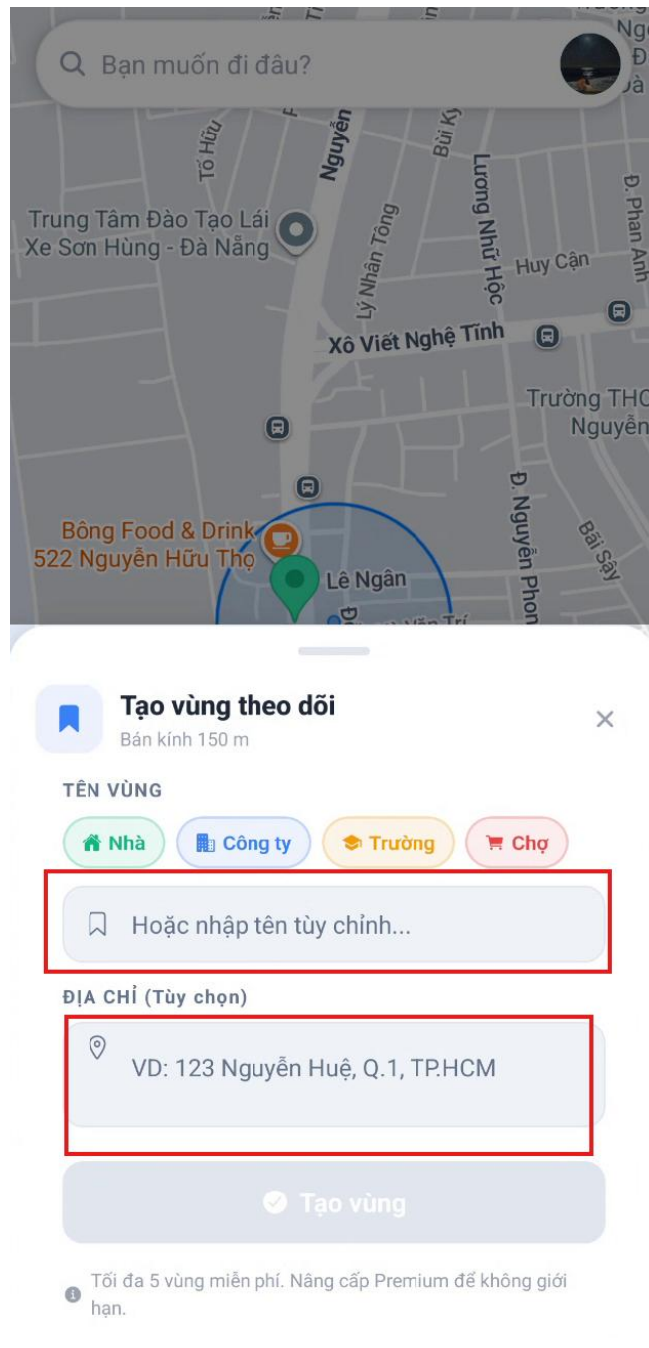


Figure 45 Register Monitored Areas

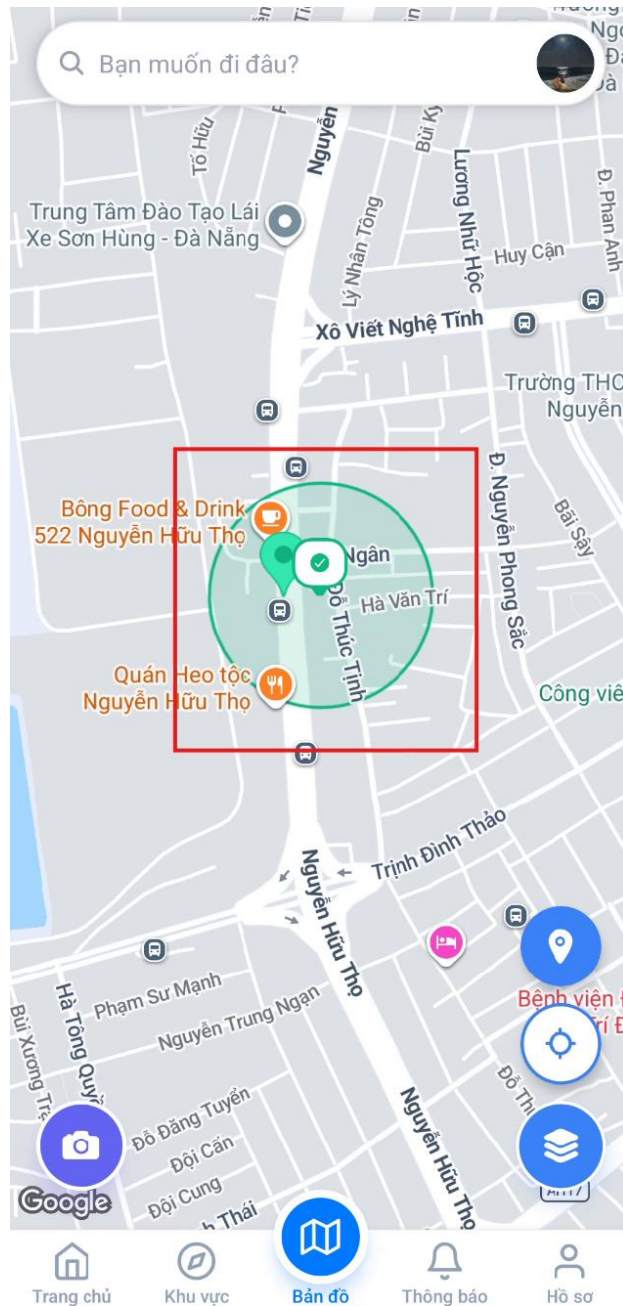


Figure 46 Register Monitored Areas

Function details:

- **Data:** Area Name, Latitude (-90 to 90), Longitude (-180 to 180), Radius (100m - 5000m), Address Text, Timestamps
- **Validation:**
 - Geometry: Valid latitude/longitude ranges
 - Radius: $50m \leq \text{radius} \leq 250m$
 - Name: Required, max 255 chars, unique per user (case-insensitive)
 - Duplicate Prevention: No areas within 50m of existing area center
- **Business Rules:**
 - Free users limited to 5 areas
 - Premium users have unlimited areas
 - Users can only access/modify their own areas

- Duplicate location check uses Haversine formula (50m threshold)
- **Normal Case:**
 - **Option 1** - GPS: User opens "Areas" tab → Taps "Create" → Option sheet appears → Selects "Current location" → App requests GPS permission → Gets current location → Map centers on user location with 150m radius circle → User can tap on map to adjust position → Adjusts radius slider (50m-150m) → Taps "Confirm location" → Enters area name and address (optional, pre-filled if searched) → Taps "Create location" → Area appears in list with current flood status within 2 seconds.
 - **Option 2** - Search location: User opens "Areas" tab → Taps "Create" → Option sheet appears → Selects "Search location" → Address search sheet opens → User enters address → Taps search button → Geocoding converts address to coordinates → Map centers on searched location with 150m radius circle → User can tap on map to adjust position → Adjusts radius slider (50m-150m) → Taps "Confirm location" → Enters area name (address pre-filled) → Taps "Create Area" → Area appears in list with current flood status within 2 seconds

3.10 Manage Monitored Areas

Function trigger:

- **Navigation path:** Home Screen (Monitored Areas section) → Areas Tab (“Khu vực của tôi”) → Area Detail Screen / Create Area Flow / Edit Area Sheet / Delete Confirmation
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen, Moderator and Admin (Authenticated users)
- **Purpose:** Allows authenticated users to view, create, edit, and delete their personal Monitored Areas, enabling them to track flood conditions for any location they care about.
- **Interface:**
 - **Areas Tab – My Areas sub-tab:** Displays all user-owned areas as WaterLevelAreaCards showing area name, address, flood status badge (Normal / Watch / Warning / Unknown), current water level, and monitoring radius.
 - **WaterLevelAreaCard (long-press or context menu):** Exposes quick-access Edit and Delete actions for the selected area.
 - **Area Detail Screen:** Full-detail view with a collapsible animated header (water level gauge, status badge), contributing sensor list, affected streets, weather section, and flood history/trend charts.
 - **Edit Area Sheet:** A bottom sheet modal pre-filled with the selected area’s name, radiusMeters, and addressText.
 - **Confirm Delete Modal:** A modal dialog displaying the area name and a permanent-deletion warning, with Confirm and Cancel actions.
 - **Error Modal:** Displays API or network error messages without losing current screen state.

Data processing:

- **Actors/Roles:** Citizen, Moderator and Admin (Authenticated users)
- **View - List:**
 - The system calls GET /api/v1/areas/me?pageNumber=1&pageSize=100 to retrieve all areas owned by the authenticated user.

- For each area, GET /api/v1/area/areas/{areaid}/status returns the current flood status (Normal / Watch / Warning / Unknown), severity level, summary, and contributing stations.
- Status badge color is derived from the status value: Normal → #10B981 (green), Watch → #FBBF24 (yellow), Warning → #EF4444 (red), Unknown → #9CA3AF (gray).
- **View - Detail:**
 - The system calls GET /api/v1/areas/{id} to load full area data (name, location, radius, address, timestamps).
 - The contributing stations list is sourced from AreaStatusResponse.contributingStations (stationId, stationCode, distance, waterLevel, severity, weight).
 - Flood history chart data: GET /api/v1/flood-history with the area's coordinates and radius as filter parameters.
 - Flood trend chart data: GET /api/v1/flood-trends with the same spatial parameters.
- **Create:**
 - The user pins a point on the map picker to set latitude and longitude.
 - The user fills in: name (required), radiusMeters (required), addressText (optional).
 - The system calls POST /api/v1/areas/area with body { name, latitude, longitude, radiusMeters, addressText }.
 - On success (HTTP 201), the new area is prepended to the My Areas list and its status is fetched asynchronously.
- **Edit:**
 - The EditAreaSheet pre-populates with the selected area's current { name, radiusMeters, addressText }.
 - On Save, the system calls PUT /api/v1/areas/{id} with the updated field values.
 - On success (HTTP 200), the area card and detail header are refreshed with the latest data.
- **Delete:**
 - The ConfirmDeleteModal displays the area name and a non-reversible warning message.
 - On Confirm, the system calls DELETE /api/v1/areas/{id}.
 - On success (HTTP 204), the area is removed from the Redux store and the user is navigated back to the Areas Tab.

Screen layout:

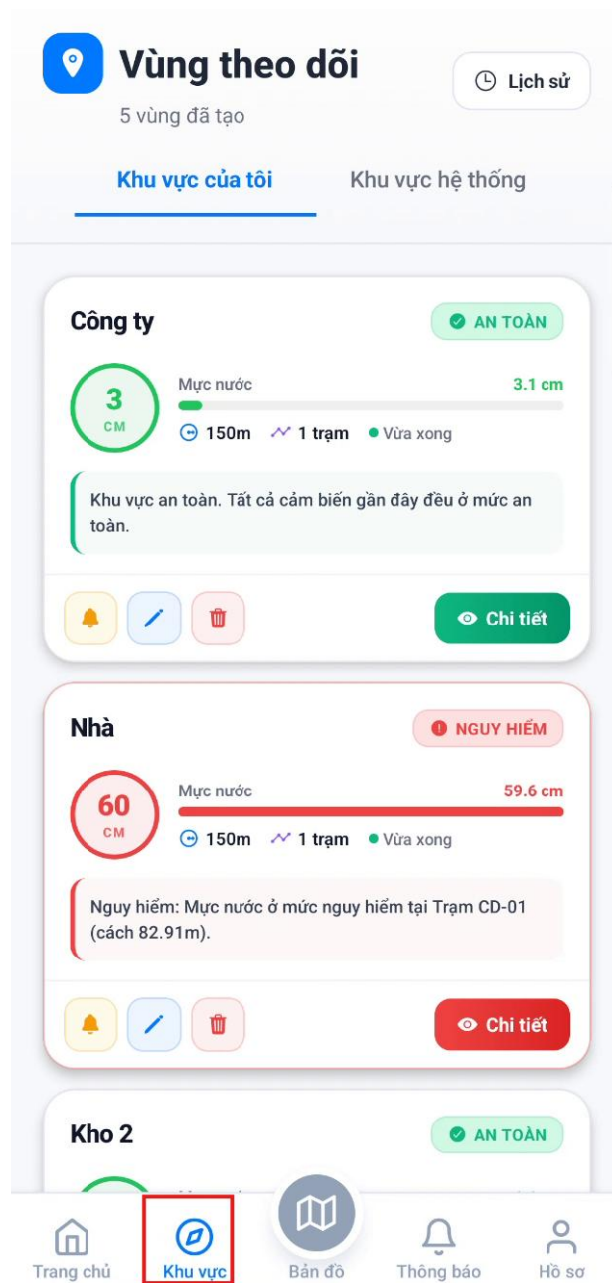


Figure 47 Areas Tab: My Areas sub-tab with WaterLevelAreaCards and flood status badges

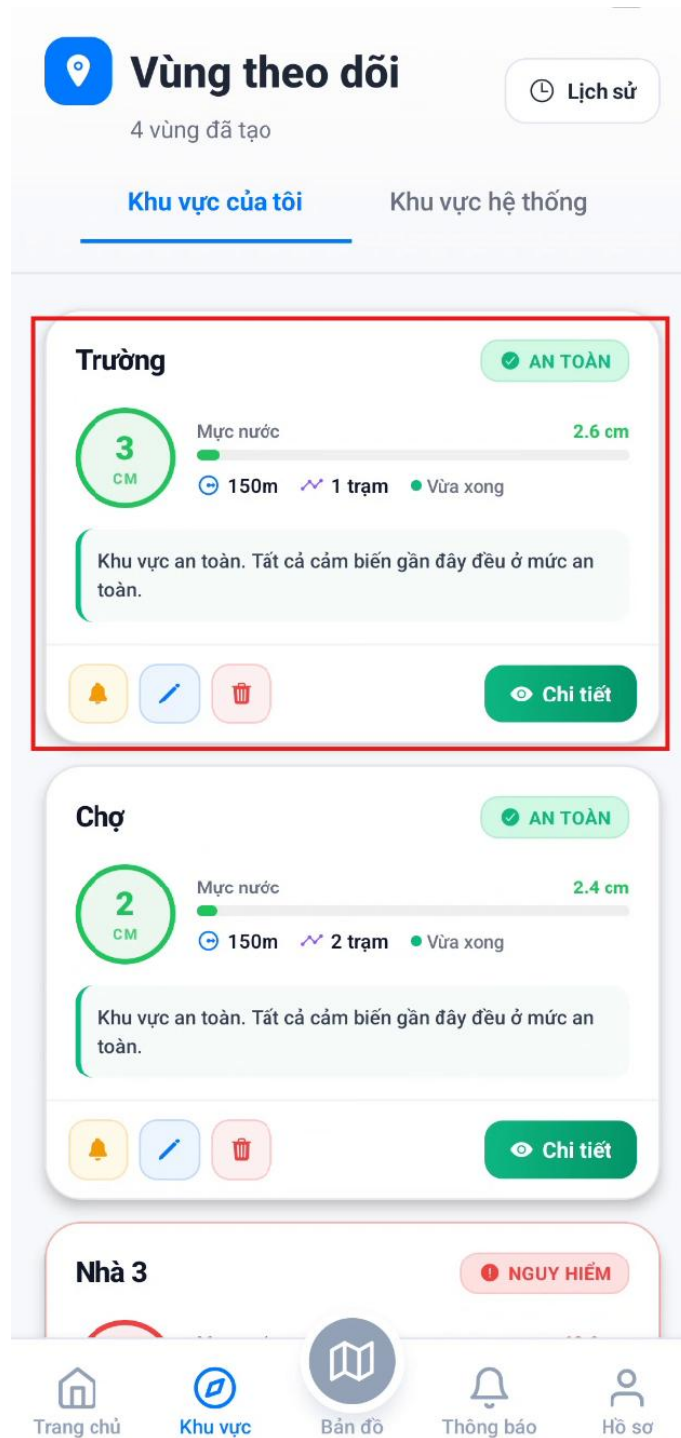


Figure 48 Area Detail Screen: animated header, water level gauge, contributing sensors list

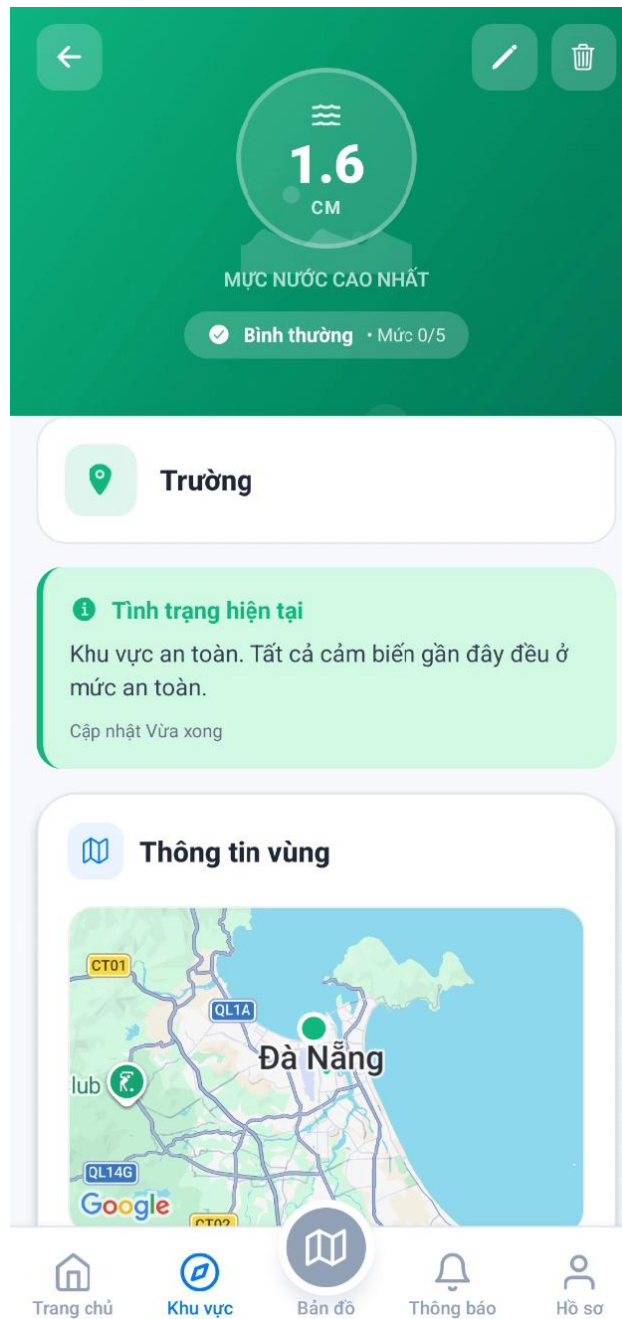


Figure 49 Area Detail Screen: animated header, water level gauge, contributing sensors list

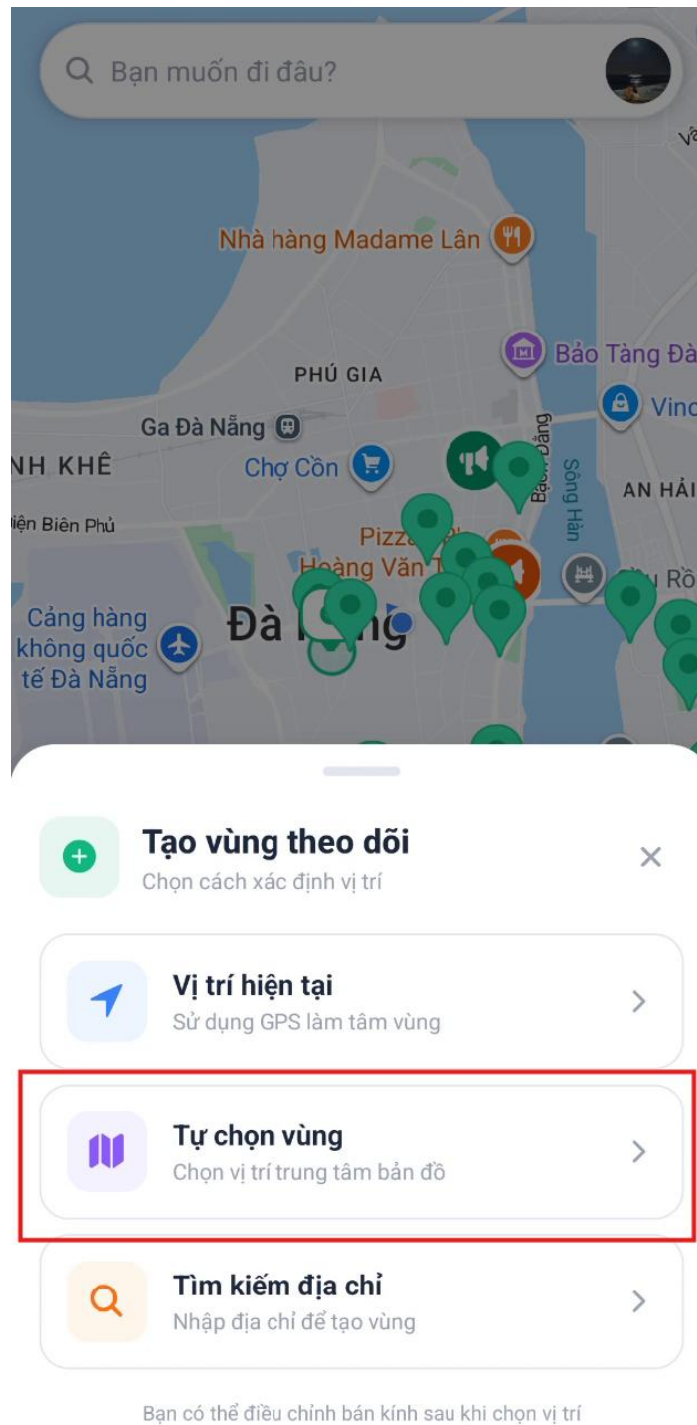


Figure 50 Create Area Step 1: Map picker screen — user pins a location on the Mapbox map

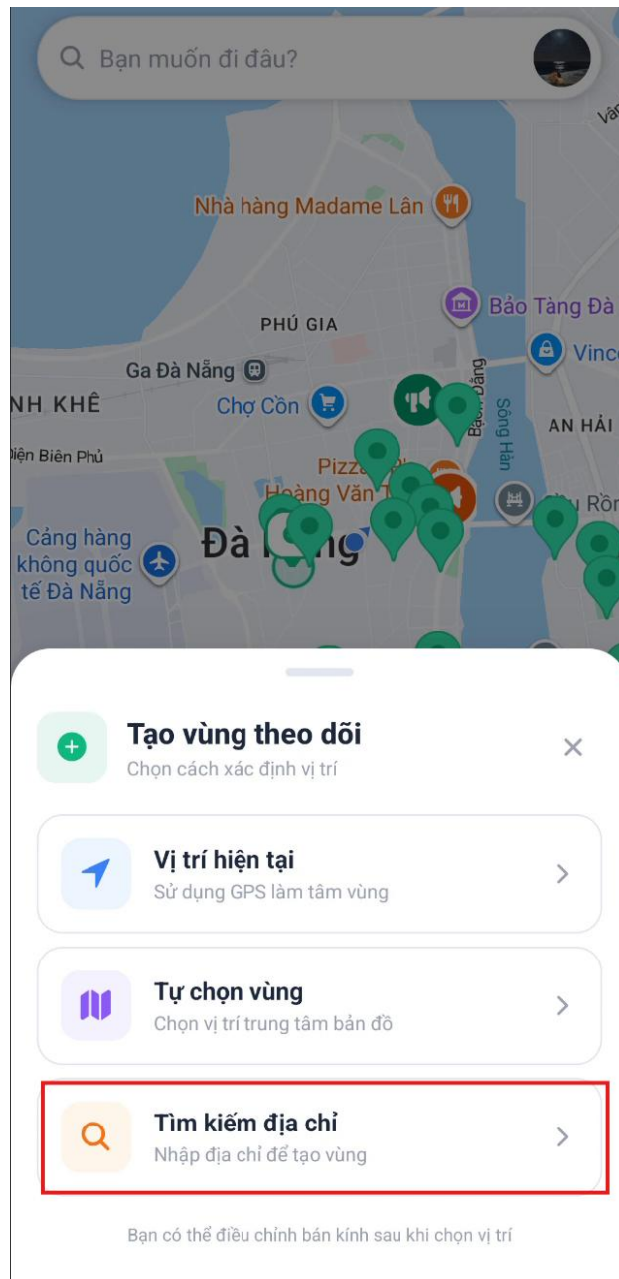


Figure 51 Create Area Step 2: Name, address, and radius input form with inline validation

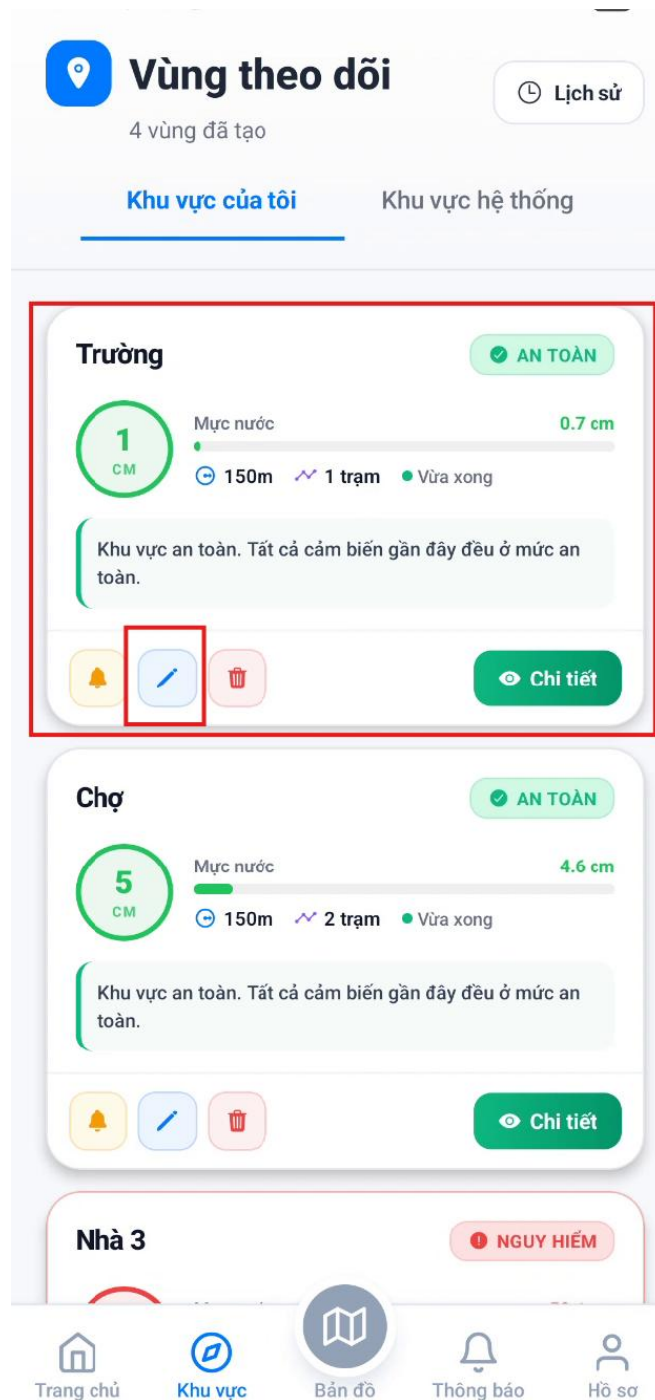


Figure S2 Edit button

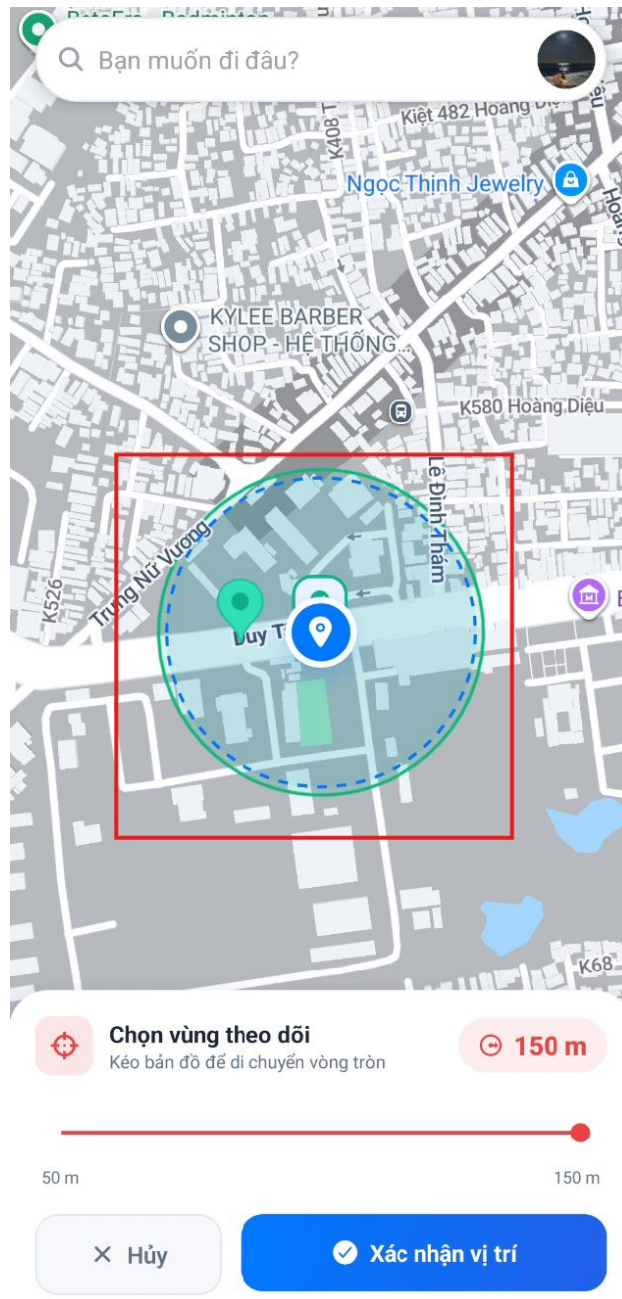


Figure 53 Edit Area: EditAreaSheet bottom sheet pre-filled with existing area data

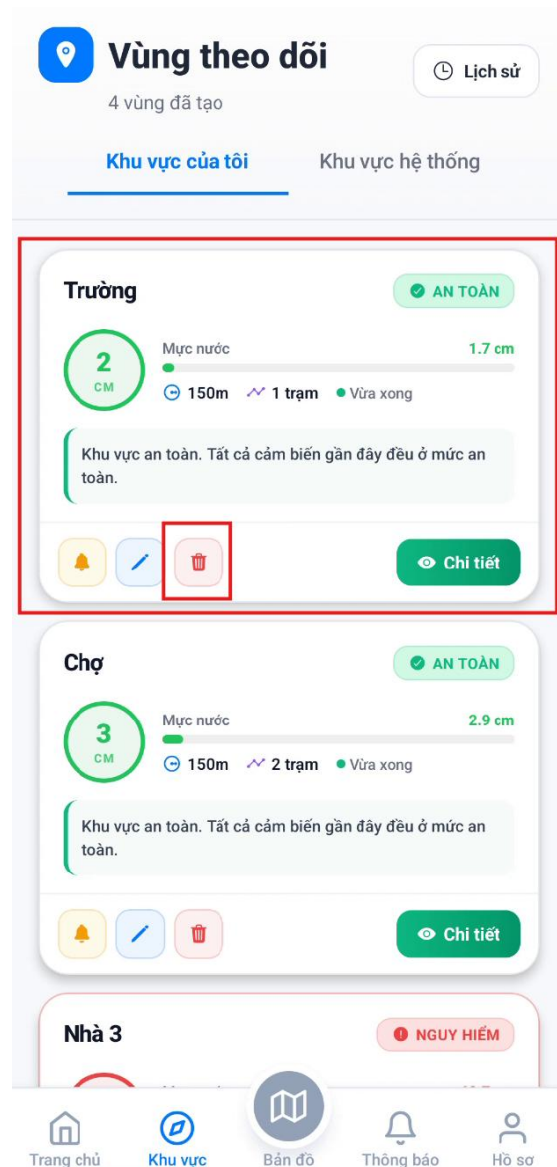


Figure 54 Delete Area: ConfirmDeleteModal with area name, warning, Confirm / Cancel buttons

Function details:

- **Data:**
 - Area identity: Id (UUID), Name (string, max 100 chars), AddressText (string, optional, max 255 chars)
 - Location: Latitude (number, -90 to 90), Longitude (number, -180 to 180), RadiusMeters (positive integer, 100–50000 m)
 - Flood status: Status (Normal | Watch | Warning | Unknown), SeverityLevel (integer 0–5), Summary (string), EvaluatedAt (ISO timestamp)
 - Water data: WaterLevel (cm), MaxLevel (cm)
 - Sensors: SensorCount (integer), ContributingStations[] (StationId, StationCode, Distance, WaterLevel, Severity, Weight)
 - Affected streets: AffectedStreets (string[])
 - Timestamps: CreatedAt (ISO), UpdatedAt (ISO)
- **Validation:**
 - Name: required; 1–100 characters; must not be blank or whitespace-only.

- Latitude: required; numeric; within [−90, 90].
- Longitude: required; numeric; within [−180, 180].
- RadiusMeters: required; positive integer; recommended range 100–50000 m.
- AddressText: optional; maximum 255 characters.
- Id (edit / delete): must be a valid UUID owned by the authenticated user (enforced server-side).
- **Business rules:** Only authenticated users can create and manage their own Monitored Areas.
- **Normal case:**
 - View List: The user opens the Areas tab and selects My Areas to see all saved areas, along with their current water level and status.
 - View Detail: The user taps an area to view more details, including key information, related sensors, and recent flood history.
 - Create: The user taps the + button, chooses a location on the map, enters an area name and radius, then taps Save. The new area is added to My Areas.
 - Edit: The user opens an area, taps Edit, updates the name or radius, and taps Save. The changes are then shown in the area details.
 - Delete: The user opens an area, taps Delete, and confirms. The area is removed from the list.
- **Abnormal case:**
 - API failure: System displays ErrorModal with the server error message; current screen state remains unchanged.
 - Network timeout on area status fetch: Area card shows Unknown status badge (gray) and displays the last-known data if available.

3.11 Evaluate Administrative Area-based Flood Updates

Function trigger:

- Admin/Moderator opens the administrative-area evaluation view from the monitoring/coordination portal.
- Admin/Moderator selects an administrative area (e.g., ward/district) and requests the current aggregated flood status.

Function description:

The system enables Admin and Moderator to evaluate flood conditions by administrative area, using water-level measurements from monitoring stations located within each respective area. The system aggregates station readings to compute and present an area-level flood status used for operational decision-making.

- **Actors/Roles:** Admin, Moderator
- **Purpose:**
 - Provide an area-level operational view (ward/district) for coordination and warning decisions.
 - Reduce cognitive load by summarizing station-level signals into administrative-area risk states.
- **Interface:**
 - Area selection:
 - Administrative area list and filters (e.g., ward/district) sourced from existing administrative area data services.

- Area overview panel:
 - Aggregated status for selected area (e.g., Safe/Caution/Warning/Critical/Unknown)
 - Station count in area, number of active stations, and latest update time
 - Links to underlying station readings (drill-down)
- Optional map overlay:
 - Highlight the selected area boundary and render contributing station markers.

Data processing:

- Fetch administrative areas:
 - Endpoint available in codebase: `GET /admin/administrative-areas` (requires auth), with pagination and optional `level` filter.
- Fetch station readings and station-to-area association:
 - Station objects may include `administrativeAreaId` (data model alignment).
- For a selected area:
 - Identify all stations mapped to the area.
 - Retrieve latest water levels and station severity indicators.
 - Compute aggregated area status using defined aggregation rules (see Business rules).
- Display area-level status and supporting evidence (stations and their readings).

Screen layout:

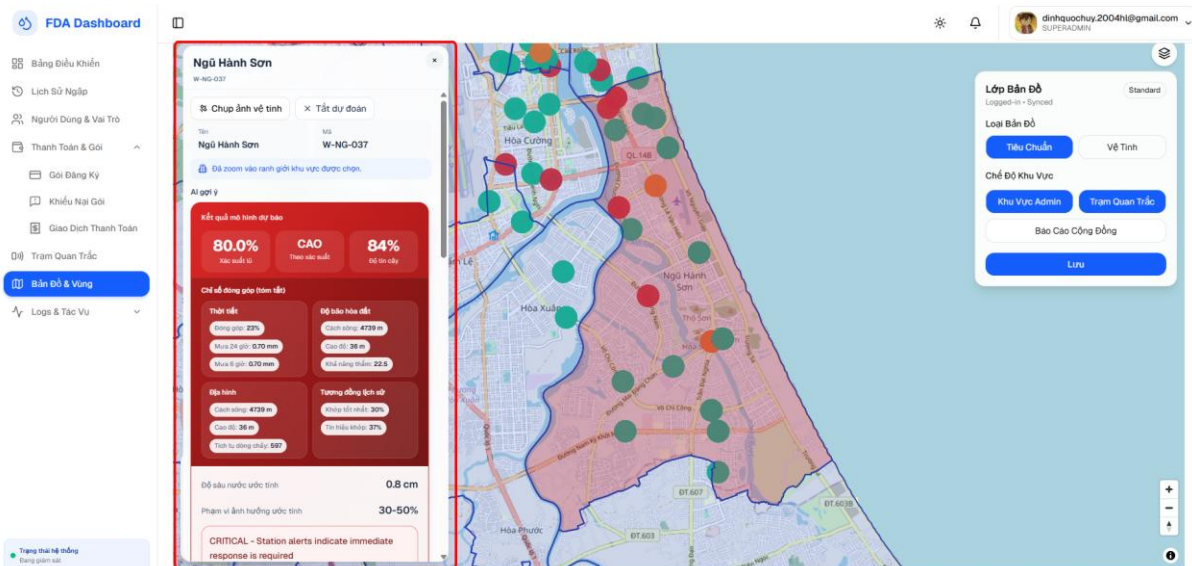


Figure 55 Evaluate Administrative Area-based Flood Updates

Function details:

- Data:
 - AdministrativeArea: id, name, level, geometry/boundary (optional)
 - Station: id, name, code, administrativeAreaId, latestWaterLevel, measuredAt, severity/status
 - AggregatedAreaStatus: severity, supporting stats, computedAt

- **Validation:**
 - Selected administrative area must exist.
 - Water level values must be numeric; missing values mark station as unknown.
 - Geometry/boundary must be valid if map visualization is enabled.
- **Business rules:**
 - **Aggregation rule (recommended baseline):**
 - If any station is Critical → area is Critical.
 - Else if any station is Warning → area is Warning.
 - Else if any station is Caution → area is Caution.
 - Else if all contributing stations are Safe → area is Safe.
 - Else → area is Unknown.
 - **Data freshness rule (recommended):**
 - If latest reading is older than a threshold, station is treated as Unknown for aggregation.
 - **Access control:**
 - Only Admin and Moderator can evaluate administrative-area updates.
- **Error Handling:**
 - No stations mapped to the area → display “No monitoring stations available for this area”.
 - Station data fetch failure → show partial results where possible and indicate degraded state.
 - Backend errors → display generic error and allow retry.
- **Normal case:**
 - Admin/Moderator opens area evaluation.
 - Selects an administrative area.
 - System aggregates station readings and shows area status.
 - Admin/Moderator reviews stations contributing to the status.
- **Abnormal case:**
 - Selected area has no stations → system shows empty/unknown status with explanation.
 - Data stale/unavailable → system marks status as unknown and prompts retry.

3.12 Receive Flood Alerts (Push / SMS / Email)

Function trigger:

- **Navigation path:** Tabs → Alerts (modal) → Alert Settings → Notification Channels section.
- **Timing Frequency:** On demand, triggered by the user when viewing/updating alert preferences for a specific area.

Function description:

- **Actors/Roles:** Citizen
- **Purpose:** Manage notification delivery settings for flood alerts across three communication channels (Push notifications, Email, SMS) and configure severity filtering and quiet hours.
- **Interface:**
 - **Notification Channels Section:** Three toggles for Push Notifications, Email Alerts, and SMS Messages.

- **Minimum Severity Section:** Radio-button style selector for Caution, Warning, or Critical severity levels
- **Quiet Hours Section:** Time picker fields for start/end times of quiet hours (optional do-not-disturb period).
- **Save Settings Bar:** "Lưu thay đổi" button to persist all preferences.
- **Alert Settings Header:** Shows area name and option to tap through to threshold settings

Data processing:

- **On screen load:** System retrieves stored settings from AsyncStorage using areald key. If none exist, defaults are applied (Push enabled, Email/SMS disabled, Warning severity, quiet hours 22:00-06:00).
- **For push channel:** System requests notification permission (Android 13+ requires POST NOTIFICATIONS runtime permission; iOS uses system prompt).
- When user modifies toggles: Form state updates locally without persistence yet.
- When user saves: Form state is packaged into AlertSubscriptionRequest payload containing: MinSeverity, EnablePush, EnableEmail, EnableSms, QuietHoursStart, QuietHoursEnd.
- If provided, the system applies filters:
 - API call: PUT /api/v1/alerts/subscriptions/{areald} with payload.
 - On success: Settings are persisted to AsyncStorage with key areald and success alert is shown.
 - On error: Error message is displayed; user can retry. Original settings remain in AsyncStorage.
- **Notification delivery:** When flood alert is triggered for the area, backend uses saved preferences to determine which channel(s) to send alert through, respecting enabled channels and severity filters.
- **Push notifications:** Delivered via Firebase Cloud Messaging (FCM). Requires device token registration (obtained during app init/auth). Foreground messages show in-app banners; background/killed taps navigate to alert details.
- **Email/SMS:** Backend sends via configured email/SMS provider when respective channels are enabled.

Screen layout:

Figure 56 Receive Flood Alerts

Function details:

- **Data:**
 - NotificationChannels: { push: boolean, email: boolean, sms: boolean }
 - QuietHours: { StartTime: string (HH:MM:SS), EndTime: string (HH:MM:SS) }
 - Areald: string (UUID of monitored area)
 - areaName: string (display name of area)
 - AlertSubscriptionRequest: API payload with minSeverity, enablePush, enableEmail, enableSms, quietHoursStart, quietHoursEnd
- **Validation:**
 - Authenticated user required: Request must include valid access token in Authorization header.
 - Valid areald required: areald must exist and belong to authenticated user's subscribed areas.
 - Severity values must be one of: "Caution", "Warning", "Critical" (maps to lowercase in API).
 - Time format: HH:MM:SS (24-hour format), validated by time picker component.

- Permission requirement for push: Android 13+ requires POST NOTIFICATIONS permission; iOS uses standard notification prompt.
- **Business rules:**
 - Only authenticated users can configure alert notification settings.
 - Notification preferences are area-specific; each monitored area has separate settings.
 - Push notifications require device token registration; system requests permission on app init.
 - Email and SMS channels require stored email/phone in user profile (backend-validated).
 - Quiet hours setting is respected: backend may suppress notifications outside specified hours.
- **Normal case:**
 - The user opens an area from the Areas tab and taps on the area card.
 - AlertSettings modal opens; areaId and areaName are passed as route params.
 - System loads stored settings from AsyncStorage for that areaId, or uses defaults.
 - Settings are displayed: channel toggles show on/off state, severity selector shows current level, time pickers show quiet hours.
 - User toggles one or more channels (e.g., enables Email, disables SMS).
 - The user adjusts the minimum severity level using the severity selector.
 - The user opens a time picker and adjusts quiet hours if needed.
 - User taps "**Lưu thay đổi**" button.
- **Abnormal case:**
 - Permission denied: User denies notification permission (Android 13+ or iOS). FCM token unavailable. Push channel cannot function. Logged but does not block save.
 - API request fails (network error, 401 unauthorized, 400 bad request): Error caught in try/catch. Error message displayed to user. Settings not saved. Users can retry.
 - Invalid areaId: Route params missing areaId. Screen returns null; nothing renders..

3.13 View Flood Alert History

Function trigger:

- **Navigation path:** Bottom Tab > “Thông báo” > “Lịch sử cảnh báo”
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen
- **Purpose:** To provide a chronological log of all flood events relevant to the user's monitored areas, serving as a personal safety record.
- **Interface:**
 - **List View:** Cards showing Location Name, Severity Icon (Color-coded), Time Sent, and Peak Water Level.

Screen layout

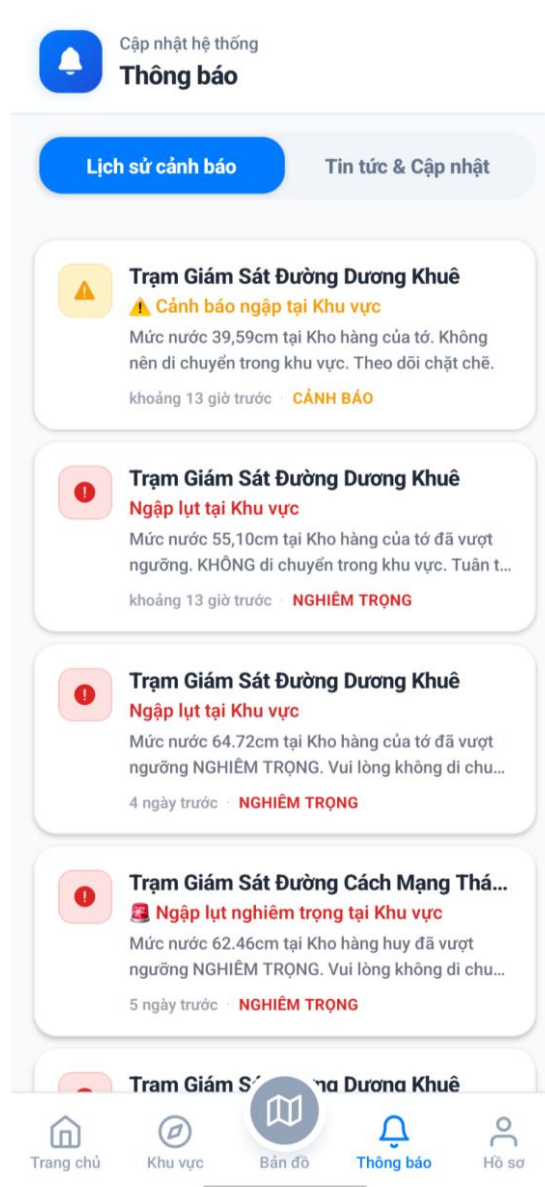


Figure 57 View Flood Alert History

Data processing:

- The system queries the NotificationLogs table joined with Alerts.
- **Filtering:** Selects records where UserId matches the current user.
- **Sorting:** Orders by CreatedAt descending (newest first).
- **Status Update:** Marks unread notifications as "Read" upon viewing.

Function details:

- **Data:** Alert Title, Body, Timestamp, Severity.
- **Business Rules:** History is retained for 30 days for Free users and 1 year for Premium users.
- **Normal Case:** User opens the history tab and sees a list: "Trạm giám sát đường Dương ", "13 giờ trước, Cảnh báo ngập trại khu vực".

3.14 View Flood History & Trends

Function trigger:

- **Navigation path:** Map View > Station Details > History Tab
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen, Moderator
- **Purpose:** To visualize historical water level data over time (24h, 7 days, 1 month), helping users understand flood patterns and trends.
- **Interface:**
 - **Biểu đồ mực nước:** X-axis (Time), Y-axis (Water Level cm).
 - **Reference Lines:** Horizontal lines indicating Warning/Danger thresholds.
 - **Date Picker:** Select custom range.

Screen layout:

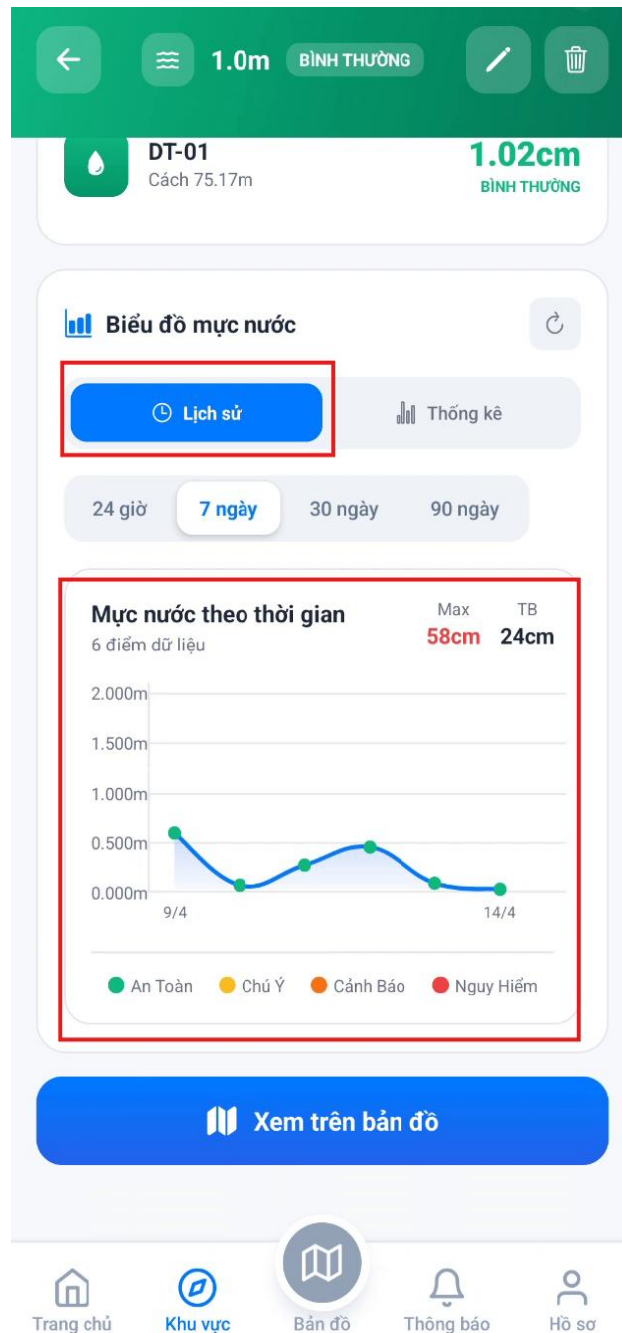


Figure 58 View Flood History

Data processing:

- User selects a station and a time range (e.g., Last 7 Days).
- **Query Strategy:**
 - For "24 giờ": Query **SensorReadings** (Raw data) for granular detail.
 - For "> 24 giờ": Query **SensorDailyAgg** (Aggregated data: Min/Max/Avg) to optimize performance.
- The system formats the dataset for the charting library (e.g., Recharts/Chart.js).

Function details:

- **Data:** Time series data points [timestamp, value].
- **Validation:** Date range cannot exceed the data retention policy (12 months).

- **Normal Case:** User selects "24h". The chart displays a curve showing water rising from 10cm to 45cm during last night's rain.

3.15 View AI-based Flood Prediction

Function trigger:

- **Navigation path:** Main Map or Flood Prediction Panel
- **Timing Frequency:** On demand (and automatically refreshed).

Function description:

- **Actors/Roles:** Citizen (Premium), Moderator
- **Purpose:** To display the calculated flood forecasts for the next 1–3 hours from the AI model.
- **Interface:** Dynamic map overlay and time selector (1 hr, 2 hr, 3 hr).

Data processing:

- The system first checks the user's active subscription in UserSubscriptions to confirm 'Premium' access.
- The system queries the PredictionResults table for the latest data associated with the most recent successful run in PredictionRuns.
- Data is filtered by HorizonMinutes (e.g., 60, 120, 180) and retrieved for all StationIds.
- The system uses the SensorReading and RiskLevel from PredictionResults to color-code and visualize the predicted flood extent on the map.

Screen layout:

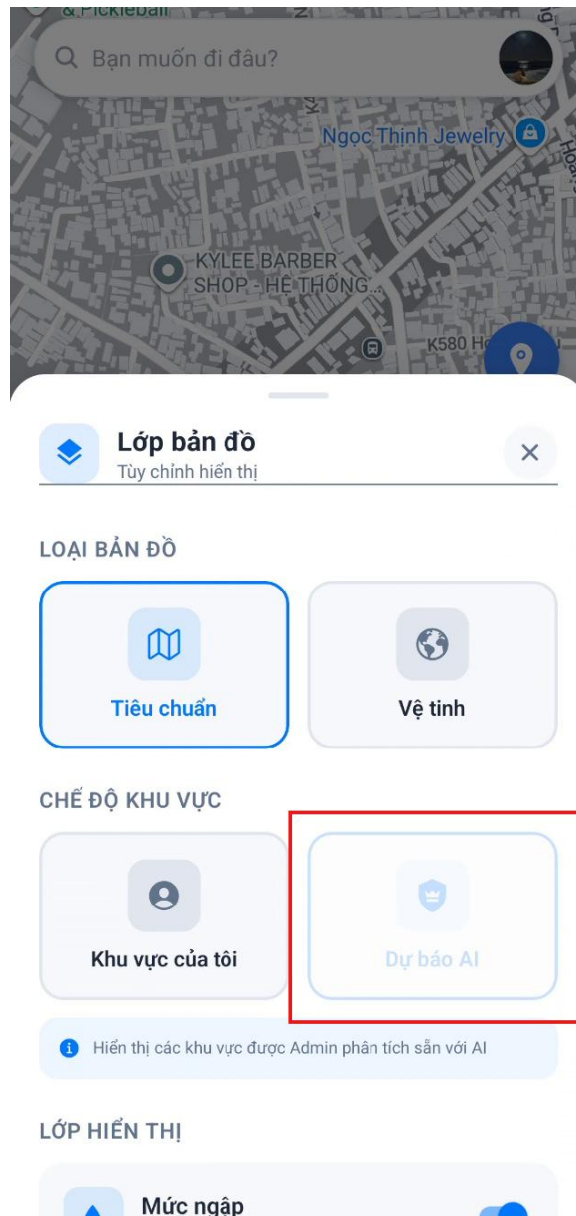


Figure 59 View AI-based Flood Prediction

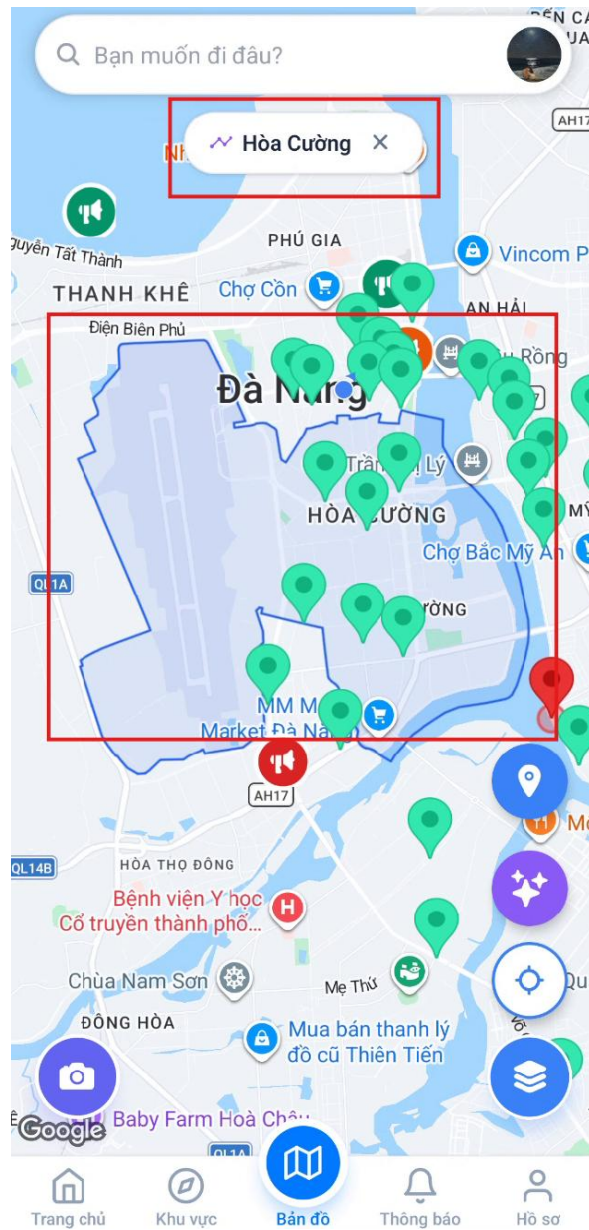


Figure 60 View AI-based Flood Prediction

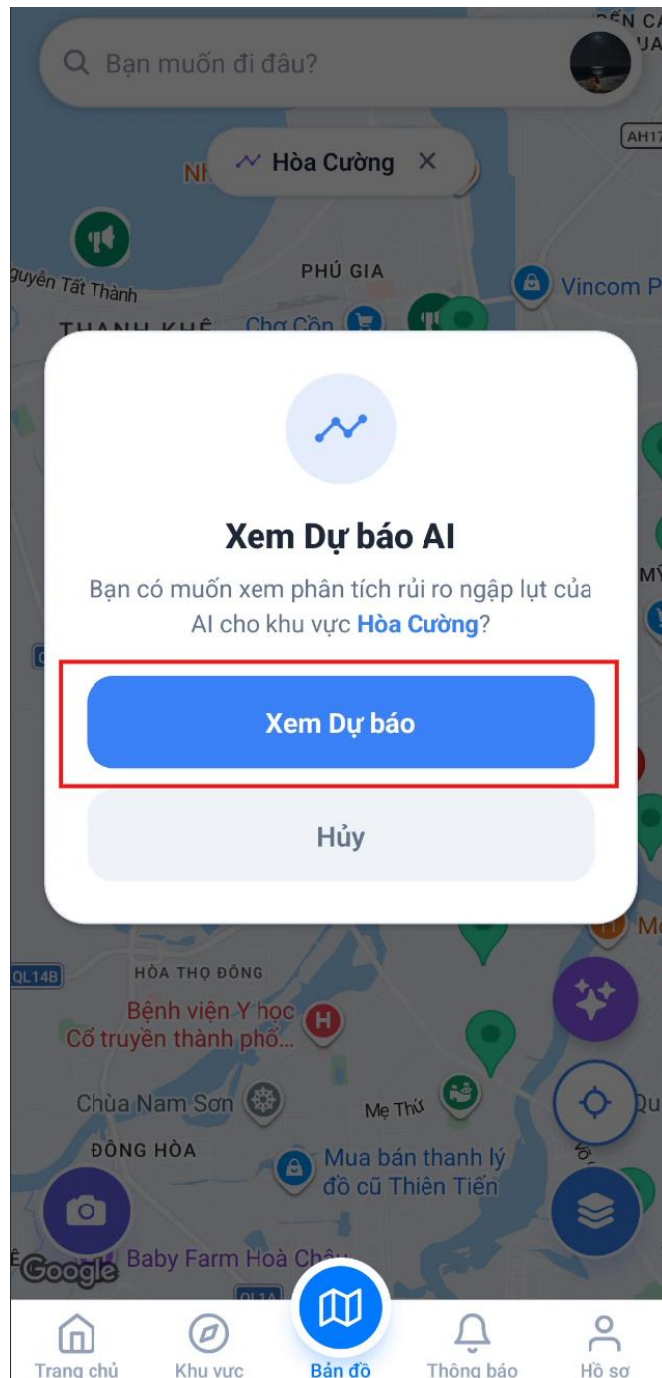


Figure 61 View AI-based Flood Prediction

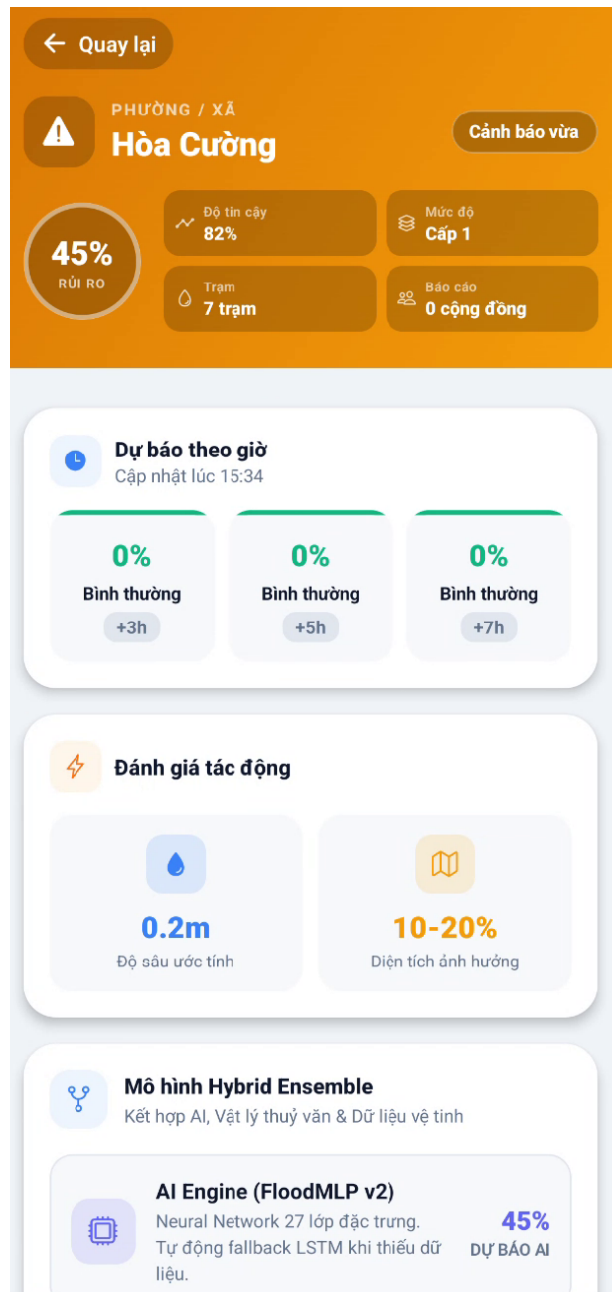


Figure 62 View AI-based Flood Prediction

Function details:

- **Data:** Predicted Water Level, Risk Level, Prediction Time.
- **Validation:** Requires a valid, active subscription for Citizen users.
- **Normal case:** the "+3h" forecast => The map updates, showing the WaterLevelPred and RiskLevel for 3 h ahead, based on the PredictionResults data.

3.16 Compare AI Predictions with Actual Data

Function trigger:

- **Navigation path:** "Logs & Tác Vụ" > "Tác Vụ"
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** To assess the performance of the AI model by comparing predicted water levels against real-time sensor measurements.
- **Interface:** Chart visualization with two time-series lines (Predicted vs. Actual) and error metrics display (e.g., MAE).

Data processing:

- The user selects a **stations.id** and a historical date range.
- The system queries:
 - **Actual Data:** SensorReadings.CalibratedVal for the selected station and time range.
 - **Predicted Data:** PredictionResults.WaterLevelPred that corresponds to the same station and time points.
- The application layer calculates the difference between the two data streams and displays the comparison.

Screen

layout:

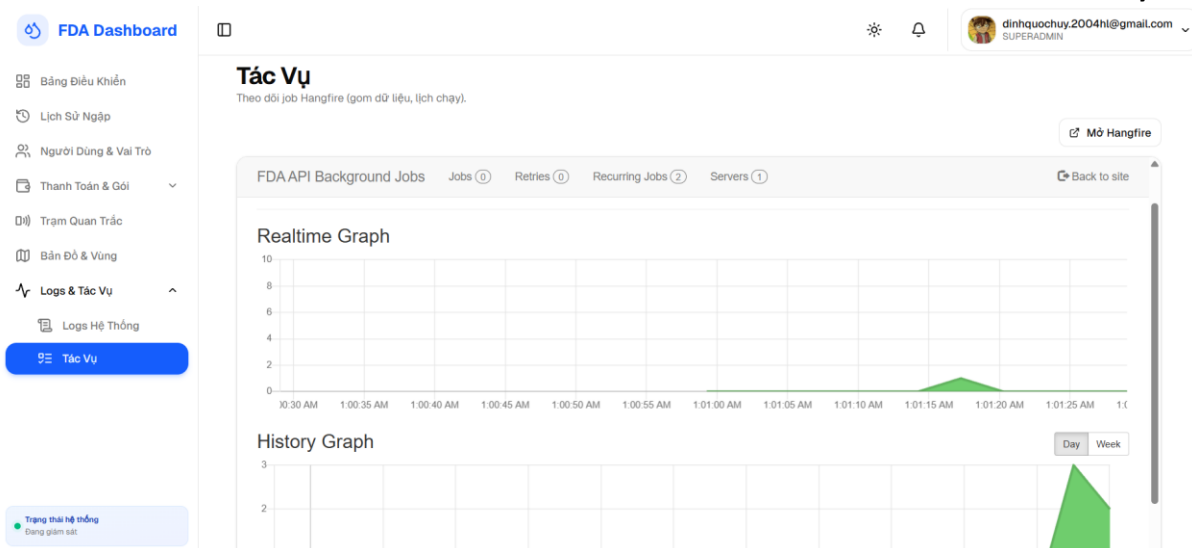


Figure 63 Compare AI Prediction with Actual Data

Function details:

- **Data:** Measured Water Level, Predicted Water Level, Time.
- **Validation:** Requires SensorReadings and PredictionResults to exist for the selected station and time range.
- **Normal case:** Moderator selects a station and the last flood event. The system generates a dual-axis chart comparing SensorReadings.CalibratedVal vs. PredictionResults.WaterLevelPred.

3.17 Identify High-risk Flood Zones (AI)

Function trigger:

- **Navigation path:** Analytics > Risk Identification Map
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** To display areas categorized as high-risk based on long-term analysis of flood frequency and intensity.
- **Interface:** Static map overlay highlighting zones with color-coded risk levels.

Data processing:

The system queries the FloodStatistics table, aggregating metrics like MaxDepthM and FloodHours over an extended period.

- Stations (stations.latitude, stations.longitude) exceeding defined long-term thresholds are flagged as High-Risk Zones.
- The system visualizes these areas on the map.

Screen layout:

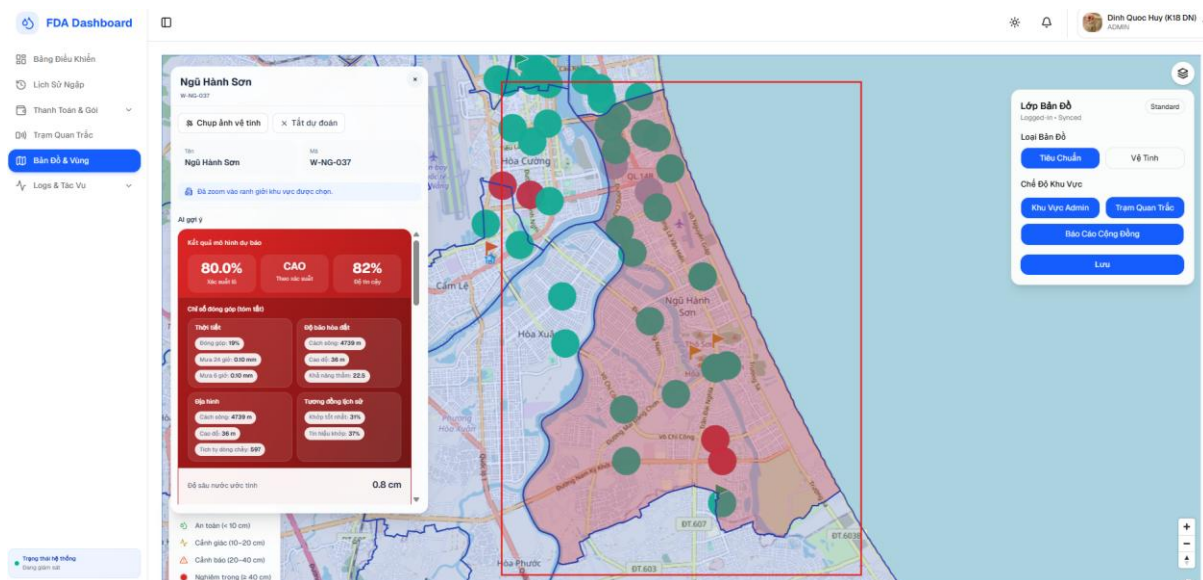


Figure 64 Identify High-risk Flood Zones (AI)

Function details:

- **Data:** Station Coordinates, Aggregated Max Depth, Aggregated Flood Hours.
- **Validation:** Requires sufficient historical data in the FloodStatistics table.
- **Normal case:** Moderator views the risk map. The system highlights the areas around the stations that have the highest FloodStatistics.FloodHours over the last year.

3.18 Request Safe Route Suggestions

Function trigger:

- **Navigation path:** Map Tab > Safe Route Button / Navigation Bar > Find Safe route
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** Generates safe travel routes that avoid flooded areas based on real-time flood conditions and historical flood data. Provides alternative route options when primary routes pass through flood-prone areas.
- **Interface:**

- **Map View:** Interactive map displaying safe routes, flood zones, and waypoints
- **Route Input Panel:** Start/destination point selection via search or map tap
- **Route Results Panel:** List of available routes with safety indicators
- **Direction Panel:** Turn-by-turn navigation instructions

Data processing:

- The user selects start and destination points on the map or via search.
- The user selects transport mode (car/motorbike/bike/walk).
- The user taps the “**Tìm đường an toàn**” button to initiate the request.
- The system sends a POST request to /api/v1/routing/safe-route with:
 - startLatitude, startLongitude (start point coordinates)
 - endLatitude, endLongitude (destination coordinates)
 - RouteProfile (car/bike/foot)
 - maxAlternatives (number of alternative routes)
 - avoidFloodedAreas(boolean flag)
- The server queries the FloodStatistics table for current water levels.
- The server queries the Stations table for station coordinates and flood zone boundaries.
- The server calculates routes using GraphHopper routing engine, avoiding areas with active flood warnings.
- The server generates alternative routes when the primary route passes through flood zones.
- The system parses GeoJSON response and renders routes on the map.
- The system displays flood zone warnings for routes near flooded areas.

Screen layout:

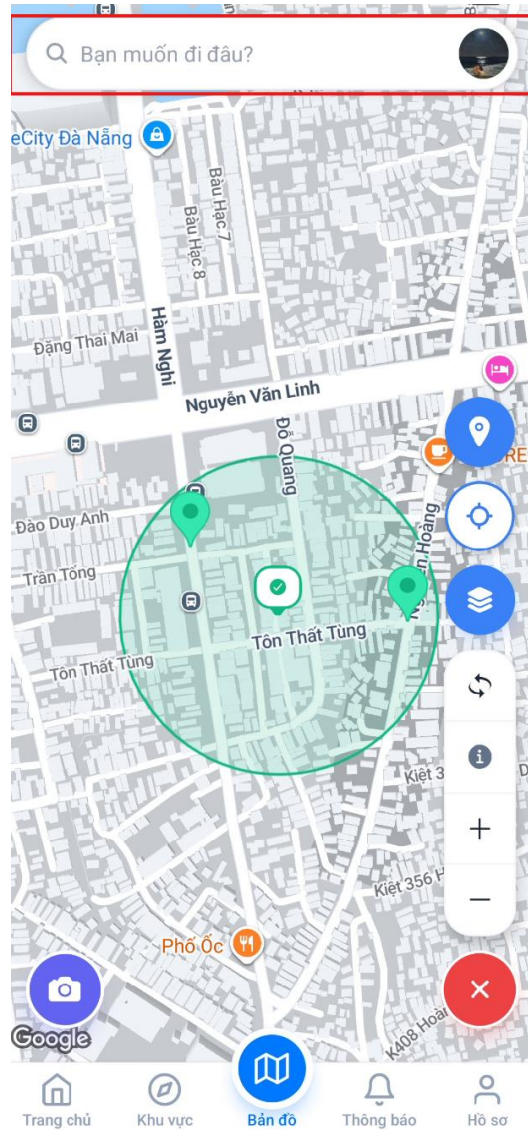


Figure 65 Request Safe Route Suggestions

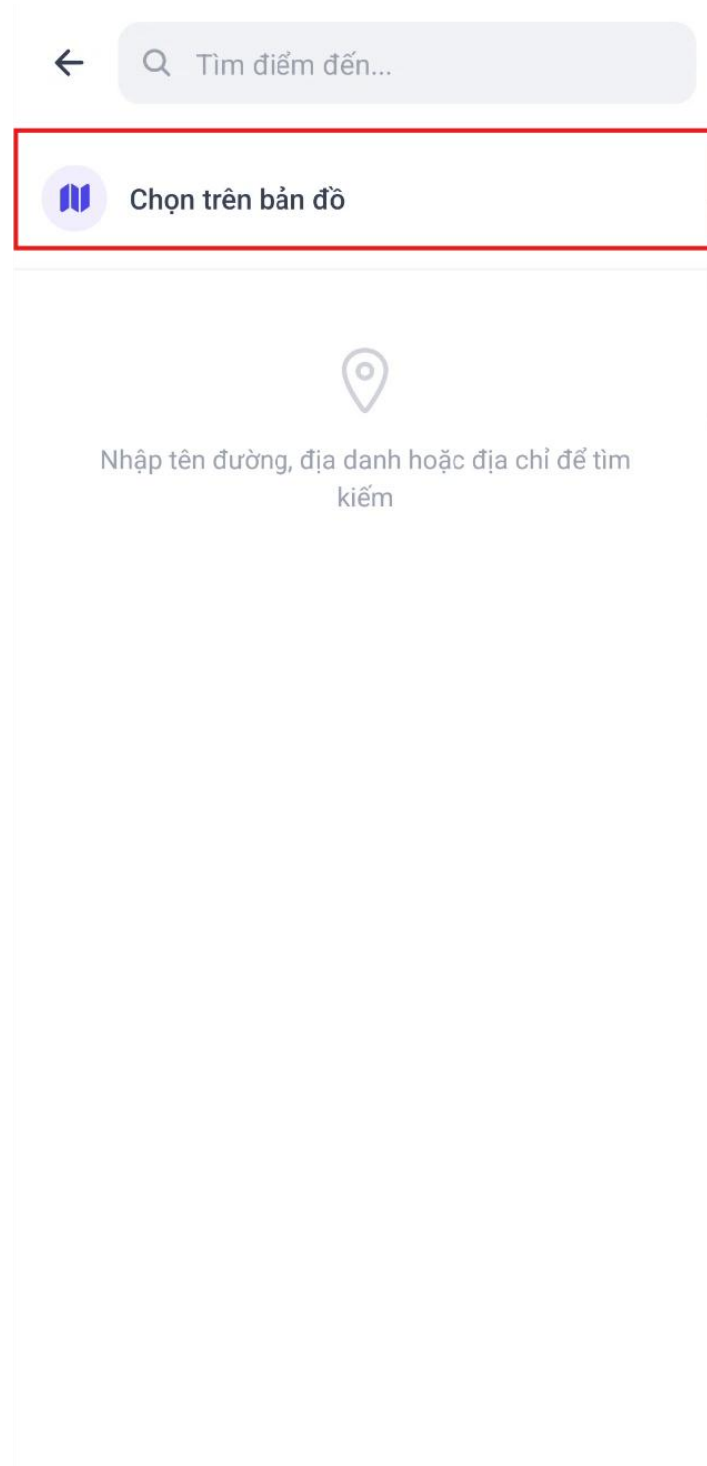


Figure 66 Request Safe Route Suggestions

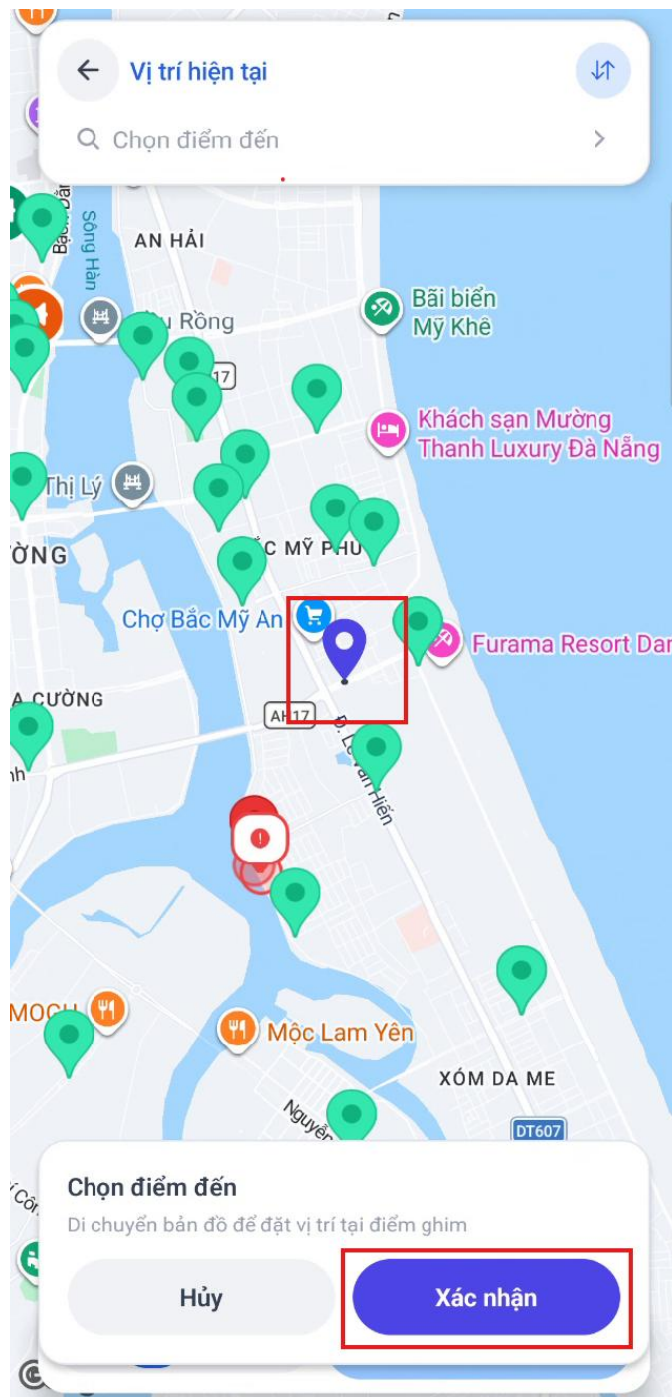


Figure 67 Request Safe Route Suggestions

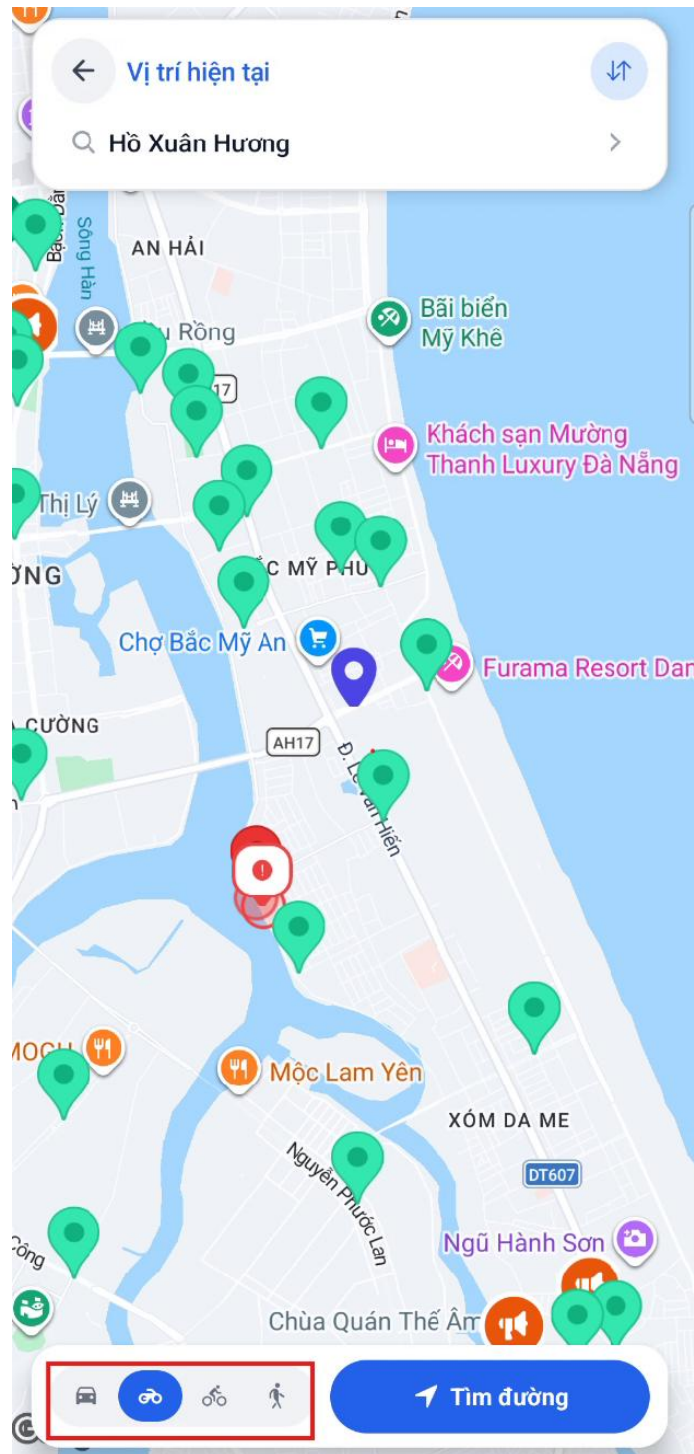


Figure 68 Request Safe Route Suggestions

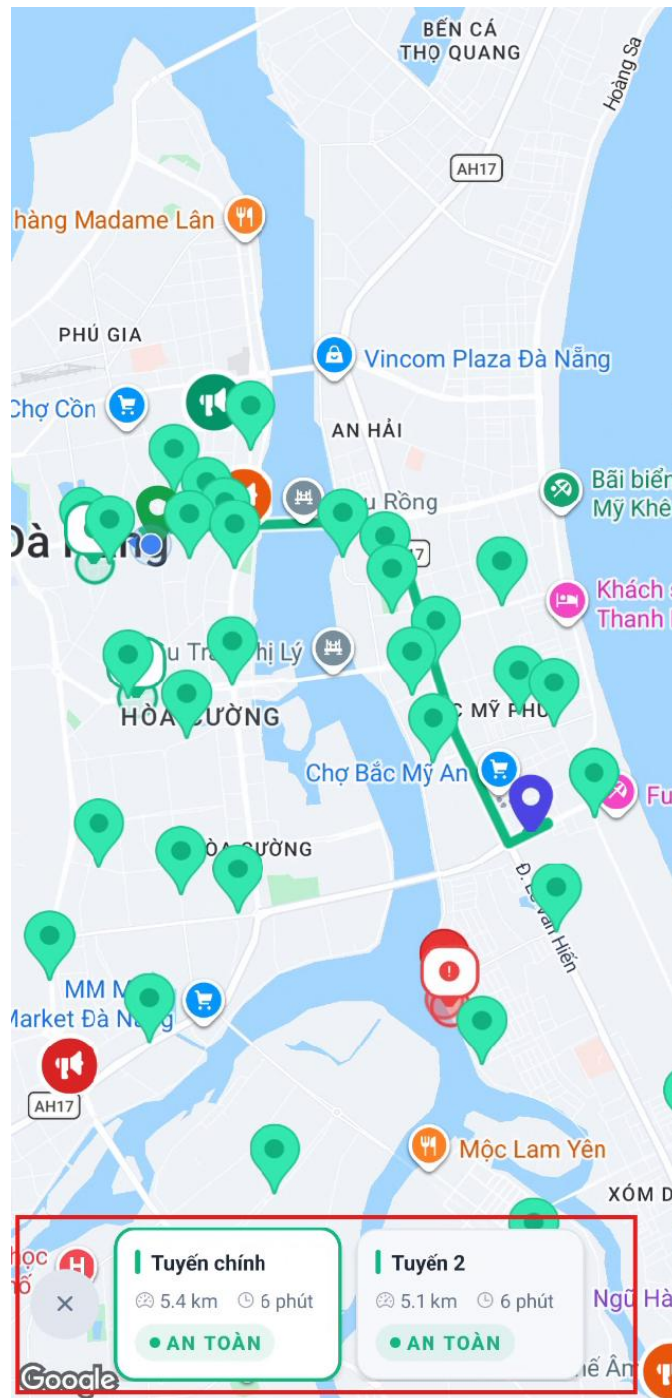


Figure 69 Request Safe Route Suggestions

Function details:

- **Data:** Start Coordinates, Destination Coordinates, Transport Mode, Route Distance, Route Duration, Flood Risk Score, Safety Status, Flood Zone Information.
- **Validation:** Start point must be selected, End point must be selected, Coordinates must be valid (latitude: -90 to 90, longitude: -180 to 180), Distance must not exceed 100km, Transport mode must be valid.
- **Business Rules:**
 - 1. Routes automatically avoid areas with "Critical" severity flood zones
 - 2. "Caution" level flood zones are highlighted but route may pass through

- 3. Alternative routes are generated when primary route has flood warnings
- 4. If all routes are blocked, display error message with suggestion
- 5. Routes are cached for 5 minutes to reduce API calls
- 6. Historical flood data (last 30 days) affects risk score calculation
- **Normal Case:** User selects start and destination points on the map, chooses transport mode as "car", and taps "Tìm đường an toàn". The system displays a green primary route with "An toàn" status and shows alternative route options. No flood zone warnings are displayed.
- **Abnormal Case:** User attempts to find a route without selecting a start point. System displays error: "Vui lòng chọn điểm xuất phát." User attempts to find a route where all paths are blocked by flood zones. System displays error: "Tất cả các tuyến đường đều bị ngập. Vui lòng thử lại sau."

3.19 Basic safe route

Function trigger:

- **Navigation path:** Map Tab > Safe Route Button > Route Direction Panel > Input Origin and Destination
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, User
- **Purpose:** Allows citizens to input start and end locations for route planning. Users can input locations via text search, GPS current location, or by picking a point on the map.
- **Interface:**
 - Origin Input Field: Text input with placeholder "Bạn muốn đi đâu?"
 - Destination Input Field: Text input with placeholder "Tìm điểm đến"
 - GPS Button: Quick button to use current location
 - Map Pin Button: Button to pick location on map
 - Swap Button: Button to swap origin and destination
 - Transport Mode Selector: Options for car/motorbike/walk

Data processing:

- The user taps on the origin input field.
- The user can type a location name for autocomplete search results.
- The user can tap the GPS icon to use the current location as origin.
- The user can tap the map pin icon to pick a location on the map.
- The user repeats the process for the destination field.
- The user can tap the swap button to exchange origin and destination.
- The system stores the coordinates (latitude, longitude) for both points.
- The system enables the "Tìm đường" button when both origin and destination are set.

Screen layout:

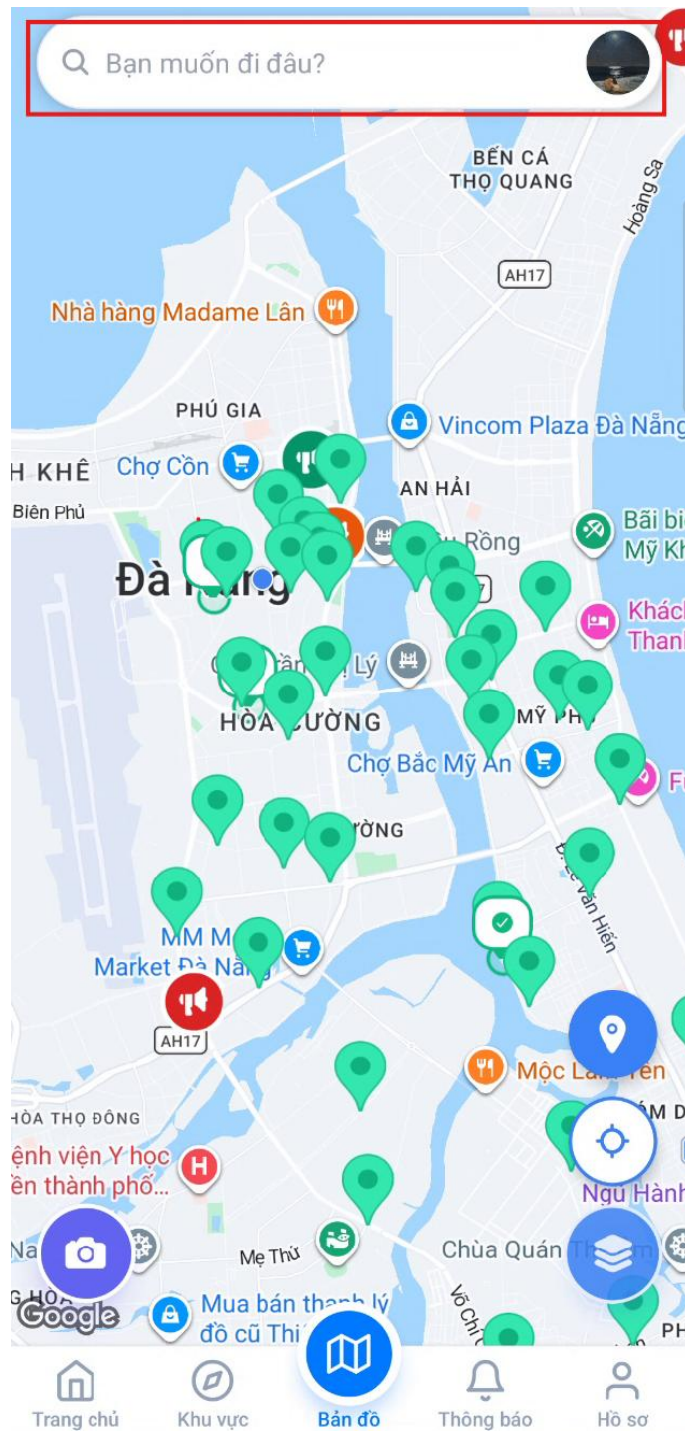


Figure 70 Input Origin and Destination

An Hải

Chọn trên bản đồ

KẾT QUẢ TÌM KIẾM

Phường An Hải

Phường An Hải, Quận An Dương, Thành Phố Hải Phòng

Xã An Hải

Xã An Hải, Huyện Ninh Phước, Tỉnh Ninh Thuận

Phường An Hải Nam

Phường An Hải Nam, Quận Sơn Trà, Thành Phố Đà Nẵng

Phường An Hải Bắc

Phường An Hải Bắc, Quận Sơn Trà, Thành Phố Đà Nẵng

Phường An Hải

Phường An Hải, Quận An Dương, Thành Phố Hải Phòng

Figure 71 Input Origin and Destination

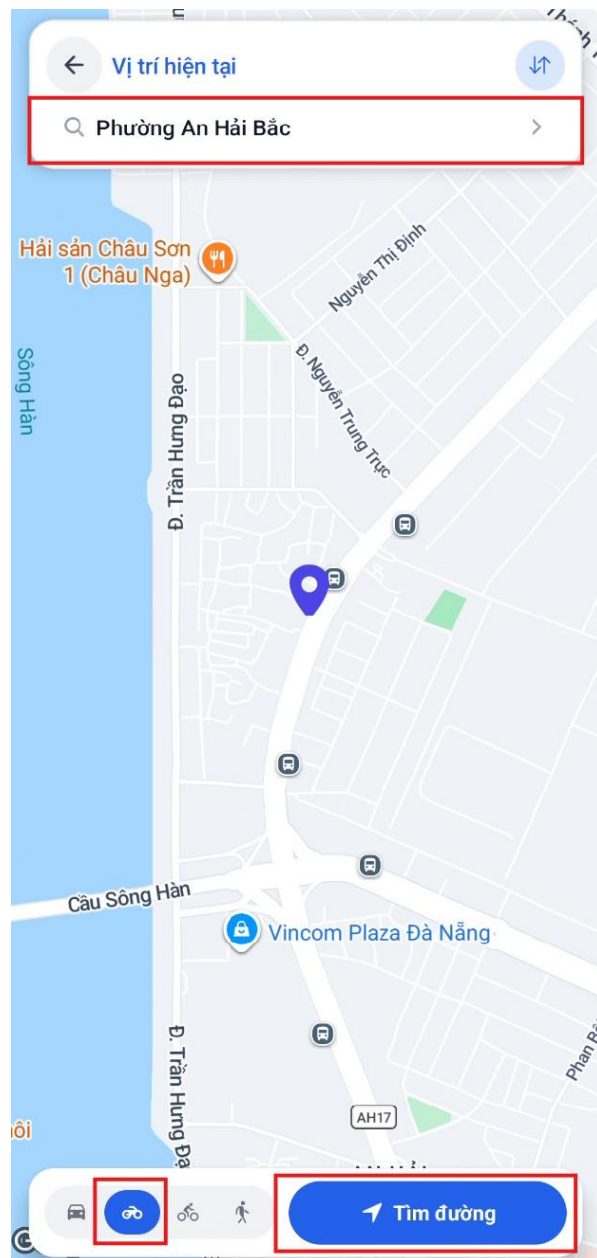


Figure 72 Input Origin and Destination

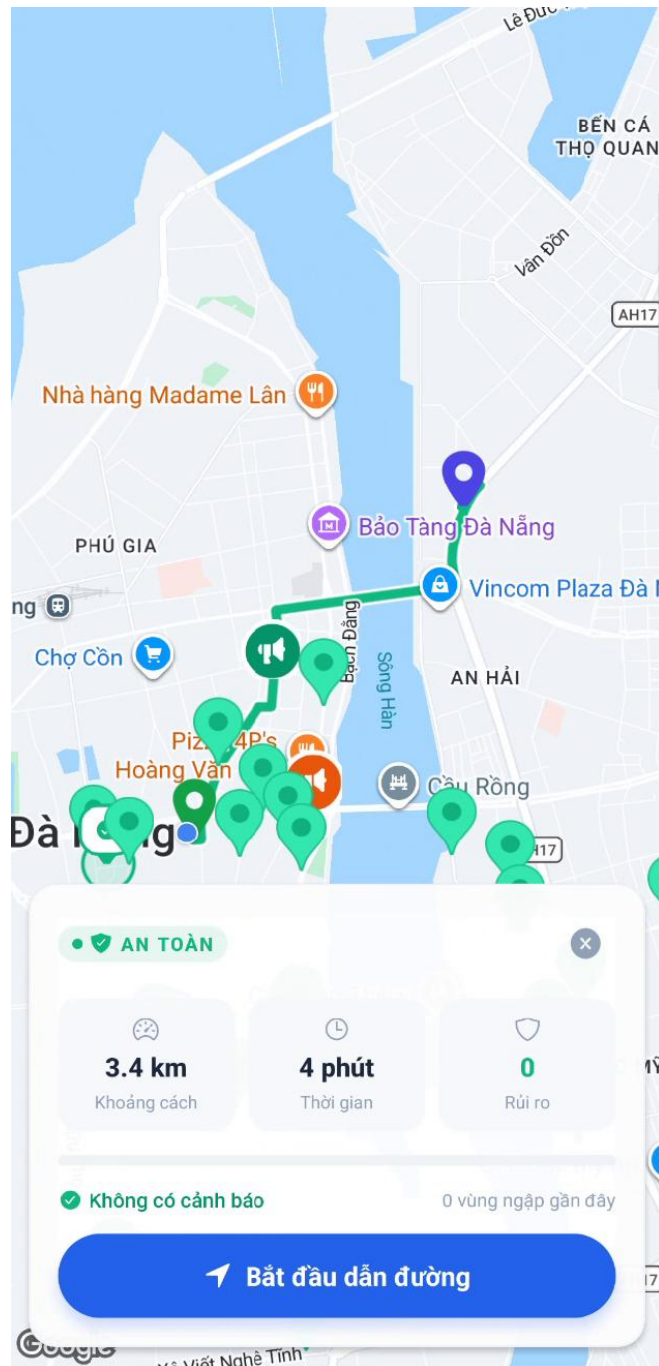


Figure 73 Input Origin and Destination

Function details:

- **Data:** Origin Text, Origin Coordinates, Destination Text, Destination Coordinates, GPS Status, Transport Mode.
- **Validation:** Origin must be set (either text, GPS, or map pin), Destination must be set (either text, GPS, or map pin).
- **Business Rules:**
 - Default origin is set to "Current Location" with GPS enabled by default
 - User can manually type to override GPS origin
 - Both origin and destination support GPS, text input, and map pin selection
 - Swap button exchanges all properties between origin and destination

- Transport mode defaults to "motorbike" but can be changed to car or walk
- Search autocomplete suggestions should appear as user types (future enhancement)
- When using GPS, the system requires location permission to be granted
- Input fields show visual feedback with color indicators (green for origin, purple for destination)
- **Normal Case:** User opens the Route Direction Panel. The origin field is automatically set to "Vị trí hiện tại" with GPS coordinates. The user taps the destination field and types "An Hải". The user selects a result from the dropdown. The user taps the "Tìm đường" button. The system proceeds to calculate safe routes.
- **Abnormal Case:** User attempts to find a route without setting a destination. System displays error: "Hãy nhập lại địa chỉ" User has location permission disabled. The system prompts users to enable location services to use GPS features. The user enters an invalid or unrecognized location. System displays error: "Không tìm được đường. Hãy thử lại"

3.20 Compare Alternative Routes

Function trigger:

- **Navigation path:** Map Tab > Safe Route Button > Route Direction Panel > Find Safe Route > Route Results > Compare Alternative Routes.
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, User
- **Purpose:** Displays multiple route options with safety comparisons. Allows users to compare different route alternatives based on distance, duration, and flood risk to choose the most suitable route.
- **Interface:**
 - **Route Cards:** Horizontal scrollable list showing all available route options
 - **Route Card Content:** Route name, distance, duration, safety status badge
 - **Selected Route Indicator:** Visual highlight for currently selected route
 - **Safety Status Badge:** Color-coded badge (green=An toàn, yellow=Chú ý, red=Nguy hiểm)
 - **Map Update:** Map automatically pans to show the selected route

Data processing:

- The system receives route alternatives from the safe route API response.
- The system parses each route's properties including distance, duration, and flood risk score.
- The system displays routes as horizontal scrollable cards at the bottom of the screen.
- The user taps on a route card to select it.
- The system highlights the selected route on the map with a different color.
- The system updates the route polylines to show only the selected route.
- The system updates the warning panel to show flood warnings for the selected route.

Screen layout:

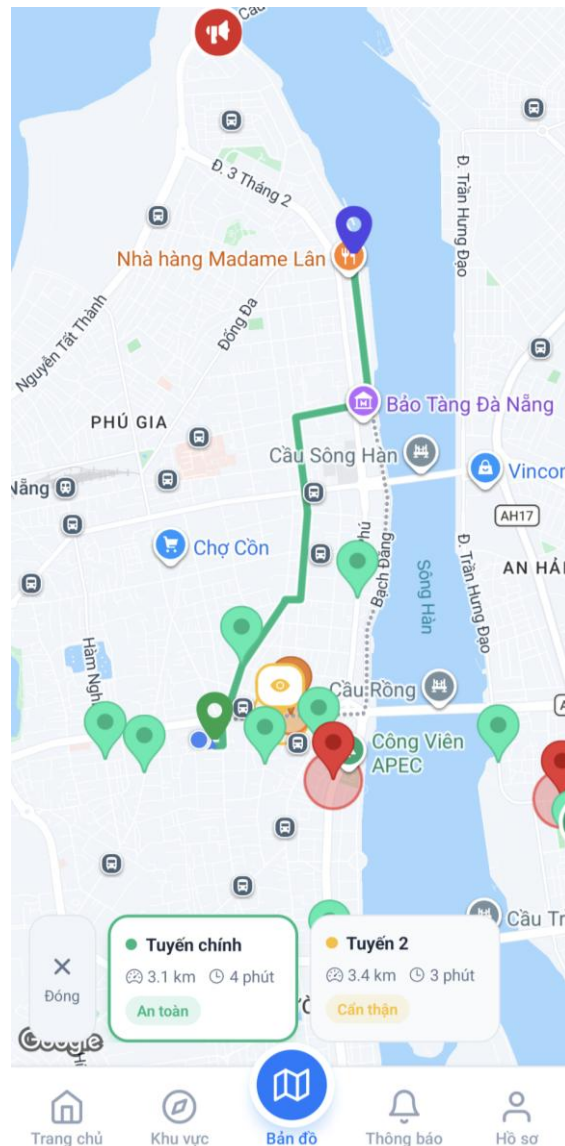


Figure 74 Compare Alternative Routes

Function details:

- **Data:** Route Name, Route Distance (meters), Route Duration (seconds), Safety Status, Flood Risk Score, Route Coordinates.
- **Validation:** At least one route must be available to display.
- **Business Rules:**
 - Primary route (safe route) is displayed first in the list
 - Alternative routes are numbered sequentially (Route 2, Route 3, etc.)
 - Routes are sorted by safety status priority: Safe > Caution > Dangerous > Blocked
 - The currently selected route has a visual highlight (white background with colored border)
 - When a route is selected, the map automatically centers to show the entire route
 - If only one route is available, the alternative comparison panel is hidden
 - Each route card shows the safety status color as a small dot indicator
- **Normal Case:** User searches for a safe route. The system displays the Route Direction Panel with results showing 3 route alternatives. The primary route is automatically selected. The

user scrolls horizontally and taps on "Tuyến 2" card. The map updates to show Route 2 polyline. The warning panel shows flood warnings for Route 2.

- **Abnormal Case:** No alternative routes available. The system displays only the primary route without the comparison panel. All available routes are blocked by flood zones. The system displays all routes with "Chặn" status and shows an error message "Tất cả các tuyến đường đều bị chặn. Hãy thử lại sau"

3.21 Report Flood Points (Photo / Video)

Function trigger:

- **Navigation path:** Citizen App > Report Flood > Add New Report
- **Timing Frequency:** On demand (User-triggered)

Function description:

- **Actors/Roles:** Citizen, Admin, Moderator
- **Purpose:** Enable citizens to actively participate in flood monitoring by reporting real-time flood conditions with visual evidence, supporting faster emergency response and better flood situation awareness.
- **Interface:**
 - **Report List Screen:**
 - Report ID / Reference Number
 - Location (address or map pin)
 - Submission Date/Time
 - Status (Pending, Verified, Rejected)
 - Thumbnail preview of attached media
 - **Editor Form:** Input fields for creating/editing a flood report:
 - Location Selection
 - Current location (GPS auto-detect)
 - Manual address search/input
 - Pin on map
 - Media Upload
 - Camera capture (Photo)
 - Video recording
 - Gallery selection
 - Support multiple files (max 5)
 - Description Text input for additional details
 - Severity Assessment Optional dropdown (Low, Medium, High)
 - **Actions:**
 - Add New Report
 - View Report Details
 - Edit Own Report (only if status is Pending)
 - Delete Own Report (only if status is Pending)
 - Verify Report (Moderator/Admin)
 - Reject Report with Reason (Moderator/Admin)

Data processing:

- The user navigates to the "Cộng đồng Đà Nẵng" screen.
- The system requests location permission (if not granted).
- User selects location (auto-detect or manual).
- The user captures/selects photos/videos.
- User adds description (optional).
- The user submits a report.
- Create/Update: When the user saves a form:
 - System validates that location is required
 - System validates at least 1 media file is attached
 - System validates media file size < 50MB per file
 - System validates video duration < 60 seconds
 - System uploads media to storage
 - System inserts or updates the record in the FloodReports table
 - Report status defaults to "Pending"
- Delete: The system performs a soft delete (sets status to 'Deleted').

Screen layout:

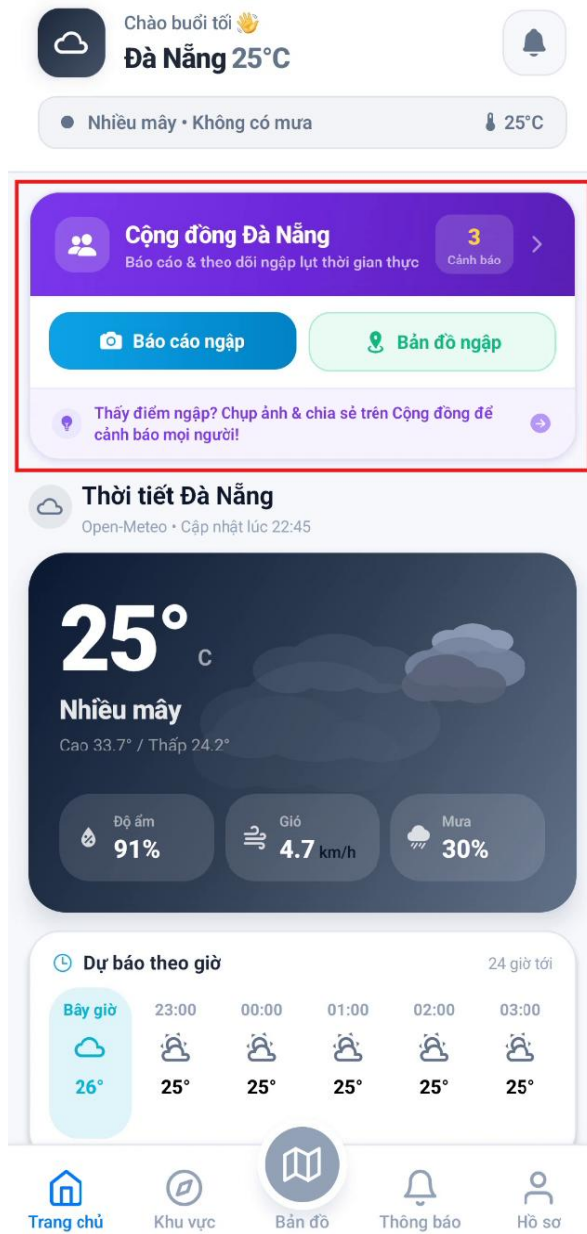


Figure 75 Report Flood Points

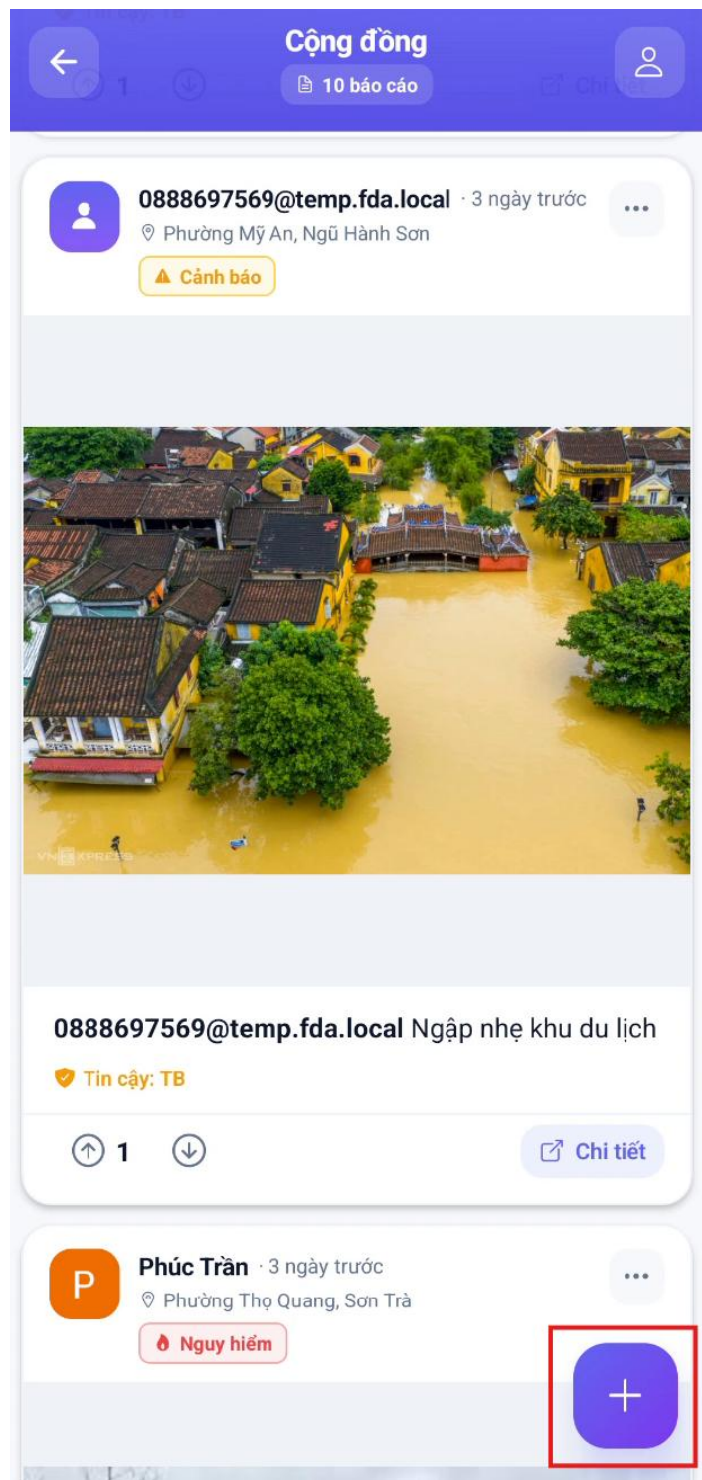


Figure 76 Report Flood Points

X

Tạo báo cáo

Đăng

10, K254 Hoàng Diệu, Hải Châu, Đà Nẵng

Hải Châu

Bạn

Báo cáo ngập lụt

Mức độ ngập lụt

Thấp

Trung bình

Cao

Mô tả tình hình ngập lụt tại khu vực...

Thêm vào báo cáo

Chụp ảnh / Video

Đã có vị trí

Figure 77 Report Flood Points

×

Tạo báo cáo

Đăng

📍

12, Hoàng Diệu, Hải Châu, Đà Nẵng

Hải Châu

👤

Bạn

Báo cáo ngập lụt

Mức độ ngập lụt

Thấp

Trung bình

Cao

✓

Hải Châu — hệ thống: An toàn

Ngập lụt

×

Bia

Thêm vào báo cáo

📷

Chụp ảnh / Video

📍

Đã có vị trí

Figure 78 Report Flood Points

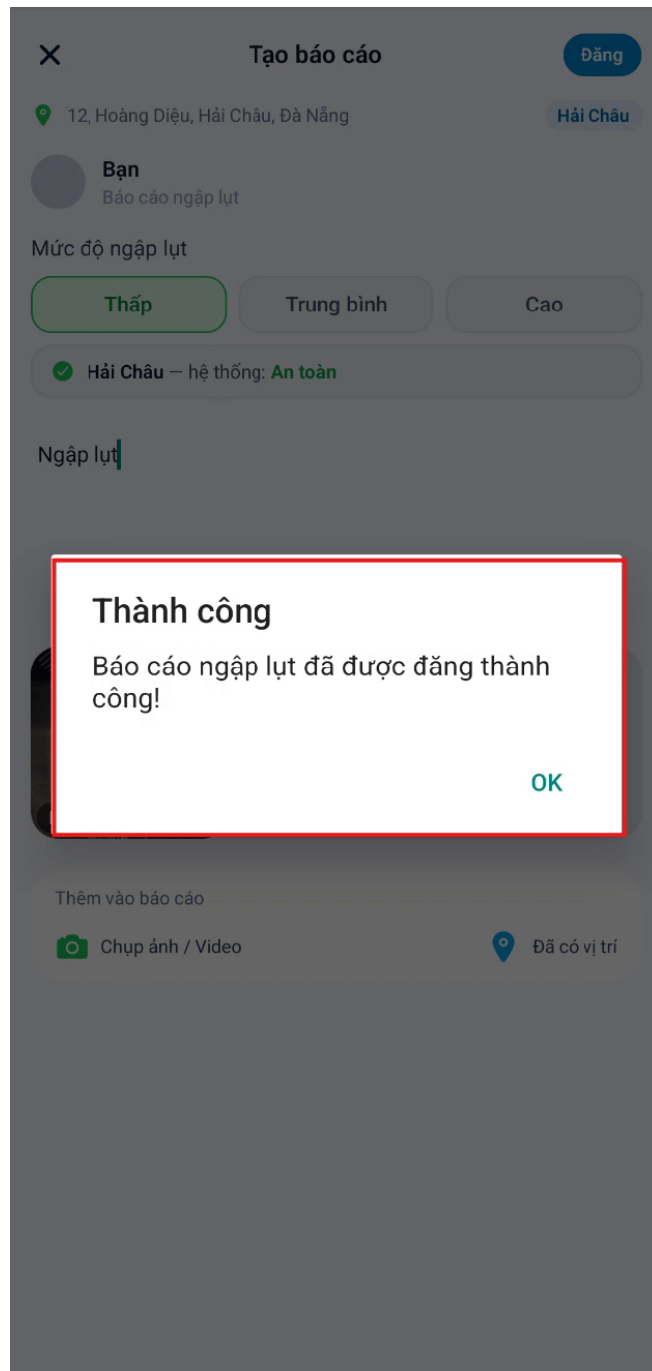


Figure 79 Report Flood Points

Function details:

- **Data:**
 - FloodReports: Template ID, User ID, Latitude, Longitude, Address, Description, Severity, Status, Rejection Reason, Created At, Updated At, Verified By, Verified At
 - FloodReportMedia: ID, Report ID, Media Type (photo/video), Media URL, Thumbnail URL, Created At
- **Validation:**
 - Location: Required, must be within service area bounds

- Media: Minimum 1 file required, Maximum 5 files per report
- Photo formats: JPEG, PNG, WEBP
- Video formats: MP4, MOV
- Max file size: 50MB per file
- Max video duration: 60 seconds
- Description: Max 500 characters
- **Business Rules:**
 - Citizen Restrictions: Can only view/edit/delete own reports, cannot modify verified/rejected reports, maximum 10 reports per day
 - Status Rules: Pending (awaiting verification), Verified (confirmed by Moderator/Admin), Rejected (rejected with reason)
 - Critical Rule: Reports with "Critical" severity cannot be deleted by citizens, only edited
 - Retention Policy: Reports retained for 2 years, media files archived after 1 year
- **Normal Case:** Citizen captures photo of flooded street. System auto-detects GPS location. Citizen adds a brief description. The system validates and saves reports. Display success message: "Báo cáo ngập lụt đã được đăng thành công!". The report appears in user's list as "Pending".
- **Abnormal Case:**
 - User tries to save without location: System displays "Cần phải nhập vị trí. Vui lòng chọn một vị trí trên bản đồ."
 - User tries to save without media: System displays "Vui lòng đính kèm ít nhất một ảnh hoặc video vào báo cáo của bạn."
 - User tries to upload file > 50MB: System displays "Kích thước tệp vượt quá giới hạn 50MB. Vui lòng nén hoặc chọn tệp có kích thước nhỏ hơn."
 - User records video > 60 seconds: System displays "Video có thời lượng vượt quá 60 giây. Vui lòng cắt bớt hoặc quay video ngắn hơn."
 - User denies location permission: System displays "Để gửi báo cáo, bạn cần có quyền truy cập vị trí. Vui lòng bật dịch vụ định vị trong Cài đặt."
 - Network error during upload: System displays "Tải lên không thành công. Vui lòng kiểm tra kết nối internet và thử lại." and retains form data for retry.

3.22 View Community Flood Reports

Function trigger:

- **Navigation path:** Citizen App > Community Reports / Map View
- **Timing Frequency:** On demand (User-triggered) or Auto-refresh when viewing map

Function description:

- **Actors/Roles:** Moderator, Citizen
- **Purpose:** Enable citizens to view flood reports from other community members to stay informed about flood situations in their area. Supports situational awareness and helps authorities communicate flood risks effectively.
- **Interface:**
 - **List View:**
 - Table of community flood reports showing:
 - Report ID / Reference Number
 - Location (address or distance from user)
 - Submission Date/Time

- Severity Level (Thấp, Trung bình, Cao)
- Description excerpt
- Media thumbnail (if available)
- Vote count (upvotes/downvotes)
- **Filter Controls:**
 - Time range selector (Last 1h, 6h, 24h, 48h, 7 days)
 - Severity filter (All, Low, Medium, High)
 - Map bounds (auto-filter by visible area)
- **Actions:**
 - View Report Details
 - View All Media Attachments
 - Get Directions to Report Location
 - Filter by Time Range
 - Filter by Severity Level
 - Refresh Report List

Data processing:

- The user navigates to the Community Reports screen or opens map view.
- The system queries the FloodReports table with the following filters:
 - Status = 'Published'
 - Score >= visibilityThreshold (default: 0)
 - CreatedAt >= fromTime (default: last 24 hours)
 - Latitude/Longitude within bounds (if provided)
- The system returns paginated results sorted by newest first.
- For Map View: The system clusters nearby reports to improve performance.
- For List View: The system displays reports in table format with pagination.
- Create/Update: N/A (read-only feature)
- Delete: N/A (read-only feature)

Screen layout:

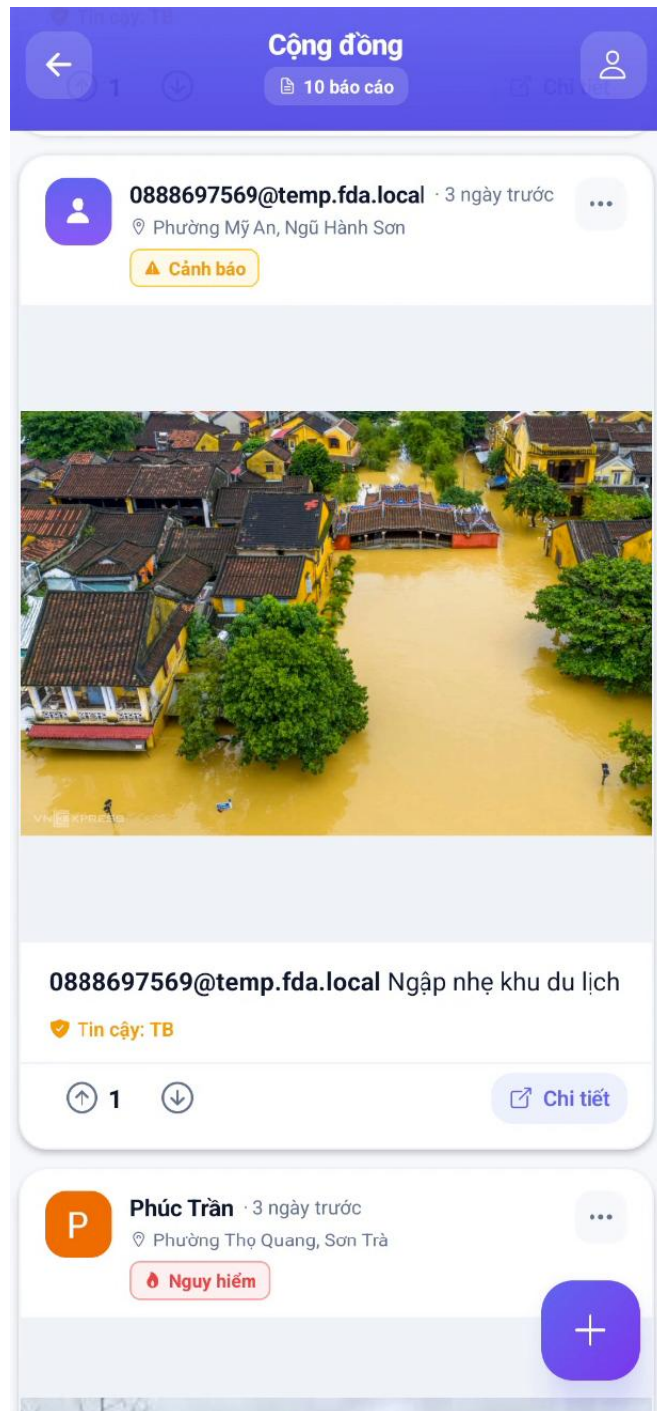


Figure 80 View Community Flood Reports

Function details:

- **Data:**
 - FloodReports table: ID, User ID, Latitude, Longitude, Address, Description, Severity, Status, Score, Created At
 - FloodReportMedia table: ID, Report ID, Media Type, Media URL, Thumbnail URL
- **Validation:**
 - User must be authenticated

- Bounds must be valid coordinates
- fromHours must be positive integer
- page must be ≥ 1
- pageSize must be ≤ 100
- **Business Rules:**
 - Hidden reports (marked by FE-25/FE-27) must never be returned
 - Reports with status other than 'Published' are excluded
 - Reports with score below threshold are excluded
 - Maximum 100 reports returned per page
- **Normal Case:** Citizen opens the app and navigates to Community Reports. The system displays a map with recent flood reports from the community. The user taps on a report marker to see details. Users can switch to a list view to see more reports in table format. User filters by "Last 24 hours" to see only recent reports.
- **Abnormal Case:**
 - No reports in area: System displays "No flood reports in this area. Be the first to report!"
 - All reports hidden: System displays "No visible reports in this area."
 - Invalid bounds parameters: System ignores invalid bounds and returns default area reports.
 - Network error: System displays "Unable to load reports. Please check your connection." with a retry button.
 - Authentication expired: System redirects to login screen.
 - Too many reports (clustering): System clusters reports and shows count badge. The user taps the cluster to zoom in.

3.23 Vote Flood Reports

Function trigger:

- **Navigation path:** Citizen App > Community Feed > Flood Report Detail > Vote / Flag
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Citizen
- **Purpose:** Harness community wisdom to surface credible flood reports and filter out inaccurate or malicious submissions, reducing the verification burden on Moderators and Admins while improving overall report quality.
- **Interface:**
 - **List View:** Display of community flood reports with:
 - Report ID / Reference Number
 - Location (address or map pin)
 - Submission Date/Time
 - Status (Pending, Verified, Rejected)
 - Thumbnail preview of attached media
 - Vote Score: Net upvotes minus downvotes
 - Flag Count: Number of flags (visible only to Moderator/Admin)
 - User's own vote state (upvoted / downvoted / neutral)
 - **Report Detail View:** Full report details with vote/flag actions:
 - Full media gallery (photos/videos)
 - Report metadata (location, time, description, severity)

- Current Vote Score Displayed prominently
- Flag Button: Opens flag dialog
- Flag Reasons: Predefined list + custom text
- **Actions:**
 - Upvote a Report
 - Downvote a Report
 - Remove own Vote
 - Flag a Report (with reason)
 - View Flagged Reports (Moderator/Admin)
 - Dismiss Flags (Moderator/Admin)
 - Clear Votes (Admin)

Data processing:

- **Voting Flow**
 - User views a report in the Community Feed
 - User taps Upvote or Downvote button
 - System validates user has not exceeded daily vote limit
 - System checks if user already voted on this report
 - If same direction: remove vote (toggle off)
 - If different direction: update vote direction
 - If no existing vote: insert new vote
 - System recalculates vote score for the report
 - System updates UI optimistically

Screen layout:

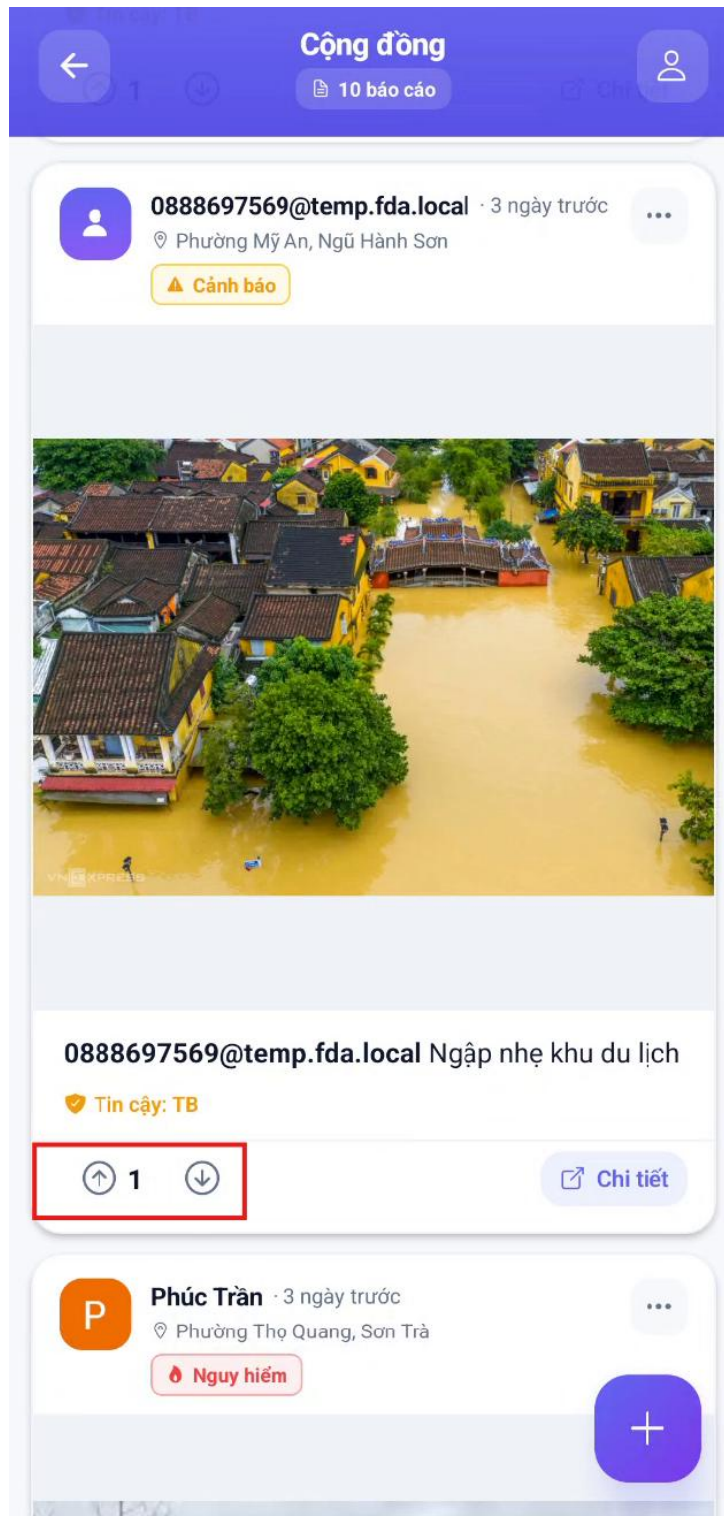


Figure 81 Vote Flood Reports

Function details:

- **Data:**
 - **ReportVotes table:** ID, Report ID, User ID, Vote Type (upvote/downvote), Created At, Updated At
 - **ReportFlags table:** ID, Report ID, User ID, Flag Reason, Additional Details, Status (active/dismissed), Created At, Dismissed By, Dismissed At

- **Validation:**
 - **Vote:** User cannot vote on own report
 - **Vote Limit:** Maximum 50 votes per day per user
- **Business Rules:**
 - **Vote Score:** Calculated as (upvote count) - (downvote count), displayed as integer
 - **Vote Display:** Reports sorted by vote score in Community Feed (highest first), with Pending status prioritized over Verified/Rejected
 - **Auto-Prioritization:** Reports with vote score ≥ 5 are auto-flagged for Moderator review
 - **Visibility:** Flag count and flag details are only visible to Moderator/Admin roles
 - **No Self-Voting:** Users cannot vote on their own reports (button disabled)
- **Normal Case:** Citizen browses Community Feed. Taps on a flood report showing flooding on Nguyen Hue Street. Reviews photos. Taps Upvote. System records vote, updates score from 3 to 4. Citizens then spot a report about the same flood location but from a different angle. Flag it as "Duplicate report". System records flag with reason. Confirmation shown. Moderator later reviews flagged report, confirms duplicate, and merges/rejects.
- **Abnormal Case:**
 - Users try to vote on their own report: System disables vote buttons and displays tooltip: "You cannot vote on your own report."
 - User reaches daily vote limit (50): System displays "You've reached your daily vote limit of 50. Please try again tomorrow." and disables vote buttons.
 - User tries to flag a report they already flagged: System displays "You have already flagged this report." and disables the flag button.
 - User tries to flag without selecting reason: System displays "Please select a flag reason before submitting."
 - User exceeds flag rate limit (10 flags/24h): System displays "You've been temporarily blocked from flagging due to excessive flags. Please try again in 24 hours."
 - Network error during vote: System reverts optimistic UI update and displays "Vote failed. Please check your internet connection."
 - Report reaches auto-hide threshold (-6 score): System hides report from public feed with message: "This report has been hidden due to community feedback."
 - Report reaches flag threshold (3 flags): System hides report from public feed, displays to Moderator/Admin queue for review.

3.24 Hide Invalid Flood Report

Function trigger:

- Moderator opens the community flood reports moderation view.
- Moderator selects ****Hide**** on a report identified as invalid/spam/inappropriate/duplicate/false information.

Function description:

The system allows the Moderator to hide an invalid community flood report from public-facing feeds and maps while keeping the record for audit purposes. Hiding is a soft-moderation action that changes report visibility state rather than deleting the record.

- **Actors/Roles:** Moderator
- **Purpose:**
 - Protect citizens from misinformation and inappropriate content.

- Maintain operational integrity of community-sourced flood signals.
- Preserve auditability for moderation actions.

Interface:

• Route/UI:

- Moderator page: `GET /moderator/community-report`
- Report list with filters:
 - Status: published / hidden
 - Severity: low / medium / high
 - Min trust score
 - Time range (from/to)
- Report card actions:
 - Hide button (disabled when already hidden)
- Confirmation modal: “Hide report”

Data processing:

- Moderator loads reports list with pagination and filtering:
 - Endpoint (community list): `GET /flood-reports/community?...`
 - Filters include status, severity, minTrustScore, from, to, pageNumber, pageSize.
- Moderator chooses a report and confirms hide.
- System sends hide request:
 - Endpoint: `PATCH /admin/flood-reports/:reportId/hide`
- On success, the report is updated to hidden state and is no longer shown to citizens.

Screen layout:

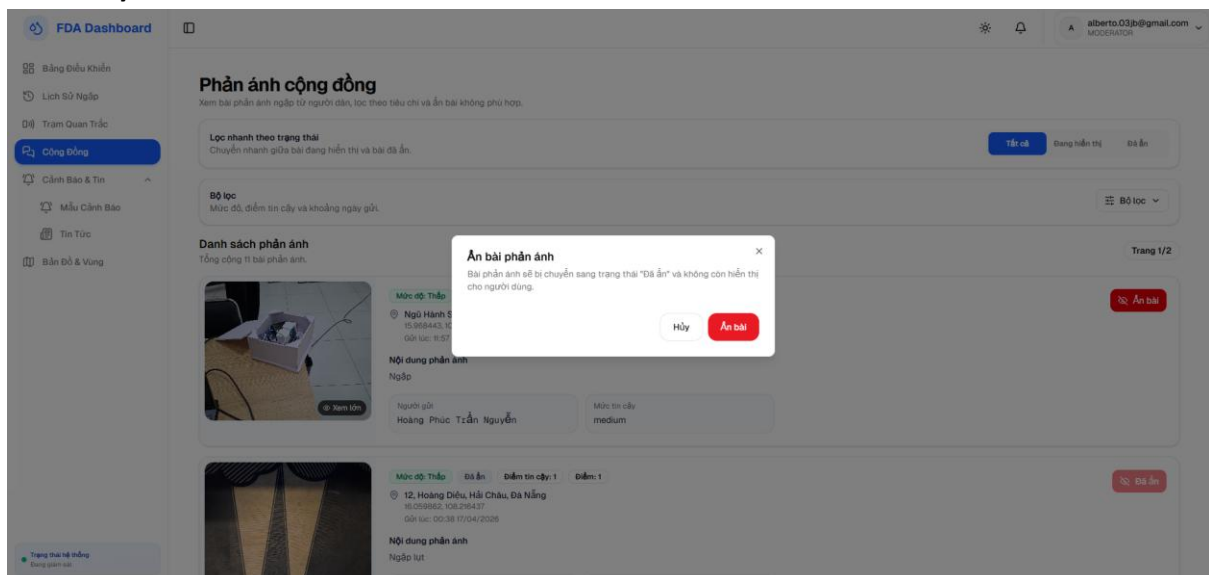


Figure 82 Hide Invalid Flood Report

Function details:

- **Data:**
 - Report: id, reporterUserId, coordinates, address, description, severity, trustScore, status, createdAt, media
 - Moderation action: reportId, actionType=hide, actorId, timestamp

- Hide reason: recommended (TBD in current UI)
- **Validation:**
 - Only a Moderator can execute hide action.
 - A report already hidden cannot be hidden again.
 - Report id must exist.
- **Business rules:**
 - Hide criteria include (non-exhaustive):
 - Spam, duplicate content, clearly incorrect location, inappropriate media, fabricated information.
 - Hiding is a soft action:
 - The record is preserved; only visibility changes.
 - The system should store moderation metadata (recommended):
 - Who hid the report, when, and an optional reason.
- **Error Handling:**
 - Forbidden operation (non-moderator) → show “Insufficient permissions”.
 - Report already hidden → disable action and show status.
 - Backend hide failure → show error and keep report visible in current list until refreshed.
- **Normal case:**
 - Moderator opens community reports view.
 - Filters or finds a suspicious report.
 - Moderator selects Hide and confirms.
 - System updates report state to hidden and refreshes list/status badges.
- **Abnormal case:**
 - Moderator tries to hide an already hidden report → system prevents duplicate action.
 - API error/network failure → system shows error and allows retry.

3.25 Create Alert Message Templates

Function trigger:

- **Navigation path:** Admin/Moderator App > Alert Template Management > Create New Template
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** Replace hardcoded notification messages with configurable, reusable templates stored in the database. Enables dynamic message generation with contextual data and supports multi-channel, multi-severity notification strategies.
- **Interface:**
 - Create Form: Template Name, Channel, Severity Level, Title Template, Body Template, Active, Sort Order
 - Actions: Create new template, Preview template with sample data before saving, Cancel create.

Data processing:

- **Create Flow**

- Admin/Moderator fills in the create form
- System validates required fields (name, channel, titleTemplate, bodyTemplate)
- System validates channel is one of: Push, Email, SMS, InApp
- System validates severity is one of: info, caution, warning, critical, or null
- System validates titleTemplate and bodyTemplate do not contain broken variable syntax
- System checks for duplicate template (same name + channel + severity combination)
- System inserts record into AlertTemplates table
- System returns created template ID
- New template takes effect immediately for subsequent alert dispatches

Screen layout:

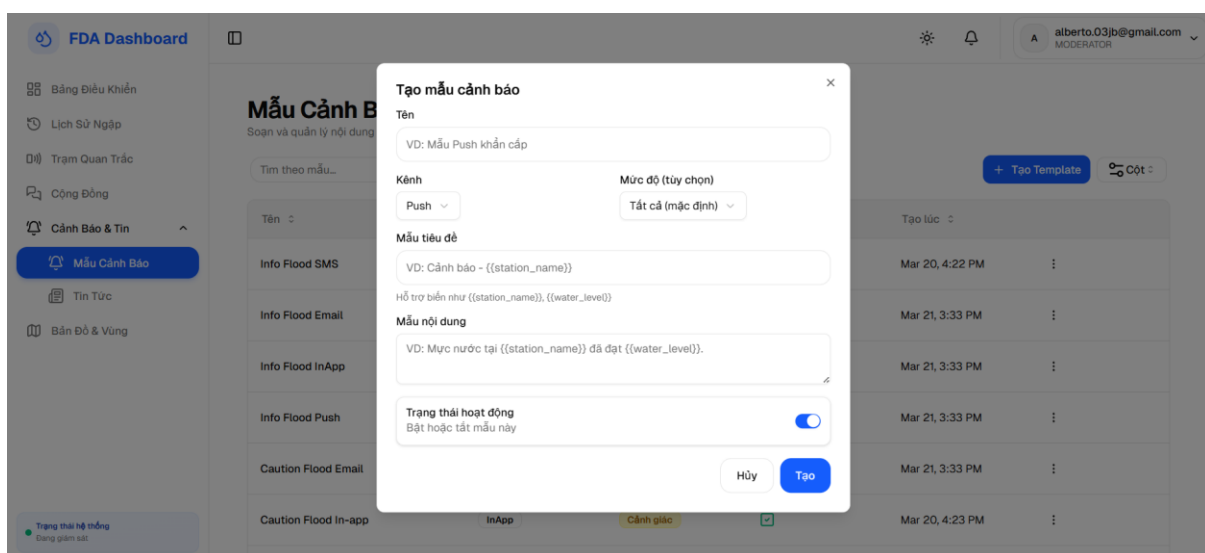


Figure 83 Create Alert Message Templates

Function details:

- **Data:** ID, Name, Channel, Severity, Title Template, Body Template, Is Active, Sort Order, Created By, Created At, Updated By, Updated At.
- **Validation:**
 - Template Name: Required, max 100 characters, unique per channel+severity combination
 - Channel: Required, must be one of: Push, Email, SMS, InApp
 - Severity: Optional, must be one of: info, caution, warning, critical, or null (means "all severities")
 - Title Template: Required, max 200 characters, must contain "{" and "}" properly closed for each variable
 - Body Template: Required, max 2000 characters, must contain "{" and "}" properly closed for each variable
 - Sort Order: Optional, integer ≥ 0
- **Business Rules:**
 - Channel + Severity Uniqueness: No two active templates can share the same channel + severity combination

- Inheritability: A template with severity = null acts as a fallback for all severity levels within its channel
- Template Matching Priority: System searches for exact channel + severity match first, then falls back to channel + null
- Sort Order: Templates are ordered by sortOrder ASC in the list view
- Preview Before Save: System recommends (but does not enforce) Admin to use Preview before saving
- Created By Tracking: System records which user created the template (Created By field)
- Audit Trail: All template changes are logged in an audit table
- **Normal Case:** Admin navigates to Alert Template Management. Taps "+ Create Template". Selects channel = "Push", severity = "critical". Enters title: "Cảnh báo ngập lụt nghiêm trọng - {{StationName}}". Enters body: "Mức nước tại {{StationName}} đã đạt {{WaterLevel}} lúc {{Time}}. Vượt ngưỡng {{Threshold}}m." Taps Preview. The system renders with sample data and displays output. Admin confirms it looks correct and taps Save. The system validates, saves, and returns a success message: "Template created successfully."
- **Abnormal Case:**
 - Template name is empty: System displays "Template name is required. Please enter a name."
 - Channel not selected: System displays "Channel is required. Please select a channel."
 - Title template is empty: System displays "Title template is required. Please enter a title."
 - Body template exceeds 2000 characters: System displays "Body template exceeds 2000 character limit." and prevents saving.
 - Duplicate template (same channel + severity + name): System displays "A template with this name, channel, and severity already exists. Please use a different name or change the severity."
 - Broken variable syntax (e.g., "{{StationName" missing closing "}}"): System detects and warns "Unclosed variable found in title/body. Variables must use {{VariableName}} format."
 - Network error during save: System displays "Failed to create template. Please check your connection and try again." and retains form data.

3.26 View Alert Message Templates

Function trigger:

- **Navigation path:** Admin/Moderator App > Alert Template Management > Template List
- **Detail View:** Admin/Moderator App > Alert Template Management > Template Detail (triggered by tapping a row)
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** To provide full visibility into the notification strategy. This allows Admins/Moderators to understand current messaging, plan new templates, and audit changes over time.
- **Interface:**
 - List View: A data table displaying the following columns:
 - Template Name

- Channel (Push, Email, SMS, InApp)
- Severity (Info, Caution, Warning, Critical, All)
- Status (Active / Inactive)
- Sort Order
- Last Updated
- Actions (Edit, Delete, Preview)
- Filter Bar:
 - Channel Dropdown: All, Push, Email, SMS, InApp.
 - Severity Dropdown: All, Info, Caution, Warning, Critical.
 - Status Toggle: All, Active, Inactive.
 - Search Bar: Search by Template Name (text string).

Data processing:

- **List Loading Logic**
 - The system fetches all records from the AlertTemplates table.
 - Applies default "All" status filter.
 - Renders the list sorted by sortOrder (ASC).
- **Filter Application**
 - The user selects a filter parameter.
 - System re-queries the database with the selected parameters.
 - View refreshes with the filtered result set.

Screen layout:

Mẫu Cảnh Báo
Soạn và quản lý nội dung gửi cảnh báo tới người dùng.

Tim theo mẫu... Kênh Mức cảnh báo + Tạo Template Cột

Tên	Kênh	Mức cảnh báo	Kích hoạt	Tạo lúc
Info Flood SMS	SMS	Thông tin	☒	Mar 20, 4:22 PM
Info Flood Email	Email	Thông tin	☒	Mar 21, 3:33 PM
Info Flood InApp	InApp	Thông tin	☒	Mar 21, 3:33 PM
Info Flood Push	Push	Thông tin	☒	Mar 21, 3:33 PM
Caution Flood Email	Email	Cảnh giác	☒	Mar 21, 3:33 PM
Caution Flood In-app	InApp	Cảnh giác	☒	Mar 20, 4:23 PM

Trạng thái hệ thống
Đang giám sát

Figure 84 View Alert Message Templates

Function details:

- **Data:** Id, Name, Channel, Severity, TitleTemplate, BodyTemplate, IsActive, SortOrder, CreatedBy, CreatedAt, UpdatedBy, UpdatedAt.
- **Validation:**
 - Input Integrity: Filters must strictly match predefined enums (Channel/Severity).
 - Access Control: Access is restricted to Admin/Moderator roles via server-side session validation.

- **Business Rules:**
 - Read-Only Integrity: The "View" function must not modify any database state.
 - Default Sorting: Priority 1: SortOrder (ASC); Priority 2: UpdatedAt (DESC).
 - Data Transparency: No content redaction for Admin/Moderator roles.
 - Empty State: If no data is found, display: "No templates found. Create your first template to get started."
- **Normal Case:** Admin opens the screen, filters by "Email", finds the "Welcome User" template, and views the details. Everything loads within the defined latency.
- **Abnormal Case:**
 - No Results: Display: *"No templates match your filter criteria. Try adjusting the filters."*
 - Record Not Found: Display: *"Template not found"* and redirect to List View.
 - Network Error (List): Display: *"Failed to load templates. Please check your connection"* with a Retry button.
 - Unauthorized Access: Server returns 403 Forbidden and redirects to the login/unauthorized page.

3.27 Update Alert Message Templates

Function trigger:

- **Navigation path:** Admin/Moderator App > Alert Template Management > Template List > Edit Template
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** To enable dynamic adjustments of notification content without requiring code redeployment. This allows for fine-tuning of messaging, severity targeting, and operational status (Activation/Deactivation).
- **Interface:**
 - **List View:** Table of existing templates showing Template Name, Severity Level, and Last Modified.
 - **Editor Form:**
 - Template Name: Text input (Pre-filled).
 - Channel: Dropdown (Pre-selected, Read-only to maintain consistency).
 - Severity Level: Dropdown (Pre-selected).
 - Title Template: Text input with variable support (Pre-filled).
 - Body Template: Multi-line text area with variable support (Pre-filled).
 - Active Status: Toggle switch (Pre-set).
 - Sort Order: Integer input (Pre-filled).
 - **Actions:** Save changes, Preview template (with sample data before saving), Cancel editing (discard changes), Delete template (navigate to delete confirmation)

Data processing:

- Update Workflow
 - Admin/Moderator taps Edit on a template row.
 - The system loads current template data into the edit form.
 - The user modifies fields and taps Save.
 - The system validates all fields (same rules as Create).

- System checks for duplicate templates (same name + channel + severity, excluding current ID).
- System updates record in AlertTemplates table (Updated By, Updated At).
- The system returns success.
- Changes take effect immediately for subsequent alert dispatches.

Screen layout:

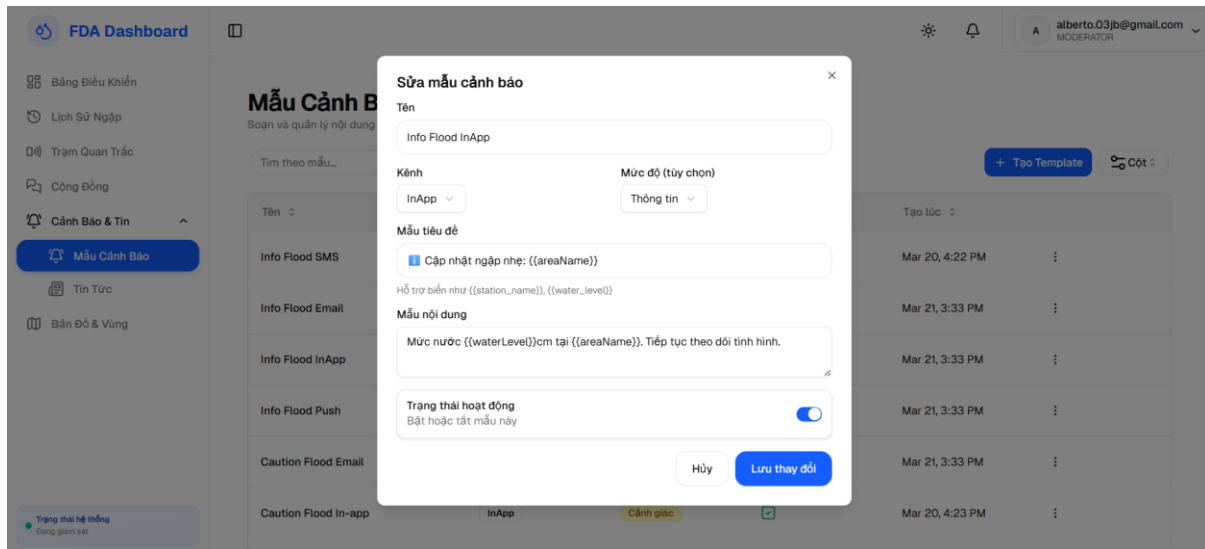


Figure 85 Update Alert Message Templates

Function details:

- **Validation:**
 - Template Name: Required, max 100 characters, unique per channel + severity.
 - Channel: Required, must be Push/Email/SMS/InApp, read-only on edit.
 - Severity: Optional, must be info/caution/warning/critical/null.
 - Title Template: Required, max 200 characters, valid variable syntax.
 - Body Template: Required, max 2000 characters, valid variable syntax.
- **Business Rules:**
 - Read-Only Channel: Channel cannot be changed after creation to prevent orphaning active dispatches.
 - Immutability of Referenced Templates: Templates actively used in pending dispatches are versioned before update (future scope).
 - Immediate Effect: Updated templates are used immediately on next dispatch — no versioning or scheduling in this phase.
 - Updated By Tracking: System records which user made the change.
 - Version History: All changes are recorded in the audit log (future scope).
 - Deactivation vs Deletion: If a template needs to be retired, deactivating (isActive=false) is preferred over deletion to preserve audit trail.
- **Normal Case:** Admin navigates to Alert Template Management. Taps Edit on "Critical Push Notification". Changes severity from "critical" to "warning". Updates title. Taps Preview to verify. System renders sample output. Admin taps Save. System validates and saves. Success message: "Template updated successfully."
- **Abnormal Case:**

- Channel changed: System prevents change and displays "Channel cannot be changed after creation. Please create a new template if you need a different channel."
- Duplicate name after update: System displays "A template with this name and severity already exists for this channel. Please use a different name."
- Required fields cleared: System validates required fields before saving and displays specific error for each empty field.
- Broken variable syntax: System detects and warns "Unclosed variable found. Variables must use {{VariableName}} format."
- Network error during save: System displays "Failed to update template. Please check your connection." and retains all form data for retry.
- Save on already-deleted template: System returns 404 "Template not found. It may have been deleted."

3.28 Delete Alert Message Templates

Function trigger:

- **Navigation path:** Admin/Moderator App > Alert Template Management > Template List > Delete Template
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** Enable cleanup of obsolete, duplicate, or incorrect alert templates. The soft-delete approach preserves the audit trail and allows recovery of accidentally deleted templates if needed.
- **Interface:**
 - **List View:** Delete button on each template row:
 - Trash/Delete icon button.
 - Enabled for all templates regardless of status.
 - **Delete Confirmation Dialog:** Modal dialog shown before deletion:
 - Title: "Delete Template?"
 - Message: "Are you sure you want to delete '{{TemplateName}}'? This action cannot be undone."
 - Warning: "This template is currently ACTIVE. Deleting it may affect notification delivery. Consider deactivating it instead."
 - Confirmation Button: "Delete" (red/danger style).
 - Cancel Button: "Cancel".
 - **Soft Delete Behavior:**
 - Templates are not permanently removed from the database.
 - isActive is set to false (or a deletedAt timestamp is set).
 - Template is hidden from the active list view.
 - Template does not appear in template matching for new dispatches.
 - Template remains queryable via "Show Deleted" filter (Admin only, future scope).
 - **Actions:**
 - Delete a template (with confirmation).
 - Cancel deletion.
 - Deactivate template as alternative to deletion.

Data processing:

- **Delete Flow:**
 - Admin/Moderator taps Delete on a template row.
 - The system checks if the template is currently active.
 - If active: System shows warning in confirmation dialog.
 - Admin/Moderator confirms deletion.
 - System performs soft delete: sets isActive = false and records deletedAt, deletedBy.
 - System removes template from active template matching.
 - The system returns success.
 - Template disappears from active list.
- **Soft Delete Details:**
 - isActive is set to false.
 - deletedAt is set to the current timestamp.
 - deletedBy is set to the acting user's ID.
 - Templates are excluded from the GET /alert-templates list by default.
 - The template remains in the database for audit/recovery purposes.

Screen layout:

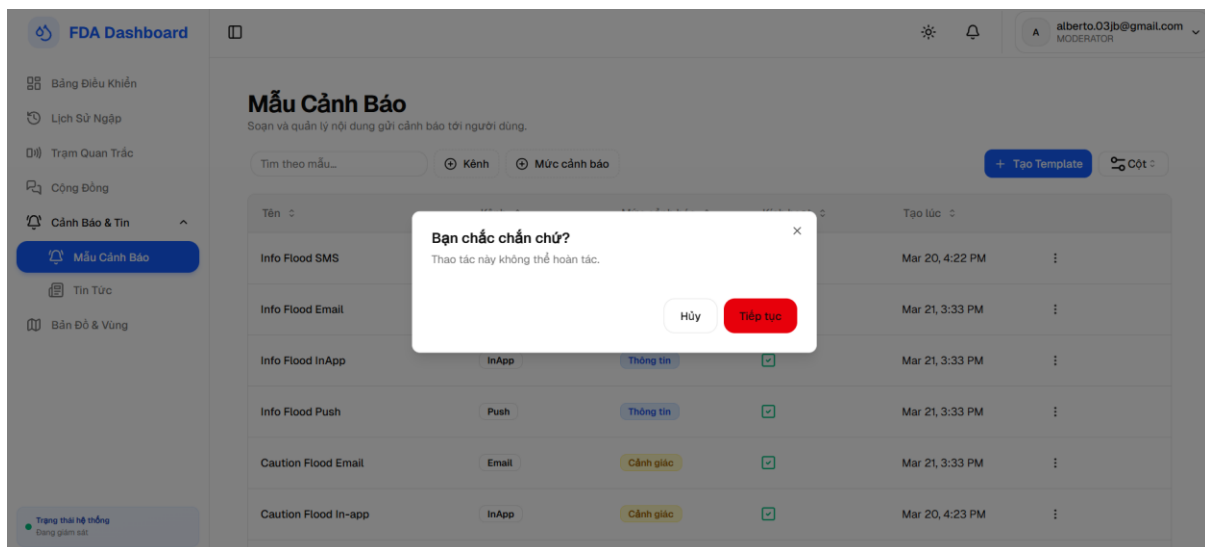


Figure 86 Delete Alert Message Templates

Function details:

- **Data:** IsActive, DeletedAt, DeletedBy fields added.
- **Validation:**
 - Template ID: Must exist in the database.
 - Delete Permission: User must be Admin or have delete permission on this template.
 - No Hard Dependencies: System checks if template is referenced by scheduled dispatches (future scope).
- **Business Rules:**
 - Soft Delete Only: No hard delete — all templates are retained in the database.
 - Active Warning: If template isActive = true, show warning before deletion.
 - Audit Preservation: Soft delete preserves the full history of template changes.
 - Deletion is Final for UI: Deleted templates do not appear in the active list, but can be recovered by Admin (future scope).

- Notification Matching Exclusion: Templates with isActive = false are excluded from notification template matching.
- Sort Order After Delete: Deletion does not auto-reorder remaining templates.
- **Normal Case:** Admin browses the Alert Template Management list. Finds a duplicate template named "Warning Push 2" that is inactive. Taps Delete. Confirmation dialog shows no warning (template is inactive). Admin taps Delete. System performs soft delete, template disappears from list. Success toast: "Template deleted successfully."
- **Abnormal Case:**
 - User tries to delete an active template: System shows confirmation dialog with warning: "This template is currently active. Deleting it may affect notification delivery. Consider deactivating it instead."
 - User cancels during confirmation: System closes dialog, no changes made.
 - Template not found during delete: System displays "Template not found. It may have already been deleted." and a refreshed list.
 - Network error during delete: System displays "Failed to delete template. Please try again." and the template remains in the list.
 - Delete permission denied: System displays "You do not have permission to delete this template." and the delete button remains disabled.
 - Concurrent deletion: System returns 404 and refreshes list.

3.29 Preview Alert Message Templates

Function trigger:

- **Navigation path:** Admin Dashboard > Alert Configuration > Message Templates
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** Allows authorized users to define and maintain standard message templates for various flood severity levels (Warning, Danger, Critical) to ensure consistent and rapid communication during emergencies.
- **Interface:**
 - **List View:** Table of existing templates showing Template Name, Severity Level, and Last Modified.
 - **Editor Form:** Input fields for Template Name, Severity Dropdown (Yellow/Orange/Red), Message Subject, and Message Body (supporting placeholders like {Location}, {WaterLevel}, {Time}).
 - **Actions:** Add New, Edit, Delete.

Data processing:

- The user navigates to the template management screen.
- The system queries the AlertTemplates table (or associated configuration table) to display existing records.
- **Create/Update:** When the user saves a form:
 - The system validates that the MessageBody contains required placeholders.
 - The system inserts or updates the record in the database.
 - The system logs the change in AdminActions.
- **Delete:** The system performs a soft delete or hard delete depending on policy.

Screen layout:

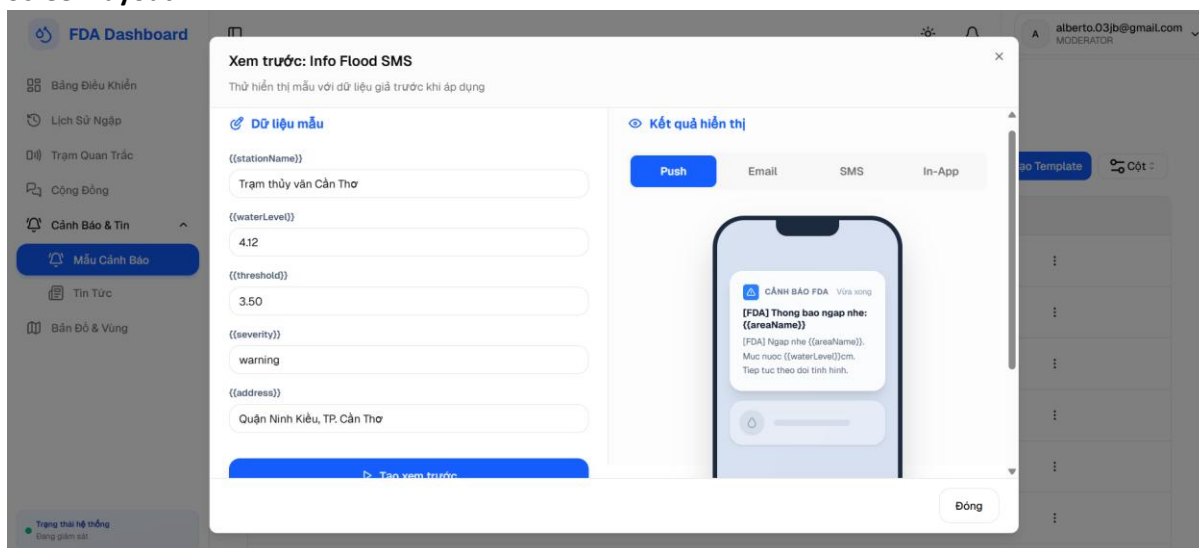


Figure 87 Preview Alert Message Templates

Function details:

- **Data:** Template ID, Severity, Subject, Body, Created By.
- **Validation:** Message body length must not exceed SMS limits (if applicable, e.g., 160 chars for SMS segments) or App notification limits.
- **Business Rules:** Templates for "Critical" severity cannot be deleted, only edited.
- **Normal Case:** Admin updates the "Danger Level" template. The system saves changes and displays "Template updated successfully."
- **Abnormal Case:** User tries to save a template without the **{location}** placeholder. System displays error: "Template must include {location} placeholder."

3.30 Create Sensor Stations

Function trigger:

- **Navigation path:** Admin App > Station Management > **Add New Station**
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Admin
- **Purpose:** Enable administrators to expand the sensor station network by adding new monitoring points with configurable alert thresholds and calibration settings, supporting comprehensive flood coverage across the service area.
- **Interface:**
 - **Create Form:** Input fields for creating a new sensor station:
 - **Station Code:** Text input (unique identifier, e.g., "ST_HCM_001").
 - **Station Name:** Text input for display name (e.g., "Trạm đo Nguyễn Trãi").
 - **Station Type:** Dropdown (UrbanLowland, Rural, DrainagePump, Riverside).
 - **Location Description:** Text input (e.g., "Cạnh cống Nguyễn Trãi").
 - **Latitude:** Number input (GPS coordinate).
 - **Longitude:** Number input (GPS coordinate).

- **Road Name:** Text input.
- **Direction:** Dropdown (upstream, downstream, crossflow).
- **Threshold Warning:** Number input in cm (triggers warning-level alert).
- **Threshold Critical:** Number input in cm (triggers critical-level alert).
- **Calibration Offset:** Number input in cm (0–50, default 5, display-only offset).
- **Installed At:** Date picker.
- **Actions:** Create new station, Cancel creation

Data processing:

- **Create Flow:**
 - Admin fills in the create station form.
 - System validates required fields (code, name, latitude, longitude, thresholds).
 - System validates latitude is within service area bounds (-90 to 90).
 - System validates that the longitude is within service area bounds (-180 to 180).
 - System validates thresholdCritical > thresholdWarning.
 - System checks for duplicate station code.
 - System inserts record into stations table.
 - System sets station status to "active" by default.
 - System returns created station ID.
 - Station is immediately available for receiving sensor data.

Screen layout:

The screenshot displays the 'Tạo trạm mới' (Create New Station) form in the FDA Dashboard. The form is a modal window with the following fields:

- Mã trạm ***: Text input with value 'ST_DN_DRAGON_01'.
- Tên trạm ***: Text input with value 'Trạm Cầu Rồng'.
- Trạng thái ***: Dropdown menu with value 'Đang hoạt động'.
- Tên đường**: Text input with value 'Đường 2 Tháng 9'.
- Mô tả vị trí**: Text area with placeholder 'Ví dụ: Nằm phía tây Cầu Rồng...' and a character count '0 / 500'.
- Hướng**: Text input with value 'thượng lưu / hạ lưu'.
- Vĩ độ**: Text input with value '16.061153'.
- Kinh độ**: Text input with value '108.221589'.

The background shows the dashboard with a sidebar menu and a table of existing stations:

#	Mã	Tên
01	ST_DEMO_02	
02	LVH-DEMO-01	
03	CD-DEMO-01	

Figure 88 Create Sensor Stations

Function details:

- **Data:** ID, Code, Name, Type, LocationDesc, Latitude, Longitude, RoadName, Direction, Status, ThresholdWarning, ThresholdCritical, CalibrationOffset, InstalledAt, LastSeenAt, CreatedBy, CreatedAt, UpdatedBy, UpdatedAt.
- **Validation:**
 - Station Code: Required, max 50 characters, unique.
 - Station Name: Required, max 200 characters.
 - Station Type: Required (UrbanLowland, Rural, DrainagePump, Riverside).

- Latitude: Required, decimal, -90 to 90.
- Longitude: Required, decimal, -180 to 180.
- Threshold Warning: Required, integer ≥ 0 (cm).
- Threshold Critical: Required, integer $>$ Threshold Warning (cm).
- Calibration Offset: Optional, decimal, 0 to 50 cm, default 5.
- Installed At: Optional, date (cannot be future date).
- **Business Rules:**
 - Unique Code: Station code must be globally unique — duplicates rejected.
 - Threshold Hierarchy: Critical threshold must always be greater than warning threshold.
 - Status Default: New stations are set to "active" by default.
 - Calibration Display Only: CalibrationOffset is for display purposes only — does NOT affect WaterLevel values or alert logic.
 - Location Bounds: Latitude/longitude must fall within the configured service area boundaries.
 - CreatedBy Tracking: System records which admin created the station.
- **Normal Case:** Admin navigates to Station Management. Taps "+ Add Station". Enter code "ST_HCM_042", name "Trạm đo Lê Lai", type "UrbanLowland". Sets latitude 10.7769, longitude 106.7009. Sets warning threshold = 200cm, critical threshold = 300cm. Leaves calibration offset at default 5cm. Taps Save. The system validates all fields. Station created successfully. The station appears in the list and begins receiving data.
- **Abnormal Case:**
 - Station code already exists: System displays "A station with this code already exists. Please use a unique code."
 - Lat/Lng out of range: System displays "Latitude must be between -90 and 90. Longitude must be between -180 and 180."
 - Threshold critical $\leq$ warning: System displays "Critical threshold must be greater than warning threshold."
 - Missing required fields: System displays specific error for each missing field.
 - Calibration offset > 50 cm: System displays "Calibration offset must be between 0 and 50 cm."
 - Future installed date: System displays "Installed date cannot be in the future."
 - Network error during save: System displays "Failed to create station. Please check your connection." and retains form data.

3.31 View Sensor Stations

Function trigger:

- **Navigation path:**
 - Public / Admin App: Station List
 - Admin App: Station Management > Station Detail
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Admin, Moderator
- **Purpose:** Provide visibility into the sensor station network so that public users can understand flood monitoring coverage in their area, and administrators can audit station configuration, manage components, and plan maintenance.

- **Interface:**
 - **List View(Public):** Table of stations showing
 - Station Code
 - Station Name
 - Location (latitude/longitude or address)
 - Status (active, offline, maintenance)
 - Alert Thresholds (warning / critical in cm)
 - Calibration Offset (displayed as "±X cm")
 - **List View (Admin):** Enhanced table with additional columns
 - Station Type
 - Direction
 - Road Name
 - Installed At
 - Last Seen At
 - Actions (Edit, Delete, View Components)
 - **Station Detail View:**
 - Station Code, Name, Type.
 - Location: Latitude, Longitude, Address/Road Name, Direction.
 - Status with last seen timestamp.
 - Alert Thresholds: Warning (cm), Critical (cm).
 - Calibration Offset: ±X cm.
 - Components: Count + list of hardware components (Admin only).
 - Installed At, Created By, Created At.
 - Updated By, Updated At.
 - **Actions:**
 - View station list.
 - Filter stations by status / type / search.
 - View station detail.
 - Navigate to Edit (Admin).

Data processing:

- List Load (Public):
 1. User opens station list (no auth required).
 2. System fetches all active stations from the stations table.
 3. System filters out stations with isIncidentActive = true.
 4. System applies a default sort (name ASC).
 5. System renders paginated list (20 per page).
- List Load (Admin):
 - Admin opens station list (auth required).
 - System fetches all stations including inactive/deleted.
 - System applies filters and sort parameters.
 - System renders paginated list with all columns.
- Detail View Load:
 - The user taps on a station row.
 - System fetches station records by ID.
 - System fetches station components (if Admin).
 - System renders a full detailed view.

Screen layout:

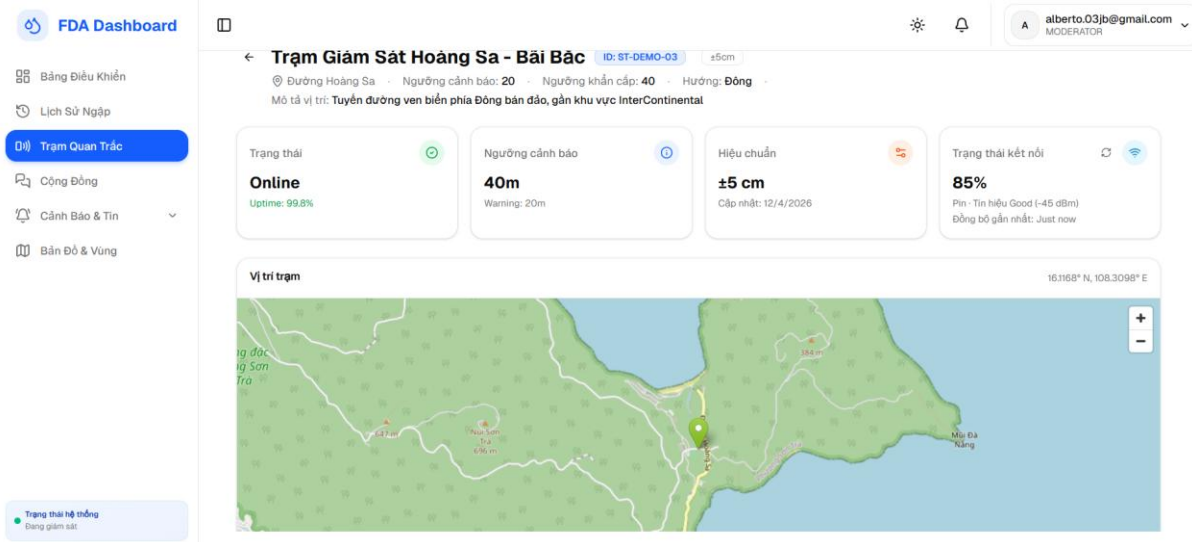


Figure 89 View Sensor Station

Function details:

- **Data:**
 - Stations table: ID, Code, Name, Type, LocationDesc, Latitude, Longitude, RoadName, Direction, Status, ThresholdWarning, ThresholdCritical, CalibrationOffset, InstalledAt, LastSeenAt, CreatedBy, CreatedAt, UpdatedBy, UpdatedAt.
 - StationComponents table: ID, StationID, ComponentType, Name, Model, SerialNumber, FirmwareVersion, Status, InstalledAt, LastMaintenanceAt, Notes, CreatedAt, UpdatedAt.
- **Validation:**
 - Station ID: Must exist in the database.
 - Filter Values: Status in {active, offline, maintenance, all}; Type in {urbanlowland, rural, drainagepump, riverside, all}.
- **Business Rules:**
 - Public Visibility: Station list/detail is publicly accessible — no authentication required.
 - Incident Filtering: Stations with active incidents (isIncidentActive=true) are hidden from the public list but visible to Admin.
 - Last Seen Update: LastSeenAt is updated by background heartbeat job — may be stale if station is offline.
 - Calibration Display: CalibrationOffset shown as "Sai số có thể: $\pm X$ cm" — purely informational.
 - Empty State: If no stations exist, display "No stations found in the monitoring network."
 - Offline Indication: If LastSeenAt is older than 10 minutes, display status as "offline".
- **Normal Case:** Public user opens the app. Browse the station map/list. Taps on "Trạm đo Nguyễn Trãi" to see details. Sees current thresholds (200/300cm), calibration offset (± 5 cm), and status (Active). No components shown (public view). Admin opens Station Management and sees the same station with a full component list and audit metadata.
- **Abnormal Case:**
 - Station not found: System displays "Station not found." and redirects to a list.

- Network error (List): System displays "Failed to load stations. Please check your connection." with the Retry button.
- Network error (Detail): System displays "Failed to load station details. Please try again."
- Filter returns no results: System displays "No stations match your filter criteria."
- No components: System shows "No components installed." in the components section.
- Station offline: If LastSeenAt > 10 min, system displays status badge as "Offline" with last seen timestamp.

3.32 Update Sensor Stations

Function trigger:

- **Navigation path:** Admin taps "Edit" on a station row, or taps "Save" in the edit form.
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator, Admin
- **Purpose:** Enable dynamic adjustment of station settings without requiring station hardware changes or redeployment. Allows Admin to fine-tune alert thresholds, correct station metadata, or activate/deactivate stations based on operational needs..
- **Interface:**
 - Edit Form (Pre-populated): Input fields for editing an existing station:
 - Station Code: Text input (pre-filled, read-only).
 - Station Name: Text input (pre-filled).
 - Station Type: Dropdown (pre-selected).
 - Location Description: Text input (pre-filled).
 - Latitude: Number input (pre-filled).
 - Longitude: Number input (pre-filled).
 - Road Name: Text input (pre-filled).
 - Direction: Dropdown (pre-selected).
 - Status: Dropdown (active, offline, maintenance).
 - Threshold Warning: Number input (pre-filled).
 - Threshold Critical: Number input (pre-filled).
 - Calibration Offset: Number input (pre-filled, 0–50 cm).
 - Installed At: Date picker (pre-filled).
 - **Actions:**
 - Save changes
 - Cancel editing (discard changes)
 - Delete station (navigate to delete confirmation)

Data processing:

- Update Flow
 - Admin taps Edit on a station row.
 - System loads current station data into the edit form.
 - Admin modifies fields and taps Save.
 - System validates all fields (same rules as Create).
 - System updates record in stations table (UpdatedBy, UpdatedAt).
 - System returns success.

- Changes take effect immediately for subsequent sensor data and alerts.

Screen layout:

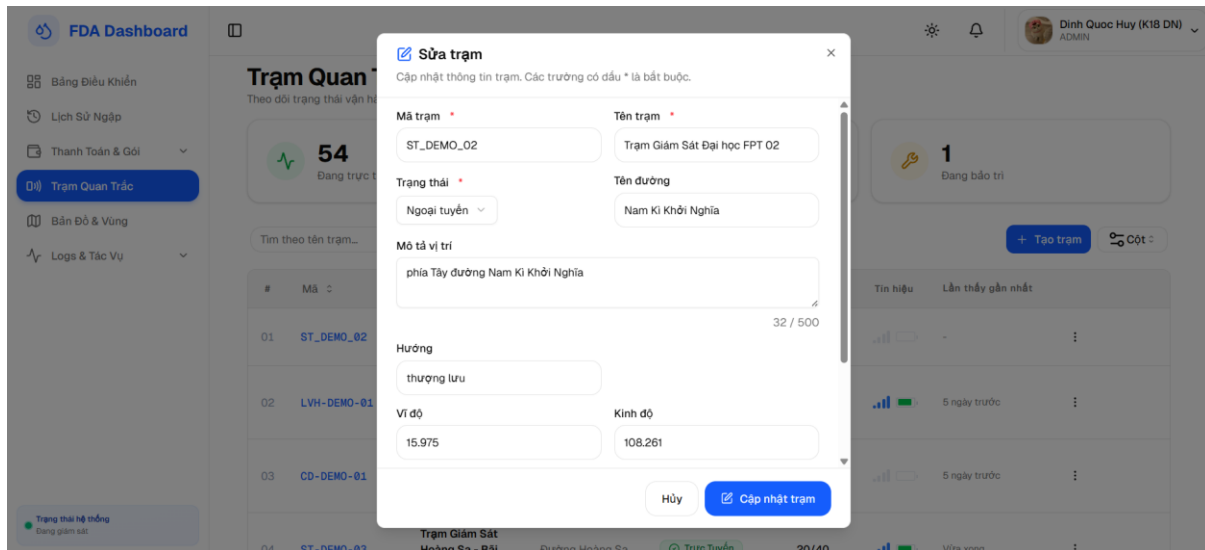


Figure 90 Update Sensor Station

Function details:

- **Validation:**
 - Station Name: Required, max 200 characters.
 - Latitude: Required, decimal, -90 to 90.
 - Longitude: Required, decimal, -180 to 180.
 - Threshold Warning: Required, integer ≥ 0 (cm).
 - Threshold Critical: Required, integer $\geq$ Threshold Warning (cm).
 - Calibration Offset: Optional, decimal, 0 to 50 cm.
 - Status: Required, one of: active, offline, maintenance.
 - Installed At: Optional, date (cannot be future date).
- **Business Rules:**
 - Read-Only Code: Station code cannot be changed after creation to preserve data integrity.
 - Immediate Effect: Updated thresholds are applied immediately — sensor readings from that point use the new values.
 - Status Transitions: Admin can manually set status to offline/maintenance even if the station is still sending data.
 - Threshold Re-evaluation: When thresholds are updated, existing active alerts are re-evaluated against new values.
 - UpdatedBy Tracking: System records which admin made the change.
 - Audit Trail: All changes are logged for audit purposes.
 - Threshold Safety: If the new critical threshold is $\leq$ warning threshold, validation error is thrown.
- **Normal Case:** Admin notices station "Trạm Nguyễn Trãi" has frequent false alarms. Open Edit form. Changes warning threshold from 200cm to 250cm, critical threshold from 300cm to 350cm. Updates calibration offset from 5cm to 8cm. Taps Save. System validates critical warning. The station was updated successfully. New thresholds apply to incoming sensor data.

- **Abnormal Case:**

- Station code changed: System prevents change and displays "Station code cannot be modified after creation."
- Threshold critical $\leq$ warning: System displays "Critical threshold must be greater than warning threshold."
- Missing required field: System validates required fields before saving and displays specific error for each empty field.
- Calibration offset > 50cm: System displays "Calibration offset must be between 0 and 50 cm."
- Network error during save: System displays "Failed to update station. Please check your connection." and retains all form data for retry.
- Save on already-deleted station: System returns 404 "Station not found. It may have been deleted."
- Lat/Lng out of range: System displays "Latitude must be between -90 and 90. Longitude must be between -180 and 180."
- Status change to maintenance without note: System shows a warning (not blocking) "Consider adding a note when setting status to maintenance."

3.33 Delete Sensor Stations

Function trigger:

- **Navigation path:** Admin App > Station Management > Station List > **Delete Station**
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Admin
- **Purpose:** Enable cleanup of decommissioned, duplicate, or incorrectly provisioned sensor stations. Soft delete preserves historical sensor data and alert records while removing the station from active monitoring and template matching.
- **Interface:**
 - **List View-Delete Action:** Delete button on each station row:
 - Trash/Delete icon button.
 - Shown only to Admin users.
 - **Delete Confirmation Dialog:** Modal dialog shown before deletion:
 - **Title:** "Delete Station?"
 - **Message:** "Are you sure you want to delete '{{StationName}}' ({{station_code}})? This action cannot be undone."
 - **Warning (conditional):** "This station is currently ACTIVE and may have ongoing alerts. Deleting it may affect flood monitoring coverage. Consider setting it to 'Maintenance' status instead."
 - **Warning (conditional):** "This station has {ComponentCount} components installed. Deleting the station will also remove all component records."
 - **Warning (conditional):** "This station has unresolved alerts. Deleting may affect alert tracking. Consider resolving or dismissing alerts first."
 - **Confirmation Button:** "Delete" (red/danger style).
 - **Cancel Button:** "Cancel".
 - **Actions:**
 - Delete a station (with confirmation).
 - Cancel deletion.

- Set to Maintenance as alternative to deletion.

Data processing:

- **Delete Flow**
 - Admin taps Delete on a station row.
 - System checks station status.
 - If active: System shows warning in confirmation dialog.
 - If has unresolved alerts: System shows warning in confirmation dialog.
 - If has components: System shows component count warning.
 - Admin confirms deletion.
 - System performs soft delete: sets status = "deleted", records deletedAt, deletedBy.
 - System cascades soft delete to all associated components.
 - System excludes station from sensor data processing.
 - System returns success.
 - Station disappears from active lists.

Screen layout:

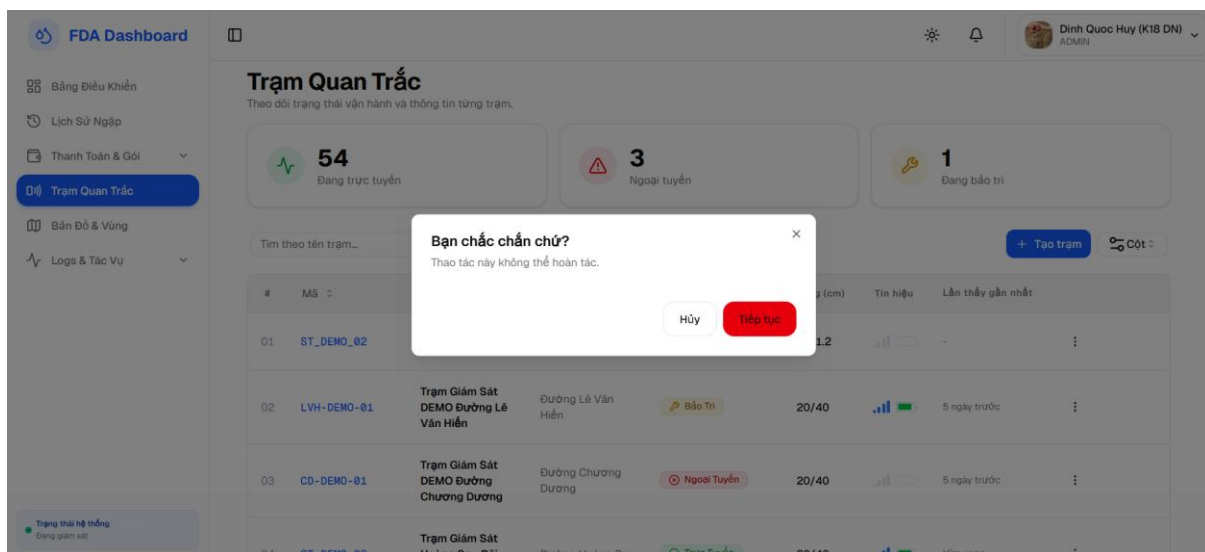


Figure 91 Delete Sensor Station

Function details:

- **Data:**
 - Stations table (updated): status = "deleted", deletedAt, deletedBy fields.
 - StationComponents table (cascade): status = "deleted", deletedAt, deletedBy.
- **Validation:**
 - Station ID: Must exist in the database and not already deleted.
 - Delete Permission: User must be Admin.
 - Alert Resolution: No hard block — warnings shown but deletion allowed even with unresolved alerts.
 - Component Cascade: System automatically soft-deletes associated components.
- **Business Rules:**
 - Soft Delete Only: No hard delete — all stations retained in the database for historical reporting.
 - Active Warning: If status = "active", show warning before deletion.

- Alert Warning: If a station has unresolved alerts, show warning before deletion (non-blocking).
- Component Cascade: Deleting a station also soft-deletes all its components.
- Historical Preservation: Sensor readings and alert history are preserved even after station deletion.
- Deletion is Final for UI: Deleted stations do not appear in the active list, but remain queryable for reporting.
- Audit Preservation: Soft delete preserves the full history of station changes.
- Template Matching Exclusion: Deleted stations are excluded from notification template matching.
- Sensor Data Retention: Historical sensor readings are NOT deleted.
- **Normal Case:** Admin identifies a decommissioned station "ST_HCM_OLD_01" that has been replaced. Open Station List. Find the station. Taps Delete. Confirmation dialog shows warning about active status. Admin confirms. System performs soft delete, station disappears from list. Historical sensor data for this station remains accessible in reports.
- **Abnormal Case:**
 - Delete active station: System shows confirmation dialog with warning: "This station is currently ACTIVE. Deleting it may affect flood monitoring coverage. Consider setting it to 'Maintenance' instead."
 - Delete station with unresolved alerts: System shows non-blocking warning: "This station has {{count}} unresolved alerts. Deleting the station will not resolve these alerts."
 - User cancels: System closes dialog, no changes made.
 - Station not found: System displays "Station not found. It may have already been deleted." and a refreshed list.
 - Network error: System displays "Failed to delete station. Please try again." and the station remains in list.
 - Permission denied: System displays "You do not have permission to delete this station." and the delete button remains disabled.
 - Concurrent deletion: System returns 404 and refreshes list.

3.34 Monitor Sensor & Device Status

Function trigger:

- **Navigation path:** Admin App > Station Management > Station Status Dashboard
 - Admin App > Station Management > Station Detail > Status Tab
 - Admin/Monitor App > Dashboard > Offline Stations Panel
- **Timing Frequency:**
 - **Background job:** Automated every 10 minutes.
 - **User queries:** On demand (user-triggered via API).

Function description:

- **Actors/Roles:** System (Background Job), Admin, Moderator, User.
- **Purpose:** Provide visibility into the health and connectivity of the sensor station network so that Admins and Moderators can quickly identify and respond to station failures, low battery, or poor signal conditions before they impact flood monitoring coverage.
- **Interface:**

- Dashboard — Status Overview: Summary cards showing Online Count (active), Offline Count (offline), and Maintenance Count.
- Dashboard — Offline Stations List: Table showing Station Name, Offline Duration (Xh Ym), Last Seen At, and "Create Incident" action.
- Station Detail — Status Section: * Status Badge: Active (green), Offline (red), Maintenance (orange).
 - Indicators: Last Seen, Battery Level (0–100%, null=100%), Signal Strength (RSSI with quality labels), Offline Duration, Last Reading.
- Status History Timeline: Chronological list of status changes (Online, Offline, Back Online, Maintenance set).
- Actions: View dashboard, View individual status, View offline list, Create manual incident, Filter by status.

Data processing:

- **Background Detection Flow:**
 1. CheckStationStatusJob runs every 10 minutes.
 2. The system calculates time since LastSeenAt.
 3. If > 10 mins and status $\neq$ "maintenance": Set status = "offline".
 4. If ≤ 10 mins and status = "offline": Set status = "active".
 5. Log changes to StationStatusHistory.
- **Auto-Incident Creation Flow:**
 1. If offline > 360 mins (6 hours) and no active incident exists:
 - Create record in SensorIncidents.
 - Set station.isIncidentActive = true (Exclude from map).
 2. If the station comes back online: Auto-resolve incident (status = "resolved").
- **MQTT Data Ingestion Flow:**
 1. The system receives MQTT messages { StationId, WaterLevel, Battery, Rssi, ... }.
 2. Update LastSeenAt = now and Status = "active".
 3. Save SensorReadings (Defaults: battery = 100 if null, signal = 0 if null).
- **User Query Flow:** System authenticates user and returns calculated status data/offline duration.

Screen layout:

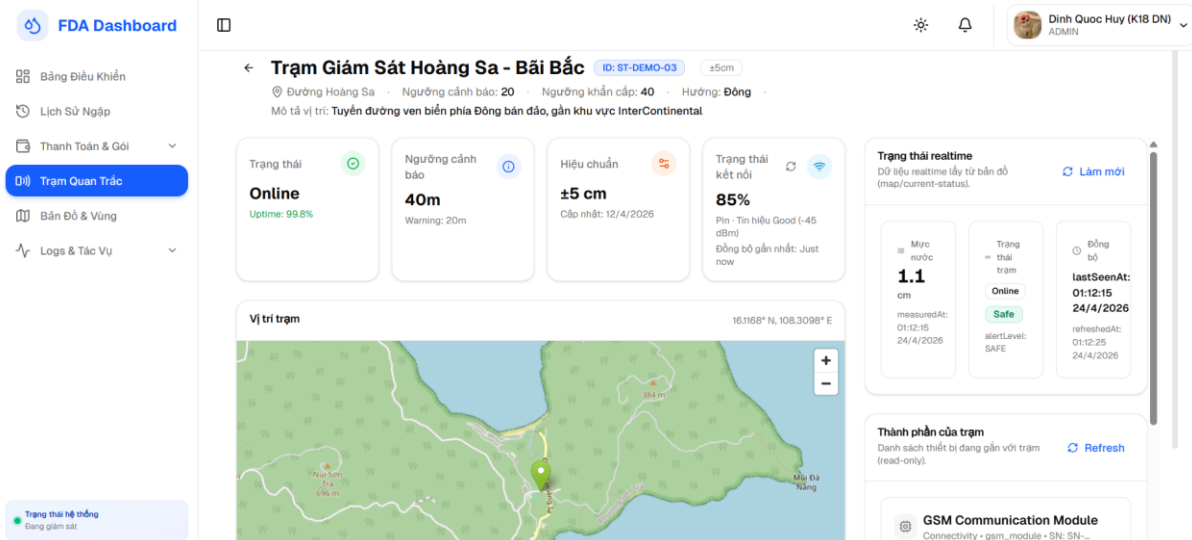


Figure 92 Monitor Sensor and Device Status

Function details:

- **Data:** Station ID, Status, LastSeenAt, IsIncidentActive, BatteryLevel, SignalStrength, Thresholds.
- **Validation:** * **Status Query:** User must be authenticated (USER+).
 - **Battery/Signal:** 0–100 for battery, dBm for signal; null values use system defaults.
- **Business Rules:**
 - **Auto-Revival:** MQTT data triggers "active" status immediately.
 - **Maintenance Override:** "Maintenance" stations never auto-mark as offline.
 - **Incident Management:** Auto-resolve on recovery; isIncidentActive hides station from public map.
 - **Thresholds:** Offline (>10m), Incident (>360m), Battery Critical (≤20%), Signal Poor (<-85 dBm).
- **Normal Case:** Admin checks dashboard, sees "Trạm Nguyễn Huệ" offline for 14h 30m, and clicks "Create Incident". Background job runs and confirms data consistency.
- **Abnormal Case:** * MQTT arrives with null values: System applies defaults (100% battery/0 signal).
 - Station offline exactly 10 mins: System waits until > 10 mins to avoid flapping.
 - Duplicate incident: System skips if an active incident already exists.

3.35 View Subscription Plans

Function trigger:

- Navigation path: Profile Tab -> "Bảng giá" / "Gói dịch vụ" entry -> route /plans OR Home banner / paywall CTA -> deep-link to /plans (handled by app/plans/index.tsx).
- Timing Frequency: On demand. Initial fetch on screen mount; React Query refetch on manual pull-to-refresh (RefreshControl) or when the cache becomes stale (staleTime = 5 minutes).

Function description:

- **Actors/Roles:** Citizen (both authenticated users and guest users — the screen is public, but upgrade actions require authentication).

- **Purpose:** Display the full pricing catalog (FREE, PREMIUM, MONITOR) so citizens can compare features, prices, and benefits before choosing a plan. The screen also surfaces the user's current subscription so they can see which plan is active and which plan they can move to.
- **Interface:**
 - Header: LinearGradient header ("Bảng giá") with back button, rendered inside app/plans/index.tsx.
 - Feature Comparison Table: FeatureComparisonTable.tsx — lists every PlanFeature (featureKey, featureName, featureValue) across the 3 plans for a side-by-side comparison.
 - Billing Cycle Toggle: BillingToggle.tsx — a 2-state switch (monthly / yearly) that re-computes the displayed price on each pricing card.
 - Pricing Cards: PricingCard.tsx, one per plan, sorted by FREE → PREMIUM → MONITOR (via getPlanRank). Each card shows the plan icon, name, description, price (formatted to vi-VN), badges ("Phổ biến", "Nâng cao", "Đang dùng"), and a dynamic CTA button.
 - Loading / Error / Empty states: ActivityIndicator during loading, error card with "Thử lại" retry button on failure, and a pricetag placeholder when the catalog is empty.

Data processing:

- The screen identifies guest status via the useUser() hook.
- usePricingPlans executes GET /api/v1/plans and caches the PricingPlan[] with a staleTime of 5 minutes.
- In parallel, UseCurrentSubscription executes GET /api/v1/plan/subscription/current; 401 responses are treated as "no subscription" without retries.
- PricingPlansList.tsx sorts plans by rank and passes subscription data to cards to compute isCurrentPlan, isUpgrade, and isDowngrade.
- Tapping a CTA on a non-current plan triggers the following hand-offs to FE-48:
 - Guest -> Alert for sign-in -> navigate to /(auth)/sign-in.
 - PREMIUM / MONITOR -> open DurationSelectionModal (upgrade flow).
 - FREE -> open DowngradeConfirmDialog (downgrade flow).
 - MONITOR -> Alert with contact instructions.
- Pull-to-refresh invokes both refetchPlans() and refetchSubscription() in parallel.
- The billing-cycle toggle updates local state (monthly | yearly) to drive immediate price re-renders without server roundtrips.

Screen layout:

Bảng giá

So sánh chi tiết

	Miễn phí	Premium
Số vùng theo dõi	5	Không giới hạn
Kênh thông báo	Push, Email	Push, Email, SMS
Dự báo AI	Có	Có
Tìm đường an toàn	Có	Có
Báo cáo cộng đồng	Có	Có (Uy tín cao)

Tháng

Năm

Miễn phí

Gói cơ bản

Hạ cấp

Phổ biến

Đang dùng

45.000 /tháng

Gói nâng cao

Đang dùng

Miễn phí

Figure 93 View Subscription Plans

Function details:

- **Data:** - Request: GET /api/v1/plans (no body or query params).
 - Response: PricingPlan objects containing ID, code, name, description, pricing, tier, and PlanFeature details.
 - Subscription: UserSubscription details including tier, status, and dates.
- **Validation:**
 - Plan code: Must be FREE | PREMIUM | MONITOR; unknown codes fall back to rank 99 and a generic icon.
 - Prices: Must be non-negative; 0 renders as "Miễn phí".
 - Billing cycle: Constrained strictly to monthly or yearly.
 - Authentication state: Derived from user existence; guests are restricted from taking action.
- **Business rules:**
 - All users can view the catalog, but only one plan is marked "Đang dùng" based on currentSubscription.tierCode.
 - The FREE card is the target for downgrades from higher tiers.
 - Comparison tables and pricing cards must remain synchronized using the same sortedPlans array.
 - MONITOR is an enterprise tier requiring direct contact rather than an automated payment link.
- **Normal case:**
 - User opens /plans -> System loads plans and subscription data in parallel.
 - ActivityIndicator displays during data fetching; pricing cards and comparison tables render upon success.
 - User toggles billing cycle -> prices update immediately using cached data.
 - User taps a new plan -> relevant upgrade/downgrade modal opens.
- **Abnormal case:**
 - Plans fetch fails -> Screen renders "Không thể tải thông tin gói cước" with a retry button.
 - Subscription fetch returns 401 -> User is treated as unauthenticated; CTAs show "Chọn" and prompt sign-in when tapped.
 - Plans list is empty -> Placeholder "Không có gói cước nào" is displayed.
 - Network offline during refresh -> Refresh indicator stops and the screen continues to show the last cached data.
 - Missing tierCode -> Card defaults to "Chọn" or uses a case-insensitive tier fallback.

3.36 Upgrade Subscription

Function trigger:

- **Navigation path:** Plans Screen (/plans) -> tap a non-current PricingCard CTA -> handleActionPress in PricingPlansList.tsx -> routes to either DurationSelectionModal (upgrade) or DowngradeConfirmDialog (downgrade).

- **Timing Frequency:** On demand, triggered when the user taps "Nâng cấp" or "Hạ cấp". The payment screen polls GET `/api/v1/payment/status/{orderCode}` until a terminal state is reached.

Function description:

- **Actors/Roles:** Authenticated Citizen (guests are redirected to sign-in).
- **Purpose:** To allow users to upgrade to a higher tier (PREMIUM / MONITOR via PayOS) or downgrade back to the FREE tier (immediate effect).
- **Interface:**
 - Upgrade Modal (DurationSelectionModal.tsx): Slide-up modal displaying plan name, price per month, and radio options for 1, 3, 6, or 12 months with discount badges.
 - Downgrade Dialog (DowngradeConfirmDialog.tsx): Center modal with a warning icon and a confirmation button to move to the Free plan.
 - Payment Processing Screen (/payment/processing): Opens the PayOS paymentUrl via WebView/system browser and polls for status.
 - Result Screens: /payment/success and /payment/cancel screens entered via deep-links (fda-mobile://payment/success or cancel).

Data processing:

- **Upgrade Path:**
 - User selects durationMonths {1, 3, 6, 12}.
 - handleUpgradeConfirm calls POST `/api/v1/payment/create` with plan details and return/cancel URLs.
 - On success, the app routes to the processing screen with the paymentUrl and orderCode.
 - Upon a paid status, the subscription cache (plansSubscriptionCurrentQueryKey) is invalidated to reflect the new tier.
- **Downgrade Path:**
 - handleDowngradeConfirm calls POST `/api/v1/plan/subscription/subscribe` with { planCode: "FREE", durationMonths: 12 }.
 - On success, the subscription query is invalidated and a success alert is shown.
- Mutation loading states (paymentLoading, downgradeLoading) are used to disable CTAs and prevent double-submission.

Screen layout:

<

Bảng giá

Số vùng theo dõi	5	Không giới hạn
Kênh thông báo	Push, Email	Push, Email, SMS
Dự báo AI	Có	Có
Tìm đường an toàn	Có	Có

Chọn thời hạn

X

Gói nâng cao — 45.000đ/tháng

☒

1 tháng

45.000đ

☐

3 tháng

Giảm 5%135.000đ

☐

6 tháng

Giảm 10%270.000đ

☐

12 tháng

Tiết kiệm nhấtGiảm 20%540.000đ

Tổng thanh toán

45.000đ

Tiếp tục →

Figure 94 Upgrade Subscription

157 | Page

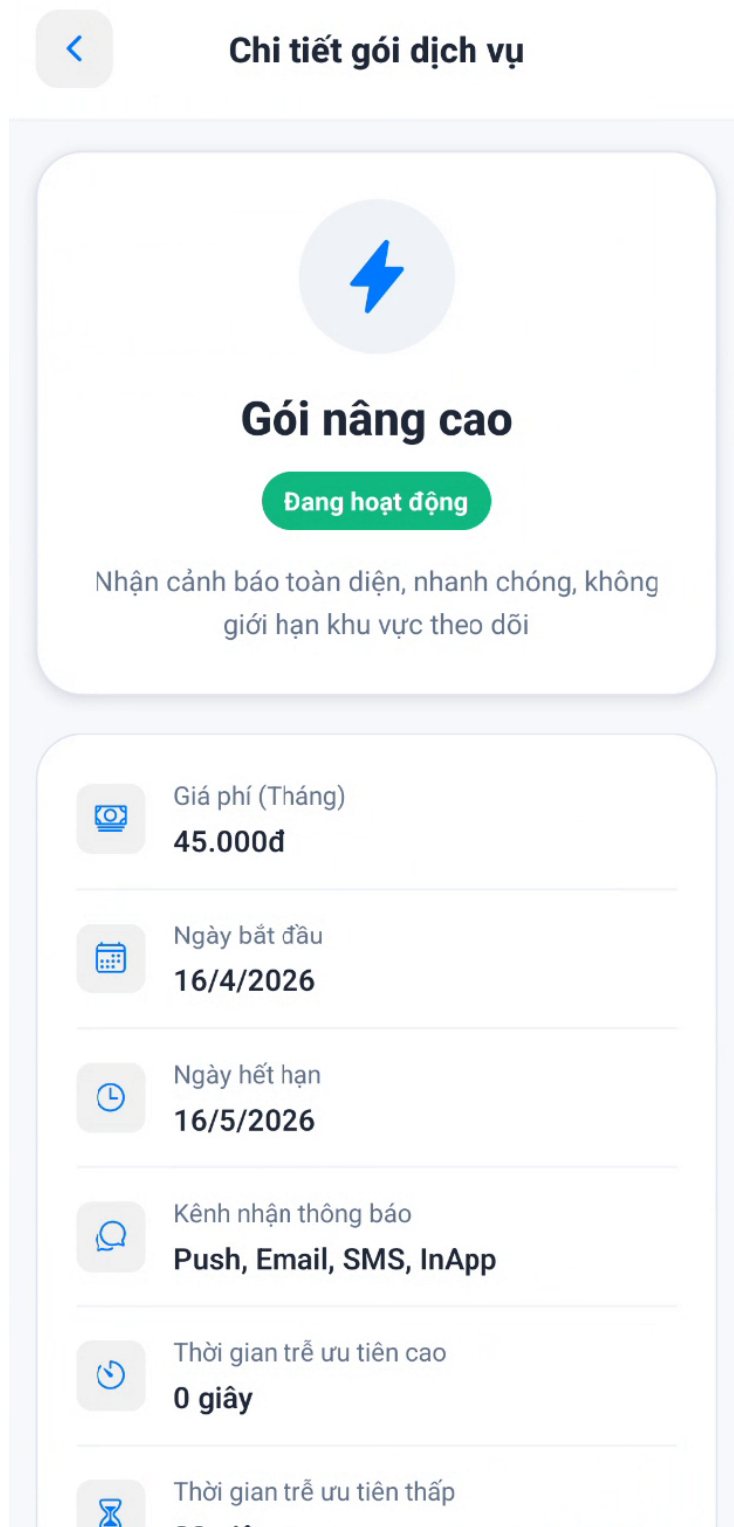


Figure 95 Current Subscription

Function details:

- **Data:**
 - Upgrade Request: planCode (PREMIUM | MONITOR), durationMonths, returnUrl, cancelUrl.

- Status Polling: SubscriptionPaymentDto with status { pending | paid | cancelled | failed }.
- Downgrade Request: { planCode: "FREE", durationMonths: 12 } (duration is hardcoded for the endpoint contract).
- **Validation:**
 - Plan Code: Upgrade is strictly restricted to PREMIUM or MONITOR.
 - Duration: Must be exactly 1, 3, 6, or 12 months.
 - Same-tier clicks: If the user taps their current plan, the action returns early and no modal opens.
 - Authentication: Guest users are blocked and redirected to sign-in.
- **Business rules:**
 - Upgrades always require a successful PayOS payment; tiers are not flipped optimistically on the client.
 - Downgrades to the FREE tier are free of charge and take effect immediately upon server confirmation.
 - MONITOR is handled as an enterprise tier where the CTA shows "Liên hệ" and triggers a contact information alert instead of the payment flow.
 - The subscription cache is invalidated on every successful tier change to ensure UI consistency.
- **Normal case (upgrade):**
 - The user taps "Nâng cấp" on PREMIUM -> selects "12 tháng" -> POST /api/v1/payment/create returns paymentUrl.
 - PayOS completes -> app receives success deep-link -> poller confirms paid -> user sees PREMIUM as "Đang dùng".
- **Normal case (downgrade):**
 - User taps "Hạ cấp" on FREE -> DowngradeConfirmDialog opens -> User confirms.
 - POST /api/v1/plan/subscription/subscribe returns success -> cache invalidated -> "Hạ cấp thành công" alert appears.
- **Abnormal case:**
 - Payment creation fails: An alert notifies the user, the modal stays open, and the loading state is cleared for retry.
 - User cancels PayOS: Deep-link routes to /payment/cancel and the UI reflects "Thanh toán đã bị hủy".
 - Payment failure/expiry: Status maps to failed and the user is prompted to re-initiate the flow from the Plans screen.
 - Polling timeout: State becomes timeout and UI reflects that confirmation is still processing asynchronously on the server.
 - Downgrade fails: Error is logged and a generic "Vui lòng thử lại sau" alert is shown.

3.37 Cancel Subscription

Function trigger:

- **Navigation path:** Profile Tab "Gói hiện tại" route /plans/current tap "Hủy gói dịch vụ" (only for non-FREE tiers) CancelSubscriptionDialog slides up.
- **Timing Frequency:** On demand; fires once per user confirmation and takes effect immediately on the server side.

Function description:

- **Actors/Roles:** Authenticated Citizen (PREMIUM or MONITOR tiers only; FREE users do not see this option).
- **Purpose:** To allow citizens to voluntarily cancel their active paid subscription, collect optional feedback, and move back to the FREE tier.
- **Interface:**
 - Current Plan Screen: Displays a hero card with plan status (Active/Free/Cancelled/Expired), description, and detailed info rows for dates and channel limits.
 - Cancel Confirmation Sheet: Bottom sheet containing a warning header, a "Loss list" of specific tier benefits, a 200-character reason textarea, and action buttons.
 - Success feedback: Native Alert confirming the transition to the Free plan, followed by a redirect to the main Plans screen.

Data processing:

- The system conditionally renders the cancel button based on whether the tierCode is not "FREE".
- handleCancelSubscription triggers the DELETE /api/v1/plan/subscription/cancel request with the optional cancelReason.
- Upon success, the plansSubscriptionCurrentQueryKey is invalidated to ensure all screens reflect the new subscription state.
- The loading state (isCancelling) disables UI controls and displays an ActivityIndicator to prevent double-submissions or accidental closure.
- The cancellation is irreversible on the client; returning to a paid tier requires a new upgrade flow.

Screen layout:



Chi tiết gói dịch vụ

Nhận cảnh báo toàn diện, nhanh chóng, không
giới hạn khu vực theo dõi



Giá phí (Tháng)

45.000đ



Ngày bắt đầu

16/4/2026



Ngày hết hạn

16/5/2026



Kênh nhận thông báo

Push, Email, SMS, InApp



Thời gian trễ ưu tiên cao

0 giây



Thời gian trễ ưu tiên thấp

20 giây



Số lần gửi lại tối đa

3 lần



Hủy gói dịch vụ

Figure 96 Cancel Subscription

Function details:

- **Data:**
 - Request: CancelSubscriptionRequest containing an optional cancelReason string.
 - Response: CancelSubscriptionResponse containing success status and CancelledSubscriptionData (ID, plan name, previous tier, timestamp, reason).
 - HTTP: DELETE method using JSON body delivered via Axios data options.
- **Validation:**
 - Reason length: Client-side cap at 200 characters via TextInput maxLength.
 - Tier guard: The cancel button is hidden for FREE users to prevent invalid or no-op API calls.
 - Subscription existence: The flow returns early if no active subscription data is found.
- **Business rules:**
 - Only authenticated citizens with an active PAID subscription (PREMIUM/MONITOR) may perform a cancellation.
 - Cancellation takes effect immediately, resulting in the immediate loss of paid benefits such as high-priority SMS and 0-second dispatch delays.
 - A successful cancellation always forces a re-fetch of the authoritative subscription state.
 - The cancelReason is optional and utilized strictly for backend analytics and retention insights.
- **Normal case:**
 - User on PREMIUM opens current plan details taps "Hủy gói dịch vụ" enters reason confirms.
 - DELETE request returns success subscription cache is invalidated "Hủy thành công" alert is shown.
 - User taps OK app replaces the current route with /plans, where FREE is now marked as "Đang dùng".
- **Abnormal case:**
 - API returns success: false Alert displays the server error message and the dialog remains open for retry.
 - 401 Unauthorized Handled by the auth interceptor to redirect the user to the sign-in screen.
 - Network/500 error Caught by a try/catch block which triggers a "Vui lòng thử lại" alert.
 - User on FREE via deep-link Logic short-circuits and no API call is made.

3.38 View Subscription Status & Billing History

Function trigger:

- **Navigation path: - Subscription status:**
 - Profile Tab -> "Gói hiện tại" -> route /plans/current (app/plans/current.tsx).
 - Billing history: Profile Tab -> "Lịch sử thanh toán" -> route /billing (app/billing/index.tsx).
- **Timing Frequency:** On demand. Initial fetch occurs on screen mount; React Query refetch is triggered by pull-to-refresh, page changes, or cache invalidation from successful upgrade, downgrade, or cancellation flows.

Function description:

- **Actors/Roles:** Authenticated Citizen (401 responses redirect to sign-in or indicate "no subscription").
- **Purpose:** Provide a single source of truth for the citizen's subscription state and complete transaction history to verify charges and manage plan decisions.
- **Interface:**
 - Current Plan Screen (/plans/current): Displays a hero card with tier icons (cloud/flash/shield), plan name, status badges (Active, Free, Cancelled, Expired), and detailed fields via InfoRow (price, dates, channels, delays, retries).
 - Billing History Screen (/billing): Features an animated collapsing header, a status filter pill row (All, Success, Pending, Cancelled), and a paginated list of BillingRow items showing order codes and VND amounts.
 - Profile Preview: A compact entry card on the Profile screen labeled "Xem lịch sử thanh toán" that navigates to the billing list.
 - States: Includes ActivityIndicator for loading, error cards with retry buttons, and an EmptyState with a CTA to explore plans.

Data processing:

- **Subscription Status:** useCurrentSubscription fetches GET /api/v1/plan/subscription/current; labels and colors are dynamically derived from the status and tier code.
- **Billing History:** useQuery calls GET /api/v1/payment/history?page={page}&pageSize=10. Pagination is server-side, while filtering (All/Paid/Pending/Cancelled) is performed client-side on the currently loaded page.
- **Formatting:** Dates are formatted to vi-VN locale (e.g., toLocaleDateString); missing end dates render as "Vô thời hạn".
- **Cache Invalidation:** Actions in FE-48 and FE-49 automatically invalidate the current subscription query key to ensure data remains authoritative.

Screen layout:

Bảng giá

Có hàng đổi	5	Không giới hạn
Kênh thông báo	Push, Email	Push, Email, SMS
Dự báo AI	Có	Có
Tìm đường an toàn	Có	Có
Báo cáo cộng đồng	Có	Có (Uy tín cao)

Tháng

Năm

Miễn phí

Gói cơ bản

Hạ cấp

⚡

Phổ biến

Đang dùng

45.000 /tháng

Gói nâng cao

Đang dùng

Miễn phí

Enterprise Plan

Liên hệ

Figure 97 View Subscription Status

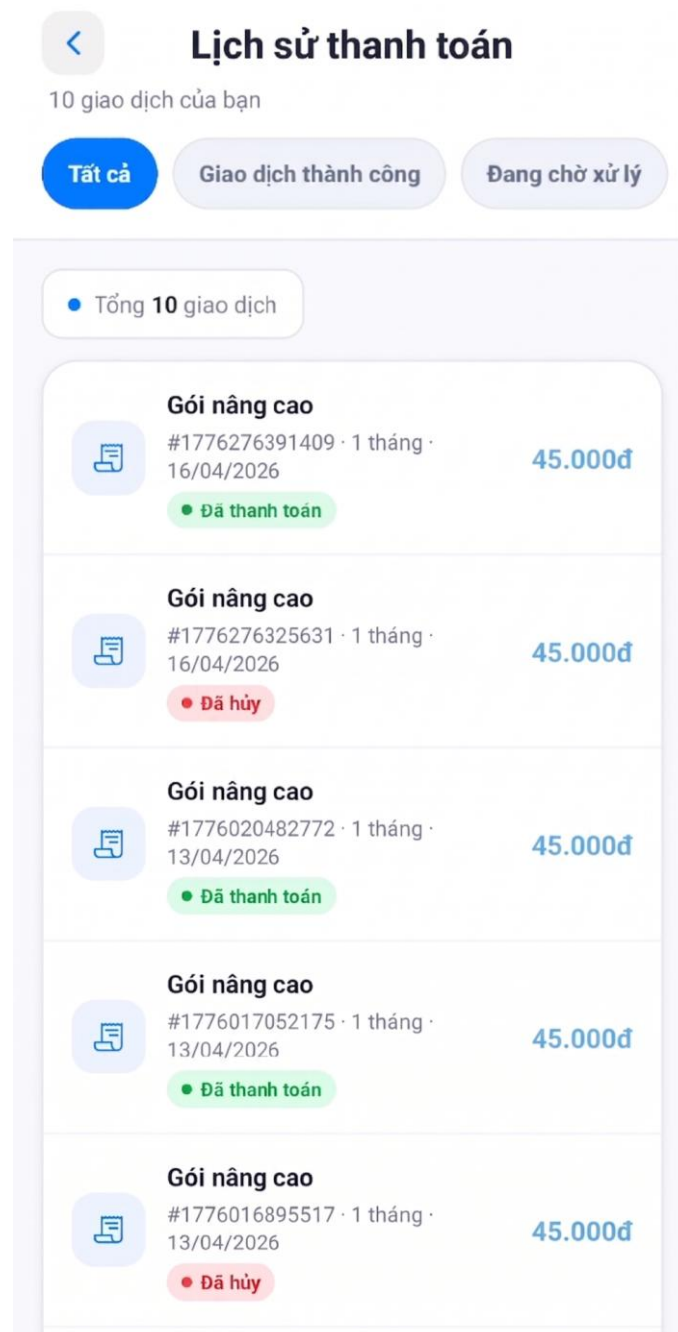


Figure 98 View Billing History

Function details:

- **Data:**
 - Subscription Response: UserSubscription containing tier, status ({active | cancelled | expired | free}), pricing, and technical limits (dispatch delays, max retries).
 - History Response: PaymentRecord objects containing orderCode, amount, currency (VND), status ({paid | pending | cancelled}), and durationMonths.
- **Validation:**
 - Status enum: Unknown values fallback to default styling/labels via getStatusLabel.
 - Dates: Must be ISO-8601; invalid strings default to "Invalid Date" without crashing the UI.

- Pagination: Page number is clamped ≥ 1 and $\leq$ totalPages; navigation buttons are disabled at boundaries.
- **Business rules:**
 - Only authenticated users can access these screens; billing history is private to the specific user.
 - Payment records are strictly read-only; they cannot be edited or deleted by the user.
 - Paid-tier benefits (channels, retries) are driven entirely by the API response rather than hardcoded client logic.
 - A cancelled subscription retains a Cancelled status until the backend explicitly transitions it to the FREE tier.
- **Normal case:**
 - User opens /plans/current -> Hero card and InfoRow list render after the loading state clears.
 - User navigates to /billing -> First 10 records are displayed with an animated header.
 - User toggles the "Đã hủy" filter -> Current page rows narrow to matching records and the summary count updates.
 - User taps "Tiếp" -> page increments, a new API call fetches page 2, and the list updates.
- **Abnormal case:**
 - Fetch error: Screen displays a "Không thể tải" message with a retry button to trigger refetch().
 - Empty history: EmptyState renders with a CTA to "Khám phá các gói dịch vụ".
 - Filter mismatch: If a filter yields no matches on the current page, a "Không khớp kết quả" view appears with a reset button.
 - Server Enum Mismatch: If an unknown status is returned, the UI defaults to pending styling (amber dot) to maintain rendering.
 - 401 Unauthorized: Handled by the shared Axios interceptor for refresh or redirection.

3.39 Create Subscription Plans

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Gói Đăng Ký ("/admin/plan-subscriptions") > Button "Tạo Gói"
- **Timing Frequency:** On demand (Admin-triggered).
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: Create a new pricing plan to configure subscription tiers, pricing, and included features. The UI uses Vietnamese page title "Gói Đăng Ký" and create dialog "Tạo gói mới".
- **Interface:**
 - **List View:** Page header: "Gói Đăng Ký" — "Cấu hình gói, mức giá và tính năng đi kèm."
 - **Actions:**
 - Click "Tạo Gói" to open the create dialog.
 - Optional: toggle "Ẩn Inactive" / "Hiện Inactive" and search plans (affects list only).

Data processing:

- The user clicks "Tạo Gói" → System opens CreatePlanDialog. → The user fills in the form and clicks "Tạo Gói". → FE validates inputs (e.g. code format, required fields). → FE builds

"CreatePlanPayload" and calls "planSubscriptionApi.createPlan(payload, token)". → On success: invalidate query "[plans]", show toast "Tạo gói thành công!", close dialog and refresh list.

- Create/Update: Create via "CreatePlanDialog".
- Fields (UI labels): "Code", "Plan Name", "Description", "Price / Month (VND)", "Price / Year (VND)", "Tier", "Sort Order", "Features".
- Feature editor buttons: "Thêm Feature" and per-row delete.
- Footer buttons: "Hủy", "Tạo Gói" (loading text: "Creating...").
- Delete: N/A.

Screen layout:

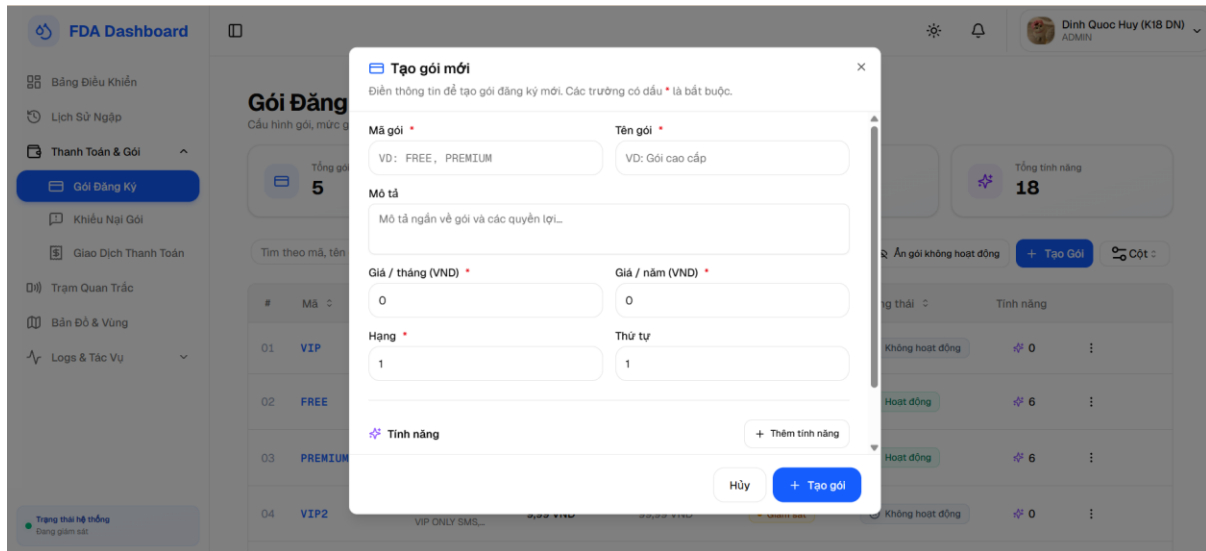


Figure 99 Create Subscription Plans

Function details:

- **Data:**
 - Types/payloads: "CreatePlanPayload", "FeatureInput", "PricingPlan".
 - Endpoint: "POST {API_BASE_URL}/admin/plans".
 - Auth: "Authorization: Bearer {accessToken}" from "getAccessToken()".
- **Validation:**
 - "code" is required and must match "[A-Z0-9_]+\$".
 - "name" is required.
 - "priceMonth", "priceYear" must be ≥ 0 .
 - "featureKey" is required for each feature row.
 - (Validation messages are currently English in UI for several fields: e.g. "Code is required", "Name is required".)
- **Business Rules:**
 - Codes are stored as uppercase.
 - Empty features are filtered out before submit.
 - On success, the plans list is refreshed by invalidating "[plans]".
- **Normal Case:** Admin opens "Gói Đăng Ký", clicks "Tạo Gói", enters plan data and features, then submits. System creates the plan and shows "Tạo gói thành công!".
- **Abnormal Case:**

- Missing/invalid form data: system blocks submit and shows field errors.
- Authentication missing/expired: FE throws "Cần đăng nhập. Vui lòng đăng nhập lại".
- API error: show toast "Tạo gói thất bại" with server message.

3.40 View Subscription Plans for Web

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Gói Đăng Ký ("/admin/plan-subscriptions")
- **Timing Frequency:** On demand (page view).
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: View and manage the list of subscription pricing plans, including status (Active/Inactive), tier labels, features count, and pricing.

Interface:

- **Overview cards:** "Total Plans", "Active Plans", "Inactive Plans", "Total Features" (English labels).
- **Table controls:**
 - Search input for column "Plan Name" (placeholder: "Tìm kiếm code, plan name ...").
 - Toggle button: "Ẩn Inactive" / "Hiện Inactive".
 - Create button: "Tạo Gói".
- **Table columns:** #, "Code", "Plan Name", "Price / Month", "Price / Year", "Tier" (labels: "Miễn phí", "Cơ bản", "Cao cấp", "Giám sát"), "Status" (Active/Inactive), "Features", "Order", "Actions".

Data processing:

- 1) On page load, FE attempts "planSubscriptionApi.getPlans(token)".
 - 2) If response shape mismatches or call fails, FE falls back to "MOCK_PLANS".
 - 3) FE sorts plans by "sortOrder" and renders overview + table.
- **Create/Update:** N/A (this use case is view-only; create/update are covered in 3.43/3.45).
 - **Delete:** N/A (deactivation covered in 3.46).

Screen layout:

Gói Đăng Ký
Cấu hình gói, mức giá và tính năng đi kèm.

Overview Cards:

- Tổng gói: 5
- Gói đang hoạt động: 3
- Gói không hoạt động: 2
- Tổng tính năng: 18

Search: Tìm theo mã, tên gói...

Controls: Ẩn gói không hoạt động, + Tạo Gói, Cột

#	Mã	Tên gói	Giá / tháng	Giá / năm	Hạng	Trạng thái	Tính năng
01	VIP	VIP2026	Miễn phí	Miễn phí	Cao cấp	Không hoạt động	0
02	FREE	Gói cơ bản Sử dụng các chức...	Miễn phí	Miễn phí	Miễn phí	Hoạt động	6
03	PREMIUM	Gói nâng cao Nhận cảnh báo...	45.000 VND	195.000 VND	Cao cấp	Hoạt động	6
04	VIP2	Vip Plan2 VIP ONLY SMS...	9,99 VND	99,99 VND	Giám sát	Không hoạt động	0
05	ENTERPRISE	Enterprise Plan Giải pháp doanh...	Miễn phí	Miễn phí	Giám sát	Hoạt động	6

Figure 100 View Subscription Plans for web

Function details:

- **Data:**
 - Type: "PricingPlan" with fields "id", "code", "name", "description", "priceMonth", "priceYear", "tier", "isActive", "sortOrder", "features".
 - Endpoint: "GET {API_BASE_URL}/admin/plans".
- **Validation:**
 - Token required for real API call; otherwise page may show mocked data.
 - Search and show/hide inactive are client-side filters.
- **Business Rules:**
 - "priceMonth == 0" displays as "Miễn phí".
 - "showInactive=false" filters plans where "isActive=true".
- **Normal Case:** Admin opens the page and sees plans list, can search by code/name, and can hide inactive plans via "Ẩn Inactive".
- **Abnormal Case:**
 - API call fails: FE shows fallback mock data.
 - Authentication missing: FE logs error and falls back (depending on token availability).

3.41 Update Subscription Plans

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Gói Đăng Ký ("/admin/plan-subscriptions") > Row "Thao tác" > "Sửa gói"
- **Timing Frequency:** On demand.
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: Edit an existing subscription plan's name, pricing, tier, activation status, sort order, and features. Plan code is read-only in UI ("Mã gói").
- **Interface:**
 - **Actions:**
 - Open row menu "Thao tác" → click "Sửa gói".
 - Edit fields and click "Save Changes".
 - Toggle activation status via switch showing "Active" / "Inactive".
 - **Edit dialog:** Dialog title: "Sửa gói".
 - Description shows plan code badge and note "(mã gói chỉ đọc)".
 - Footer buttons: "Cancel", "Save Changes" (loading: "Saving...").

Data processing:

- FE opens "EditPlanDialog" with selected "PricingPlan".
- FE pre-fills form state from "plan".
- On submit, FE validates and builds "UpdatePlanPayload".
- FE calls "planSubscriptionApi.updatePlan(plan.id, payload, token)".
- On success: invalidate "['plans']", toast "Cập nhật gói thành công!", close dialog.

Screen layout:

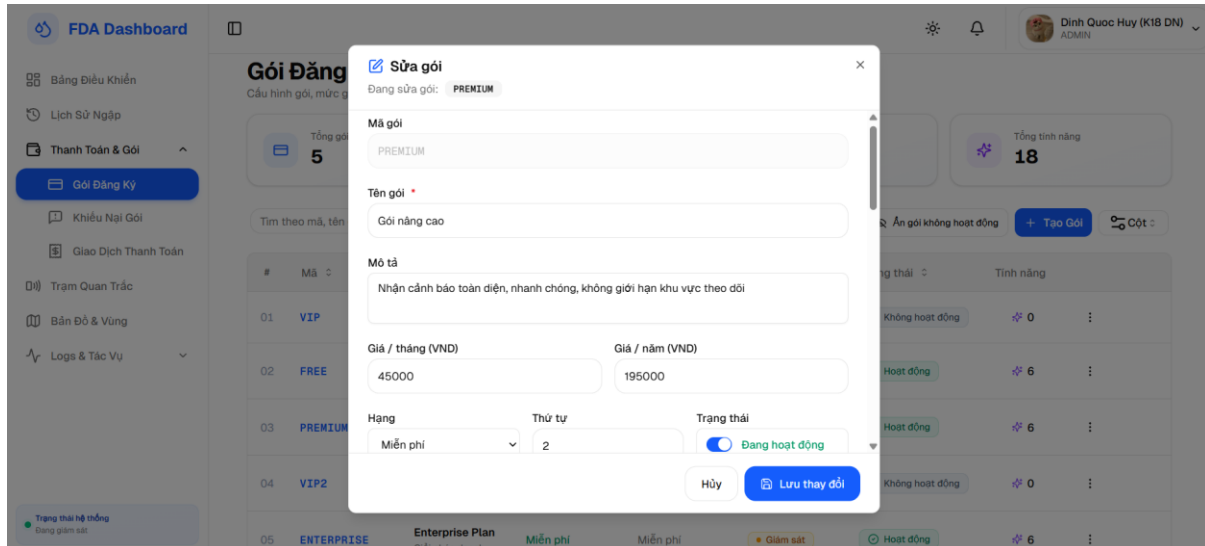


Figure 101 Update Subscription Plans

Function details:

- **Data:**
 - Endpoint: "PUT {API_BASE_URL}/admin/plans/{id}".
 - Auth: Bearer token from "getAccessToken()".
- **Validation:**
 - "name" required.
 - "priceMonth", "priceYear" ≥ 0 .
 - Each feature row must have "featureKey".
 - (Validation messages are English in UI.)
- **Business Rules:**
 - Plan "code" cannot be edited.
 - Empty feature rows are filtered out before submit.
 - Changes reflect after invalidating "[plans]".
- **Normal Case:** Admin edits a plan and saves. System updates and shows toast "Cập nhật gói thành công!".
- **Abnormal Case:**
 - Validation fails: show errors, block submit.
 - API error: show toast "Cập nhật gói thất bại".
 - Authentication missing: error "Cần đăng nhập. Vui lòng đăng nhập lại".

3.42 Delete Subscription Plan

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Gói Đăng Ký ("/admin/plan-subscriptions") > Row "Thao tác" > "Ngừng kích hoạt"
- **Timing Frequency:** On demand.
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: Remove a plan from being available to users by deactivating it (soft delete). The UI wording is "Ngừng kích hoạt", not a permanent delete.
- **Interface:**

- **Actions:**
 - Open row menu "Thao tác".
 - Click "Ngừng kích hoạt" (disabled when already inactive, label becomes "Đã ngừng kích hoạt").
 - Confirm in dialog "Ngừng kích hoạt gói".
- **Confirmation dialog:** Title: "Ngừng kích hoạt gói".
 - Description: "Bạn có chắc muốn ngừng kích hoạt {plan.name}?".
 - Warning note: "Việc này sẽ ẩn gói khỏi người dùng. Người đang đăng ký sẽ không bị ảnh hưởng."
 - Buttons: "Hủy", "Ngừng kích hoạt" (loading: "Đang ngừng kích hoạt...").

Data processing:

- Admin clicks "Ngừng kích hoạt".
- FE calls "planSubscriptionApi.deactivatePlan(plan.id, token)".
- On success: invalidate "[plans]", toast success, and close dialog.

Screen layout:

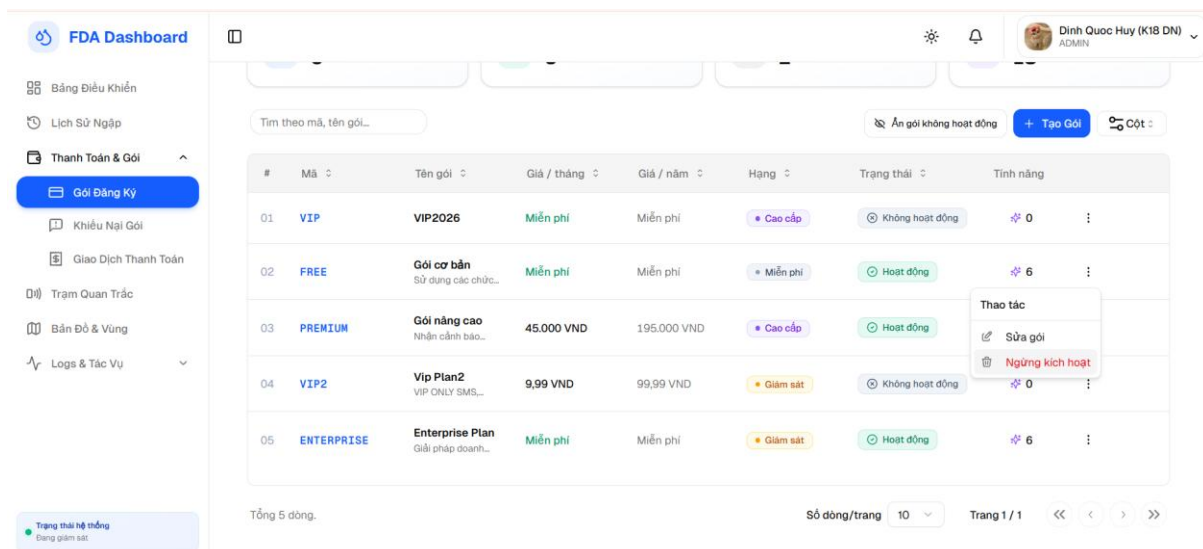


Figure 102 Delete Subscription Plans

Function details:

- **Data:**
 - Endpoint: "DELETE {API_BASE_URL}/admin/plans/{id}".
 - Auth: Bearer token from "getAccessToken()".
- **Validation:**
 - Deactivate action is disabled when "data.isActive" is false.
 - Token required.
- **Business Rules:** Deactivation hides the plan from end users but does not impact existing subscriptions (per UI warning).
- **Normal Case:** Admin deactivates an active plan; the plan becomes inactive and disappears when "Ẩn Inactive" is enabled.

- **Abnormal Case:**

- API failure: show toast "Ngừng kích hoạt gói thất bại".
- Authentication missing: error "Cần đăng nhập. Vui lòng đăng nhập lại."

3.43 Handle Subscription Disputes

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Khiếu Nại Gói ("/admin/subscription-disputes")
- **Timing Frequency:** On demand.
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: Review and resolve subscription complaints (khiếu nại). Page title "Khiếu Nại Gói" with description "Xem xét và xử lý khiếu nại liên quan đến gói đăng ký."
- **Interface:**
 - **Table controls:**
 - Search by "Subject" (placeholder: "Tìm kiếm subject...").
 - Filter by "Status" (Open/Resolved/Rejected).
 - **Actions:**
 - Click complaint "Subject" to view details.
 - If the status is "open", click "Resolve" to open the resolve dialog.
 - If not open, the action shows "Resolved" text.
 - **Resolve dialog:**
 - Dialog title: "Resolve Subscription Complaint".
 - Fields: "Admin Response" (textarea, required), "New Status" (radio: "Resolve" or "Reject").
 - Buttons: "Cancel", "Submit" (loading: "Submitting...").

Data processing:

- FE loads complaints via "subscriptionDisputeApi.getAdminComplaints({page, pageSize, status}, token)".
- Admin opens "ResolveDisputeDialog" for a complaint.
- Admin submits "ResolveComplaintPayload {adminResponse, newStatus }".
- FE calls "subscriptionDisputeApi.resolveComplaint(complaint.id, payload, token)".
- On success: invalidate query "[subscription-disputes]", toast success, close dialog.

Screen layout:

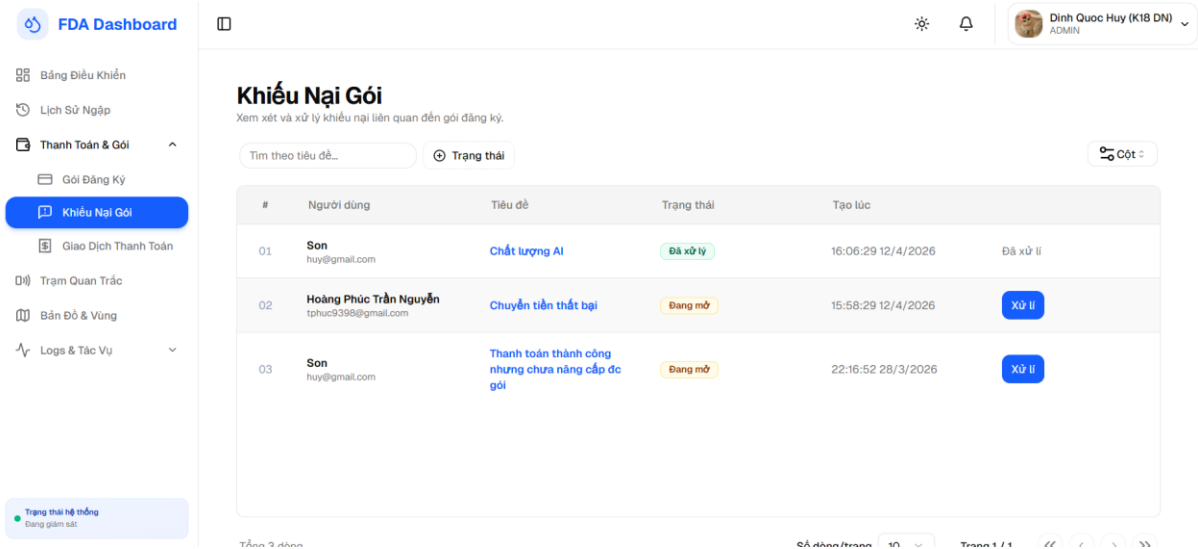


Figure 103 Handle Subscription Disputes

Function details:

- **Data:**

- Types: "AdminComplaint", "ResolveComplaintPayload", "ResolveNewStatus".
- Endpoints (by type docs):
 - "GET /api/v1/admin/complaints" (list)
 - "PUT /api/v1/admin/complaints/{id}/resolve" (resolve)

- **Validation:**

- "Admin Response" must be non-empty (submit disabled until trimmed content exists).
- Token required.

- **Business Rules:**

- Only "open" complaints can be resolved from the UI.
- New status can be "resolved" or "rejected".

- **Normal Case:** Admin filters "Open" complaints, clicks "Resolve", writes "Admin Response", selects status and submits. Complaint becomes "Resolved" or "Rejected".

- **Abnormal Case:**

- API error: toast "Failed to resolve complaint" and dialog shows error message.
- Authentication missing: error "Authentication required. Please log in again."

3.44 View Billing & Payment Records

Function trigger:

- **Navigation path:** Admin Portal > Thanh Toán & Gói > Giao Dịch Thanh Toán ("/admin/billing-payment")
- **Timing Frequency:** On demand.
- **Function description:** Actors/Roles: ADMIN, SUPERADMIN. Purpose: View billing/payment history and open a read-only record details dialog. Page title "Giao Dịch Thanh Toán".
- **Interface:**
 - **Overview cards:** "Total Transactions", "Total Revenue", "Pending", "Cancelled" (English labels).

- **Table controls:**
 - Search input for "Order Code" filter (placeholder: "Tìm kiếm order code, user, plan...").
 - Select filter "Status" with options: Paid / Pending / Cancelled.
- **Table columns:** #, "Order Code", "User", "Plan", "Amount", "Duration", "Status", "Payment Date".
- **Details dialog:** Click an "Order Code" value to open "Payment Details" dialog (read-only). Footer button: "Close".

Data processing:

- FE fetches via "billingPaymentApi.getAdminPayments({page, pageSize, status}, token)".
- FE applies client-side filtering for search query across orderCode/user/plan.
- Clicking a row order code opens "PaymentDetailsDialog" populated with "AdminPaymentRecord".

Screen layout:

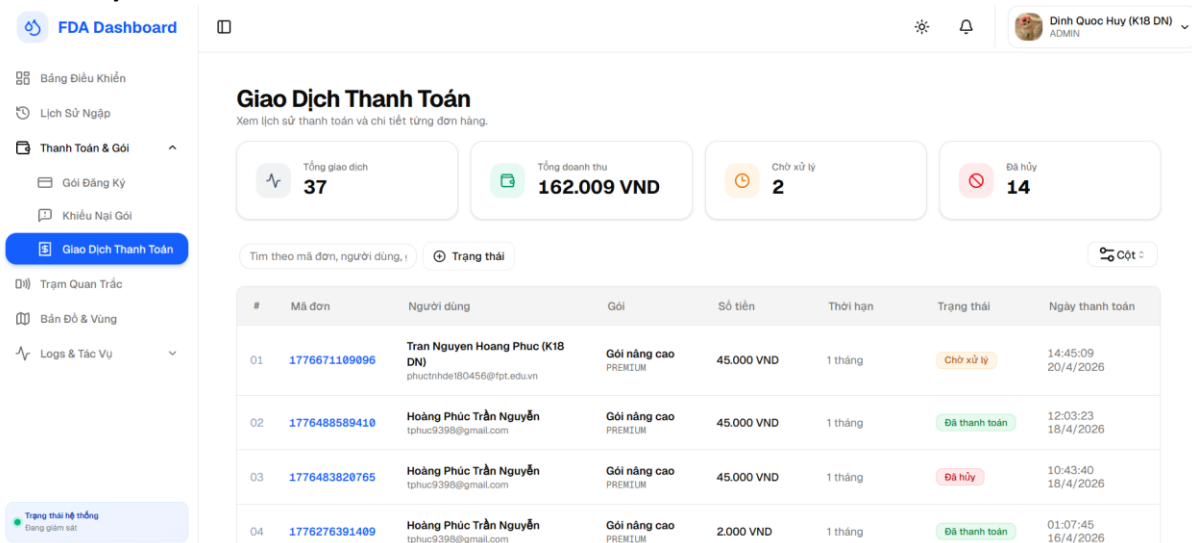


Figure 104 View Billing and Payment Records

Function details:

- **Data:**
 - Type: "AdminPaymentRecord" (fields include "orderCode", "planName", "planCode", "amount", "currency", "paymentMethod", "status", "durationMonths", "userFullName", "userEmail", timestamps).
 - Endpoint: "GET /api/v1/admin/payments" (via FE client).
- **Validation:**
 - Token required.
 - Filters are applied by query parameters (status) and client-side search (orderCode/user/plan).
- **Business Rules:**
 - Clicking "Order Code" is the primary way to open details.
 - Records are read-only in the details dialog.

- **Normal Case:** Admin opens the page, filters by "Status", searches by order code, then opens "Payment Details" to review a transaction.
- **Abnormal Case:**
 - API error: UI shows "Retry" action.
 - No results: shows "No payment records found." or "No payments match the selected filter."

3.45 Publish NewsFeed

Function trigger:

- **Navigation path:** Admin Web Portal -> Sidebar "News" / "Announcements" -> "Publish new article" button -> Publish form modal / dedicated /admin/news/new route.
- **Timing Frequency:** On demand; triggered when a Moderator decides to broadcast an official flood-related announcement (e.g., typhoon warning, evacuation order) to all citizens.

Function description:

- **Actors/Roles:** Moderator (authenticated admin user with moderator/admin scope); Guest and Citizen roles are blocked at the route-guard and API level.
- **Purpose:** Enables authorized Moderators to compose and publish official announcements to the newsfeed and trigger FCM push notifications for high or urgent priority items.
- **Interface:**
 - Publish Form: Includes a rich-text editor for HTML content, title, summary, and optional image/attachment fields.
 - Priority Selector: Dropdown restricted to low | normal | high | urgent with corresponding color chips.
 - Schedule / Publish-now toggle: Uses an ISO-8601 datetime picker for scheduled posts or a "publish immediately" default.
 - Preview Panel: Side-by-side preview rendering HTML using the same component as the mobile app.
 - Primary actions: Save Draft, Publish, and Cancel.
 - Post-publish confirmation: Toast message and redirect to the article's admin detail view showing live stats (delivery, read, and view counts).

Data processing:

- On submission, the portal issues a POST /api/v1/announcements request with the article payload.
- The backend persists the AnnouncementDetail record with server-assigned fields: id, authorName, createdAt, and interaction counters initialized to zero.
- For high or urgent priority, the backend initiates an FCM push notification fan-out to all subscribed devices.
- The record is served to the mobile app via GET /api/v1/announcements (list) and GET /api/v1/announcements/{id} (detail).
- Attachments are stored as a JSON string of URLs, which the mobile app parses into a tappable list.
- Edits or deletions are performed via PUT or DELETE on the resource, with changes appearing on the next mobile list refetch.

Screen layout:

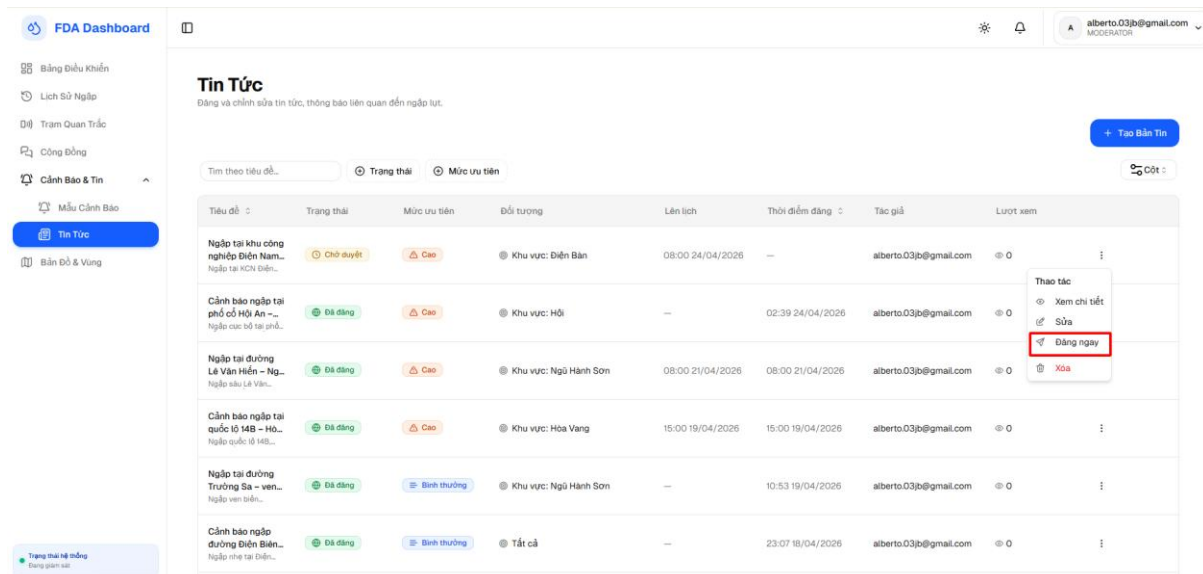


Figure 105 Publish NewsFeed

Function details:

- **Data:**
 - Request: Required fields include title, summary, content, and priority; optional fields include imageUrl, attachments, and publishedAt.
 - Response: Newly created AnnouncementDetail object containing the GUID, author name, and timestamps.
- **Validation:**
 - Required Fields: Title, summary, and content must be non-empty; backend enforces length bounds.
 - Content: HTML is sanitized server-side to strip unsafe scripts or attributes.
 - Priority: Must match one of the four enum values or the request is rejected as a 400 Bad Request.
 - Authentication: Requires a Moderator JWT; citizen tokens receive a 403 Forbidden response.
- **Business rules:**
 - Only users with the Moderator role may create, edit, or delete announcements.
 - Announcements are visible to all citizens immediately after publishing (or at the scheduled time).
 - High/Urgent priority triggers push notifications; lower priorities are silent.
 - Interaction counts (viewCount, readCount, etc.) are server-incremented and cannot be modified by clients.
- **Normal case:** Moderator fills out the form with "urgent" priority -> Clicks Publish -> POST request succeeds -> Backend dispatches FCM pushes -> Article appears at the top of the mobile newsfeed with an "URGENT" badge.
- **Abnormal case:**
 - Validation fails (400): Portal displays field-level errors and the form is preserved for correction.
 - Auth expired (401): Portal redirects to the admin sign-in page; the draft is saved in local storage.

- Forbidden (403): Non-moderator sessions are blocked from publishing.
- Server error (500): Shows a retry button; no partial record is created due to the transactional nature of the publish action.
- Malformed Attachments: Pre-validated by the portal; if passed, the mobile app silently drops malformed attachment rows during parsing.

3.46 View Newsfeed

Function trigger:

- **Navigation path:** Bottom Tab Bar -> "Notifications" tab -> NotificationTabToggle -> "News" segment (activeTab === "news") -> Tapping a news item navigates to /notifications/news/[id].
- **Timing Frequency:** On demand. Initial load occurs when the Notifications tab "News" segment is mounted; refetches occur on pull-to-refresh, pagination changes, or when the cache is invalidated by useMarkNewsRead / useMarkAllNewsRead mutations.

Function description:

- **Actors/Roles:** Guest, Citizen, Moderator. Announcement listing is public, but marking items as read requires an authenticated session.
- **Purpose:** To display the official flood-related announcement stream, allowing users to triage messages by priority (LOW to URGENT), view full HTML articles, and manage unread statuses.
- **Interface:**
 - **Tab Toggle:** Segmented control to switch between "Alerts" and "News".
 - **Header:** Includes an unread-count chip (dot indicator) and a filter button slot.
 - **Mark-All-Read Button:** Visible only when unread news exists; disabled during pending mutations.
 - **Card List (NewsCard.tsx):** Displays priority badges (colored pills), CHƯA ĐQC (Unread) chips, thumbnails, truncated titles/summaries, and meta rows (view count and relative time).
 - **Pagination Control:** Footer with {currentPage}/{totalPages} and page-change callbacks.
 - **Detail Screen:** Full-screen view with priority chips, HTML content, attachment cards, and engagement stats.

Data processing:

- The list uses useNewsfeed to call GET /api/v1/announcements with a fixed pageSize of 10.
- Pagination is managed via local newsPage state; changing pages scrolls the list back to the top.
- Unread state is detected if item.isRead === false (null is also treated as unread for the UI).
- The detail screen uses useEffect to trigger a markReadMutation (POST /api/v1/announcements/{id}/read) if an unread article is opened.
- "Mark all read" calls POST /api/v1/announcements/read-all and invalidates the news list cache upon success.
- HTML content for articles is rendered using react-native-render-html with custom styles for dark-mode and typography.

Screen layout:

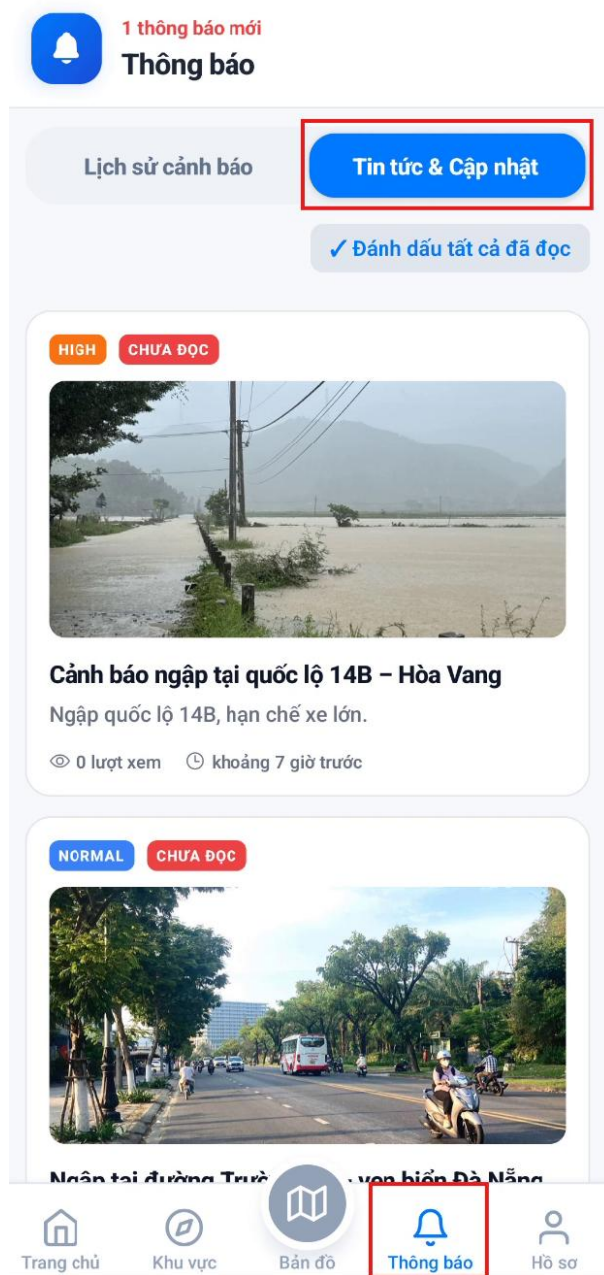


Figure 106 New Feed of Mobile

Function details:

- **Data:**
 - **List Request:** GET /api/v1/announcements with optional filters for priority, dates, and search terms.
 - **AnnouncementItem:** Contains ID, title, summary, priority, image URL, publication date, and read status.
 - **Detail Response:** Includes full HTML content, attachments (JSON string), and author information.
- **Validation:**

- **Priority:** Must be low | normal | high | urgent; unknown values default to "normal" styling.
- **Dates:** Wrapped in try/catch; invalid strings fallback to "Gần đây" (Recent) or "Không xác định" (Unknown).
- **Attachments:** Parsed defensively from JSON; malformed strings produce an empty list without crashing the screen.
- **Read Guard:** A hasMarkedRead flag ensures the /read API is only called once per detail screen mount.
- **Business rules:**
 - Any user can view news, but read-state tracking is user-scoped and requires authentication.
 - Priority levels (URGENT/HIGH vs. NORMAL/LOW) drive UI coloring and push notification behavior.
 - Pagination is entirely server-driven; the client does not perform local slicing.
- **Normal case:**
 - User selects "News" segment -> list of 10 items appears -> user taps an item with a CHƯA ĐỌC chip.
 - Detail screen opens -> full content renders -> background mutation marks the item as read.
 - User returns to the list -> the CHƯA ĐỌC chip is removed and the header unread dot clears.
- **Abnormal case:**
 - **Fetch failure:** EmptyNotificationsState renders with a pull-to-refresh option.
 - **Article not found (404):** Detail screen shows "Không tìm thấy bài viết" (Article not found) with a back button.
 - **Unauthenticated user:** Article content renders normally, but read mark mutations return 401 and are silently ignored.
 - **Malformed JSON:** Attachment cards are silently omitted from the detail view.

3.47 Create Newsfeed

Function trigger:

- **Navigation path:** Admin Portal: "/admin/news" (page title "Tin Tức") OR Moderator Portal: "/moderator/news" (page title "Tin Tức") > Button "Tạo Bản Tin"
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** Create a newsfeed flood to send to users.
- **Interface:**
- **Actions:**
 - Click "Tạo Bản Tin" to open create dialog.
 - Fill required fields and click "Tạo thông báo".
 - **Create dialog:** Dialog title "Tạo thông báo".
 - Dialog description: "Tạo thông báo mới để gửi đến người dùng."
 - Footer buttons: "Hủy", "Tạo thông báo" (loading: "Đang tạo...").

- **Form fields (UI labels):** "Tiêu đề *" (required), "Nội dung *" (supports HTML), "Tóm tắt" (optional), "Hình ảnh đại diện" (mode: "Nhập URL" or "Upload file"), "Đối tượng nhận *" (Tất cả người dùng / Theo khu vực / Theo vai trò), optional "Mã khu vực" / "Tên vai trò", "Độ ưu tiên *" (Thấp/Bình thường/Cao/Khẩn cấp), "Đặt lịch đăng" (datetime-local).

Data processing:

- FE validates input (e.g. title/content required, length constraints).
- If image mode is upload, FE uploads to "NEXT_PUBLIC_UPLOAD_IMAGE_ENDPOINT" and fills "imageUrl".
- FE builds "CreateAnnouncementPayload" and calls `newsApi.createAnnouncement(payload)`.
- On success: toast "Tạo thông báo thành công!", close dialog and refresh listing.

Screen layout:

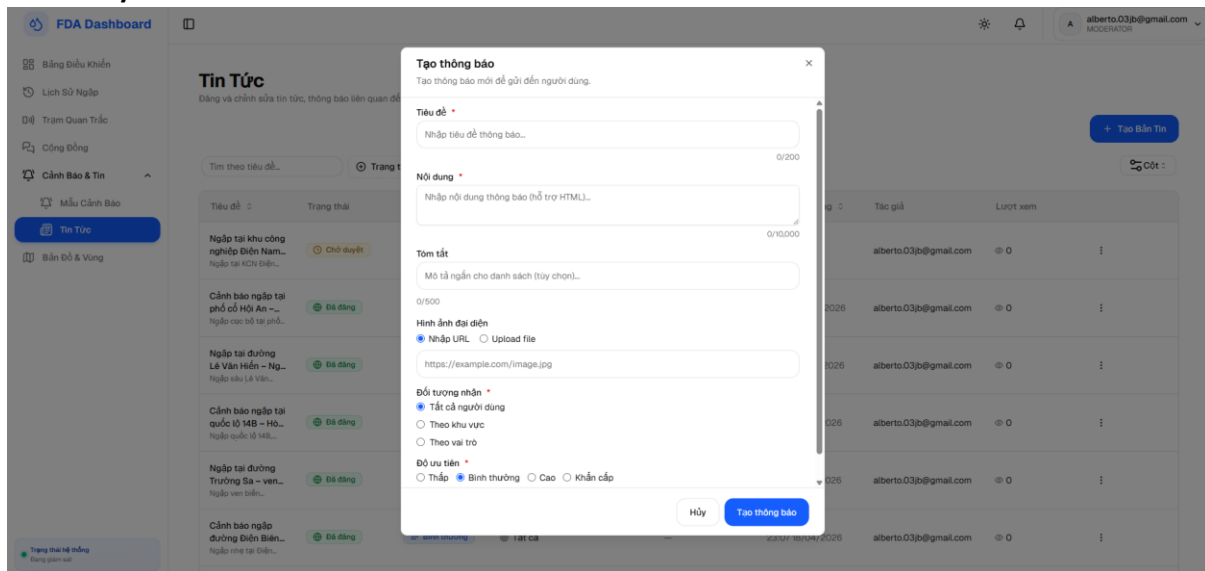


Figure 107 Create Newsfeed

Function details:

- **Data:**
 - Types/payloads: "Announcement", "CreateAnnouncementPayload".
 - Endpoint: "POST /admin/announcements".
- **Validation:**
 - "title" required (max 200).
 - "content" required (max 10,000).
 - If scheduled, date-time must be greater than current time.
 - If image upload selected, must upload successfully to obtain "imageUrl".
 - (Validation messages are Vietnamese in this dialog, e.g. "Tiêu đề là bắt buộc".)
- **Business Rules:**
 - Target is "all" by default.
 - "scheduledAt" is converted to ISO string on submit when scheduling enabled.
- **Normal Case:** Moderator/Admin creates a news item with title, content, priority, target, optionally schedules it. System creates and shows "Tạo thông báo thành công!".

- **Abnormal Case:** Upload endpoint missing: error "Thiếu cấu hình NEXT_PUBLIC_UPLOAD_IMAGE_ENDPOINT".

3.48 Update Newsfeed

Function trigger:

- **Navigation path:** • "Tin Tức" ("/admin/news" or "/moderator/news") > Row "Thao tác" > "Sửa"
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** Update an existing announcement. UI restricts editing to announcements with status "draft" or "pending".
- **Interface:**
 - Actions: - Open row menu "Thao tác".
 - Click "Sửa" (only visible when status is "draft" or "pending").
 - Update fields and click "Lưu thay đổi".
 - Edit dialog: Dialog title: "Chỉnh sửa thông báo".
 - Dialog description: "Cập nhật nội dung thông báo. Chỉ có thể sửa khi trạng thái là Bản nháp hoặc Chờ đăng."
 - Footer buttons: "Hủy", "Lưu thay đổi" (loading: "Đang lưu...").

Data processing:

- 1) FE pre-fills "UpdateAnnouncementPayload" from selected "Announcement".
- 2) FE validates required fields.
- 3) FE calls "newsApi.updateAnnouncement(announcementData.id, payload)".
- 4) On success: toast "Cập nhật thông báo thành công!" and refresh listing.

Screen layout:

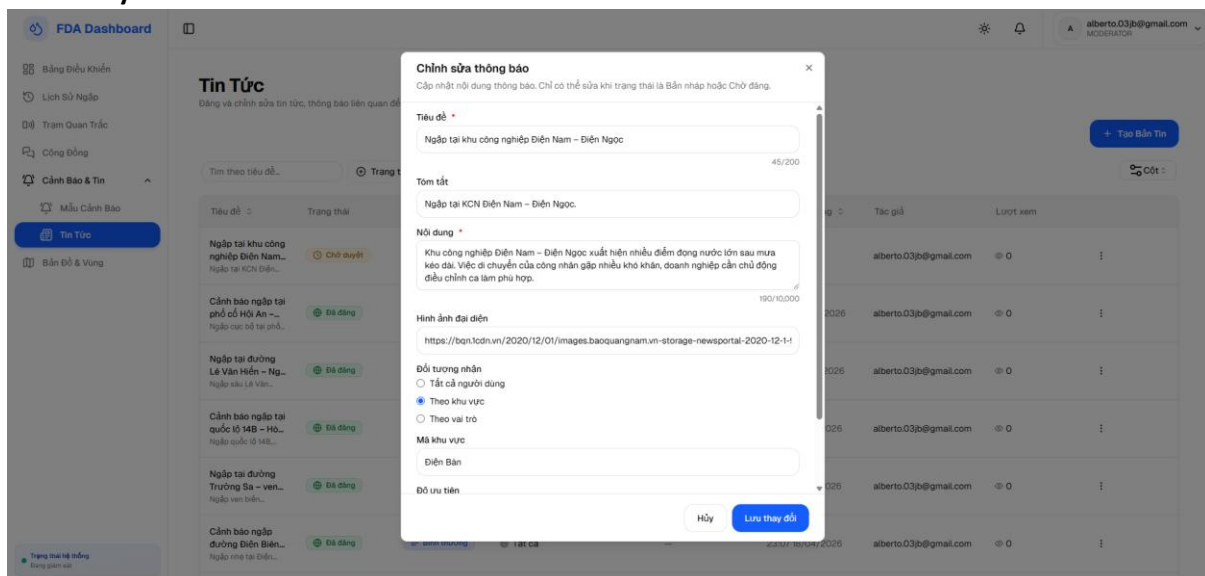


Figure 108 Update Newsfeed

Function details:

- **Data:**
 - Types/payloads: "Announcement", "UpdateAnnouncementPayload".
 - Endpoint: "PUT /admin/announcements/{id}".
- **Validation:** - "title" required (max 200).
 - "content" required (max 10,000).
 - If scheduled, date-time must be greater than current time.
- **Business Rules:**
 - Can edit only when status is "draft" or "pending".
 - Scheduling converts local datetime to ISO string.
- **Normal Case:** User edits a draft announcement, saves changes, and sees "Cập nhật thông báo thành công!".
- **Abnormal Case:** Status not editable: "Sửa" action is not available.

3.49 Delete Newsfeed

Function trigger:

- **Navigation path:** "Tin Tức" ("/admin/news" or "/moderator/news") > Row "Thao tác" > "Xóa"
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Moderator
- **Purpose:** Update an existing announcement. UI restricts editing to announcements with status "draft" or "pending".
- **Interface:**
 - **Actions:** - Open row menu "Thao tác" → click "Xóa".
 - Confirm modal shows dynamic description:
 - If status is "published" or "cancelled": soft delete ("xóa mềm"): "sẽ bị ẩn khỏi danh sách".
 - Otherwise: hard delete ("xóa cứng"): "sẽ bị xóa vĩnh viễn".
 - **Confirmation modal:** Title: "Xóa thông báo".
 - Buttons: "Hủy", "Xóa" (loading: "Đang xóa...").

Data processing:

- 1) FE calls "newsApi.deleteAnnouncement(announcement.id)".
- 2) On success: toast "Đã xóa thông báo" and refresh listing.
- 3) On failure: toast "Xóa thông báo thất bại".

Screen layout:

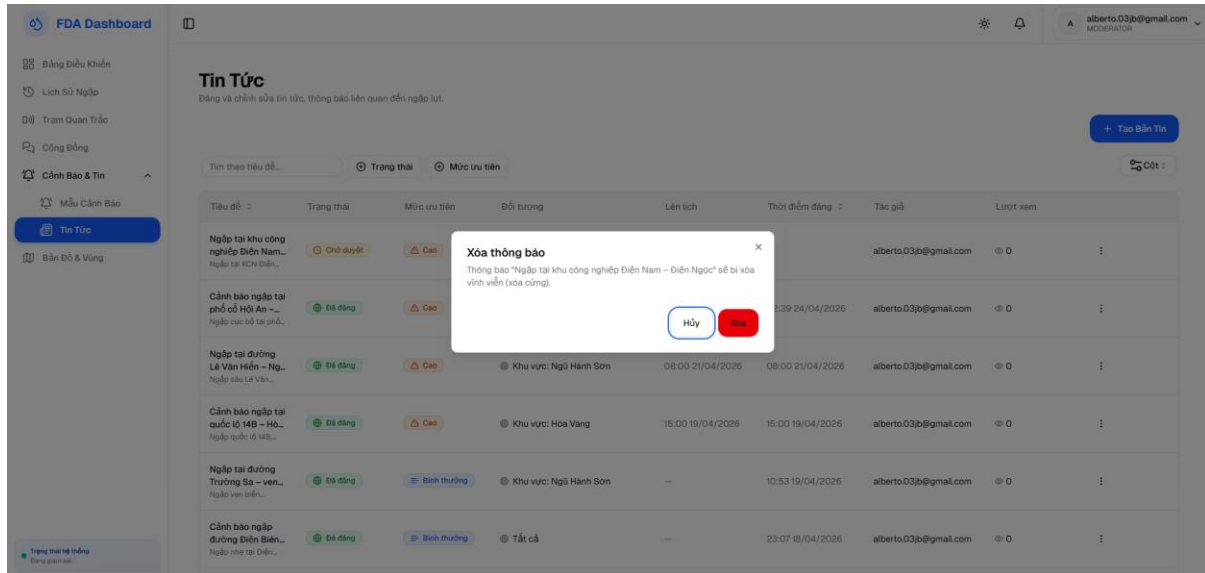


Figure 109 Delete Newsfeed

Function details:

- **Data:**
 - Endpoint: "DELETE /admin/announcements/{id}".
 - Type: "DeleteAnnouncementResponse".
- **Validation:** -
 - Confirmation required.
 - Token/auth handled by "apiFetch".
- **Business Rules:**
 - Behavior described as soft/hard delete depends on announcement "status" in UI messaging.
 - After delete, listing refreshes.
- **Normal Case:** User deletes an announcement, confirms in modal, system deletes and shows "Đã xóa thông báo".
- **Abnormal Case:**
 - (1) API failure: toast "Xóa thông báo thất bại".
 - (2) Network error: listing remains unchanged until refreshed.

3.50 Use Offline Mode (Mobile)

Function trigger:

- **Navigation path:** App Launch OR any in-app navigation to screens consuming persisted React Query (Map, Areas, Alerts, Community).
- **Timing Frequency:** Continuous. Offline Mode activates implicitly when @react-native-community/netinfo reports no connection.

Function description:

- **Actors/Roles:** All Users (Authenticated + Guest).
- **Purpose:** To allow users to view the latest flood conditions, saved areas, and alerts even without connectivity during disaster conditions. Scope is strictly read-only.
- **Interface:** * Offline Banner: Animated banner "Đang offline — hiển thị dữ liệu đã lưu" slides down when offline.

- **Cached Data Badge:** Displays `dataUpdatedAt` (e.g., "Cập nhật X phút trước") on screens serving cached data.
- **Write Interactions:** Submit buttons show an inline "Cần kết nối mạng" hint and are disabled.

Data processing:

- On app start, the system rehydrates the React Query cache from AsyncStorage (key `fda_query_cache`).
- Queries use `networkMode = "offlineFirst"` to return cached values immediately when offline and pause fetching.
- The system whitelists specific query keys for persistence: `floodSeverity`, `userAreas`, `communityReports`, and `alerts`.
- `useNetworkStatus` hook monitors connectivity to drive UI components (Banner, Disabled buttons).

Screen layout:

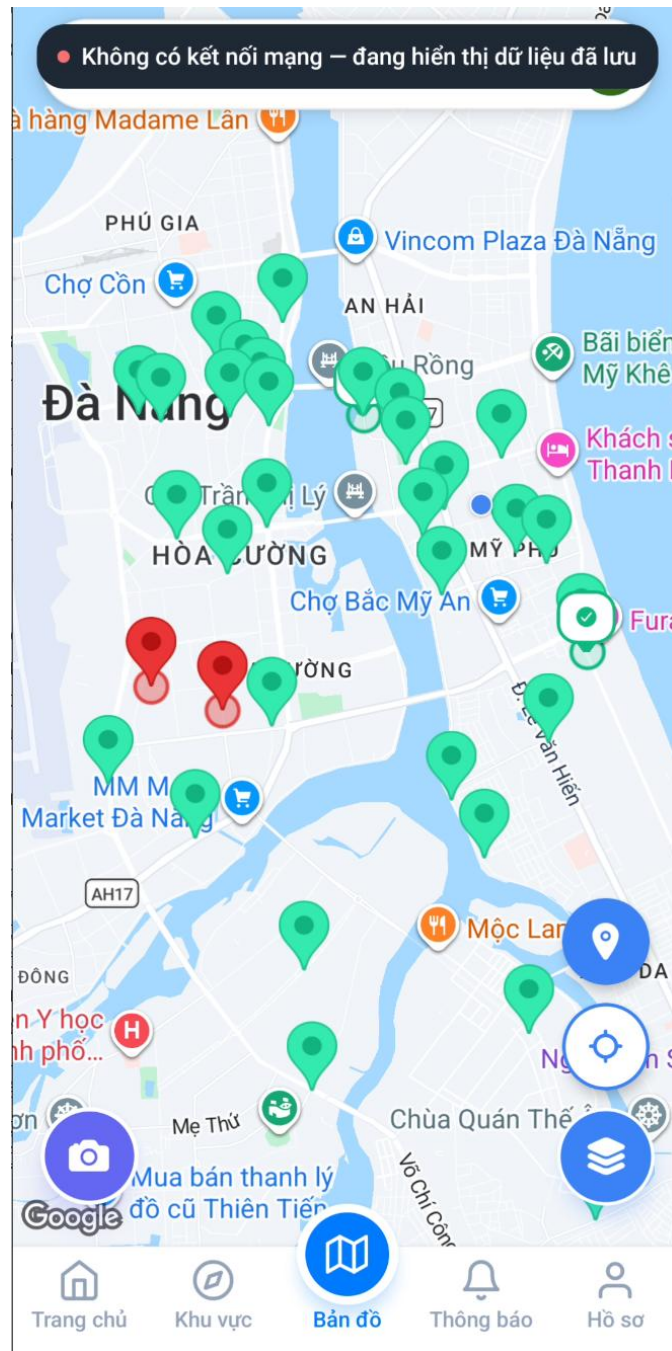


Figure 110 Use Offline Mode (Mobile)

Function details:

- **Data:** AsyncStorage serialized snapshot; Whitelisted query keys; NetInfo state.
- **Validation:**
 - Cache age: Entries older than 24h are discarded.
 - Network state: isOnline is determined by both isConnected and isInternetReachable.
- **Normal case:** User opens the map while in a flood area with no signal. The system renders markers from the 24h cache and displays the offline banner and data timestamp.
- **Abnormal case:**
 - No cache exists: Screen shows "Không có dữ liệu đã lưu" empty state.
 - Corrupted payload: System discards bad snapshot and proceeds with empty cache.

3.51 View Operational Logs (Sensors / Alerts / Moderation)

Function trigger:

- **Navigation path:** Operational Logs Viewer
- **Timing Frequency:** On demand.

Function description:

- **Actors/Roles:** Admin, Moderator
- **Purpose:** To provide a single view for auditing and troubleshooting system activities across different domains.
- **Interface:** Filterable, searchable table view with filters for Log Type (Sensor, Alert, Admin) and Date Range.

Data processing:

- The system queries the appropriate log table based on user filter:
 - **Sensor/Device Logs:** Queries DevicesLogs.
 - **Alert Logs:** Queries Alerts.
 - **Moderation Logs:** Queries AdminActions.
- The results are presented chronologically.

Screen layout:

Thời điểm	Mức độ	Nhóm	Sự kiện	Người dùng	Đối tượng	Lỗi	Chi tiết
02:45:19 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:44:57 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "AlertProcessing.Job...
02:44:19 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:43:19 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:42:56 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "AlertProcessing.Job...
02:42:17 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:41:17 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:40:56 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "AlertProcessing.Job...
02:40:17 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...
02:39:17 24/4/2026	Thông tin	system	job_completed	—	—	—	{"jobName": "...

Figure 111 View Operational Logs

Function details:

- **Data:** Log Message, Event Type, Timestamp, User ID (if applicable), Station ID.
- **Normal case:** Moderator filters for 'Sensor Logs' in the last 7 days. The system displays a list of **devices_logs** entries, quickly identifying recent connectivity problems based on the **message** column.

4. Non-Functional Requirements

4.1 External Interfaces

User Interfaces:

- **Web Portal (Admin & Moderator):** The administration dashboard is built on NextJS, optimized for modern web browsers (Chrome, Edge). The interface focuses on intuitive visualization with statistical charts and heatmaps.
- **Mobile Application (Citizen):** A cross-platform mobile application developed with React Native (compatible with Android & iOS). It features a Minimalist Design style, prioritizing map visualization and emergency reporting buttons, ensuring high visibility and ease of use even in adverse weather conditions (High Contrast UI).

External Systems & Hardware:

- **IoT Hardware Integration:** Interacts with ESP32 hardware via the MQTT protocol. The system must parse JSON payloads containing real-time water levels and battery status from SR04T sensors.
- **Map Service Provider:** Integrates Mapbox GL JS API to render background maps, flood layers, and navigation routes with low latency.
- **Notification Service:** Integrates Firebase Cloud Messaging (FCM) for push notifications and eSMS/Twilio gateway for emergency SMS alerts when internet connectivity is unavailable.

4.2 Quality Attributes

4.2.1 Usability

Training Time:

- **Citizen:** No training required. Users should intuitively understand map navigation and alert reception upon first use (Intuitive Design).
- **Moderator/Admin:** Administrative tasks (e.g., configuring sensor nodes, moderating reports) should require no more than 30–60 minutes of familiarization via the user manual.

Task Efficiency:

- **Citizen Reporting:** Submitting a flood report must be completed in under 3 steps and within 1 minute to ensure timeliness.
- **Admin Operations:** Administrators must be able to add or calibrate a sensor node within 5 minutes.

Accessibility:

- The alert color coding system (Green/Yellow/Orange/Red) must adhere to contrast standards to support visually impaired users and ensure visibility in low-light/stormy conditions.

4.2.2 Reliability

Availability:

- The Server and Database must achieve 99% uptime to ensure continuous availability during critical flood events.
- **Offline Mode Support:** The Mobile App must support offline functionality, allowing citizens to view cached maps and the last known alert data even without an internet connection.

IoT Resilience:

- **Auto-reconnect:** IoT devices (ESP32) must automatically attempt to reconnect to the MQTT Broker within 30 seconds of network loss.
- **Data Buffering:** Sensor data must be buffered locally on the device during connection outages and re-transmitted immediately upon reconnection to prevent data loss.

Data Accuracy:

- Displayed water level data must maintain an accuracy variance of $\pm 4\text{cm}$ compared to actual levels (after noise filtering algorithms).

4.2.3 Performance

Response Time:

- **Alert Latency:** The time from critical water level detection by sensors to user notification (Push) must be under 10 seconds.
- **API Response:** Standard APIs (historical data, station list) must respond within 500ms under normal load conditions.
- **Map Rendering:** Flood map layers must fully render within 2 seconds on a standard 4G connection.

Concurrency:

- The backend system must support at least 1,000 concurrent users accessing the platform simultaneously during flood emergencies without crashing.
- The MQTT Broker must be capable of processing continuous data streams from 50 sensor stations at a frequency of 30 seconds per packet

4.2.4 Security

Authentication & Authorization:

- Uses JWT (JSON Web Token) for secure session authentication.
- **Strict Role-Based Access Control (RBAC):** Only Moderator users can moderate community reports; only Admins can modify sensor thresholds.

Data Privacy:

- User location data used for "Safe Routing" features must be anonymized or encrypted and must not be stored as persistent personal tracking history.

Communication Security:

- All communications between Client (App/Web) and Server must be encrypted using HTTPS/TLS 1.2.

4.2.4 Maintainability

Modular Architecture:

- The backend follows Clean Architecture principles, ensuring clear separation between Core Logic, Infrastructure, and APIs to facilitate future upgrades and maintenance.

Scalability:

- The system must support Horizontal Scaling via Docker, allowing for the rapid addition of server resources when the number of sensors or users increases significantly.

Logging & Monitoring:

- Integrates centralized logging (e.g., Serilog/ELK Stack) to track API errors and IoT device status.
- Includes a System Health Check Dashboard for Admins to detect API failures or Database overloads proactively.

5. Requirement Appendix

5.1 Business Rules

ID	Rule Definition
BR-01	Free Tier users are restricted to registering a maximum of 1 specific location (Geofence) for automatic flood monitoring.
BR-02	Flood alerts for Free Tier users must be delivered exclusively via Mobile App Push Notification. SMS delivery is disabled.
BR-03	Premium Tier users are authorized to register and monitor up to 5 distinct locations simultaneously.
BR-04	Premium Tier users must be able to enable SMS as an additional alert channel to ensure reception even without internet (3G/4G/Wifi).

ID	Rule Definition
BR-05	Upon expiration of the Premium subscription, the system must automatically revert the account to Free Tier status and disable the 4 most recently added locations, keeping only the primary one active.
BR-06	Water level data sent from sensors must be within the valid range (10cm - 200cm) to be processed.
BR-07	A sensor is automatically marked as "Offline" if no heartbeat signal is received for more than 15 minutes.
BR-08	Data transmission frequency must automatically increase (from 10 mins to 30 secs) when water level exceeds the "Warning" threshold (10cm).
BR-09	The system must NOT send duplicate flood alerts for the same location within a 30-minute window unless the alert level increases.
BR-10	Push notifications must only be sent to users currently located within a 1km radius of the flooded area.
BR-11	Upon detecting "Danger" level (>40cm) within a 2km radius of the flooded area, the system must immediately trigger notifications via all channels (App, SMS).
BR-12	A community report is considered "Verified" only after receiving at least 3 confirmations from other users.
BR-13	Users cannot submit more than 5 reports within 10 minutes to prevent spamming.
BR-14	Only Admin accounts have the permission to modify sensor calibration parameters and alert thresholds.
BR-15	Water Level Smoothing: The displayed water level must be calculated as the average of 5 consecutive valid sensor readings to filter out noise/spikes.
BR-16	Image Data Constraints: User-submitted flood images must be in JPG/PNG format, maximum size 5MB, and must contain GPS + Time for verification.
BR-17	Historical flood data must be retained in the database for a minimum of 12 months for AI model retraining purposes.

Table 8 Business Rules

5.2 Common Requirements

1. Supports PNG, JPG, JPEG image file format, maximum image size 10MB.
2. Data in the form of lists, tables, etc. will be displayed as data tables/paging supported.
3. On each web page, there should be no more than 3 fonts displayed.
4. All changes, updates, and modifications are alerted by pop-up.
5. Ensure the platform is accessible 24/7, with maintenance scheduled during off-peak hours.

5.3 Application Messages List

#	Message code	Message Type	Context	Content
1	MSG01	In-line (Red)	User leaves a mandatory field empty during form submission.	<i>The {field_name} is required.</i>
2	MSG02	In-line (Red)	User enters data in an invalid format (e.g., wrong email, phone number).	<i>Invalid format. Please enter a valid {field_name}.</i>

3	MSG03	In-line (Red)	Login failed due to wrong credentials.	<i>Incorrect username or password. Please try again.</i>
4	MSG04	Toast (Success)	Data saved/updated successfully (Profile, Settings, Station Config).	<i>{object_name} updated successfully.</i>
5	MSG05	Toast (Error)	System error or API failure (500 Error).	<i>Something went wrong. Please try again later.</i>
6	MSG06	Banner (Warning)	[Offline Mode] App detects network loss..	<i>You are currently offline. Showing cached flood data.</i>
7	MSG07	Toast (Success)	[Offline Sync] Network restored and data synchronized.	<i>You are back online. Data synchronized successfully.</i>
8	MSG08	Modal (Alert)	[Subscription] Free user tries to add 2nd location.	<i>Free Limit Reached. Upgrade to Premium to monitor more locations.</i>
9	MSG09	Toast (Success)	[Community] User submits a flood report successfully.	<i>Report submitted! Thank you for contributing to the community.</i>
10	MSG-10	Modal (Error)	[Community] User uploads image > 5MB or wrong format.	<i>Upload failed. The image must be JPG/PNG and under 5MB.</i>
11	MSG-11	Toast (Info)	[Map Routing] No safe route found due to heavy flooding.	<i>No safe route found. All paths are currently flooded > 20cm.</i>
12	MSG-12	In-line (Red)	[IoT Config] Admin sets threshold lower than permitted range.	<i>Threshold must be between 0cm and 200cm.</i>
13	MSG-13	Toast (Info)	[GPS] App requests location permission but user denies.	<i>Location access is needed to show flood alerts near you.</i>
14	MSG-14	Modal (Confirm)	[Admin] Admin tries to delete a critical sensor station.	<i>Are you sure you want to delete this station? This action cannot be undone.</i>
15	MSG-15	Toast (Success)	[Subscription] Payment processed successfully.	<i>Payment successful! Your Premium features are now active.</i>

Table 9 Application Messages List